WNDRVR
SUPPORT NETWORK

VXWORKS 7 NETWORK STACK PROGRAMMER'S GUIDE, SR0650 EDITION 3

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Network Stack Programmer's Guide, SR0650 Edition 3

27 April 2022

1. OVERVIEW

1. About the VxWorks 7 Network Stack

The VxWorks® Network Stack is a full featured dual IPv4/IPv6 TCP/IP stack that supports multithreading and simultaneous use of raw Internet Protocol (IP), User Datagram Protocol (UDP), and Transmission Control Protocol (TCP) over IPv4 and IPv6.

The network stack is delivered as a Red Hat Package Manager RPM (package) and is part of the core networking layer. A package can be updated individually without updating the overall product. You can also select a specific package version to use when configuring your VxWorks source build. For more information, see Configuration and Build Workflow on page 6. For more information on downloading and installing packages, see the *VxWorks 7 Configuration and Build Guide*.

The core networking layer provides basic networking functionality. When this layer is added to your VxWorks installation (along with any required networking device drivers), basic networking functionality, such as ping support and packet forwarding, is provided. Additional advanced networking technologies are available in other layers and supported by the core networking layer.

For further information on setting socket options, see the *VxWorks 7 Sockets Programmer's Guide*.

Next topic: About VxWorks 7 Network Stack Technologies on page 1

2. About VxWorks 7 Network Stack Technologies

The core networking layer includes the VxWorks 7 Network Stack.

The following protocols and features are implemented as part of the core networking layer:

- Address Resolution Protocol (ARP)
  - Gratuitous ARP
  - Proxy ARP
  - Reverse ARP
  - Dynamic ARP
- Internet Control Message Protocol (ICMP)
- Internet Control Message Protocol Version 6 (ICMPv6)
- Internet Group Management Protocol (IGMP)
  - IGMPv1 proxy and router
  - IGMPv2 proxy and router
  - IGMPv3 proxy and router
- Internet Protocol (IP)
- Internet Protocol Version 6 (IPv6)
- Management Information Base (MIB-II)
- Multicast Listener Discovery (MLD)
  - MLDv1 proxy and router
  - MLDv2 proxy and router
- Multicast Routing

WNDRVR

- Neighbor Discovery Protocol (NDP)
- Quality of Service (QoS)
  - Egress network interface queues
  - DiffServ classifier and markers
- Stream Control Transmission Protocol (SCTP)
- Transmission Control Protocol (TCP)
- Tunneling
- User Datagram Protocol (UDP)
- Lightweight User Datagram Protocol (UDP-Lite)
- Virtual local area network (VLAN)
- Virtual Router Redundancy Protocol (VRRP)
- Virtual Routing

Previous topic: About the VxWorks 7 Network Stack on page 1
Next topic: Supported RFCs for the VxWorks 7 Network Stack on page 2

3. Supported RFCs for the VxWorks 7 Network Stack

The VxWorks 7 Network Stack request for comments (RFCs) contain standards and design information defined by the Internet Engineering Task Force (IETF).

These RFCs are accessible from the following Web site:

http://www.ietf.org.

The VxWorks 7 Network Stack complies with the following IETF RFCs:

- RFC 147: *Definition of a socket*
- RFC 768: *User Datagram Protocol*
- RFC 781: *Specification of the Internet Protocol (IP) timestamp option*
- RFC 791: *Internet Protocol*
- RFC 792: *Internet Control Message Protocol*
- RFC 793: *Transmission Control Protocol*
- RFC 826: *Ethernet Address Resolution Protocol: Or Converting Network Protocol Addresses to 48.bit Ethernet Address for Transmission on Ethernet Hardware*
- RFC 903: *A Reverse Address Resolution Protocol*
- RFC 919: *Broadcasting Internet Datagrams*
- RFC 922: *Broadcasting Internet datagrams in the presence of subnets*
- RFC 925: *Multi-LAN Address Resolution*
- RFC 950: *Internet Standard Subnetting Procedure*
- RFC 1027: *Using ARP to implement transparent subnet gateways*
- RFC 1058: *Routing Information Protocol*
- RFC 1071: *Computing the Internet checksum*
- RFC 1112: *Host extensions for IP multicasting.*

- RFC 1122: *Requirements for Internet Hosts -- Communication Layers*. The Wind River Network Stack does not implement the following elements of this RFC:
  - 2.3.3 - Cannot send or receive RFC-1042 packets
  - 3.2.2.3 - Source Quench not implemented
  - 3.2.2.9 - Address Mask Request/Reply not implemented
  - 3.3.1.3 - It is possible to route based on TOS, but the TOS field is not part of the route entries.
- RFC 1191: *Path MTU discovery*
  - The Path MTU will be lowered to the (configurable) minimum MTU if the destination unreachable/need frag ICMP message does not contain the Next-Hop MTU field.
- RFC 1256: *ICMP Router Discovery Messages*
- RFC 1282: *BSD Rlogin*
- RFC 1323: *TCP Extensions for High Performance*
- RFC 1517: *Applicability Statement for the Implementation of Classless Inter-Domain Routing CIDR*
- RFC 1518: *An Architecture for IP Address Allocation with CIDR*
- RFC 1519: *Classless Inter-Domain Routing (CIDR): an Address Assignment and Aggregation Strategy*
- RFC 1624: *Computation of the Internet Checksum via Incremental Update*
- RFC 1701: *Generic Routing Encapsulation (GRE)*
- RFC 1724: *RIP Version 2 MIB Extension*
- RFC 1812: *Requirements for IP Version 4 Routers*
- RFC 1924: *A Compact Representation of IPv6 Addresses*
- RFC 1981: *Path MTU Discovery for IP version 6*
- RFC 2003: *IP Encapsulation within IP*
- RFC 2004: *Minimal Encapsulation within IP*
- RFC 2005: *Applicability Statement for IP Mobility Support*
- RFC 2018: *TCP Selective Acknowledgment Options*
- RFC 2113: *IP Router Alert Option*
- RFC 2338: *Virtual Router Redundancy Protocol* (obsoleted by RFC 3768)
- RFC 2375: *IPv6 Multicast Address Assignments*
- RFC 2385: *Protection of BGP Sessions via the TCP MD5 Signature Option*
- RFC 2450: *Proposed TLA and NLA Assignment Rule*
- RFC 2460: *Internet Protocol, Version 6 (IPv6) Specification*
- RFC 2464: *Transmission of IPv6 Packets over Ethernet Networks*
- RFC 2473: *Generic Packet Tunneling in IPv6 Specification*
- RFC 2507: *IP Header Compression*
- RFC 2581: *TCP Congestion Control* (obsoletes RFC 2001; obsoleted by RFC 5681)
- RFC 2583: *Guidelines for Next Hop Client (NHC) Developers*
- RFC 2597: *Assured Forwarding PHB Group*
- RFC 2710: *Multicast Listener Discovery (MLD) for IPv6*
- RFC 2711: *IPv6 Router Alert Option*
- RFC 2780: *IANA Allocation Guidelines For Values In the Internet Protocol and Related Headers*
- RFC 2890: *Key and Sequence Number Extensions to GRE*
- RFC 2933: *Internet Group Management Protocol MIB*
- RFC 2988: *Computing TCP's Retransmission Timer* (obsoleted by RFC 6298)
- RFC 2991: *Multipath Issues in Unicast and Multicast Next-Hop Selection*

- RFC 3021: *Using 31-Bit Prefixes on IPv4 Point-to-Point Links*
- RFC 3376: *Internet Group Management Protocol, Version 3* (obsoletes RFC 2236)
- RFC 3484: *Default Address Selection for Internet Protocol version 6 (IPv6)*
- RFC 3493: *Basic Socket Interface Extensions for IPv6* (obsoletes RFC 2553)
- RFC 3513: *Internet Protocol Version 6 (IPv6) Addressing Architecture*
- RFC 3542: *Advanced Sockets Application Program Interface (API) for IPv6*
- RFC 3587: *IPv6 Global Unicast Address Format* (obsoletes RFC 2374)
- RFC 3590: *Source Address Selection for the Multicast Listener Discovery (MLD) Protocol*.
- RFC 3768: *Virtual Router Redundancy Protocol (VRRP)* (obsoletes RFC 2338)
- RFC 3769: *Requirements for IPv6 Prefix Delegation*
- RFC 3810: *Multicast Listener Discovery Version 2 (MLDv2) for IPv6*
- RFC 3828: *The Lightweight User Datagram Protocol (UDP-Lite)*
- RFC 3873: *Stream Control Transmission Protocol Management Information Base (MIB)*
- RFC 3879: *Deprecating Site Local Addresses*
- RFC 3927: *Dynamic Configuration of IPv4 Link-Local Addresses*
- RFC 4007: *IPv6 Scoped Address Architecture*
- RFC 4193: *Unique Local IPv6 Unicast Addresses*
- RFC 4213: *Basic Transition Mechanisms for IPv6 Hosts and Routers* (does not implement the optional (MAY) feature described in 3.2.2., Dynamic Tunnel MTU) (obsoletes RFC 2893)
- RFC 4291: *IP Version 6 Addressing Architecture* (obsoletes RFCs 2373 and 3513)
- RFC 4293: *Management Information Base for the Internet Protocol (IP)*
- RFC 4294: *IPv6 Node Requirements*
- RFC 4443: *Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification* (obsoletes RFC 2463)
- RFC 4604: *Using Internet Group Management Protocol Version 3 (IGMPv3) and Multicast Listener Discovery Protocol Version 2 (MLDv2) for Source-Specific Multicast*
- RFC 4605: *Internet Group Management Protocol (IGMP) / Multicast Listener Discovery (MLD)-Based Multicast Forwarding ("IGMP/MLD Proxying")* The operation of a multicast proxy is described in RFC 4605. The Wind River multicast proxy implements the server part of the RFCs listed in this section.
- RFC 4607: *Source-Specific Multicast for IP*
- RFC 4692: *Considerations on the IPv6 Host Density Metric*
- RFC 4704: *The Dynamic Host Configuration Protocol for IPv6 (DHCPv6) Client Fully Qualified Domain Name (FQDN) Option*
- RFC 4861: *Neighbor Discovery for IP version 6 (IPv6)* (obsoletes RFC 2461)
- RFC 4862: *IPv6 Stateless Address Autoconfiguration* (obsoletes RFC 2462)
- RFC 4941: *Privacy Extensions for Stateless Address Autoconfiguration in IPv6* (obsoletes RFC 3041)
- RFC 4960: *Stream Control Transmission Protocol*
- RFC 5095: *Deprecation of Type 0 Routing Headers in IPv6*
- RFC 5156: *Special-Use IPv6 Addresses*
  - not supported for Tunneling IPv6 over UDP through Network Address Translations (Teredo service)
  - not supported for an IPv6 Prefix for Overlay Routable Cryptographic Hash Identifiers (ORCHID)
- RFC 5175: *IPv6 Router Advertisement Flags Option*
- RFC 5227: *IPv4 Address Conflict Detection.* The Wind River Network Stack implements all MUST and SHALL functionality, as applicable to 802.3 and 802.11 hosts.)
- RFC 5681: *TCP Congestion Control* (obsoletes RFC 2581). The Wind River Network Stack implements all MUST and SHALL functionality.

- RFC 5798: *Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6*. (Obsoletes RFC 3768.)
- RFC 5925: *The TCP Authentication Option*
- RFC 5927: *ICMP Attacks against TCP*
- RFC 5961: *Improving TCP's Robustness to Blind In-Window Attack*
- RFC 6298: *Computing TCP's Retransmission Timer* (obsoletes RFC 2988). The Wind River Network Stack implements all MUST and SHALL functionality.
- RFC 6528: *Defending against Sequence Number Attacks*

Related RFCs

- RFC 0854: *Telnet Protocol Specification*
- RFC 0951: *Bootstrap Protocol*
- RFC 1014: *XDR: External Data Representation standard*
- RFC 1542: *Clarifications and Extensions for the Bootstrap Protocol*
- RFC 1700: *Assigned Numbers*
- RFC 1831: *RPC: Remote Procedure Call Protocol Specification Version 2*
- RFC 2131: *Dynamic Host Configuration Protocol*
- RFC 2132: *DHCP Options and BOOTP Vendor Extensions*
- RFC 2242: *NetWare/IP Domain Name and Information*
- RFC 2849: *The LDAP Data Interchange Format (LDIF) - Technical Specification*
- RFC 3152: *Delegation of IP6.ARPA*
- RFC 3232: *Assigned Numbers: RFC 1700 is Replaced by an On-line Database*
- RFC 3633: *IPv6 Prefix Options for Dynamic Host Configuration Protocol (DHCP) version 6*

Internet Engineering Task Force Internet-Drafts

- *Sockets API Extensions for Stream Control Transmission Protocol (SCTP)*, **draft-ietf-tsvwg-sctpsockets-19.txt**

Previous topic: About VxWorks 7 Network Stack Technologies on page 1

2. CONFIGURATION AND BUILD

1. Configuration and Build Workflow

Your VxWorks® Network Stack installation includes source code but does not include pre-built kernel libraries.

Before beginning any VxWorks development, you must build the VxWorks kernel libraries. You must also build a VxWorks runtime kernel image based on the kernel libraries.

For specific instructions on how to configure and build a VSB project and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the *VxWorks 7 Configuration and Build Guide*.

For a list of available source configuration options for the core networking layer of the Wind River Network Stack, see Source Build Configuration Options on page 6.

For a list of available kernel configuration components for Wind River Network Stack, see Required VIP Components for a Minimal Network Stack on page 11.

For information about feature-specific configuration, see the respective chapter and it's configuration topics.

Next topic: Source Build Configuration Options on page 6

2. Source Build Configuration Options

The VxWorks source build (VSB) configuration options enable the configuration of the kernel libraries for your operating environment.

Core Networking Layer

For information on a minimal configuration, see Basic Networking Configuration Example on page 14.

Name	Description
END	Use this option to enable Ethernet network device support. You must have network driver support for basic networking functionality.
END_DRV	This option provides a driver for the network device.
END_LIB	This option provides the network driver support library.
IPNET	Use this option to add support for the core networking stack and networking support infrastructure.
IPNET_COREIP	This option enables the core networking stack. The core stack can be further configured using the options described in *Core Networking Build Options*.

Name	Description
MIB2	This option enables the SNMP Management Information Base (MIB) for the networking stack and for network interface card drivers.
NET_BASE	This option enables the basic network libraries.

Core Networking Build Options

For information on core networking source build caveats, see *Core Networking Source Build Caveats.*

Name	Description
IPNET_BUILD_MODE_CHOICE - BUILD_MODE_speed - BUILD_MODE_debug	These options allow you to override the global debug mode set with the Global Debug Flag option. Use the BUILD_MODE_speed option to compile the network stack with the debug flag (-g flag) disabled and with normal optimizations enabled (optimization set to level 2). Use the BUILD_MODE_debug option to compile the network stack with the debug flag (-g flag) enabled and with normal optimizations disabled. Assert and full debug instrumentation are enabled.
IPNET_INET_CHOICE - FEATURE_IPNET_INET4_ONLY - FEATURE_IPNET_INET6	These options allow you to build the networking libraries with support for IPv4 (FEATURE_IPNET_INET4_ONLY) or both IPv4 and IPv6 (FEATURE_IPNET_INET6). In a wrtool environment, IPv4 and IPv6 libraries are supported by default. To select IPv4 libraries only, use the -inet4 command option as part of the vsb create command. It is not possible to include IPv6 support alone. If you build the networking libraries with support for IPv4 (FEATURE_IPNET_INET4_ONLY), then subsequently rebuild them with support for IPv6 (FEATURE_IPNET_INET6), you will have to create a new VIP based on the VSB with IPv6 support. An existing VIP will not support IPv6, even when the VSB on which it is based is rebuilt with IPv6 support.
IPNET_DEFAULT_SYSLOG_PRIORITY_CHOICE	Use this option to set the default priority level for all networking modules. The default level is IPCOM_DEFAULT_SYSLOG_PRIORITY_WARNING. See also IPCOM_DEBUG_ADVANCED.
- IPCOM_DEFAULT_SYSLOG_PRIORITY_EMERG	Emergency level
- IPCOM_DEFAULT_SYSLOG_PRIORITY_CRIT	Critical level
- IPCOM_DEFAULT_SYSLOG_PRIORITY_ERR	Error level
- IPCOM_DEFAULT_SYSLOG_PRIORITY_WARNING	Warning level

Name	Description
- IPCOM_DEFAULT_SYSLOG_PRIORITY_NOTICE	Notice level
- IPCOM_DEFAULT_SYSLOG_PRIORITY_INFO	Informational level
- IPCOM_DEFAULT_SYSLOG_PRIORITY_DEBUG	Basic debug level
- IPCOM_DEFAULT_SYSLOG_PRIORITY_DEBUG2	Detailed debug level
IPCOM_DEBUG_ADVANCED	To set the default debugging level for modules that support advanced debugging (such as PPP with IPPPP_DEBUG), enable this option. Networking modules with advanced debugging capabilities can have a different default debugging level than that set with the IPNET_DEFAULT_SYSLOG_PRIORITY_CHOICE option. The default level is IPCOM_DEBUG_SYSLOG_PRIORITY_DEBUG.
- IPCOM_DEBUG_SYSLOG_PRIORITY_EMERG	Emergency level
- IPCOM_DEBUG_SYSLOG_PRIORITY_CRIT	Critical level
- IPCOM_DEBUG_SYSLOG_PRIORITY_ERR	Error level
- IPCOM_DEBUG_SYSLOG_PRIORITY_WARNING	Warning level
- IPCOM_DEBUG_SYSLOG_PRIORITY_NOTICE	Notice level
- IPCOM_DEBUG_SYSLOG_PRIORITY_INFO	Informational level
IPCOM_DEBUG_SYSLOG_PRIORITY_DEBUG	Basic debug level
- IPCOM_DEBUG_SYSLOG_PRIORITY_DEBUG2	Detailed debug level

Priority Levels

The available priority levels in ascending order are emergency (minimum logging), critical, error, warning, notice, info, debug, and debug2. When you select the emergency level, only the emergency log messages are included in the VSB. To include all log messages in the VSB, select the debug2 priority level.

You can change the default priority for all modules at VIP creation time, at boot time, and at run time. At VIP creation time, set the IPCOM_SYSLOGD_DEFAULT_PRIORITY parameter to the desired priority level. At boot time or run time, use the syslog command.

Layer 2 Link Options

Use the COMPONENT_VLAN_STACKING option to enable 802.1ad VLAN stacking in the network stack. For more information on this feature, see *Working with 802.1ad Stacked VLANs*.

Use the COMPONENT_USE_VLAN option to enable 802.1q virtual LAN in the network stack.

Layer 3 Network Options

The table below lists the VSB options that enable layer 3 network functionality.

Name	Description
COMPONENT_NAT	Use this option to enable network address translation (NAT) support in the network stack. For more information on NAT, see the *Wind River NAT for VxWorks 7 Programmer's Guide*.
COMPONENT_TUNNEL_SUPPORT	Use this option to enable various tunneling mechanisms in the network stack. For example, Simple Internet Transition (SIT), IPv6 to IPv4 (6to4), IPv6 over IPv4 (6over4), and Generic Routing Encapsulation (GRE). For more information, see *Tunneling on page 104*.
COMPONENT_IPNET_ROUTE	Use this option to enable IPNET route mode.

Layer 4 Transport Options

The table below lists the VSB options that enable layer 4 transport functionality.

Name	Description
COMPONENT_IPSCTP	Use this option to enable Stream Control Transmission Protocol (SCTP) support in the networking stack.
COMPONENT_IPMCP	Use this option to include a multicast proxy that operates according to RFC 4605 - *Internet Group Management Protocol (IGMP) /Multicast Listener Discovery (MLD)-Based Multicast Forwarding ("IGMP/MLD Proxying")*.
COMPONENT_VIRTUAL_ROUTER	Use this option to enable virtual router support.

Quality of Service

The table below lists the QoS options that enable quality of service and differentiated services.

Name	Description
IPNET_QOS	Use this option to enable quality of service (QoS) in the network stack. For more information, see 8. Quality of Service.
IPNET_DIFFSERV	Use this option to enable differentiated services in the network stack.

Virtual Router Redundancy Protocol

Enable the IPNET_VRRP option to include the virtual router redundancy protocol (VRRP) library in your build.

User Networking Library Options

The table below lists the VSB options that enable user networking libraries.

Name	Description
IPNET_USRSPACE USR_INET4_ONLY USR_INET6	These options allow you to build user networking libraries with support for IPv4 (USR_INET4_ONLY) or both IPv4 and IPv6 (USR_INET6). It is not possible to include IPv6 support alone. IPv6 is only supported in combination with IPv4 support.

Configuring the Network Stack as a Router or Host

You can configure the network stack to act as a router or host.

To configure IPNET as a router, use the **IPNET_IS_ROUTER** macro in *installDir* **/vxworks-7/pkgs_v2/net/ipnet/coreip/src/ipnet2/config/ipnet_config.h**.

Define this macro to configure the network stack to act as a router. You must also define it to include the IPv6 router advertisement daemon. A node can be configured to act as a host if this macro is defined, but it cannot be configured as a router if it is not defined.

To configure the system to act as a host, use ipcom_sysctl( ) to set the required system variables.

Core Networking Source Build Caveats

In some cases, VxWorks source build configuration settings outside of core networking can have an impact on the core networking configuration.

You should be aware of the following items when configuring your VxWorks source build project:

Determinism Settings

When you compile your VxWorks source code, the DETERMINISTIC_SMP option provides a way to tune VxWorks for determinism or performance. It is applicable only to Symmetric Multiprocessing (SMP) systems.

When this option is selected, it ensures that VxWorks is deterministic. When this option is not selected, it causes VxWorks to prioritize performance over deterministic behavior.

Disabling this option improves the performance of the network stack. It also provides greater latitude in the development of your application. If you decide to sacrifice determinism for performance, you can use nondeterministic algorithms (for example, nondeterministic spinlock functions) in your application code.

Global Debug Flag Settings

The VxWorks Global Configuration Options > Global Debug Flag option affects the default settings of the Networking Build Options > Set the IPNET build mode option.

WNDRVR

When the Global Debug Flag option is set to yes, the networking debug mode option is automatically set to yes (debug mode). If the Global Debug Flag option is set to no, the networking speed mode option is enabled.

The Allow Debug -g and Optimization to co-exist option has no effect on the state of the network stack optimization settings.

Including a Network Driver

To successfully include core networking functionality in your application, you must also include a driver for your networking interface device. Network drivers are delivered as an RPM and are included as their own layer.

Note: MUX support is provided as part of the core networking layer and is included with basic stack functionality.

Previous topic: Configuration and Build Workflow on page 6
Next topic: Required VIP Components for a Minimal Network Stack on page 11

3. Required VIP Components for a Minimal Network Stack

In your VxWorks image project (VIP), you can configure the minimal set of network related components required to create a network stack with enough resources to respond to a ping or ping6 request.

This lets a remote system test the network connection to your target. Because you cannot test network connectivity from a target running a minimal stack without ping components.

Basing your VxWorks source build (VSB) on any available Board Support Package (BSP) or CPU type and accepting the default configuration is sufficient to provide basic networking functionality at the source build level. You can then create a VIP (kernel image) based on your VSB. Accepting the default configuration options for networking results in a VIP with basic networking functionality.

For information on how to configure a VSB project and a VIP for basic networking functionality, see Basic Networking Configuration Example on page 14.

The table below lists the VIP Components for Basic Networking Functionality.

Name	Description
INCLUDE_IPNET	This component automatically includes additional components for the common networking infrastructure and support for the routing network stack, the networking daemon, the MUX and network drivers, job queuing, and memory pool buffers. This component also includes IPNET_MAX_REASSEMBLY_LIST_LEN and IPNET_MAX_DGRAM_FRAG_LIST_LEN. IPNET_MAX_REASSEMBLY_LIST_LEN specifies the maximum number of partly reassembled unique IP-datagrams that the stack will keep track of. This parameter defends against the case where an attacker sends a large number of fragmented IP-datagrams, but does not include one or more of the fragment(s). IPNET_MAX_DGRAM_FRAG_LIST_LEN specifies the maximum number of fragment the stack will allow for a single IP-datagram. This parameter defends against the case where an attacker sends as little as 8 bytes of IP payload in each fragment, which would cause a single IP-datagram to use an inordinate number of

Name	Description
	packet buffers. Use these two parameters together to defend against denial-of-service (DoS) attacks.
INCLUDE_IPCOM_USE_INET INCLUDE_IPCOM_USE_INET6	These components pull in UDPv4, ICMPv4, as well as the standard support for IPv4 or UDPv6, ICMPv6, as well as the standard support for IPv6.
INCLUDE_IPTCP	This component pulls in libraries and modules that provide support for TCP in either the IPv4 or IPv6 domains. Which support is included depends on how the network stack source code is compiled. For more information on including IPv4 and IPv6 support in your source build, see Source Build Configuration Options on page 6.
INCLUDE_INETLIB	This component pulls in **inetLib** and other modules that implement functions for manipulating Internet addresses, including the UNIX BSD inet_( ) functions. It includes functions for converting between character addresses in Internet standard dotted decimal notation and integer addresses, functions for extracting the network and host portions out of an Internet address, and functions for constructing Internet addresses given the network and host address parts. There are no component dependencies or configuration parameters associated with this component. For information on the externally callable functions associated with this component, see the **inetLib** reference entry.
INCLUDE_IPATTACH INCLUDE_IP6ATTACH	This component provides ipAttach( ) or ip6Attach( ) for attaching network interfaces at run time.
INCLUDE_IPCOM_USE_ETHERNET	This component pulls in Ethernet support and denotes the type of link layer interface to support.
INCLUDE_IPPING_CMD INCLUDE_IPPING6_CMD	These components pull in the modules for ping or ping6. If you want to run an application, you probably want the default stack components (at a minimum), which also include INCLUDE_APPL_LOG_UTIL.
INCLUDE_NET_BOOT_CONFIG	This component configures a network interface based on the device configuration parameters in the boot line. This component has no associated configuration parameters.
INCLUDE_NET_REM_IO	This component pulls in modules that initialize systems in support of file access on the boot host. This component supports the activities of components such as INCLUDE_NET_DRV and requires the following components: • INCLUDE_NET_DRV • INCLUDE_BOOT_LINE_INIT • INCLUDE_NET_BOOT • INCLUDE_NET_HOST_SETUP

WNDRVR

Name	Description
	For information on how to use the network remote file I/O driver, see the netDrv reference entry.
INCLUDE_NET_SYSCTL	This component pulls in the sysctlLib module and adds network system control support. This component requires INCLUDE_SYSCTL and has no configuration parameters. See the **sysctlLib** reference entry.
INCLUDE_PING INCLUDE_PING6	These components pull in support for the ping utility for IPv4 or IPv6.
INCLUDE_SOCKLIB	This component includes the VxWorks socket library. The network stack is dependent on OS socket support.

inetAddrLib Library Caveats

The return values of inet_aton are not compatible with some other BSD socket implementations. inet_aton returns **OK** for success or **ERROR** for failure, whereas **vxworks.h** defines **OK** as 0 and **ERROR** as (-1).

Like other BSD implementations, the internal API ipcom_inet_aton( ) returns -1 for success and 0 for failure. However, to maintain backwards compatibility for VxWorks socket applications, these values have been reversed by the public wrapper code inet_aton.

Previous topic: Source Build Configuration Options on page 6
Next topic: Adding Network Utilities to an RTP Project on page 13

4. Adding Network Utilities to an RTP Project

If you plan to include the network utilities in a real-time process (RTP) project, you must manually add the network library.

To minimize the footprint of RTP processes, VxWorks does not automatically include the **libnet** network library. If you are building an RTP project and you plan to include the socket APIs and some networking utilities, you must manually add **libnet** by configuring the RTP project's build properties.

Adding Network Utilities to RTP Build Properties in Workbench on page 13

Previous topic: Required VIP Components for a Minimal Network Stack on page 11
Next topic: Basic Networking Configuration Example on page 14

4.1. Adding Network Utilities to RTP Build Properties in Workbench

About This Task

This procedure tells you how to add the network utilities to your RTP project, using Workbench.

Procedure

1. Right-click the RTP project name and select **Properties**.
2. Select **Build Properties** and select the **Libraries** tab.

3. Add the **libnet** library by adding **-lnet** to the library directives.
4. Click **Apply and Close**.

Parent topic: Adding Network Utilities to an RTP Project on page 13

5. Basic Networking Configuration Example

To configure VxWorks 7 with basic networking support, you must build a VxWorks source build (VSB) that supports a minimal network configuration, create a VxWorks image project (VIP) based on the VSB, add the necessary networking components, and build your project.

About This Task

To create a VSB and VIP for this configuration, do the following:

Procedure

1. Create a VSB for your target.
2. Enable the following VSB options:
   - IPNET_COREIP
   - END_LIB
   - MIB2

   MIB2 (note below here - Depending upon your BSP or CPU choice, some or all of the VSB options listed may be enabled by default.)

   For information on these options, see Source Build Configuration Options on page 6.)

3. Build the VSB.
4. Create a VIP and base it on the VSB.
5. Add the following components to the VIP:

   INCLUDE_IPCOM

   INCLUDE_IPCOM_USE_ETHERNET

   INCLUDE_IPCOM_USE_INET

   INCLUDE_IPNET

   INCLUDE_IPNET_IFCONFIG_1

   INCLUDE_IPNET_LOOPBACK_CONFIG

   INCLUDE_IPNET_SOCKET

   INCLUDE_IPNET_USE_LOOPBACK

   INCLUDE_IPNET_USE_MCAST_ROUTING

   INCLUDE_IPNET_USE_NETLINKSOCK

WNDRVR

INCLUDE_IPNET_USE_ROUTESOCK

INCLUDE_IPNET_USE_SOCK_COMPAT

INCLUDE_IPNET_USE_VLAN

INCLUDE_IPPING_CMD

INCLUDE_IPTCP

INCLUDE_IPWRAP_GETIFADDRS

INCLUDE_IPWRAP_IFCONFIG

INCLUDE_IPWRAP_INETLIB

INCLUDE_IPWRAP_IPPROTO

INCLUDE_MBUF_UTIL1

INCLUDE_MUX

INCLUDE_MUX2

INCLUDE_MUX2_OVER_END

INCLUDE_MUXTK

INCLUDE_MUXTK_OVER_END

INCLUDE_MUX_COMMON

INCLUDE_NETBUFLIB

INCLUDE_NETBUFPOOL

INCLUDE_NETMASK_GET

INCLUDE_NETWORK

INCLUDE_NET_BOOT

INCLUDE_NET_DAEMON

INCLUDE_NET_HOST_SETUP

Note: Depending upon your VSB configuration, some or all of the components listed may be enabled by default.

6. Build your VIP.

Results

The kernel image produced can now be used to support simple networking for VxWorks.

For example usage of the basic networking configuration in a simulated networking environment, see Basic Networking Example on page 45.

Previous topic: Adding Network Utilities to an RTP Project on page 13

Next topic: Setting the Memory Limit on page 16

6. Setting the Memory Limit

Use the parameter **IPNET_MEMORY_LIMIT** to define the maximum amount of memory the network stack is allowed to use.

When you configure memory for the VxWorks network stack, the only parameter you set is the overall memory limit. You cannot define the number or size of packets. The network stack is given a maximum amount of memory, which it uses to create the packet pool layout, and the layout of the packet pool changes dynamically to adapt to the current workload.

For example, if an application suddenly starts sending or receiving very large UDP datagrams, the stack adjusts the packet pool to contain more large (that is, 10KB or 64KB) packets and decreases the number of 1500-byte packets, if it needs to stay within the total memory limit.

Define the maximum memory limit at build time by setting **IPNET_MEMORY_LIMIT**, which is a parameter of **INCLUDE_IPNET**.

This parameter specifies the maximum amount of dynamic memory the network stack can use. The default value is 16MB.

Finding the Current Limit

To find the current value of **IPNET_MEMORY_LIMIT**, check the variable using the VxWorks C shell.

Evaluating Memory Needs

When you set the memory limit, choose a value large enough to accommodate all sockets, packet headers, packet buffers, timers, events, TCP segment control blocks (one per non-packed TCP segment), IPv4/IPv6 address blocks, and poll signals used to implement select( ) and blocking calls to sendmsg( ) and recvmsg( ).

The following table lists the nominal sizes of each item. These values may vary between VxWorks releases.

Table 1. Memory Sizes

Item	Size (bytes)
Socket	544
Packet header	672
Packet buffer	Takes 1682, 3182, 10182, 65718 (or 1500, 3000, 10 KB and 64 KB buffers)[1] on page 17
Timer	20
Event	24
TCP segment control block	32
IPv4 address block	112
IPv6 address block	140

Item	Size (bytes)
Poll signal	72

The sizes of the packets were chosen for the following reasons:

- 1500 bytes is, by far, the most commonly used size. This size matches the MTU of Ethernet and loopback devices.
- 3000 bytes was chosen because a PPP frame can, in the worst case, grow to twice its original size during TX framing. Thus, 3000 represents 2 times 1500.
- 10,000 bytes is big enough to hold most fragmented UDP datagrams. There are a number of protocols that generate UDP datagrams up to 8192 bytes.
- 65,535 bytes is the largest packet supported by IPv4 and IPv6, without the IPv6 jumbogram extension header.

The total amount of memory assigned to the stack must never be so small that it cannot fill:

- the send and receive buffers of at least one socket
- all RX and TX descriptors of at least one network interface

For example, a socket with 64KB send and receive buffers using the **gei** driver would need about 800KB for just the buffer and header. This does not include any memory for the socket, timers, and so on.

Thus, the minimum amount of memory would be 1MB for the system to function properly when using one socket under high load with no fragmentation, or using a few (10 or so) sockets under light load.

The default value of 16MB should work for most systems that are using the default VxWorks limit of 50 open file descriptors in the system, which also limits the number of sockets to 50. In most cases, even 10MB should be sufficient.

Previous topic: Basic Networking Configuration Example on page 14
Next topic: Setting the Slab Cache Descriptors on page 17

[1] on page 16

A packet that can hold 1500 bytes of L3 header + L4 header + L4 payload will occupy 1682 bytes of memory. The packet has to be larger than the maximum number of L3+L4 data since there has to be room for L2 headers and also room for putting the data at any alignment specified by the network stack.

7. Setting the Slab Cache Descriptors

You can configure the size of the slab memory caches, which provide the network stack with buffers of a fixed size.

The Wind River Network Stack and the network drivers that work with it use several types of memory pools for memory allocation. Memory allocated by the IP network stack comes ultimately from the system heap. The stack creates multiple “slab memory caches” based upon the system heap; each cache provides the stack with buffers of a fixed size, sometimes partially initialized as objects of a particular kind. These memory caches can grow dynamically, as needed, with a limit on total allocation, by the stack, across all the memory caches. If the maximum is reached, a limited form of “garbage collection” may occur, which returns any completely free memory slabs held in a memory cache to the system pool. The stack allocates both packet buffers and various fixed-size control structures from these memory caches. The stack allocates several other control structures directly from the system heap; these direct allocations do not have an explicit bound.

To specify the size of the slab memory caches, use the configuration parameter **IPNET_SLAB_DESC**_*N*. There are six of these parameters, numbered from 0 to 5.

IPNET_SLAB_DESC_*N* has three fields, separated by commas:

- The first field specifies the size of the memory objects in the slab. A single bigger chunk of memory is split into memory areas of this size and put into the slab.
- The second field gives the system a hint on how big the requested memory region that is split into "first filed" sized objects should be. The format for this field is **IPCOM_SLAB_F_[AVERAGE|ONE|FEW|MANY]**.
  - **IPCOM_SLAB_F_ONE** allocates enough memory to hold a single object.
  - **IPCOM_SLAB_F_[FEW|AVERAGE|MANY]** allocates enough memory to hold more than one object, hinting for increasingly larger allocation. **IPCOM_SLAB_F_ONE** does not mean that the slab can only hold a single object. It means that that the slab will add and remove memory at the granularity of a single memory region worth of data. So **IPCOM_SLAB_F_MANY** is the most speed-efficient, but it can waste some memory if the target never uses as many memory regions a single allocation result in.
- The third field specifies the number of unused memory regions the system will allow in a slab before starting to release some memory regions. This is also a reason to avoid **IPCOM_SLAB_F_MANY** unless really needed because the memory area cannot be freed before all memory objects that memory area was split into are unused. One can see this as a form of memory fragmentation. One main reason to use slab caches apart from performance is that the use of slabs tend avoids memory fragmentation, because they only request memory of few different sizes from the system.

In almost all cases, the default configuration will provide the best performance. In rare cases, it may be necessary to adjust the **IPNET_SLAB_DESC_*N*** parameters to accommodate hardware with a particular requirement for buffer sizes for which a perfect slab match is required.

If you do need to depart from the default configuration of **IPNET_SLAB_DESC_*N***, contact Wind River Professional Services for guidance.

Previous topic: Setting the Memory Limit on page 16
Next topic: Configuring Network Memory Buffers on page 18

8. Configuring Network Memory Buffers

Configuring network memory buffers is an important part of developing a robust networking application. A *memory buffer* (*mbuf*) is a basic unit of memory management in the kernel and store network packets and socket buffers. An *mbuf* contains a variable-sized header and a small internal buffer for data.

Ingress buffers are queued on sockets until an application calls recv( ), therefore, it is difficult to estimate the number of *mbufs* required. In any case, the number of *mbufs* must be at least the size of the RX + TX ring, but also handle cases where ingress packets get queued in fragmentation lists, TCP-out-of-order queues, and socket receive buffers.

You can estimate the number of *mbufs* required for your application using the following equation:

$$mbuf = (RX_ring_slots * RX_ring_cnt) + (max_sockets * socket_buffer_size / MTU)$$

Where:

RX_ring_slots = the number of slots in the RX ring

RX_ring_cnt = the number of RX rings

max_sockets = the maximum number of sockets

socket_buffer_size = buffer size

MTU = maximum transmission unit, typically 1500 bytes

To set the number of *mbufs* for your network, find the function call to endPoolCreate( ) in your network driver code found in **vxworks-7/pkgs_v2/net/end/drv/src/end**. The tupleCnt parameter in the call below is the number of *mbufs* each instance of the driver will allocate.

```
(
int tupleCnt,
NET_POOL_ID * ppNetPool
)
```

Note: The tupleCnt is represented by the *RX_ring_slots * RX_ring_cnt* part of the equation above. The parameter is called tupleCnt, instead of mbufCnt because it allocates an *mbuf* along with its associated cluster storage buffer. For more information, see the *VxWorks 7 Network Driver Developer's Guide: Utility Functions for MAC Drivers*section *Buffer Management*.

For example, the FEI driver (**vxbFei8255xEnd.c**) uses endPoolCreate( ) to allocate mBlk tuples as shown below:

```
if (endPoolCreate ((FEI_RXCNT * 4) + FEI_CBCNT,
    &pDrvCtrl->feiEndObj.pNetPool) == ERROR)
    {
    FEI_LOGMSG("%s%d: pool creation failed\n", FEI_NAME,
                          pDrvCtrl->unit, 0, 0, 0, 0);
     return (NULL);
     }
```

The macros **FEI_RXCNT** and **FEI_CBCNT** are defined in **vxbFei8255xEnd.h**.

```
#define FEI_RXCNT 64
```

In this case, the driver allocates four times *mbuf*s needed by the driver to account for the *mbuf*s that may be lent to the network stack.

Note: The FEI driver in the example is a simple device and only has one slot, therefore, **FEI_RXCNT** is represented by RX_ring_cnt in the equation above. However, advanced network cards such as TEI/GEI can have multiple rings.

The tupleCnt should be increased if your device uses one network interface and the combined socket RX buffer size in your application exceeds:

FEI_RXCNT * 3 * MTU = 288,000 bytes (by default).

Note: The value of 3 is used in this equation because the driver allocates four times as many *mbufs* it needs for itself, therefore, up to 3 out of 4 can be held by the network stack.

You can also specify the maximum number of sockets and default socket buffer size using these VIP parameters.

IPNET_SOCK_DEFAULT_RECV_BUFSIZE (default 60000)

NUM_FILES (socket and other file descriptors share file-descriptor space) (default 50)

You can also change the default size of any socket dynamically by using the setsockopt( ) SO_RCVBUF option. For more information, see the *VxWorks 7 Socket Programmer's Guide: Socket Options*.

Test your application for all possible use cases to ensure you have allocated enough *mbufs*. Do not over allocate *mbufs* because this may cause an inefficient use of memory.

Previous topic: Setting the Slab Cache Descriptors on page 17

3. MULTITHREADED NETWORK STACK

1. About VxWorks 7 Multithreaded Network Stack

The VxWorks 7 multithreaded Network Stack supports a single network stack with multiple network tasks (tNet*N*) that can concurrently execute network protocol and network driver processes on separate CPUs, thus improving network throughput for symmetric multiprocessing (SMP) systems hosting multiple network interfaces.

Previous versions of VxWorks for SMP systems supported only a single network task, but could still improve throughput over single processor systems by allowing the network task and socket application tasks to operate in parallel on different CPUs. However, the limitation with this architecture lies in a single network task having to perform all network protocol and network driver processing.

In SMP systems with multithreaded network stack support, multiple network tasks operating on multiple CPUs can process in parallel both socket requests from multiple applications and driver requests from multiple network interfaces to run network protocol and driver processes. This parallelism can improve overall networking throughput compared to configurations with a single network task.

Note:

For both single processor and SMP builds, the default VxWorks 7 configuration specifies a single network task, with all END interfaces and sockets being assigned to that single network task. Enabling multiple network tasks is allowed for single processor systems as well as multiprocessor systems.

However, operating multiple network tasks and application tasks on a single CPU is not conducive to parallelism and therefore no obvious benefits are expected from such an operating environment.

This topic covers the implementation and configuration of the VxWorks 7 multithreaded network stack functionality.

To configure basic functionality for the network stack, see Configuration and Build Workflow on page 6.

For information on VxWorks SMP and how to configure VxWorks for SMP, see the *VxWorks 7 Application Programmer's Guide: VxWorks SMP*.

Next topic: VxWorks 7 Network Stack Implementation Overview on page 20

2. VxWorks 7 Network Stack Implementation Overview

To use the VxWorks 7 multithreaded network stack in symmetric multiprocessing (SMP) systems, enable SMP and specify the number network tasks (tNet*N*) to create and the number of CPUs to enable for SMP. Network tasks are labelled tNet0, tNet1, tNet2, and so forth. CPUs are labelled CPU0, CPU1, CPU2, and so forth.

Network Tasks and CPU Assignments

By default, network tasks are assigned CPU affinity. tNet0 is assigned to CPU0, tNet1 is assigned to CPU1, and so forth until either the network task pool or the CPU pool is exhausted. If there are more network tasks than CPUs, the additional tasks are not assigned to a CPU. At configuration time you can override the default configuration and assign network tasks to specific CPUs, or disable CPU affinity to prevent the assignment of network tasks to CPUs.

At run time, you can manage CPU affinity with the taskCpuAffinitySet( ) ( )and taskCpuAffinityGet( ) functions.

Enabling the Multithreaded Network Stack with QorIQ DPAA-based Drivers

For some boards with QorIQ DPAA-based drivers, it may be necessary to specify the job queue ID to enable the multithreaded network stack.

All Qman portals will use netJobQueueId( )--that is, the job queue for tNet0) unless the BSP specifies a 'job-queue-id' parameter for the Qman portal. This parameter is actually the job queue index: **0** for the tNet0 job queue, **1** for the tNet1 job queue, and so forth.

The Qman initialization code (qmanMuxConnect ( ) in **vxbQorIQQman.c**) calls vxbParamGet (portal->qportalDev, "job-queue-id", VXB_PARAM_INT32, &val) to get the job queue index. It also sets **portal->qportalJobQueue** to the corresponding job queue ID if the parameter is found for the specific portal; otherwise, it leaves **portal->qportalJobQueue** as **NULL**.

qportalIntHandlingEnable( ) (in **vxbQorIQQmanPortal.c**) uses *netJobQueueId* as the default if **portal->qportalJobQueue** has not been set to a non-NULL value yet.

The job queue index to use for each Qman portal needs to be put in the appropriate DTS file for the qman portals in question if you want to use a non-default (!= netJobQueueId) portal. The parameter can be added either in the "chosen" node or in the specific device node; see **vxbParamLib.c** for the basic syntax.

For fsl_t4240, you might need to modify the **qman-portal** device nodes in the **qoriq-qbman-portals.dtsi** (or **qoriq-qbman-portals-more.dtsi**) files. For example, to assign **qman-portal** number **1** to use the tNet1 job queue, one would add the line **job-queue-id = <0x1>;** in:

```
qman-portal@4000 {
      cell-index = <0x1>;
      compatible = "fsl,qman-portal";
      reg = <0x4000 0x4000 0x1001000 0x1000>;
      interrupts = <0x6a 0x2 0x0 0x0>;
      fsl,qman-channel-id = <0x1>;
      job-queue-id = <0x1>;
};
```

END Interface Assignments

By default, all END interfaces are assigned to the primary network task (tNet0) and therefore CPU0. When you configure a VxWorks image project (VIP), you can assign specific network interfaces to particular network tasks. At run time, with the use of an API function, you can reassign specific END interfaces to particular network tasks.

Note: Only END interfaces that support the EIOCSRCVJOBQ END ioctl can be assigned to a different network task.

Socket Assignments

By default, all newly created sockets are assigned to the primary network task (tNet0) and therefore CPU0. When you configure a VIP, you can enable the assignment of most types of newly created sockets to pseudo-randomly chosen network tasks.

At run time, you can use a system variable to enable or disable random socket assignment for newly created sockets. You can also use a new socket option to reassign a socket to a different network task.

The network stack may also reassign a TCP socket to a different network task when the TCP connection involving the socket becomes established. This occurs for both active connections initiated using connect( ), and for child TCP sockets obtained from accept( ).

The stack reassigns such a socket (when necessary) to the network task handling the END interface that receives segments from the socket's TCP peer. This automatic socket migration is a one-time event for a given TCP socket.

Shared Network Resource Management

Network tasks (tNet*N*) handle the driver and protocol level processes such as packet handling, driver events, timeouts, as well as requests from socket applications. Socket application tasks handle application specific processes as well as transferring data between the internal packet buffers of the network stack and the application buffers.

The multithreaded network stack implements a single network stack with multiple network tasks. Some network stack data structures are shared by all network tasks, while other structures are replicated among (and so private to) individual network tasks. Further, some global stack resources may be assigned to particular network tasks. A network task together with its private network stack data structures is called a network stack instance.

The multithreaded network stack must protect resources shared globally among network tasks using some mechanism of mutual exclusion. Some such shared resources -- typically ones that are normally accessed in a read-only fashion, but might be modified when the stack configuration changes -- are protected by a mutual exclusion mechanism named stack suspension. An example of such resources is the set of network interfaces known to the stack and their configured IP addresses. Network tasks normally access such resources in a read-only fashion without any explicit mutual exclusion; however, sometimes (usually infrequently after initialization), the interfaces or their addresses may need to be reconfigured. In that case, the primary stack instance, tNet0, performs the modification only after causing the other network tasks to pend themselves (by blocking on semaphores at a safe point, until tNet0 terminates the stack suspension).

To implement stack suspension, the tNet0 network task sends a suspension request to all other network tasks, and must wait for each network task to finish its current work, receive the suspension request, and indicate that it has suspended operation. Finally, after modifying the shared resource, tNet0 awakens the other network tasks, allowing them to proceed.

> Note: All network tasks must be suspended, even though some might not be accessing the shared resource.

Stack suspension is used as follows:

- To protect globally shared data that must be modified when the network stack is configured or reconfigured.
- For network interface operations other than output queues and statistics.
- When the network multiplex (MUX) layer is binding a network service to, or unbinding it from, an END interface.
- For most write-mode socket ioctl operations.
- For socket option operations that may modify global configuration data or affect membership in a multicast group.

> Note: If your application has any real-time network processing requirements, you may want to avoid runtime configuration changes that cause stack suspension during those periods of processing where deadlines on network operations apply.

In previous VxWorks SMP releases, CPUs other than the boot processor (CPU0) were enabled after the network stack had completed initialization. However, in the initialization of the multithreaded network stack, all of the network tasks need to be able to respond to requests (including stack suspension requests from tNet0). However, a network task would not be able to respond if the CPU to which it has affinity is not enabled yet. To handle this, VxWorks delays those portions of network stack initialization requiring all network tasks to be responsive until after SMP initialization has enabled all configured CPUs. These portions include binding a network service through the MUX to an END device, as well as most protocol initialization.

Previous topic: About VxWorks 7 Multithreaded Network Stack on page 20
Next topic: VxWorks 7 Network Stack Configuration and Build on page 22

3. VxWorks 7 Network Stack Configuration and Build

Source Build Configuration Options

The default VxWorks source build (VSB) project configuration specifies single processor operation. To enable SMP, set the SMP option to Yes and build the VSB project. For more information on SMP and SMP configuration parameters, see the *VxWorks 7 Application Programmer's Guide: VxWorks SMP*.

Kernel Configuration Components

To configure a VIP to support a multithreaded network stack, include the INCLUDE_NET_DAEMON, INCLUDE_IPNET_SOCKET, INCLUDE_END, and INCLUDE_KERNEL components.

All of these components are enabled by default. Note that while one can remove the network stack entirely, one cannot disable the multithreaded nature of the stack, other than by setting the number of network tasks (NUM_NET_DAEMONS) to 1.

Configuration Parameters

The table below lists the multithreaded network stack configuration parameters and their default values.

Parameter Name and Description	**Default and Type**
NET_TASK_OPTIONS Specifies the options for the taskSpawn( ) function call for all network tasks. The default options define the tasks as operating in supervisor mode and to ignore breakpoints. Additional network task options are defined in the taskSpawn( ) reference entry. For information on setting task options for a spawned task, see the taskOptionsSet( ) reference entry. For more information on task options, see the *VxWorks 7 Application Programmer's Guide: Task Options*	VX_SUPERVISOR_MODE | VX_UNBREAKABLE int
NET_DAEMONS_CPU_AFFINITY For SMP systems, this parameter specifies the default assignment of CPU affinity for all network tasks. When this parameter is set to TRUE, each network task tNet*N* is assigned to CPU *N*, if CPU *N* is configured. If there are more tasks than CPUs, the excess tasks remain unassigned. When this parameter is set to FALSE, none of the tasks is given CPU affinity. In either case, you can override the default setting for each network task using the NET_DAEMONS_CPU_AFFINITIES parameter. At run time, you can manage CPU affinity with the taskCpuAffinitySet( ) and taskCpuAffinityGet( ) functions. See also ENABLE_ALL_CPUS and Network Tasks and CPU Assignments on page 26.	TRUE bool

Parameter Name and Description	Default and Type
NET_DAEMONS_CPU_AFFINITIES For SMP systems, this parameter overrides the default CPU affinity assignments established by NET_DAEMONS_CPU_AFFINITY, for selected network tasks. The format of this string parameter is a sequence of zero or more assignments of the form *networkTaskIndex*,*cpuIndex*, separated by semicolons, with whitespace optional: "*networkTaskIndex*,*cpuIndex*;*networkTaskIndex*,*cpuIndex*;... " For example, "0,0;1,2" assigns tNet0 to CPU0, and tNet1 to CPU 2. You may give a CPU index greater than or equal to the number of configured CPUs to indicate that a particular network task should have no CPU affinity. For instance, if the number of configured CPUs is 4, then "0, 4; 3, 4" indicates that neither tNet0 nor tNet3 should be given CPU affinity. If a network task index is not stated in this parameter, the default from NET_DAEMONS_CPU_AFFINITY governs that task. When NET_DAEMONS_CPU_AFFINITIES has its default empty string value, NET_DAEMONS_CPU_AFFINITY governs the affinity for all network tasks. If an invalid format or value is encountered, the part of the parameter string after the error is ignored. At run time, you can manage CPU affinity with the taskCpuAffinitySet( )and taskCpuAffinityGet( ) functions. See also ENABLE_ALL_CPUS and Network Tasks and CPU Assignments on page 26.	" " char*
NET_TASK_DEFAULT_PRIORITY Sets the default task priority for all network tasks.	50 uint
NET_TASK_PRIORITIES Specifies the task priorities for network tasks. The default empty string value sets all network tasks to the NET_TASK_DEFAULT_PRIORITY value. If a network task is not assigned a task priority by this parameter, the NET_TASK_DEFAULT_PRIORITY value is used. The format of this string parameter is a sequence of zero or more assignments of the form *networkTaskIndex*,*priorityValue*, separated by semicolons, with whitespace optional: "*networkTaskIndex*,*priorityValue*;*networkTaskIndex*, *priorityValue*; ...". For example, "0,50;1,51" assigns network task 0 a priority of 50 and network task 1 a priority of 51. If an invalid format or value is encountered, the part of the parameter string after the error is ignored.	" " char*

Parameter Name and Description	Default and Type
NET_TASK_STACKSIZE Sets the stack size for all network tasks.	10000 uint
NUM_NET_DAEMONS Number of network tasks (tNet*N*) to create. The range for this parameter is 1 to 4094. See also, VX_SMP_NUM_CPUS and Network Tasks and CPU Assignments on page 26.	1 uint
NET_JOB_NUM_CFG Number of jobs in a job queue's standard job pool. The number of standard (QJOB_STANDARD) network jobs allocated into a pool associated with the job queue for each network task. Used by API functions such as jobQueueStdPost( ) or netJobAdd( ).	85 uint
ENABLE_ALL_CPUS This parameter is part of the INCLUDE_KERNEL component and controls the activation of all CPUs that have been configured for the system based on the VX_SMP_NUM_CPUS parameter. When set to TRUE, VxWorks boots with all CPUs enabled and running. The parameter can be set to FALSE for debugging purposes, in which case only CPU0 is enabled. The kernelCpuEnable( ) function can then be used to enable a specific CPU once the system has booted. See also NET_DAEMONS_CPU_AFFINITY, NET_DAEMONS_CPU_AFFINITIES, and Network Tasks and CPU Assignments on page 26.	TRUE bool
VX_SMP_NUM_CPUS Configures the number of CPUs available to a VxWorks SMP image. The allowed range is from 1 to the maximum number of CPUs in the target system. The default value is BSP dependent. A recommended configuration is to set VX_SMP_NUM_CPUS to the number of CPUs in the target system and set NUM_NET_DAEMONS to the same value. For more information, see Network Tasks and CPU Assignments on page 26.	uint

Parameter Name and Description	Default and Type
IPNET_SOCK_ASSIGN_RANDOM This parameter is part of the INCLUDE_IPNET_SOCKET component. By default, all sockets are assigned to the primary network stack instance (tNet0). When this parameter is set to " yes ", each new socket is assigned pseudo-randomly to one of the network stack instances. Setting this parameter to " yes " can help spread the workload for handling operations on different sockets among different network tasks. For more information, see Socket Assignments for Newly Created Sockets on page 28.	"no" char *
END_NET_JOB_INIT_STRING This parameter is part of the INCLUDE_END component and specifies which network task is assigned to a given END interface. The format is as follows: *interfaceName*:*networkTaskIndex* | *interfaceName*:*networkTaskIndex* ... The setting can be overridden at runtime with the muxEndQnumSet( ) function. This parameter and the muxEndQnumSet( ) ( )function are only applicable to network drivers that support the EIOCSRCVJOBQ END ioctl. For more information, see END Interface Assignments on page 28. For information on modifying an existing driver to support EIOCSRCVJOBQ, see the *VxWorks 7 Network Driver Developer's Guide*.	"dummy0:0|dummy1:1" char *

Previous topic: VxWorks 7 Network Stack Implementation Overview on page 20
Next topic: Network Tasks and CPU Assignments on page 26

4. Network Tasks and CPU Assignments

VxWorks 7 symmetric multiprocessing (SMP) provides the ability to automatically assign a network task to a CPU according to task and CPU indices (0, 1, 2, and so forth). Hence, tNet0 is assigned to CPU0, tNet1 is assigned to CPU1, and so forth.

A common configuration is to set the number of network tasks (NUM_NET_DAEMONS) to the same number of configured CPUs (VX_SMP_NUM_CPUS).

For example, to automatically assign four network tasks to four CPUs, set the SMP VSB option to **True** and configure the following VIP parameters:

- CPU Affinity (NET_DAEMONS_CPU_AFFINITY): **True**
- Number of network tasks (NUM_NET_DAEMONS): **4**
- Enable all CPUs (ENABLE_ALL_CPUS): **True**

- Number of CPUs (VX_SMP_NUM_CPUS): **4**

Note: The VX_SMP_NUM_CPUS value must be equal or less than the number of CPUs on the target system.

You can also use the NET_DAEMONS_CPU_AFFINITIES parameter to set the CPU affinities of individual network tasks, overriding the default settings from NET_DAEMONS_CPU_AFFINITY.

This table lists the configuration options for CPU affinity.

NET_DAEMONS_CPU_AFFINITY	ENABLE_ALL_CPUS	Configuration
TRUE	**TRUE**	All CPUs are enabled. Network tasks are assigned to the enabled CPUs based on their indices. If there are more network tasks than CPUs, the additional tasks are not assigned to a CPU. If there are more CPUs than tasks, the additional CPUs do not have an assigned network task.
TRUE	**FALSE**	This configuration is not recommended.
FALSE	**TRUE**	All network tasks remain unassigned and all configured CPUs are enabled.
FALSE	**FALSE**	All network tasks remain unassigned and only CPU0 is enabled.

Performance Objectives of CPU Affinity

The potential benefits of assigning a given task to a CPU are as follows:

- Network tasks commonly handle network device interrupts, and if device interrupts are routed to the CPU to which a network task has affinity, no interprocessor interrupt is necessary for the interrupt service routine to schedule the network task.
- In systems with well defined networking workloads, on SMP systems supporting symmetric multithreading (SMT) or other similar hardware configurations in which not all pairs of CPUs can be considered equivalent, it may help performance if both network tasks and socket application tasks are assigned to particular CPUs. For instance, in some benchmarks it may help if a TCP socket application task is assigned to a CPU that shares a cache with the CPU to which the socket's assigned network task has CPU affinity.

Performance benefits of CPU affinity should be verified for each application. Setting the CPU affinity decreases the flexibility that the VxWorks scheduler has to assign network tasks to different CPUs and therefore can decrease performance.

The network stack itself does not require CPU affinity.

For information on performance optimizations, see Performance Optimization on page 31.

For more information on CPU affinity, see the *VxWorks 7 Application Programmer's Guide: CPU Affinity*.

Previous topic: VxWorks 7 Network Stack Configuration and Build on page 22
Next topic: END Interface Assignments on page 28

5. END Interface Assignments

By default the VxWorks 7 network stack assigns all END interfaces to the primary network task (tNet0).

The assigned network task handles packets received on the network interface, performs transmit cleanups, and processes events such as link status changes. Any task, including any network task, can send packets to an END interface. Only one network task can be assigned to an END interface. An END interface is identified by its driver name and unit number and a network task is identified by its index.

At VxWorks image project (VIP) configuration time, and only for END interfaces that support the EIOCSRCVJOBQ END ioctl, you can use the END_NET_JOB_INIT_STRING VIP parameter to assign a network task instance to an END interface. END interfaces that support the EIOCSRCVJOBQ END ioctl include generation 2 gei, tei, etsec, dtsec, and memac END drivers, and the VxBus legacy dtsec END driver.

At run time, you can use the muxEndQnumSet( ) function to assign a network task to an END interface. The muxEndQnumSet( )function uses the EIOCSRCVJOBQ END ioctl to set the specified network task. If the interface is operational, muxEndQnumSet( ) will stop and restart the interface device to implement the change.

From the VxWorks kernel shell C interpreter, you can reassign an END interface to a network task with the muxEndQnumSet( ) (*interfaceName* , ( ) *interfaceIndex* , ( ) *taskNumber*) function. For example, to set END interface tei0 to network task tNet2, enter the following at the kernel shell prompt:

```
-> muxEndQnumSet "tei", 0, 2
```

Previous topic: Network Tasks and CPU Assignments on page 26
Next topic: Socket Assignments for Newly Created Sockets on page 28

6. Socket Assignments for Newly Created Sockets

By default in VxWorks 7, all newly created sockets are assigned to the primary network task (tNet0) and therefore CPU0.

When random socket assignment (IPNET_SOCK_ASSIGN_RANDOM) is enabled in the VIP, sockets created with the socket( ) function are assigned pseudo-randomly to one of the available network tasks. An exception to this rule is any socket whose domain is AF_NETLINK (Linux NetLink), AF_ROUTE (internal routing protocol), or AF_SOCKDEV (network interface socket). AF_NETLINK and AF_ROUTE sockets are always assigned to tNet0 as they frequently modify the global stack configuration, and AF_SOCKDEV sockets are also always assigned to tNet0 in the current implementation.

Enabling random socket assignment can help spread the workload for handling operations on different sockets among different network tasks. However, random assignment is unlikely to be optimal when considering the peers with which such sockets communicate. Any average benefit from enabling random socket assignment should be verified on an application-specific basis.

When random socket assignment is disabled, all newly created sockets are assigned to tNet0.

At run time, you can enable and disable the random socket assignment functionality with the *ipnet.sock.AssignRandom* system variable.

Previous topic: END Interface Assignments on page 28
Next topic: Socket Assignments for TCP Sockets on page 29

7. Socket Assignments for TCP Sockets

In the VxWorks 7 network stack, a transmission control protocol (TCP) socket assigned to a network task at socket creation time can be migrated automatically to another network task at connection establishment time.

TCP automatically migrates TCP sockets when they initially enter the ESTABLISHED state. The TCP socket is migrated to the network stack instance that received the packet from the peer that moved the connection into the ESTABLISHED state. Automatic migration happens only once for a given TCP socket.

Automatic socket migration also applies to TCP child sockets obtained using the accept( ) function; they start off assigned to the network stack instance that owns the listening parent socket, but move to the network stack instance that receives TCP segments from the peer once they are in the ESTABLISHED state.

Note: TCP strongly associates a given TCP socket and control block with a particular network stack instance, the socket owner instance. All operations on the TCP protocol control block must occur in the owner instance to avoid explicit mutual exclusion for the TCP endpoint state. If a TCP segment is received by a different network stack instance than the socket owner, the TCP segment must be relayed to the socket owner instance.

Note: Listening (server) TCP sockets never enter the ESTABLISHED state and are not automatically migrated.

Previous topic: Socket Assignments for Newly Created Sockets on page 28
Next topic: Explicit Socket Assignments on page 29

8. Explicit Socket Assignments

A socket application can explicitly migrate a socket to a VxWorks 7 network stack instance using the SO_X_STACK_IDX socket option extension, provided that the following restrictions are met:

- The socket does not belong to the AF_NETLINK, AF_ROUTE or AF_SOCKDEV socket domains. These sockets are handled by tNet0 only.
- The socket is not in a poll operation where another task is blocked waiting to read from or write to the socket.
- The protocol must permit the migration. For example, TCP only allows socket migration for sockets that are not yet reachable (bound) or that are in the ESTABLISHED state.

Note: In some applications, in addition to setting the network stack instance for a socket, it may be beneficial to set CPU affinity for the application task that handles the socket.

Non-TCP Socket Applications on page 29 VxWorks 7 Network Stack non-TCP socket applications that are aware of the multithreaded nature of the network stack and of the network stack instances and their associated network interfaces may be able to improve performance by assigning sockets to particular network stack instances using the **SO_X_STACK_IDX** socket option.

Previous topic: Socket Assignments for TCP Sockets on page 29
Next topic: Shared Data Structures on page 30

8.1. Non-TCP Socket Applications

VxWorks 7 Network Stack non-TCP socket applications that are aware of the multithreaded nature of the network stack and of the network stack instances and their associated network interfaces may be able to improve performance by assigning sockets to particular network stack instances using the **SO_X_STACK_IDX** socket option.

The data for this socket option is an integer value which is the index of the network stack instance that can handle application socket requests for this socket. For example:

```
#ifdef SO_X_STACK_IDX
{
     int index = MYAPP_NETWORK_STACK_INDEX;
     if (setsockopt(mySockFd, SOL_SOCKET, SO_X_STACK_IDX, &index,
        (socklen_t)(sizeof(index))) < 0)

        {
         perror("setsockopt(SO_X_STACK_IDX)");
        }
}
#endif /* SO_X_STACK_IDX */
```

Parent topic: Explicit Socket Assignments on page 29

9. Shared Data Structures

A number of VxWorks 7 network stack data structures are shared among all network stack instances.

These include the following important data structures:

- The **sockets table** is the data structure that the stack uses to look up socket structures based upon socket file descriptor numbers. The sockets table uses an IPCOM spinlock as a mutual exclusion mechanism when opening or closing a socket, adding or removing a socket, as well as reference counts for each socket, used when accessing the socket to protect it from deletion for the duration of a socket call.
- The stack uses a **reachable sockets table** mainly to locate sockets based upon the 5-tuple (destination and source IP addresses, protocol, and when present, destination and source layer 4 port numbers) in received packets. The reachable sockets table is protected by a single read/write semaphore, which allows multiple stack instances to look up sockets in the table while allowing only one instance to access the table when binding, connecting, unbinding, or disconnecting sockets.
- The global database of routes in the forwarding information base (FIB) is protected by its own lock mechanism. The FIB is not modified with dynamic routes such as those created by Internet Control Message Protocol (ICMP) redirects; it only contains the routes configured by a system administrator or routing protocol.

  To avoid locking the FIB each time a packet is sent, some information derived from the routing table is cached as destination cache entries in each network stack instance. The destination caches and neighbor caches also serve to hold Address Resolution Protocol (ARP) entries, redirects, path Maximum Transmission Unit (PMTU) values, and similar dynamic information that might in other systems be kept in the shared routing table.
- There is a shared collection of peer information entries. Each peer information entry holds some data about an IP peer with which the network stack has communicated recently. The data stored in these entries includes:
  - The next IPv4 ID value to be used in sending to the peer.
  - A queue of partially reassembled IP datagrams from the peer.
  - Certain flags used in the path Maximum Transmission Unit (MTU) implementation.
  - Members used in implementing rate limiting of ICMP messages sent to the peer.
- Network interface structures and their IP addresses.

Previous topic: Explicit Socket Assignments on page 29
Next topic: Private Data Structures on page 31

10. Private Data Structures

A number of VxWorks 7 network stack data structures are kept local to a network stack instance.

The most important local data structures are as follows:

- Each stack instance maintains its own private hash table of destination cache entries. Each such entry corresponds to a layer 3 (L3) flow of data, and it is looked up for each packet received or sent by the network stack instance. The keys for the lookup are L3 only, and include the IP source and destination address (and address family), the egress and ingress interface indices when appropriate, the zone ID for IPv6, the TOS or traffic class, and the virtual router index when virtual routers are enabled. The destination cache entries specify how each flow should be handled, a decision made the first time that the flow is encountered. For instance, the destination cache entry tells whether packets received on an ingress flow should be dropped, delivered locally, or forwarded. The destination cache entry also contains pointers to the appropriate peer information entry, and to the neighbor cache entry used when sending on the flow. Some additional flow-specific data such as the path Maximum Transmission Unit (PMTU) is maintained in the destination cache entry.

Using a per-instance destination cache, a stack instance only needs to consult the shared forwarding information base (FIB) when creating a new destination cache entry for a flow not already stored in the cache. Subsequent look-ups touch only the per-instance cache and its entries, avoiding synchronization with other CPUs as well as cache-line bouncing.

- Each stack instance maintains its own copies of neighbor cache entries, which specify the Media Access Control (MAC) address and network interface used to reach a next-hop neighbor (which might be either the final destination or the next-hop gateway). The neighbor cache forms the basis of the Address Resolution Protocol (ARP) cache for IPv4, and the Neighbor Discovery Protocol cache for IPv6.

While each instance maintains its own neighbor cache, allowing it to look up its neighbor cache entries without contending with other instances, the neighbor caches need to be consistent among all the stack instances, and in fact responsibility for running the ARP or ND protocols is given to the primary stack instance. All received ARP or ND packets are relayed to the primary instance, which handles the ARP or ND protocol events, timeouts, and drives most transitions of the neighbor cache entry state machine. Any state changes made by the primary instance is broadcast to all the other stack instances, allowing them to keep their neighbor caches in sync with that of the primary stack instance.

- Most (although not all) statistics that the stack keeps are maintained on a per-stack-instance basis, and are only summed over all stack instances when they are reported via a network command-line utility, netlink sockets or the like, or via Simple Network Management Protocol (SNMP). Keeping the statistics separate avoids cache-line contention among the different CPUs on which the network tasks run.

Previous topic: Shared Data Structures on page 30
Next topic: Performance Optimization on page 31

11. Performance Optimization

VxWorks 7 Network Stack performance optimization tends to be specific to particular workloads and hardware. Some experimentation is necessary when attempting to maximize throughput in a multithreaded network stack environment.

The following topics provide some insight into optimizing your environment to maximize throughput in an Symmetric Multiprocessing (SMP) system with a multithreaded network stack.

Working with a Single Network Interface

If a target has only one network interface, the benefit from a multithreaded network stack may be small. Only one network task can be assigned to handle RX traffic and TX cleanup on a given network interface.

If you distribute socket request handling for non-Transmission Control Protocol (TCP) sockets among different network tasks, all non-local packets are received in the context of the single network task assigned to the single network interface. There might be some benefit on the transmit side, as any network task can send to the network interface; however, this can also lead to increased contention for the transmit resources associated with the interface, at both the stack level and the driver level.

If all the traffic that a target is concerned with belongs to a single flow, it will likewise arrive in a single network task and other network tasks may have little to do.

Working with Multiple Network Interfaces

The best performance improvements are obtained when there is more than one network interface on the target and the incoming network traffic is well distributed among the network interfaces, and the network interfaces are assigned to different network tasks.

Layer 4 workloads involve processing both by network tasks (which handle the driver and protocol level code), and socket application tasks, which handle application specific work as well as transferring data between internal packet buffers and application buffers of the network stack.

There are two main configuration concerns relating to the multithreaded stack to obtain the best performance in applications with multiple network interfaces. They are socket assignments and CPU affinity:

- A socket should be assigned to the best network task for that socket. The best network task is usually the one that handles the most received traffic destined for the socket. For connected TCP sockets, the automatic migration that occurs at connection establishment time ensures that the best network task handles the socket (except in unusual cases where traffic from the peer starts arriving on a different interface some time after connection establishment). However, for non-TCP sockets, or in fact for listening parent TCP server sockets, a single (unconnected) socket may receive traffic from more than one network interface (and so may receive traffic in more than one network task).

Note: For a listening parent TCP socket, new connections may indeed arrive on multiple interfaces, but the SYN and ACK segments from the initial handshake are all relayed to the stack instance that handles the listening parent socket. Only once a child connection becomes established is it automatically migrated to the stack instance that received the initial ACK segment from the peer.

In some cases an application may know that a given socket will receive most of its traffic from a particular network interface (and so from a particular network task). In this case it may make sense to explicitly migrate the socket to that network task using the SO_X_STACK_IDX socket option extension.

For information on END interface assignments, see END Interface Assignments on page 28.

For information on socket assignments, see Socket Assignments for Newly Created Sockets on page 28, Socket Assignments for TCP Sockets on page 29, and Explicit Socket Assignments on page 29.

- When network tasks are given CPU affinity, and on systems in which not all pairs of logical CPUs are equivalent, it may sometimes make sense to assign CPU affinity for a socket application task to a CPU chosen based upon the CPU affinity of the network task to which the socket is assigned. For instance, consider a quadcore processor in which CPUs 0 and 1 share an L2 cache, and CPUs 2 and 3 share a different L2 cache. In such a system, it may make sense if a socket application task whose TCP socket is assigned to CPU0 sets its CPU affinity to CPU1. This means in particular that the **lpcom_pkt** packet structures to which the application task on CPU1 copies its transmit data will be already in the shared cache when the network task tNet0, running on CPU0, handles the transmit request. Similarly, the other data (the SENDMSG signal, the socket structure itself) that are accessed by both tNet0 and the application task will tend to be cached after the other task has accessed it.

On the other hand, in some situations in which different logical CPUS share computational resources more intimately (such as Intel HyperThreading or similar SMT systems), it may instead be better if two CPUs that must be active together are not hyperthreads of the same core, to avoid resource conflicts. As an extreme case of this, it would usually not make sense to assign the socket application task affinity to the same CPU as the network task that handles the socket, as that would prevent the network task and the application task from running in parallel. On the other hand, it depends upon how many CPUs are available and what the other CPUs are assigned. On a dual core target with two network interfaces and two corresponding network tasks and continually active socket

WNDRVR

application tasks associated with each interface, it might indeed make sense to give the application tasks affinity to the same CPU as the network task that handles them.

Another consideration in deciding whether or how to assign affinity to socket application tasks is that for a single socket, work may not be equally balanced between the network task and the application task. For example, for stateful L4 protocols such as TCP or Stream Control Transmission Protocol (SCTP), the work in the network task is usually greater than the work needed in a socket application task that simply sinks or sources the data. In this case, the network task processing would be the bottleneck. Of course, if the application task has more serious work to do, it could be the bottleneck rather than the network task.

For information on CPU affinity, see Network Tasks and CPU Assignments on page 26.

Transmit Contention In Network Drivers

One remaining problem that can limit performance - seen especially in forwarding benchmarks - is contention between CPUs for the transmit resources of a single network interface. In particular, one network task (or in fact any task) trying to send on an END interface may contend for the mutual exclusion guarding the transmit resources of the END interface with another network task that is assigned to the same END interface and is performing transmit cleanup (freeing packets that have completed transmission).

As an example, consider a benchmark involving IP forwarding between two END interfaces, say **gei0** and **gei1**. Suppose tNet0 is assigned to **gei0** while tNet1 is assigned to **gei1**. When tNet0 receives packets on **gei0** and determines that they should be forwarded out **gei1**, it will call the **gei1** driver's send routine to transmit the packet. This will involve taking a lock to protect the software and hardware transmit resources associated with the **gei1** interface. Meanwhile, tNet1 may be trying to perform cleanup of packets transmitted earlier on **gei1** that have completed transmission. In this case tNet1 and tNet0 will contend for the transmit resources of **gei1**. The contention can be bad enough that bidirectional forwarding between the two ports forwards more packets (even on a per-direction basis) than unidirectional forwarding, simply because with bidirectional forwarding, both sides have receive work to perform and so one side cannot spend all of its time performing transmit cleanup.

There are various ways to try to address transmit contention in network drivers, and some drivers (including the generation 2 **tei** and **gei** drivers) have been optimized in some ways to reduce transmit contention:

- Use more efficient forms of locking mechanisms for the transmit resources, and do not hold the lock for extended periods, but still perform some batching of transmit cleanup.
- Perform some transmit cleanup in the driver send routine when the transmit ring is almost full.
- Take various steps to perform transmit cleanup more lazily, rather than performing transmit cleanup as soon as any single packet has completed transmission. For most purposes, prompt transmit cleanup in response to transmit completion interrupts is not critical, provided that the driver send routine also performs some transmit cleanup when the transmit ring is almost full.

In this situation, further sends will result in previous sends being cleaned. However, there may be issues with non-prompt transmit cleanup if it causes protocols to stop transmitting before filling the transmit ring. For instance, IPNET's current mechanism for User Datagram Protocol (UDP) socket transmit flow control depends on the signal of previously transmitted packets being freed by the driver to decrease the amount of outstanding data charged against a socket's send buffer and allow more sending. In this case, depending on the socket send buffer size and the sizes of individual sends, a sending UDP socket may see decreased transmit throughput if cleanup of transmit packets is not prompt.

Unfortunately, contention between a sender and the network task performing transmit cleanup can still impact performance significantly in those cases where it occurs. There may not be much an application can do to decrease such contention. However it is important to recognize the issue if it is seen.

Transmit Contention Between CPUs for a Network Interface

Contention between multiple CPUs sending to the same network interface is also possible, both at the END interface and in IPNET. At the END interface, the steps mentioned in Transmit Contention In Network Drivers on page 33 help to limit the contention.

At the IPNET level, this contention is greater when the Quality of Service (QoS) feature is enabled and an output queue is configured for the interface. By default (non-QoS mode), a simple FIFO queue is used and contention at the IPNET level is not usually serious unless sending is fast enough that the END driver send routine starts returning **END_ERR_BLOCK** and the FIFO queue starts to fill.

For information on QOS, see About VxWorks 7 Network Stack Quality of Service (QoS) on page 73.

Previous topic: Private Data Structures on page 31

4. TASK PRIORITIES FOR NETWORK SERVICE TASKS

1. Service Task Priorities

A VxWorks 7 network service task is a process that can be executed by the operating system to manage a network service such as a dynamic host configuration protocol (DHCP) client or server.

VxWorks 7 makes a distinction between network service tasks and the network tasks (tNet*N*), that handle low-level protocol and network driver operations.

Task scheduling relies on individual task priorities of all system tasks including network stack instance tasks, kernel application tasks, driver tasks, and network service tasks.

The VxWorks kernel provides 256 priority levels, numbered 0 through 255. Priority 0 is the highest and priority 255 is the lowest.

By default, all network service tasks are set to a task priority of 50, which is the same as the default priority of the network tasks (tNet *N*). You can define network service task priorities at VIP configuration time and at run time. The VxWorks stack can be configured with multiple network tasks at different priorities, although most commonly the same task priority is used for all network tasks; and if there is more than one network task, the different network tasks often run on different CPUs in an Symmetric Multiprocessing (SMP) system. To avoid potential starvation of the network tasks, it is generally recommended to set the task priority values of network service tasks no lower than the task priority value of any network task. However, what the stack actually enforces when starting a network service task is that the task priority value for a network service task is no lower than the NET_TASK_DEFAULT_PRIORITY parameter value.

For information on task priorities for network stack tasks in a multithreaded network stack environment, see About VxWorks 7 Multithreaded Network Stack on page 20.

For information on task creation and management, including task priorities for kernel application and driver tasks, see the *VxWorks 7 Application Programmer's Guide: Tasks and Multitasking*.

For information on debug modes for network service tasks, see About Debug Mode for Network Service Tasks on page 41.

Next topic: Task Priority Assignments for Network Service Tasks on page 35

2. Task Priority Assignments for Network Service Tasks

Most VxWorks 7 Network Stack applications can retain the default settings for network service tasks, but if needed you can change the priorities.

A task priority value for a VxWorks 7 Network Stack service task is not allowed to be less than the value of the NET_TASK_DEFAULT_PRIORITY parameter value. This priority value is the default task priority used for the network tasks (tNet*N*), although it can be overridden for particular network tasks using the NET_TASK_PRIORITIES parameter. But consider the following, for the case of a single processor system with just one network task tNet0:

- An interrupt service routine (ISR) interrupt will interrupt a task with a task priority 0 (highest priority).
- When tNet0 is the highest priority task ready to run, it runs.
- If a user task with a higher priority than tNet0 is ready to run, it runs instead of tNet0.
- While tNet0 is not running, packets are not processed, although packets may continue to arrive.

While the situation for an symmetric multiprocessing (SMP) system with multiple network tasks is more complex, if network service tasks are assigned priorities higher (numerically lower) than those of a network task, there is a danger of preventing that network task

from running, particularly when the network tasks have affinity to particular CPUs. Caution is recommended when assigning numerically lower task priority values to network service tasks than those assigned to any network task.

You can assign a task priority at VxWorks image project (VIP) configuration time and at run time.

> Note: If you launch a new user task that depends on network services, ensure that the task priority of the user task is set to a higher value (lower priority) than that of tNet0.

Previous topic: Service Task Priorities on page 35
Next topic: Task Priority Configuration for Network Service Tasks on page 36

3. Task Priority Configuration for Network Service Tasks

This table lists the parameter name, system variable name, and provides a description for each task priority parameter of the VxWorks 7 Network Stack services.

For information on managing system variables with the sysvar shell command, see About System Variables on page 209.

The default task priority for each network service task is 50. The lowest task priority value for a network service task is set by the NET_TASK_DEFAULT_PRIORITY parameter.

The highest task priority value for a network service task is 255 (lowest priority).

> Note: Task priority is assigned to a task at task creation time. If you change the system variable *serviceName .priority* governing the priority of a network service task that is already running, its task priority is not changed. Only if the service is stopped and restarted will the new service task get the modified priority. However, if you change the priority of a running task using the taskPrioritySet( ) function, its priority changes immediately.
>
> Restarting the network service task using taskRestart( ) is not safe and will not change the priority of a task.

> Note: When you configure a task priority value lower than that of NET_TASK_DEFAULT_PRIORITY for a network service task, the assigned value is automatically changed to the value set by NET_TASK_DEFAULT_PRIORITY. When you configure a task priority value higher than 255, the assigned value is automatically changed to 255.
>
> When the system changes a task priority value to a valid value, the associated *serviceName.priority* system variable is not automatically updated to reflect the assigned value. For these conditions and when the debug level of IPCOM_SYSLOGD_DEFAULT_PRIORITY is set to *IPCOM_LOG_DEBUG2* (highest debug level), a warning log is created and stored in the system log. For example:
>
> ```
> MON MAY 11
> 08:42:21 2015: ipcom[38378358]: Warning: The configured priority of task ipdhcpc is i
> nvalid.
> The task priority has been automatically adjusted to a valid value.
> ```
>
> To view the current task priority values for all tasks, use the i command from the kernel shell or the command shell.

The table below lists the configuration parameters for task priorities of network services:

WNDRVR

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC_TASK_PRIORITY *ipdhcpc.priority* Sets the task priority of the dynamic host configuration protocol version 4 (DHCPv4) client network service task.	"50" char *
DHCPC6_TASK_PRIORITY *ipdhcpc6.priority* Sets the task priority of the DHCPv6 client network service task.	"50" char *
DHCPR_TASK_PRIORITY *ipdhcpr.priority* Sets the task priority of the DHCP relay agent network service task.	"50" char *
DHCPS_TASK_PRIORITY *ipdhcps.priority* Sets the task priority of the DHCPv4 server network service task.	"50" char *
DHCPS6_TASK_PRIORITY *Ipdhcps6.priority* Sets the task priority of the DHCPv6 server network service task.	"50" char *
FTPS_TASK_PRIORITY *ipftps.priority* Sets the task priority of the file transport protocol (FTP) server network service task.	"50" char *
FTPS_SESSION_TASK_PRIORITY *ipftps.session_priority* Sets the task priority of the file transport protocol (FTP) network service session.	"50" char *

Parameter Name, System Variable Name, and Description	Default and Type
NTP_TASK_PRIORITY *ipntp.priority* Sets the task priority of the network time protocol (NTP) network service task.	"50" char *
SNTP_TASK_PRIORITY *ipsntp.priority* Sets the task priority of the simple network time protocol (SNTP) network service task.	"50" char *
TFTPS_TASK_PRIORITY *iptftp.priority* Sets the task priority of the trivial file transport protocol (TFTP) server network service task.	"50" char *
IPCOM_TELNET_TASK_PRIORITY *ipcom.telnet.priority* Sets the task priority of the telnet network service task.	"50" char *
MCP_TASK_PRIORITY *ipmcp.priority* Sets the task priority of the MCP network service task.	"50" char *
RADVD_TASK_PRIORITY *ipnet.inet6.radvd.priority* Sets the task priority of the router advertisement daemon (RADVD) network service task.	"50" char *
PPP_TASK_PRIORITY *ipppp.priority* Sets the task priority of the point-to-point protocol (PPP) network service task.	"50" char *

Parameter Name, System Variable Name, and Description	Default and Type
IPIKE_TASK_PRIORITY *ipike.priority* Sets the task priority of the internet key exchange (IKE) network service task.	"50" char *
L2TP_TASK_PRIORITY *ipl2tp.priority* Sets the task priority of the layer 2 tunneling protocol (L2TP) network service task.	"50" char *
IP8021X_TASK_PRIORITY *ip8021x.priority* Sets the task priority of the 802.1x port-based network access control network service task.	"50" char *
IPWPS_TASK_PRIORITY *ipwps.priority* Sets the task priority of the Wifi protected setup (WPS) network service task.	"50" char *
IPRIP_TASK_PRIORITY *iprip.priority* Sets the task priority of the routing information protocol (RIP) network service task.	"50" char *
IPRIPNG_TASK_PRIORITY *ipripng.priority* Sets the task priority of the routing information protocol next generation (RIPng) network service task.	"50" char *
VRRP_TASK_PRIORITY *ipnet.vrrp.priority* Sets the task priority of the virtual router redundancy protocol (VRRP) network service task.	"50" char *

Previous topic: Task Priority Assignments for Network Service Tasks on page 35

5. DEBUG MODES FOR NETWORK SERVICE TASKS

1. About Debug Mode for Network Service Tasks

By default, the debug mode for all network service tasks is disabled (unbreakable). Enabling the debug mode allows breakpoints within a task. The debug mode can be changed at VxWorks image project (VIP) configuration time and at run time.

Note: Network tasks (tNet*N*) are also unbreakable by default.

For information on debugging, see the *Using breakpoints, watchpoints, and breakpoint actions* topic within Workbench help. For additional information on watchpoints, including how to set watchpoints from the command line, see *Workbench 4 User Tutorials* accessible from the knowledge library at:

https://knowledge.windriver.com

For more information on debugging, see the *VxWorks 7 Debugging Guide*.

For information on task priorities for network service tasks, see Service Task Priorities on page 35.

Next topic: Debug Mode Configurations for Network Service Tasks on page 41

2. Debug Mode Configurations for Network Service Tasks

The table below lists the parameter name, system variable name, and provides a description for each debug mode parameter of the VxWorks 7 network services.

The default debug mode for each network service task is set to disabled (*"0"*). To enable, set to *"1"*.

At run time, use taskSpawn( ) to spawn a task and set the task options. Once a task is spawned, use taskOptionsSet( ) to change the task options. For more information, see the taskSpawn( ) and taskOptionsSet( ) reference entries.

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC_DEBUG_ENABLE *ipdhcpc.debug* Sets the debug mode of the dynamic host configuration protocol version 4 (DHCPv4) client network service task.	"0" char *
DHCPC6_DEBUG_ENABLE *ipdhcpc6.debug* Sets the debug mode of the DHCPv6 client network service task.	"0" char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPR_DEBUG_ENABLE *ipdhcpr.debug* Sets the debug mode of the DHCP relay agent network service task.	"0" char *
DHCPS_DEBUG_ENABLE *ipdhcps.debug* Sets the debug mode of the DHCPv4 server network service task.	"0" char *
DHCPS6_DEBUG_ENABLE *ipdhcps6.debug* Sets the debug mode of the DHCPv6 server network service task.	"0" char *
FTPS_DEBUG_ENABLE *ipftps.debug* Sets the debug mode of the file transport protocol (FTP) server network service task.	"0" char *
NTP_DEBUG_ENABLE *ipntp.debug* Sets the debug mode of the network time protocol (NTP) network service task.	"0" char *
SNTP_DEBUG_ENABLE *ipsntp.debug* Sets the debug mode of the simple network time protocol (SNTP) network service task.	"0" char *
TFTPS_DEBUG_ENABLE *iptftp.debug* Sets the debug mode of the trivial file transport protocol (TFTP) network service task.	"0" char *

Parameter Name, System Variable Name, and Description	Default and Type
IPCOM_TELNET_DEBUG_ENABLE *ipcom.telnet.debug* Sets the debug mode of the telnet network service task.	"0" char *
MCP_DEBUG_ENABLE *ipmcp.debug* Sets the debug mode of the MCP network service task.	"0" char *
RADVD_DEBUG_ENABLE *ipnet.inet6.radvd.debug* Sets the debug mode of the router advertisement daemon (RADVD) network service task.	"0" char *
PPP_DEBUG_ENABLE *ipppp.debug* Sets the debug mode of the point-to-point protocol (PPP) network service task.	"0" char *
IPIKE_DEBUG_ENABLE *ipike.debug* Sets the debug mode of the internet key exchange (IKE) network service task.	"0" char *
L2TP_DEBUG_ENABLE *ipl2tp.debug* Sets the debug mode of the layer 2 tunneling protocol (L2TP) network service task.	"0" char *
IP8021X_DEBUG_ENABLE *ip8021x.debug* Sets the debug mode of the 802.1x port-based network access control network service task.	"0" char *

Parameter Name, System Variable Name, and Description	Default and Type
IPWPS_DEBUG_ENABLE *ipwps.debug* Sets the debug mode of the Wifi protected setup (WPS) network service task.	"0" char *
IPRIP_DEBUG_ENABLE *iprip.debug* Sets the debug mode of the routing information protocol (RIP) network service task.	"0" char *
IPRIPNG_DEBUG_ENABLE *ipripng.debug* Sets the debug mode of the routing information protocol next generation (RIPng) network service task.	"0" char *
VRRP_DEBUG_ENABLE *ipnet.vrrp.debug* Sets the debug mode of the virtual router redundancy protocol (VRRP) network service task.	"0" char *

Previous topic: About Debug Mode for Network Service Tasks on page 41

6. NETWORKING TUTORIALS

1. Basic Networking Example

The following example takes you through the steps of creating a simulated network with one router and three interconnected targets.

About This Task

The workflow for creating and testing the IPv4 network environment is as follows:

Procedure

1. Create and configure a VxWorks source build (VSB) and a VxWorks image project (VIP) for the VxWorks simulator.

   Add the **INCLUDE_ROUTECMD** component to your VIP build to enable the configuration of the routing table.

2. Install the Wind River WRTAP network driver and configure network connectivity to enable IP communications between the VxWorks simulator and your host.

   See Install the Wind River TAP Driver on page 46.

3. Configure and start the VxWorks simulator daemon **vxsimnetd**.

   See Configure and Start the VxWorks Simulator Daemon on page 47.

4. Configure and start the router instance.

   See Configure and Start the Router Instance on page 48.

5. See Configure and Start the Three Simulated Targets on page 50.

   See Configure and Start the Router Instance on page 48.

6. Configure the routing table.

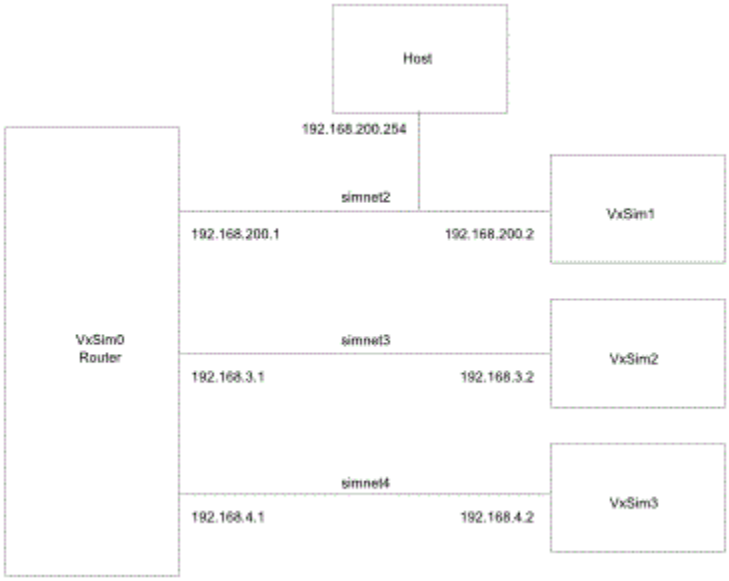
   See Configure the Routing Tables on page 51.

7. Run ping commands to verify connectivity.

   See Test Connectivity on page 52.

   For information on the VxWorks simulator, see the *VxWorks 7 VxSim User's Guide*.

   Figure 1. Basic Networking Example



Install the Wind River TAP Driver on page 46 This example makes use of the VxWorks simulator (VxSim) in place of an actual target.
Configure and Start the VxWorks Simulator Daemon on page 47 For the purposes of this example, a configuration file is used to configure three class C network subnets.
Configure and Start the Router Instance on page 48 The router instance sets up a network interface for the router on each subnet to enable packet forwarding through the specified gateway IP address.
Configure and Start the Three Simulated Targets on page 50 Launch each device with the -d **simnet** option, -e *ipAddress* , and -p *processorNumber* .
Configure the Routing Tables on page 51 To allow IP forwarding between subnets, routes between subnets must be specified in the routing table.
Test Connectivity on page 52 To verify connectivity between all targets, access the command mode and execute a sequence of ping commands from each target.

Next topic: Networking with IPv6 on page 53

1.1. Install the Wind River TAP Driver

This example makes use of the VxWorks simulator (VxSim) in place of an actual target.

About This Task

To simulate network IP connectivity, the VxWorks simulator includes a Wind River TAP (WRTAP) driver that allows VxWorks to communicate by handing off IP packets to the host operating system for processing. Once installed, the WRTAP driver is automatically initialized upon launching the VxWorks simulator.

To install a WRTAP driver, proceed as follows:

Procedure

1. Open a Windows command shell as an administrator.
2. In the command shell, use the following command syntax:

```
C:\> installDir\vxWorks-7\host\x86-win32\bin\wrtapInstaller.exe install wrtap
```

where *installDir* is the name of your VxWorks installation directory.

3. Observe the presence of a new network connection on your host. For example, **WindRiver WRTAP #***n* where *n* can be 1, 2, 3, and so on.

   Getting Help

   For more information on using the WRTAP driver, see the *VxWorks 7 VxSim User's Guide*.

Parent topic: Basic Networking Example on page 45

1.2. Configure and Start the VxWorks Simulator Daemon

For the purposes of this example, a configuration file is used to configure three class C network subnets.

About This Task

First subnet:

- Subnet name: subnet_1_200_extern_promisc
- Address range: 192.168.200.0 to 192.168.200.255
  - The subnet has 256 addresses.
- Network access: External access.
  - The subnet can communicate with the host.
- Subnet mode: External Promiscuous
  - The host can view every packet sent on the subnet. It allows you to attach a packet sniffer on the host interface to monitor traffic.

Second subnet:

- Subnet name: subnet_2_3_internal
- Address range: 192.168.3.0 to 192.168.3.255
  - The subnet has 256 addresses.
- Network access: Internal access.
  - The subnet cannot communicate with the host.

Third subnet:

- Subnet name: subnet_3_4_internal
- Address range: 192.168.4.0 to 192.168.4.255
  - The subnet has 256 addresses.
- Network access: Internal access.
  - The subnet cannot communicate with the host.

To get help for vxsimnetd, type vxsimnetd -help at the host command shell prompt.

To get help for the vxsimnetd shell, type help at the vxsimnetd shell prompt.

To configure and start the **vxsimnetd** daemon, proceed as follows:

Procedure

1. Create a file named **vxsimNetworking.conf** in *installDir* **\vxWorks-7\host\x86-win***xx* **\bin** and add the following to the file:

```
SUBNET_START subnet_1_200_external_promisc {
    SUBNET_ADDRESS = "192.168.200.0";
    SUBNET_EXTERNAL = yes;
    SUBNET_EXTPROMISC = yes;
};
SUBNET_START subnet_2_3_internal {
    SUBNET_ADDRESS = "192.168.3.0";
    SUBNET_EXTERNAL = no;
};
SUBNET_START subnet_3_4_internal {
SUBNET_ADDRESS = "192.168.4.0";
SUBNET_EXTERNAL = no;
};
```

2. Launch the VxWorks simulator daemon **vxsimnetd** using the configuration file you just created.
   1. Open a Windows command shell as an administrator.
   2. From the command shell, use the following command syntax to launch the VxWorks simulator using the configuration file (**-f**) and start the vxsimnetd shell (**-s**).

```
C:\> installDir \vxworks-7\host\x86-win32\bin\vxsimnetd -f vxsimNetworking.conf -s
          vxsimnetd Debug Shell
vxsimnetd>
```

   Display the current network configuration and status with the following commands:

```
vxsimnetd> subnet subnet_1_200_external_promisc
vxsimnetd> node subnet_1_200_external_promisc

vxsimnetd> subnet subnet_2_3_internal
vxsimnetd> node subnet_2_3_internal

vxsimnetd> subnet subnet_3_4_internal
vxsimnetd> node subnet_3_4_internal
```

Results

For more information on vxsimnetd, see the *VxWorks 7 VxSim User's Guide*

Parent topic: Basic Networking Example on page 45

1.3. Configure and Start the Router Instance

The router instance sets up a network interface for the router on each subnet to enable packet forwarding through the specified gateway IP address.

About This Task

This procedure makes use of several utilities.

- To get help for the wrenv command, type wrenv -help at the development shell prompt.
- To get help for the vxsim command, type vxsim -help at the development shell prompt. For more information on vxsim, see the VxWorks 7 VxSim User's Guide.
- To get help for the ifconfig command, type ifconfig "" at the kernel shell prompt.
- To get help for the routec command, type routec at the kernel shell prompt. You can also use the route command from the command shell (#). To get help for the route command, type route at the command shell.

Configure and start the router instance as follows:

Procedure

1. Open a development shell either from Workbench or from the command line.

   From the command line, use the following command from the VxWorks installation directory:

```
installDir>wrenv -p vxworks-7
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
 installDir>
```

2. From the development shell, access the **default** directory of the VIP.
3. From the development shell, launch the router instance with the following command:

```
> vxsim -ni simnet2=192.168.200.1;simnet3=192.168.3.1;simnet4=192.168.4.1
```

   The -ni command option creates a network interface for each subnet. The instance is booted as a passive device (**passDev**).

4. From the newly created **VxSim0** console (router instance), run the ifconfig command to view the network configuration. Observe the presence of all three network interfaces. For example:

```
-> ifconfig
...
simnet2 Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:01 Queue:none
     inet 192.168.200.1 mask 255.255.255.0 broadcast 192.168.200.255
     inet6 unicast fe80::787a:c0ff:fea8:c801%simnet2 prefixlen 64 automatic
     UP RUNNING SIMPLEX BROADCAST MULTICAST
...
simnet3 Link type:Ethernet HWaddr 7a:7a:c0:a8:03:01 Queue:none
     inet 192.168.3.1 mask 255.255.255.0 broadcast 192.168.3.255
     inet6 unicast fe80::787a:c0ff:fea8:301%simnet3 prefixlen 64 automatic
     UP RUNNING SIMPLEX BROADCAST MULTICAST
...
simnet4 Link type:Ethernet HWaddr 7a:7a:c0:a8:04:01 Queue:none
    inet 192.168.4.1 mask 255.255.255.0 broadcast 192.168.4.255
    inet6 unicast fe80::787a:c0ff:fea8:401%simnet4 prefixlen 64 automatic
    UP RUNNING SIMPLEX BROADCAST MULTICAST
...
```

5. From the **VxSim0** console (router instance), run the routec "show" command to view the routes, gateways, and interfaces (**If**). For example:

```
-> routec "show"

INET route table - vr: 0, table: 254
Destination          Gateway          Flags        Use          If          Metric
localhost            localhost        UH           4            lo0         0
192.168.3.0/24       192.168.3.1      UC           5            simnet3     0
192.168.3.1          192.168.3.1      UH           1            lo0         0
192.168.4.0/24       192.168.4.1      UC           7            simnet4     0
192.168.4.1          192.168.4.1      UH           1            lo0         0
192.168.200.0/24     192.168.200.1    UC           8            simnet2     0
192.168.200.1        192.168.200.1    UH           2            lo0         0
```

Parent topic: Basic Networking Example on page 45

1.4. Configure and Start the Three Simulated Targets

Launch each device with the -d **simnet** option, -e *ipAddress*, and -p *processorNumber*.

About This Task

This procedure makes use of these utilities:

- To get help for the vxsim command, type vxsim -help at the development shell prompt. For more information on vxsim, see the VxWorks 7 VxSim User's Guide.
- To get help for the ifconfig command, type ifconfig "" at the kernel shell prompt.

To configure and start the three simulated targets, proceed as follows:

Procedure

1. Open a development shell.
2. Access the **default** directory of the VIP.
3. Launch the first instance with the following command:

```
>vxsim -d simnet -e 192.168.200.2 -p 1
```

   The command launches the simulator instance as a simulated network boot device using the -d simnet option with IP address 192.168.200.2 (**-e 192.168.200.2**), and with processor number 1 (**-p 1**).

4. From the newly created **VxSim1** console window, run the ifconfig command to view the network configuration. For example:

```
-> ifconfig
...
simnet0 Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:02 Queue:none
     inet 192.168.200.2 mask 255.255.255.0 broadcast 192.168.200.255
     inet6 unicast fe80::787a:c0ff:fea8:c802%simnet0 prefixlen 64 automatic
     UP RUNNING SIMPLEX BROADCAST MULTICAST
```

5. From the development shell, access the **default** directory of the VIP.
6. Launch the second instance with the following command:

```
> vxsim -d simnet -e 192.168.3.2 -p 2
```


WNDRVR

7. From the newly created **VxSim2** console, run the ifconfig command to view the network configuration.
8. From the development shell, access the **default** directory of the VIP.
9. Launch the third instance with the following command:

```
> vxsim -d simnet -e 192.168.4.2 -p 3
```

10. From the newly created **VxSim3** console, run the ifconfig command to view the network configuration.
11. Observe the presence of the four instances as follows:
    1. **VxSim0** as the router.
    2. **VxSim1** as the first target at 192.168.200.2.
    3. **VxSim2** as the second target at 192.168.3.2.
    4. **VxSim3** as the third target at 192.168.4.2.

Parent topic: Basic Networking Example on page 45

1.5. Configure the Routing Tables

To allow IP forwarding between subnets, routes between subnets must be specified in the routing table.

About This Task

For help with the routec command used in this procedure, type routec at the kernel shell prompt. You can also use the route command from the command shell (#). To get help for the **route** command, type **route** at the command shell.

To configure the routing table for the simulated network, proceed as follows:

Procedure

1. From the **VxSim1** kernel shell, use the following commands:

```
-> routec "show"
-> routec ("add -net 192.168.3.0/24 192.168.200.1");
-> routec ("add -net 192.168.4.0/24 192.168.200.1");
-> routec "show"
```

The first command displays the default route entry for **VxSim1**. For example:

```
-> routec "show"
INET route table - vr: 0, table: 254
Destination              Gateway        Flags         Use          If          Metric
localhost                localhost      UH            4            lo0         0
192.168.200.0/24         VxSim1         UC            30           simnet0     0
VxSim1                   VxSim1         UH            10           lo0         0
...
```

The second command adds a routing table entry that specifies a network route (**-net**) to subnet 192.168.3.0 through gateway address 192.168.200.1 on **VxSim0** (router instance).

The third command adds a routing table entry that specifies a network route (**-net**) to subnet 192.168.4.0 through gateway address 192.168.200.1 on **VxSim0** (router instance).

The last command displays the updated routing table entries for **VxSim1**. For example:

WNDRVR

```
-> routec "show"
INET route table - vr: 0, table: 254
Destination                Gateway        Flags          Use          If          Metric
localhost                  localhost      UH             4            lo0         0
192.168.3.0/24             VxSim1         UGS            0            lo0         0
192.168.4.0/24             VxSim1         UGS            0            lo0         0
192.168.200.0/24           VxSim1         UC             41           simnet0     0
VxSim1                     VxSim1         UH             3            lo0         0
...
```

2. From the **VxSim2** kernel shell, use the following commands:

```
-> routec ("add -net 192.168.200.0/24 192.168.3.1");
-> routec ("add -net 192.168.4.0/24 192.168.3.1");
-> routec "show"
```

3. From the **VxSim3** kernel shell, use the following commands:

```
-> routec ("add -net 192.168.200.0/24 192.168.4.1");
-> routec ("add -net 192.168.3.0/24 192.168.4.1");
-> routec "show"
```

Parent topic: Basic Networking Example on page 45

1.6. Test Connectivity

To verify connectivity between all targets, access the command mode and execute a sequence of ping commands from each target.

About This Task

Proceed as follows:

Procedure

1. Access the command shell from the **VxSim1** console.

```
-> cmd
[vxWorks *]#
```

2. Run ping commands and verify connectivity to the **VxSim2** and **VxSim3** targets from **VxSim1** at 192.168.200.2 (0% packet loss). For example:

```
[vxWorks *]# ping 192.168.3.2
Pinging 192.168.3.2 (192.168.3.2) with 64 bytes of data:
Reply from 192.168.3.2 bytes=64 ttl=63 seq=0 time<1ms
--- 192.168.3.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0 ms
rtt min/avg/max = 0/0/0 ms

[vxWorks *]# ping 192.168.4.2
```

3. Access the **VxSim2** console and run the following commands to verify connectivity to **VxSim1** and **VxSim3**:

WNDRVR

```
-> cmd
[vxWorks *]# ping 192.168.200.2
[vxWorks *]# ping 192.168.4.2
```

4. Access the **VxSim3** console and run the following commands to verify connectivity to **VxSim1** and **VxSim2**:

```
-> cmd
[vxWorks *]# ping 192.168.200.2
[vxWorks *]# ping 192.168.3.2
```

Parent topic: Basic Networking Example on page 45

2. Networking with IPv6

This example show you how to configure your host system and your simulated targets to communicate using the IPv6 protocol.

About This Task

Before you begin, you must configure your host with IPv6 support. Refer to your operating system documentation for information on how to enable IPv6 for your network connections.

If not done, install a host connection driver (WRTAP driver). See Install the Wind River TAP Driver on page 46.

To create an IPv6 environment for the VxWorks simulator, proceed as follows:

Procedure

1. Create a simulator-based VSB and VIP for the example.

   See Configure VxWorks with IPv6 Support on page 54.

2. Configure and start the VxWorks simulator network daemon **vxsimnetd**.

   See Configure and Start the VxWorks Simulator Network Daemon on page 55.

3. Start four VxWorks simulator instances and configure the instances with a startup script.

   See Start the VxWorks Simulator Instances on page 55.

4. Verify the network interfaces.

   See Verify the Network Interfaces on page 57.

5. Verify that the routing advertisement services are operational.

   See Verify That the Routing Advertisement Service Is Operational on page 58

6. Test the network connections within each subnet.

   See Test the Network Connections on page 59.

7. Setup routes to interconnect subnets.

   See Create Routes to Interconnect Subnets on page 60.

Configure VxWorks with IPv6 Support on page 54 Additional components must be added to your VIP to support IPv6.
Configure and Start the VxWorks Simulator Network Daemon on page 55 Additional components must be added to your VIP to support the simulator daemon.
Start the VxWorks Simulator Instances on page 55 Start your VxWorks simulator instances using the VxWorks images you created and configure the instances at boot time with startup scripts.
Verify the Network Interfaces on page 57 Verify the network interfaces on the VxWorks simulator instances. They should have an IPv6 address (inet6).
Test the Network Connections on page 59 Use the ping6 command to test the connections between VxWorks simulator instances on the same subnet.
Create Routes to Interconnect Subnets on page 60 In addition, you can set the routing such that you can ping between the **default** subnet and subnet **sub3** through **VxSim2**.

Previous topic: Basic Networking Example on page 45

2.1. Configure VxWorks with IPv6 Support

Additional components must be added to your VIP to support IPv6.

About This Task

You can use an existing simulator-based VxWorks image project (VIP) and include additional VIP components or create a new simulator-based VxWorks source build (VSB) and VIP.

For information on how to create a VSB and VIP for the VxWorks simulator, see the *VxWorks 7 VxSim User's Guide*.

Complete the following steps to add IPv6 support to VxWorks.

Procedure

1. Add the following VIP components to your project:

   These components are required to support IPv6 with router advertisement.

   - Add INCLUDE_IPCOM_USE_INET6 to include internet protocol IPv6 and enable network connectivity with IPv6 addressing.
   - Add PRESERVE nameliteral-ucINCLUDE_PING6_CMD to access the ping6 shell command and test network connectivity in an IPv6 environment.
   - Add INCLUDE_IPCOM_SYSVAR_CMD to access the sysvar shell command and configure system variables.
   - Add the INCLUDE_IPRADVD_CMD to access the router advertisement daemon (radvdconfig) shell command and configure router advertisements.
   - Add PRESERVE nameliteral-ucINCLUDE_IPD_CMD to access the daemon control (ipd) shell command to manage the target services.
   - Add INCLUDE_STARTUP_SCRIPT to enable the creation of startup scripts that configure simulator instances at boot time.
2. Build the project.

Parent topic: Networking with IPv6 on page 53

2.2. Configure and Start the VxWorks Simulator Network Daemon

Additional components must be added to your VIP to support the simulator daemon.

About This Task

You can use an existing simulator-based VIP and include additional VxWorks image project (VIP) components or create a new simulator-based VxWorks source build (VSB) and VIP. For information see the *VxWorks 7 VxSim User's Guide*.

Procedure

1. Add the following VIP components to your project:
   - Add INCLUDE_IPCOM_USE_INET6 to include internet protocol IPv6 and enable network connectivity with IPv6 addressing.
   - Add PRESERVE nameliteral-ucINCLUDE_PING6_CMD to access the ping6 shell command and test network connectivity in an IPv6 environment.
   - Add INCLUDE_IPCOM_SYSVAR_CMD to access the sysvar shell command and configure system variables.
   - Add the INCLUDE_IPRADVD_CMD to access the router advertisement daemon (radvdconfig) shell command and configure router advertisements.
   - Add PRESERVE INCLUDE_IPD_CMD to access the daemon control (ipd) shell command to manage the target services.
   - Add INCLUDE_STARTUP_SCRIPT to enable the creation of startup scripts that configure simulator instances at boot time.
2. Build the project.

Parent topic: Networking with IPv6 on page 53

2.3. Start the VxWorks Simulator Instances

Start your VxWorks simulator instances using the VxWorks images you created and configure the instances at boot time with startup scripts.

About This Task

This procedure utilizes several utilities.

- For information on boot parameters and how to configure boot parameters, see the VxWorks 7 VxSim User's Guide.
- To get help for the vxsim command, type vxsim -help at the development shell prompt. For more information on vxsim, see the VxWorks 7 VxSim User's Guide.
- To get help for the ifconfig command, type ifconfig "" at the kernel shell prompt.
- To get help with the radvdconfig command, type radvdconfig at the command shell prompt.
- To get help with the sysvar command, type sysvar at the target command shell prompt.
- To get help with the ipd command, type ipd at the target command shell prompt.

Note: You can use the set bootline or bootChange commands to specify the startup script file to load at boot time or specify the file as a **vxsim** command-line option. Furthermore, instead of using startup scripts, you can copy and paste the commands at the command prompt (#) of the simulator instance.

WNDRVR

Procedure

1. Start the **advertiser** and **solicitor** simulator instances on subnet **default** at 192.168.200.0. Proceed as follows:
   1. Create a startup script named **default_startup.txt** that specifies the IPv6 address to use, the prefix to advertise, and starts the **ipnet_radvd** service:

```
# add IPv6 address
ifconfig "simnet0 inet6 add 2002:A01:201::787A:C0FF:FEA8:C801"

# switch to the command mode from the kernel shell mode
cmd

# configure the IPv6 router advertisement message and enable it
radvdconfig simnet0 add test 2002:0a01:0201::/64 valid 600 preferred 200 L A
radvdconfig simnet0 enable test

# start advertising prefix
sysvar set -o ipnet.inet6.radvd.interfaces simnet0
ipd start ipnet_radvd
```

   2. Start an **advertiser** VxWorks simulator instance on subnet **default** at 192.168.200.1. You can start the simulator instance from the VxWorks development shell using the **default_startup.txt** script.

```
-> vxsim -p 0 -d simnet -e 192.168.200.1 -f
installDir / vipDirectory /default/vxworks -s pathToStartupScript /default_startup.txt
```

   Where *pathToStartupScript* is the full path to your startup script location.

   This starts a VxWorks simulator instance (**VxSim0**). From the **VxSim0** console, observe that the startup script executed successfully.

   3. Start a **solicitor** VxWorks simulator instance on subnet **default** at 192.168.200.2.

```
192.168.200.2
 -> vxsim -p 1 -d simnet -e 192.168.200.2 -f
installDir / vipDirectory /default/vxworks
```

   This starts a VxWorks simulator instance (**VxSim1**).

2. Start the **advertiser** and **solicitor** simulator instances on subnet **sub3** at 192.168.3.0. Proceed as follows:
   1. Create a startup script (**sub3_startup.txt**) that specifies the IPv6 address to use, the prefix to advertise, and starts the **ipnet_radvd** service:

```
# add IPv6 address
ifconfig "simnet0 inet6 add 2002:A01:202::787A:C0FF:FEA8:C801"

# switch interpreter
cmd

# configure the IPv6 router advertisement message and enable it
radvdconfig simnet0 add test 2002:0a01:0202::/64 valid 600 preferred 200
L A
radvdconfig simnet0 enable test

# start advertising prefix
```

```
sysvar set -o ipnet.inet6.radvd.interfaces simnet0
ipd start ipnet_radvd
```

2. Start an **advertiser** VxWorks simulator instance on subnet **sub3** at 192.168.3.1 with an interface to subnet **default** at 192.168.200.3.

```
-> vxsim -p 2 -ni simnet0=192.168.3.1;simnet1=192.168.200.3 -f
installDir / vipDirectory /default/vxworks -s pathToStartupScript /sub3_startup.txt
```

where *pathToStartupScript* is the full path to your startup script location.

This starts a VxWorks simulator instance (**VxSim2**). From the **VxSim2** console, observe that the startup script executed successfully.

3. Start the **solicitor** VxWorks simulator instance on subnet **sub3** at 192.168.3.2.

```
-> vxsim -p 3 -d simnet -e 192.168.3.2 -f installDir /
vipDirectory /default/vxworks
```

This starts a VxWorks simulator instance (**VxSim3**).

Parent topic: Networking with IPv6 on page 53

2.4. Verify the Network Interfaces

Verify the network interfaces on the VxWorks simulator instances. They should have an IPv6 address (inet6).

About This Task

This procedure uses the ifconfig utility. To get help, type ifconfig "" at the kernel shell prompt.

Complete the following steps:

Procedure

1. In the **VxSim0** console, type the following:

```
-> ifconfig "simnet0"
simnet0 Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:01 Queue:none
        inet 192.168.200.1 mask 255.255.255.0 broadcast 192.168.200.255
        inet 224.0.0.1 mask 240.0.0.0
        inet6 unicast 2002:A01:201::787A:C0FF:FEA8:C801 prefixlen 64
        inet6 unicast FE80::787A:C0FF:FEA8:C801%simnet0 prefixlen 64
...
value = 0 = 0x0
->
```

2. In the **VxSim1** console, type the following:

```
-> ifconfig "simnet0"
simnet0 Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:02 Queue:none
        inet 192.168.200.2 mask 255.255.255.0 broadcast 192.168.200.255
        inet 224.0.0.1 mask 240.0.0.0
```


WNDRVR

```
        inet6 unicast 2002:A01:201::787A:C0FF:FEA8:C802 prefixlen 64
        inet6 unicast FE80::787A:C0FF:FEA8:C802%simnet0 prefixlen 64
...
value = 0 = 0x0
->
```

3. In the **VxSim2** console, type the following:

```
-> ifconfig
lo0  Link type:Local loopback Queue:none
...
simnet0 Link type:Ethernet HWaddr 7a:7a:c0:a8:03:01 Queue:none
        inet 192.168.3.1 mask 255.255.255.0 broadcast 192.168.3.255
        inet 224.0.0.1 mask 240.0.0.0
        inet6 unicast 2002:A01:202::787A:C0FF:FEA8:C801 prefixlen 64
        inet6 unicast FE80::787A:C0FF:FEA8:301%simnet0 prefixlen 64
...
simnet1 Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:03 Queue:none
        inet 192.168.200.3 mask 255.255.255.0 broadcast 192.168.200.255
        inet 224.0.0.1 mask 240.0.0.0
        inet6 unicast 2002:A01:201::787A:C0FF:FEA8:C803 prefixlen 64
        inet6 unicast FE80::787A:C0FF:FEA8:C803%simnet1 prefixlen 64
...
value = 0 = 0x0
->
```

4. In the **VxSim3** console, type the following:

```
-> ifconfig "simnet0"
simnet0 Link type:Ethernet HWaddr 7a:7a:c0:a8:03:02 Queue:none
        inet 192.168.3.2 mask 255.255.255.0 broadcast 192.168.3.255
        inet 224.0.0.1 mask 240.0.0.0
        inet6 unicast 2002:A01:202::787A:C0FF:FEA8:302 prefixlen 64
        inet6 unicast FE80::787A:C0FF:FEA8:302%simnet0 prefixlen 64
...
value = 0 = 0x0
->
```

Verify That the Routing Advertisement Service Is Operational on page 58 In the **VxSim0** and **VxSim2** consoles, observe that the **ipnet_radvd** service is operational (**started**).

Parent topic: Networking with IPv6 on page 53

Verify That the Routing Advertisement Service Is Operational

In the **VxSim0** and **VxSim2** consoles, observe that the **ipnet_radvd** service is operational (**started**).

Type the following in the **VxSim0** console and then in the **VxSim2** console:

```
-> cmd
[vxWorks *]# ipd list
Services:
ipnet                 started
ipnet_radvd           started
iptcp                 started
```


WNDRVR

```
iptelnets            started
ipradius             started
```

Getting Help

To get help with the ipd command, type ipd at the target command shell prompt.

Parent topic: Verify the Network Interfaces on page 57

2.5. Test the Network Connections

Use the ping6 command to test the connections between VxWorks simulator instances on the same subnet.

About This Task

The default subnet (**default**) includes **VxSim0**, **VxSim1**, and the host interface. Subnet 3 (**sub3**) includes **VxSim2** and **VxSim3**.

This procedure uses the ping and ping6 utilitities.

- To get help for the ping command, type ping at the host command shell prompt.
- To get help for the ping6 command, type ping6 at the target command shell prompt.

Procedure

1. Ping **VxSim1** from **VxSim0**. In the **VxSim0** console, enter the following:

```
    -> cmd
    [vxWorks *]# ping6 2002:A01:201::787A:C0FF:FEA8:C802

    Pinging 2002:a01:201::787a:c0ff:fea8:c802
(2002:a01:201::787a:c0ff:fea8:c802) with 64 bytes of data:
    Reply from 2002:a01:201::787a:c0ff:fea8:c802 bytes=64 hlim=64 seq=0 time=0ms

    --- 2002:a01:201::787a:c0ff:fea8:c802 ping
statistics ---
    1 packets transmitted, 1 received, 0% packet loss, time 0 ms
    rtt min/avg/max = 0/0/0 ms
```

2. In the **VxSim1** console, you can ping **VxSim0** with the local address as follows:

```
    -> cmd
    [vxWorks *]# ping6 FE80::787A:C0FF:FEA8:C801%simnet0
    ...
```

3. From **VxSim1**, you can also ping **VxSim0** at the automatically configured address as follows:

```
    -> cmd
    [vxWorks *]# ping6 2002:A01:201::787A:c0FF:FEA8:C801
    ...
```

4. From your host you can ping the simulator instances. For example, view the local interface on your host and ping **VxSim0**.

WNDRVR

```
C:\> ipconfig
...
Ethernet adapter Local VxSim0:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::ac42:f964:e82e:8810%47
   IPv4 Address. . . . . . . . . . . : 192.168.200.254
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :
...

C:\> ping 2002:a01:201::787a:c0ff:fea8:c801
Pinging 2002:a01:201:0:787a:c0ff:fea8:c801 with 32 bytes of data:
Reply from 2002:a01:201:0:787a:c0ff:fea8:c801: time=3ms
Reply from 2002:a01:201:0:787a:c0ff:fea8:c801: time<1ms
Reply from 2002:a01:201:0:787a:c0ff:fea8:c801: time<1ms
Reply from 2002:a01:201:0:787a:c0ff:fea8:c801: time<1ms

Ping statistics for 2002:a01:201:0:787a:c0ff:fea8:c801:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 3ms, Average = 0ms
```

Parent topic: Networking with IPv6 on page 53

2.6. Create Routes to Interconnect Subnets

In addition, you can set the routing such that you can ping between the **default** subnet and subnet **sub3** through **VxSim2**.

For example usage of routing commands to set up routes, see Basic Networking Example on page 45 and the *VxWorks 7 VxSim User's Guide*.

Parent topic: Networking with IPv6 on page 53

7. TRANSPORT AND NETWORK PROTOCOL CONFIGURATION

1. About Configuring Transport and Network Protocols

After you have included the appropriate VxWorks source build (VSB) options and VxWorks image project (VIP) components for the network stack, you can tailor your VxWorks configuration for basic TCP/IP support and to support various transport and network protocols.

Some of the protocols are automatically included as part of the core stack, while others can be individually added to a network stack.

The available transport and network protocols are:

- Internet Control Message Protocol (ICMP), ICMPv4 and ICMPv6 automatically included
- Internet Protocol (IPv4, IPv6)
- Transmission Control Protocol/Internet Protocol (TCP/IP)
- User Datagram Protocol (UDP)
- Address Resolution Protocol (ARP), automatically included
- Neighbor Discovery Protocol (NDP), automatically included
- CGA
- SEND

Most protocols can be configured at build time or at run time (using the sysvar or other shell commands).

For more information on specific configuration parameters, see About VxWorks Network Stack Configuration Parameters on page 213.

Next topic: Adding ICMP Support to a VIP (v4 and v6) on page 61

2. Adding ICMP Support to a VIP (v4 and v6)

Internet Control Message Protocol (ICMP) is part of the VxWorks 7 Network Stack.

The appropriate version of ICMP is automatically included when you include the IPv4 or IPv6 components. For more information, see IPv4 Support on page 61 or IPv6 Support in a VIP on page 64.

Previous topic: About Configuring Transport and Network Protocols on page 61
Next topic: IPv4 Support on page 61

3. IPv4 Support

To add VxWorks 7 Network Stack IPv4 support, you must include the INCLUDE_IPCOM_USE_INET component in your VxWorks image project (VIP).

This component includes User Datagram Protocol (UDP) and Internet Control Message Protocol (ICMP) support.

Adding Support for Automatic Network Interface Attachment

At boot time, the network boot device is automatically attached to all supported network layer protocols built into the stack. To attach additional IPv4 devices at run time using IpAttach( ), include the INCLUDE_IPATTACH VIP component in your project.

Configuring IPv4

You should customize the default values for the configuration parameters associated with the INCLUDE_IPCOM_USE_INET component to suit the needs of your application. Configuration parameters can be updated at build time by updating your VIP kernel configuration or by using the sysvar command at run time.

For more information on using the sysvar command, see About System Variables on page 209.

For a list of available configuration parameters and system variables, see About VxWorks Network Stack Configuration Parameters on page 213.

About Automatic IPv4 Interface (Auto IP) Configuration

To join a wide-area network, a host must be configured with IP addresses for its interfaces. Such configuration can be performed either manually by the user or automatically by a network resource, such as a DHCP server.

In some cases, address configuration by user assignment or DHCP is unavailable, but a host must be still able to rely on a useful subset of IP networking functions. For these scenarios, the Wind River Network Stack supports dynamic configuration of IPv4 link-local addresses, or Auto IP. When this feature is enabled, a host can automatically configure an interface with an IPv4 address within the 169.254/16 prefix. The address is valid for communication with other devices connected to the same physical (or logical) link.

IPv4 link-local addresses are not suitable for communication with devices not directly connected to the same physical (or logical) link, and they are only used where stable, routable addresses are not available, such as ad hoc or isolated networks. IPv4 link-local addresses and routable addresses should not be configured simultaneously on the same interface.

When you configure a host with Auto IP, it can join a local network automatically, without being either manually configured or built with hard-coded IPv4 addresses. In addition, if a DHCP agent later assigns a routable IPv4 address to an interface, the host uses that address instead of the Auto IP-configured address. In such a case, the target retains the Auto IP address, defends it in case another interface on the network attempts to assume it, and uses it if the routable IPv4 address is removed.

About Address Conflict Detection (ACD)

Two hosts on the same link may attempt to use the same IP address. To prevent the problems that can result from this situation, address conflict detection (ACD) can be used to determine if an address is likely to lead to an addressing conflict, or is inadvertently being used by another host on the network, and to limit the number of address acquisition attempts in the case of repeated conflicts.

Configuring the VxWorks for Auto IP on page 63 You configure a system with Auto IP at build time. Once it is so configured, you can make further modifications at run time.
Auto IP Configuration on page 63 You should customize the default values for the configuration parameters associated with the INCLUDE_IPAIP_GLOBAL_CONFIGS and the INCLUDE_IPAIP_INTERFACE_CONFIGS components to suit the needs of your application. Configuration parameters can be updated at build time by updating your VIP kernel configuration using Workbench 4 or the wrtool command-line facility, or by using the sysvar command.
Address Conflict Detection (ACD) Configuration on page 64 VxWorks 7 Network Stack ACD is controlled by the *ipnet.inet.AddressConflictDetect* system variable, which can be used globally or for individual interfaces.

Previous topic: Adding ICMP Support to a VIP (v4 and v6) on page 61
Next topic: IPv6 Support in a VIP on page 64

3.1. Configuring the VxWorks for Auto IP

You configure a system with Auto IP at build time. Once it is so configured, you can make further modifications at run time.

About This Task

Auto IP is included in a VxWorks image project by default. However, Auto IP is not enabled, or active, on an interface until you specify that particular interface for Auto IP configuration (using the INET_IPAIP_IFNAME_LIST parameter).

Auto IP is automatically disabled if an interface is configured with an additional address (either manually, using an IFCONFIG_*N* parameter, or automatically by DHCP). If the additional address is subsequently removed from that interface, Auto IP is automatically re-enabled.

To enable Auto IP, proceed as follows:

Procedure

1. Include the **IPv4 AutoIP Components** (FOLDER_AUTOIP).
2. Include the INCLUDE_IPAIP component and specify one or more interfaces for Auto IP, using the **autoIP interface list** (INET_IPAIP_IFNAME_LIST). If you are listing multiple interfaces for Auto IP, separate them with a space, for example: **"eth0 eth1"**.
3. Include one or both of the following optional components:
   - INCLUDE_IPAIP_GLOBAL_CONFIGS (optional). This component lets you configure Auto IP globally. If you do not include this component, the system still uses its default values, which are the values from RFC 5227.
   - INCLUDE_IPAIP_INTERFACE_CONFIGS (optional). This component lets you configure Auto IP on a per-interface basis.
4. Specify values for INCLUDE_IPAIP_GLOBAL_CONFIGS and INCLUDE_IPAIP_INTERFACE_CONFIGS, if you have included them.
5. Build the project.
6. If any of the interfaces you specified for Auto IP also have addresses assigned statically or by a DHCP server, you must delete those addresses before Auto IP can configure the interface. To do this, proceed as follows:
   1. Run the **ifconfig** shell command to determine the assigned IP address, for example:

```
-> ifconfig "eth0"
eth0 Link type:Ethernet HWaddr 00:1e:a0:a0:1e:01
Queue:none inet 192.168.195.7 mask 255.255.0.0
```

   2. Delete that IP address, for instance with the following command:

```
-> ifconfig "eth0 inet delete 192.168.195.7"
```

   3. Auto IP will start automatically. You can see the new address assignment with ifconfig, for example:

```
-> ifconfig "eth0"
eth0 Link type:Ethernet HWaddr 00:1e:a0:a0:1e:01
Queue:none inet 169.254.134.89 mask 255.255.0.0
```

Parent topic: IPv4 Support on page 61

3.2. Auto IP Configuration

You should customize the default values for the configuration parameters associated with the INCLUDE_IPAIP_GLOBAL_CONFIGS and the INCLUDE_IPAIP_INTERFACE_CONFIGS components to suit the needs of your application. Configuration parameters can be

updated at build time by updating your VIP kernel configuration using Workbench 4 or the wrtool command-line facility, or by using the sysvar command.

For more information on using the sysvar command, see About System Variables on page 209.

For a list of available configuration parameters and system variables, see Global Configuration Parameters for Auto IP and Address Conflict Detection (ACD) on page 217.

Note: The same set of configuration parameters are used to control both ACD and Auto IP behavior. By default, the values of the configuration parameters used by ACD are those configured with Auto IP at build time.

Parent topic: IPv4 Support on page 61

3.3. Address Conflict Detection (ACD) Configuration

VxWorks 7 Network Stack ACD is controlled by the *ipnet.inet.AddressConflictDetect* system variable, which can be used globally or for individual interfaces.

By default, ACD is not enabled to prevent removing an IP address should that address already be present on the network.

Note: Not enabling ACD allows the DHCP client to obtain a duplicate IP address from another node on the network causing unexpected issues.

To use ACD, you must also configure VxWorks with Auto IP, see Configuring the VxWorks for Auto IP on page 63 and Global Configuration Parameters for Auto IP and Address Conflict Detection (ACD) on page 217.

Using the sysvar command at run time to change a system variable changes it for both ACD and Auto IP.

The same set of configuration parameters are used to control both ACD and Auto IP behavior. By default, the values of the configuration parameters used by ACD are those configured with Auto IP at build time. For information on configuring Auto IP support, see Auto IP Configuration on page 63.

Parent topic: IPv4 Support on page 61

4. IPv6 Support in a VIP

To add VxWorks 7 Network Stack IPv6 support, you must include the INCLUDE_IPCOM_USE_INET6 component in your VxWorks image project (VIP). This component includes User Datagram Protocol (UDP), Internet Control Message Protocol (ICMP), and Neighbor Discovery Protocol (NDP) support.

Note: If IPv6 is enabled, IPCOM_USE_INET6 must be defined or the **Ipftps_sockcb** data structures embedded in the **Ipftps_session** data structure may not be valid. To include the IPv6 structures, you must add the following statements in your source code prior to including the **ipcom_sock.h** header file.

```
#if defined(INET6)     #define IPCOM_USE_INET6 1 #endif
```

Adding Support for Attaching Additional IPv6 Devices on page 65 At boot time, the network boot device is automatically attached to all supported network layer protocols built into the stack.
IPv6 Configuration on page 65 You should customize the VxWorks 7 Network Stack default values for the configuration parameters associated with the INCLUDE_IPCOM_USE_INET6 component to suit the needs of your application.
About IPv6 Address Scopes and Zones on page 65 VxWorks 7 Network Stack IPv6 addresses can be characterized by the properties of *scope* and *zone*.

IPv6 Address Scope and Zone Setup on page 67 You can set the VxWorks 7 Network Stack IPv6 address scope and zone for a node, using the sysvar command with a system variable of the following form:
Assigning the Default Zone on page 67 You can assign the VxWorks 7 Network Stack default zone for a scope, such as **node**, **link**, **subnet**, **admin**, **site**, or **org**.
Attach an Interface to a Zone on page 68 All of the VxWorks 7 Network Stack network interfaces are assigned to their default zones when a node first comes up. However, you can reassign an interface to a zone defined as a node, link, subnet, admin, site, or organization, using a system variable. For example:
Privacy Extensions for Stateless Address Autoconfiguration on page 68 You can configure a VxWorks 7 Network Stack IPv6 node with stateless address autoconfiguration. The system then generates addresses using a combination of locally available information and information advertised by routers, typically by combining network prefixes with an interface identifier.

Previous topic: IPv4 Support on page 61
Next topic: Adding TCP Support to a VIP on page 69

4.1. Adding Support for Attaching Additional IPv6 Devices

At boot time, the network boot device is automatically attached to all supported network layer protocols built into the stack.

To attach additional IPv6 devices at run time using ip6Attach( ), include the INCLUDE_IP6ATTACH component in your VIP project.

Parent topic: IPv6 Support in a VIP on page 64

4.2. IPv6 Configuration

You should customize the VxWorks 7 Network Stack default values for the configuration parameters associated with the INCLUDE_IPCOM_USE_INET6 component to suit the needs of your application.

Configuration parameters can be updated at build time by updating your VIP kernel configuration using Workbench 4 or the wrtool command-line facility, or by using the sysvar command.

For more information on using the sysvar command, see About System Variables on page 209.

For a list of available configuration parameters and system variables, see About VxWorks Network Stack Configuration Parameters on page 213.

Parent topic: IPv6 Support in a VIP on page 64

4.3. About IPv6 Address Scopes and Zones

VxWorks 7 Network Stack IPv6 addresses can be characterized by the properties of *scope* and *zone*.

Address Scope

Scope identifies the application suitable for the address--that is, the region where an IPv6 address can serve as the unique identifier of an interface. Unicast and multicast addresses both support the concept of scope.

Unicast addresses support the following three scopes:

- Node local (**::1**)
- Link-local (**fe80::/16**)
- Global (all others)

Multicast addresses support five scopes, as described in the table below.

Scope Field Value	Scope
0	Reserved
1	Interface-local scope
2	Link-local scope
3	Reserved
4	Administrator-local scope
5	Site-local scope
6	Unassigned
7	Unassigned
8	Organization-local scope
9	Unassigned
A	Unassigned
B	Unassigned
C	Unassigned
D	Unassigned
E	Global
F	Reserved

All other possible scope values are currently unassigned.

The scope of a multicast address is determined by the fourth nibble.

- The first byte (two first nibbles) are always 0xFF.
- The next nibble is a flag field.
- The next nibble is the scope.

Address Zones

IPv6 addresses are further characterized by *zones*, which are defined within specific scopes. Each address can belong to only one zone, which is unique within its scope.

A scope is defined by a numerical identifier called the *zone ID*. For example, a site-local zone with the ID **123** might contain the link-local zones 1, 2, 3, and 4.

WNDRVR

The default zone ID for all scopes except **node** and **link** is 0. The default zone ID for **node** is always 1--that is, the loopback device. Do not change this assignment. The zone ID for **link** is always > 1. To be more precise, it is the first non-loopback interface with an *ifindex* > 1. A target generally boots from this interface.

Parent topic: IPv6 Support in a VIP on page 64

4.4. IPv6 Address Scope and Zone Setup

You can set the VxWorks 7 Network Stack IPv6 address scope and zone for a node, using the sysvar command with a system variable of the following form:

```
ipnet.inet6.zone.[node|subnet|admin|site|org|link]Id = num
```

You can make two types of assignment using this system variable:

- You can assign the default zone for a scope, such as **node**, **subnet**, **admin**, **site**, **org**, or **link**.
- You can attach an individual interface to link that belongs to the specified zone.

For specific information on the available system variables, see IPv6 Address Scope and Zone System Variables on page 226.

Parent topic: IPv6 Support in a VIP on page 64

4.5. Assigning the Default Zone

You can assign the VxWorks 7 Network Stack default zone for a scope, such as **node**, **link**, **subnet**, **admin**, **site**, or **org**.

The default configuration has the following zone assignments:

- one link-local zone per network interface
- one zone for all interfaces for all zones with a scope that is greater than link-local (**subnet**, **admin**, **site** and **org**).

It is possible to attach more than one interface to the same link-local zone, but you should only make such an assignment for interfaces that are connected to the same physical link. Few nodes require such an assignment in practice.

However, you can alter this configuration to meet the needs of your application. For example, consider a node with at least two Ethernet interfaces. You want to assign the link-local zone to the second Ethernet interface, which is defined by **ifindex=3**. You would use the following command:

```
[vxWorks *] # sysvar set -co ipnet.inet6.zone.link.Default 3
```

You can also define default zones for specific interfaces. For example, consider a node with interfaces **lo0**, **eth0**, **eth1**, and **eth2**. Define two subnet zones:

- Zone 1 contains **lo0**, **eth0**, and **eth1**.
- Zone 2 contains the rest (only **eth2** in this case).

The following sysvar commands implement these assignments:

```
[vxWorks *] # sysvar set -co lo0.inet6.zone.subnet.Id 1
[vxWorks *] # sysvar set -co eth0.inet6.zone.subnet.Id 1
[vxWorks *] # sysvar set -co eth1.inet6.zone.subnet.Id 1
[vxWorks *] # sysvar set -co ipnet.inet6.zone.subnet.Id 2
```


WNDRVR

For most applications, the default zone IDs are correct, but in cases where two or more physical interfaces are attached to the same link and affect multicast applications, it may be appropriate to modify the zone ID settings.

> **Note:** The Wind River Network Stack does not enforce the RFC 4007 requirement that zone A must be a strict superset of zone B, where the scope of zone A is greater than the scope of zone B. The system administrator must ensure the application of this requirement.

The ability to define a default zone for a node has implications for programming practice. The default zone is used if the calling function sets the *scope_id* field of the struct **sockaddr_in6** to zero. Such a setting makes it possible to omit the link when sending data to a link-local address. For example, the following command:

```
[vxWorks *] # ping6 fe80::2:3:4:5
```

would fail without a default zone assignment, even if there is a node with the address **fe80::2:3:4:5** on the link to which the node's first physical interface is attached. With a default zone assignment, however, this command succeeds--just as if it were written in the following manner:

```
[vxWorks *] # ping6 fe80::2:3:4:5%ifname_of_first_physical_if
```

Parent topic: IPv6 Support in a VIP on page 64

4.6. Attach an Interface to a Zone

All of the VxWorks 7 Network Stack network interfaces are assigned to their default zones when a node first comes up. However, you can reassign an interface to a zone defined as a node, link, subnet, admin, site, or organization, using a system variable. For example:

```
ifname .ipnet.inet6.zone.[subnet|admin|site|link|org].Id num
```

For example, the following command:

```
[vxWorks *] # sysvar set -co eth0.ipnet.inet6.zone.link.Id 3
```

attaches the interface **eth0** to a link that is a member of the zone with zone ID **3**.

The *ifindex* qualifier is a unique identifier assigned by the network stack to all attached network interfaces. Use if_nametoindex( ) to get the *ifindex* of an interface. For example:

```
ifnametoindex( ifname );
```

Parent topic: IPv6 Support in a VIP on page 64

4.7. Privacy Extensions for Stateless Address Autoconfiguration

You can configure a VxWorks 7 Network Stack IPv6 node with stateless address autoconfiguration. The system then generates addresses using a combination of locally available information and information advertised by routers, typically by combining network prefixes with an interface identifier.

If an interface contains an embedded IEEE identifier, the node derives its interface identifier from the IEEE identifier. For other interface types, the node generates an interface identifier by other means, such as random number generation. In addition, you can also configure a node or individual interface with privacy extensions. When enabled, privacy extensions cause nodes whose interface identifier is derived from an IEEE identifier to generate global scope addresses from interface identifiers that change over time.

Note: Privacy extensions for stateless address autoconfiguration are incompatible with FIPS mode. When you compile the network stack source code in FIPS mode, privacy extensions for stateless address autoconfiguration are disabled.

Changing the interface identifier (and the global scope addresses generated from it) provides additional privacy because it is more difficult for eavesdroppers to identify when different addresses used in different transactions actually correspond to the same node.

When privacy extensions are enabled for a node or interface, the system identifies each newly generated address as the preferred address and uses this address for newly established application sessions. The lifetime of a preferred address is defined at run time by a system variable (*ipnet.inet6.TempPreferredLifetime*). Following this interval, an address goes into a deprecated, or non-preferred state, for the length of time specified by the system variable **ipnet.inet6.TempValidLifetime**. The system continues to use deprecated addresses for existing sessions until the address expires altogether. If an application session extends beyond this lifetime, the application must acquire a new temporary address.

For a list of configuration variables available for stateless address autoconfiguration, see Configuration Variables for Stateless Address Autoconfiguration on page 226.

Parent topic: IPv6 Support in a VIP on page 64

5. Adding TCP Support to a VIP

To add Transmission Control Protocol (TCP) support to your VxWorks image, you must include the INCLUDE_IPTCP component in your VxWorks image project (VIP).

This component pulls in modules that provide support for TCP in either the IPv4 or IPv6 domains. (You can designate which domain it supports when you build the VxWorks source code.)

Configuring Transmission Control Protocol (TCP) on page 69 You should customize the default values for the configuration parameters associated with the INCLUDE_IPTCP component to suit the needs of your application.

Previous topic: IPv6 Support in a VIP on page 64
Next topic: Adding ARP Support to a VIP on page 69

5.1. Configuring Transmission Control Protocol (TCP)

You should customize the default values for the configuration parameters associated with the INCLUDE_IPTCP component to suit the needs of your application.

Configuration parameters can be updated at build time by updating your VIP kernel configuration using Workbench 4 or the wrtool command-line facility, or by using the sysvar command.

For more information on using the sysvar command, see About System Variables on page 209.

For a list of available configuration parameters and system variables, see TCP Configuration Parameters on page 227.

Parent topic: Adding TCP Support to a VIP on page 69

6. Adding ARP Support to a VIP

Address Resolution Protocol (ARP) is part of the network stack and is automatically included when you build an IPv4 network stack (see IPv4 Support on page 61).

Note: You can also develop a custom link layer and address resolution functions, then register the function hooks with the network stack. The network stack then bypasses its normal Ethernet processing and ARP and calls the custom functions instead.

Adding Support for Proxy ARP on page 70
Adding Support for ARP Runtime Configuration on page 70
Configuring ARP at Run Time on page 70
Configuring ARP Entry Cache Time on page 71 Address resolution protocol (ARP) entries for IP addresses that have not received communication recently (*recently* being defined as 30 seconds by default) are cached for the length of time specified by the INET_NBR_CACHE_TO_LIVE_TIME parameter, or with the system variable *ipnet.inet.NeighborCacheToLive* .
Adding Support for Gratuitous ARP at Build Time on page 71 Address resolution protocol (ARP) can be configured to support gratuitous ARP.
Adding Support for Gratuitous ARP at Run Time on page 71 You can enable or disable gratuitous address resolution protocol (ARP) at run time, using the sysvar shell command.
Configuring Proxy ARP on page 71 You should customize the default values for the configuration parameters associated with the INCLUDE_IPPROXYARP component to suit the needs of your application.

Previous topic: Adding TCP Support to a VIP on page 69
Next topic: Neighbor Discovery Protocol (NDP) on page 72

6.1. Adding Support for Proxy ARP

To add proxy ARP support, you must include the INCLUDE_IPPROXYARP component in your VIP. This component supplies configuration parameters to define how proxy ARP works over the network.

You can also use an API to enable or disable proxy ARP on a per-interface basis. Call ifProxyArpEnable( ) to enable or ifProxyArpDisable( ) to disable.

For more information about proxy ARP, see Configuring Proxy ARP on page 71.

Parent topic: Adding ARP Support to a VIP on page 69

6.2. Adding Support for ARP Runtime Configuration

Address resolution protocol (ARP) provides a shell command and an API for run-time configuration, but they must be included at build time. For the shell command, add the **INCLUDE_IPARP_CMD** component. For the **arpLib** API, add the INCLUDE_ARP_API component. This library gives application programs access to the ARP entries so they can list, add, and delete ARP entries.

For information about the run time use of the shell command and API, see Configuring ARP at Run Time on page 70.

Parent topic: Adding ARP Support to a VIP on page 69

6.3. Configuring ARP at Run Time

Use the arp command or the **arpLib** library functions to configure ARP at run time. For further information, see the reference entry for **arpLib** in the Workbench 4 online help or the **arp** shell command in the About Network Stack Shell Commands on page 235. VxWorks must be configured with support for these features; see Adding Support for ARP Runtime Configuration on page 70.

For information on configuring Proxy ARP, see Configuring Proxy ARP on page 71.

Parent topic: Adding ARP Support to a VIP on page 69

6.4. Configuring ARP Entry Cache Time

Address resolution protocol (ARP) entries for IP addresses that have not received communication recently (*recently* being defined as 30 seconds by default) are cached for the length of time specified by the INET_NBR_CACHE_TO_LIVE_TIME parameter, or with the system variable *ipnet.inet.NeighborCacheToLive* .

This allows traffic to flow immediately once communication is resumed. The default value is 1,200 seconds.

The network stack only removes an ARP entry when a node receives no traffic for the length of time that is specified by or *ipnet.inet.NeighborCacheToLive* --or when the specified node stops responding to ARP requests.

There is a small delay associated with acquiring an IP address to link layer mapping if the mapping is not present in the cache. In general, this should not present a problem. The INET_NBR_CACHE_TO_LIVE_TIME parameter or *ipnet.inet.NeighborCacheToLive* system variable can, however, be set to a large (or infinite--with a value of zero) amount of time to reduce this delay, and the latency of the first packet transmission to that node--but this only has the desired effect if the link layer address has remained the same. If the link layer address has changed, then packets are sent to the wrong link-layer address (and to the wrong node or lost) until a new mapping is detected.

Parent topic: Adding ARP Support to a VIP on page 69

6.5. Adding Support for Gratuitous ARP at Build Time

Address resolution protocol (ARP) can be configured to support gratuitous ARP.

This support is configured by means of the configuration parameter INET_SEND_GRATUITOUS_ARP.

To enable gratuitous ARP, set this parameter to **"Yes"**. The target will send an ARP packet when an address is added to a network interface.

To disable gratuitous ARP, set this parameter to **"No"**. The target will not send an ARP packet when an address is added to a network interface.

Parent topic: Adding ARP Support to a VIP on page 69

6.6. Adding Support for Gratuitous ARP at Run Time

You can enable or disable gratuitous address resolution protocol (ARP) at run time, using the sysvar shell command.

You can control this feature either globally or on a per interface basis.

To enable or disable gratuitous ARP globally, use the following shell command:

```
[vxWorks *] # sysvar set -co ipnet.inet.SendGratuitousArp {yes|no}
```

To enable or disable gratuitous ARP for an individual interface, use the following shell command:

```
[vxWorks *] # sysvar set -co ifname.inet.SendGratuitousArp {yes|no}
```

Parent topic: Adding ARP Support to a VIP on page 69

6.7. Configuring Proxy ARP

You should customize the default values for the configuration parameters associated with the INCLUDE_IPPROXYARP component to suit the needs of your application.

Configuration parameters can be updated at build time by updating your VIP kernel configuration using Workbench 4 or the wrtool command line facility, or by using the sysvar command at runtime.

For more information on using the sysvar command, see About System Variables on page 209.

For a list of available configuration parameters and system variables, seeProxy Address Resolution Protocol (ARP) Configuration Parameters on page 232.

Parent topic: Adding ARP Support to a VIP on page 69

7. Neighbor Discovery Protocol (NDP)

NDP is part of the VxWorks 7 network stack and is automatically included when you build an IPv6 stack.

Adding NDP Support to a VIP

For information on adding support for IPv6, see IPv6 Support in a VIP on page 64.

Adding Support for the NDP Shell Command

To include support for the ndp shell command in your VxWorks image, include the **IPCOM** ndp command build component, INCLUDE_IPNDP_CMD.

Configuring NDP at Run Time

You can configure NDP using the ndp shell command. For further information, see the ndp reference entry in the *Network Stack Shell Command Reference*.

Previous topic: Adding ARP Support to a VIP on page 69
Next topic: VxWorks 7 Network Stack Remote Host Library on page 72

8. VxWorks 7 Network Stack Remote Host Library

Adding Remote Host Library Support to a VIP

To include support for the **hostShow** library in your VxWorks image project (VIP), add the INCLUDE_NET_HOST_SHOW component. The **hostShow** library supplies the hostShow( )function. This function provides a list of remote hosts, along with their Internet addresses and aliases.

Previous topic: Neighbor Discovery Protocol (NDP) on page 72

8. QUALITY OF SERVICE

1. About VxWorks 7 Network Stack Quality of Service (QoS)

Quality of Service (QoS) refers to the capability of a network to treat some traffic flows differently than others.

The Wind River Network Stack implements the Differentiated Services (DiffServ) model of QoS, which classifies traffic entering a network and conditionalizes it before treating it in an appropriate manner. The most common usage is to give a specific traffic flow a better service than the normal best effort service.

Different types of flow have different requirements; for example, some flows have restrictions in latency, while others have restrictions in minimum bandwidth, and so on. For example:

- Interactive traffic, like Telnet and Secure Socket Shell (SSH), perform better with low latency so that the user does not experience a delay when typing.
- File Transfer Protocol (FTP) traffic performs better when it can use as much bandwidth as possible; it does not matter if the latency is high or if the data arrives in bursts.

Telnet has a high latency if routers treat all traffic equally, which is the default best effort behavior used by most routers. However, routers can improve latency if each router identifies the two varieties of flow and then lets the Telnet packets move ahead of FTP packets that the router has already queued to send on the outgoing interface. Doing so ordinarily has little effect on the FTP application because telnet normally uses little bandwidth.

Next topic: About Differentiated Services on page 73

2. About Differentiated Services

The Differentiated Services (DiffServ) architecture is based on a model in which an edge router classifies traffic entering a network, possibly conditions it (for instance to reduce jitter or latency), and assigns it to different behavior aggregates.

It assigns packets to behavior aggregates by setting a single differentiated-services (DS) codepoint in the value of the DS field of the IP datagram (the type of service (ToS) field for IPv4 or the Traffic Class field for IPv6). The core routers of the network can then forward these packets according to the per-hop behavior they associate with each DS codepoint.

To control, create, and delete interface output queues, use the API and structures defined in the following files:

installDir **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_diffserv.c**

installDir **/vxworks-7/pkgs_v2/net/ipnet/qos/h/ipnet_qos.h**

Previous topic: About VxWorks 7 Network Stack Quality of Service (QoS) on page 73
Next topic: Including Differentiated Services in a Build on page 73

3. Including Differentiated Services in a Build

To enable differentiated services in a VxWorks source build (VSB), enable the IPNET_DIFFSERV option.

Use this option to enable the simple marker (see Using Existing Meter/Marker Entity Varieties on page 81) and the single-rate, three-color marker (see Using Existing Meter/Marker Entity Varieties on page 81).

To manage differentiated services from your VIP, include the following components:

INCLUDE_IPQOS_CMD

Enables the use of shell commands to configure differentiated services.

INCLUDE_IPQUEUE_CONFIG_CMD

Enables the use of shell commands to configure output queues.

Previous topic: About Differentiated Services on page 73
Next topic: Using Differentiated Services on page 74

4. Using Differentiated Services

You can configure a DiffServ traffic classifier to run either in behavior aggregate mode or in multifield mode:

In behavior aggregate mode, the classifier looks only at the DS field (the type of service (ToS) field in IPv4, or the traffic class for IPv6).

In multifield mode, the classifier may look at any field supported by the IPNET classifier--this mode is slower but more flexible.

To run in behavior aggregate mode, define the macro **IPNET_DIFFSERV_CLASSIFIER_MODE_BA** in the following file:

installDir **/vxworks-7/pkgs_v2/net/ipnet/coreip-**x.x.x.x **/src/ipnet2/config/ipnet_config.h**

The macro is defined by default.

Undefine this macro to run in multifield mode. When in the multifield mode, the differentiated services commands are available from the qos shell command. See qos Command on page 99.

Adding a Filter Rule for a Meter/Marker Entity

To add a filter rule for a meter/marker entity when running DiffServ in multifield mode, you can either call a function from within a program or use a qc command interactively.

To invoke this operation from within a program, use the following call:

ioctl (sock_fd, SIOCXADSFILTER, &filter);

Where **filter** is a **ds_filter** object that describes the filter you are adding (see Differentiated Services Classes on page 77). The network stack chooses an ID for the filter and stores it in the **id** member of this object. Use this ID number if you need to refer to this specific filter, for instance if you add it to a DiffServ meter/marker entity.

You can also use a qc command, which has the following format:

```
[vxWorks *] # qc filter add dev device  parent queueID handle filterID [filterArgs] flowid queueID
```


WNDRVR

The arguments to this command are as follows:

dev *device*

The device to which you are attaching the filter, for instance **eth0**.

parent *queueID*

The identifying number of the container queue to which you are adding the filter.

handle *filterID* [*filterArgs*]

The *filterID* is the identifying number of the filter. You may use the following arguments in the *filterArgs* argument to describe the filter:

- proto *number*
- tclass *number*
- srcport *range*
- dstport *range*
- srcaddr *address* [*/prefix*]
- dstaddr *address* [*/prefix*]

flowid *queueID*

The identifying number of the destination queue for packets that match the filter.

For example:

To add a filter identified by the number five to the container queue identified by the number one, so that all TCP packets (protocol number six) are filtered into the queue identified by the number 31, use the following qc command:

```
[vxWorks *] #qc filter add dev eth0 parent 1 handle 5 proto 6 flowid 31
```

To add a second filter (identified by the number three) to the same container queue that filters all UDP packets (protocol number 17) that are sent to 2001::/16 into the same queue, use the following qc command:

```
[vxWorks *] # qc filter add dev eth0 parent 1 handle 3 proto 17 srcaddr 2001::/16 flowid 31
```

Deleting a Filter Rule from a Meter/Marker Entity

To delete a filter rule from a meter/marker entity when running DiffServ in multifield mode, you can either call a function from within a program or use a qc command interactively.

To invoke this operation from within a program, use the following call:

```
ioctl (sock_fd, SIOCXDDSFILTER, &filter);
```

In this call, **filter** is a **ds_filter** object that describes the filter you are deleting (see Differentiated Services Classes on page 77). You only need to set the **id** member of this object in order to specify which filter you want to delete.

You can also use a qc command, which has the following format:

```
[vxWorks *] #qc filter del filterID
```

qc Command Example

In this example, you add a multi-band container (with 2 bands) as the root, use a FIFO on the first band and a token bucket container with a maximum rate of 2 Mbps.

The example looks similar to the following:

```
[vxWorks]# qc queue add dev eth0 root handle 1 mbc bands 2 default_band 0
[vxWorks]# qc queue add dev eth0 parent 1 handle 1000 htbc rate 2Mbps
[vxWorks]# qc queue show dev eth0
qc queue show dev eth0
mbc/1[0] queue at eth0
 bands: 2, default_band: 0
 htbc/1000[1] queue at eth0
    rate: 2000kbps, burst: 6000, default id: 1002
      fifo/1002[1000] queue at eth0
        limit: 16
  fifo/1001[1] queue at eth0
    limit: 16
```

In this example every packet sent on interface **eth0** is queued to the multi-band container (**mbc**) as that is the root queue.

No classification rules have been added to the MBP yet. Therefore, all packets will end up in the default band which is specified as being the first band (index starts at zero).

The packet is passed to the child queue attached to first band which, in this case, is a hierarchical token bucket container (**htbc**) where you set the rate to 2 MBps.

A **htbc** can have multiple child queues, but this is not a practical use case. **mbc** should be used for fan-out.

The **htcb** measures the rate of packets going through this queue. Packets are passed to the child queue; the fifo-queue with ID 1002, in this example.

mbc always de-queues packets from the band with the lowest number, moving to the next band when the child queue returns **NULL**.

In this example, the dequeue operation goes to the **htbc** which, in turn, verifies the rate at which it has dequeued packets recently. **NULL** is returned if the rate exceeds the configured rate. Otherwise, it calls the dequeue handler on the child queue.

The child queue, in this case, is the fifo-queue with id 1002 to which something was just enqueued. Therefore, that packet is returned and delivered to the driver for **eth0**.

Finally, you can add a rule to the **mbc** so that every UDP packet goes to the second band as follows:

```
[vxWorks]# qc filter add dev eth0 parent 1 handle 100 proto 17 flowid 1001
```

To understand this command, **parent 1** identifies that this rule applies to the (container) queue on **eth0** with ID 1, which, in this example, is the **mbc**-queue. **handle 100** is the ID for this rule which is needed later if you want to delete this rule. **proto 17** (in this example) is the actual rule. This rule matches if the protocol of the packet is 17 which is UDP.

flowid 1001 is a pointer to the child queue of the container queue the packet should be queued on if this rule matches. **1001** is the FIFO queue attached to the second band. The flow ID must refer to a queue directly attached to the container-queue the rule applies to.

In practice, rules are only applicable to **mbc** queues.

WNDRVR

Creating a Meter/Marker Entity

To create a meter/marker entity, you can call a function. To invoke this operation, use the following call:

```
ioctl (sock_fd, SIOCXDSCREATE, &entity);
```

In this call, **entity** is an object of the **ds** superclass (see Differentiated Services Classes on page 77). The network stack will fill in the **id** field of this object. Use this identifying number to identify this entity in future calls.

See Creating New Meter/Marker Entity Varieties on page 80, for descriptions of the meter/marker entities that are part of the Wind River Network Stack and for instructions on how to add new entities.

Deleting a Meter/Marker Entity

To delete a meter/marker entity, you can call a function. To invoke this operation, use the following call:

```
ioctl (sock_fd, SIOCXDSDESTROY, &entity);
```

In this call, **entity** is an object of the **ds** superclass (see Differentiated Services Classes on page 77), but you need only fill in the **id** member of this object in order to sufficiently identify the entity you want to delete.

Mapping a Filter to a Meter/Marker Entity

To map a filter (or DS codepoint if you are operating in multifield mode) to a meter/marker entity, you can call a function from within a program. To invoke this operation, use the following call:

```
ioctl (sock_fd, SIOCXADSMAP, &mapping);
```

In this call, **mapping** is an object of the **ds_map** class that describes the mapping you are establishing (see Differentiated Services Classes on page 77).

Removing a Filter-to-Meter/Marker Entity Mapping

To remove a mapping between a filter (or DS codepoint if you are operating in multifield mode) and a meter/marker entity, you can call a function from within a program. To invoke this operation, use the following call:

```
ioctl (sock_fd, SIOXDDSMAP, &mapping);
```

In this call, **mapping** is an object of the **ds_map** class that describes the mapping that you are removing (see Differentiated Services Classes on page 77).

Previous topic: Including Differentiated Services in a Build on page 73
Next topic: Differentiated Services Classes on page 77

5. Differentiated Services Classes

Description of the classes of objects associated with Differentiated Services.

The following describe classes of objects associated with DiffServ:

WNDRVR

- **ds_filter** - see ds_filter Class on page 78
- **ds** - see ds Class on page 79
- **ds_map** - see Mapping from a Filter Rule to a Meter Marker Entity on page 79

ds_filter Class

A DiffServ filter rule is instantiated as an object of the **ds_filter** class, which contains and int, read-only, ID for this filter and a **classifier_rule** structure.

The members of the **classifier_rule** class are defined as follows:

mask

A mask that indicates which fields must match in order to trigger the filter rule. Construct this mask by AND'ing together one or more of the **CLS_RULE**_*x* constants:

- **CLS_RULE_DS** - DS field
- **CLS_RULE_PROTO** - protocol field
- **CLS_RULE_SADDR** - source address
- **CLS_RULE_DADDR** - destination address
- **CLS_RULE_SPORT** - source port
- **CLS_RULE_DPORT** - destination port

ds

The value that a packet must have in its DS field in order to trigger the filter rule (assuming the **CLS_RULE_DS** flag is set in **mask**). The DS field is the traffic class field for IPv6 and the TOS field for IPv4.

proto

The value that a packet must have in its IP header's protocol field in order to trigger the filter rule (assuming the **CLS_RULE_PROTO** flag is set in **mask**).

sport_low

The lowest source port a UDP or TCP packet can come from and still trigger the filter rule (assuming the **CLS_RULE_SPORT** flag is set in **mask**).

sport_high

The highest source port a UDP or TCP packet can come from and still trigger the filter rule (assuming the **CLS_RULE_SPORT** flag is set in **mask**).

dport_low

The lowest destination port a UDP or TCP packet can be destined for and still trigger the filter rule (assuming the **CLS_RULE_DPORT** flag is set in **mask**).

dport_high

The highest destination port a UDP or TCP packet can be destined for and still trigger the filter rule (assuming the **CLS_RULE_DPORT** flag is set in **mask**).

af

The address family the packet must belong to in order to trigger the filter rule, either **AF_INET** or **AF_INET6**.

saddr_prefixlen

The prefix length (mask) that the filter rule uses when it checks whether the source address matches (assuming the **CLS_RULE_SADDR** flag is set in **mask**).

daddr_prefixlen

The prefix length (mask) that the filter rule uses when it checks whether the destination address matches (assuming the **CLS_RULE_DADDR** flag is set in **mask**).

saddr

The source address (or network) that packets must match in order to trigger the filter rule (assuming the **CLS_RULE_SADDR** flag is set in **mask**).

daddr

The destination address (or network) that packets must match in order to trigger the filter rule (assuming the **CLS_RULE_DADDR** flag is set in **mask**).

ds Class

Meter/marker entities are objects of a variety of the **ds** class. The members of this class are defined as follows:

id

The ID of this meter/marker entity. The network stack sets this during the **create** operation.

name

The name of the meter/marker entity. The names of the two entity varieties that come with the Wind River Network Stack are:

"srTCM" - single-rate, three-color marker

"SimpleMarker" - simple marker

d

An object of the specific entity class. All meter/marker entities created by Wind River have an data type that starts with **ds_**, for instance **ds_sm**. The **ds_data** pseudoclass is a union of all of these classes.

Mapping from a Filter Rule to a Meter Marker Entity

When an edge router in multifield mode creates a new classifier rule it gets an ID number in return. When it creates a meter/marker entity, it also gets an ID number in return. The router can search a database that maps between these two varieties of ID value, so that when a packet matches a rule with a particular rule ID value the router can determine the corresponding entity ID value. The database of mappings is a set of **ds_map** objects.

The members of this class are as follows:

filter_id

The ID of the filter rule.

ds_id

The ID of the meter/marker entity that applies to packets that match the filter rule.

Previous topic: Using Differentiated Services on page 74
Next topic: Creating New Meter/Marker Entity Varieties on page 80

6. Creating New Meter/Marker Entity Varieties

To create a new meter/marker entity variety (other than the simple marker and single-rate three-color marker, which already exist), use this procedure.

About This Task

Procedure

1. Implement all of the functions pointed to by function pointers in the **Ipnet_diffserv_handlers** structure.
2. Define and register a factory function for meter/marker entities.
3. Implement the Function Pointers in the Ipnet_diffserv_handlers Structure

   A meter/marker entity must implement all of the functions pointed to by function pointers in the **Ipnet_diffserv_handlers** structure, which is defined in *installDir* **/vxworks-7/pkgs_v2/net/ipnet/qos/h/ipnet_diffserv.h**.

   The members of this structure are as follows:

   meter_input

   You can set this pointer to **IP_NULL** if this entity will do no metering on the flow passing through it. You should otherwise set it to point to a function that measures some property of the flow and keeps track of the result in some private data area. The prototype of this function is as follows:

   ```
   void myMeterInput (Ipnet_diffserv_handlers * handlers, Ipcom_pkt * packet)
   ```

   marker_input

   You can set this pointer to **IP_NULL** if this entity will do no (re)marking of packets. Otherwise set this to point to a function that marks the packet based on the configuration of the meter/marker or the property measured by the meter function. Your function may write directly into the DS field of the **Ipcom_pkt** that it receives as the **pkt** argument. **pkt->ipstart** is the offset into the **pkt->data** area at which the IP header is stored. The prototype of this function is as follows:

   ```
   void myMarkerInput (Ipnet_diffserv_handlers * handlers, Ipcom_pkt * packet)
   ```

   destroy

   Set this pointer to point to a function that frees all resources held by the meter/marker. Do not set this pointer to **IP_NULL**. The prototype of this function is as follows:

   ```
   void myDestroy (Ipnet_diffserv_handlers * handlers);
   ```

4. Define and Register a Factory Function for Meter/Marker Entities

   Define a factory function for the new meter/marker entity and register it with the network stack. You must register the factory function as an **Ipnet_diffserv_handlers_template**.

   ```
   typedef struct Ipnet_diffserv_handlers_template_struct
             {
             const char * name;
             Ipnet_diffserv_ctor
             } Ipnet_diffserv_handlers_template;
   ```

   The fields in this structure are defined as follows:

name

The name of the meter/marker entity. The network stack associates this name with the **create** function below, so that the stack calls this function when meter/marker entity structures (**ds** objects) with this name are passed to the **SIOCXDSCREATE** ioctl operation (see Using Differentiated Services on page 74).

create

The **create** function the network stack calls for meter/marker entities whose **name** members match **name**, above. The stack passes data (the **d** member of the entity's **ds** object) as the first argument to this function. This function has the following prototype:

```
int myCreate (void * arg, Ipnet_diffserv_handlers ** phandler);
```

For the network stack to be able to use the new meter/marker entity type, you must register the factory function by passing it in to ipnet_diffserv_register_ctor( ) from the ipnet_diffserv_init( ) function, which is defined in:

installDir **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_diffserv.c**

For example, the single-rate three-color marker declares a function called ipnet_diffserv_srtcm_template( ), which returns a static variable of type **Ipnet_diffserv_handlers_template** that it initializes with a name and a pointer to a constructor function, so its factory registration call looks like this:

```
ipnet_diffserv_register_ctor (ipnet_diffserv_srtcm_template ())
```

Previous topic: Differentiated Services Classes on page 77
Next topic: Using Existing Meter/Marker Entity Varieties on page 81

7. Using Existing Meter/Marker Entity Varieties

The simple marker and marker and single-rate, three-color marker are already implemented and part of the VxWorks 7 network stack.

- a simple marker
- a single-rate, three-color marker

SimpleMarker

The simple marker, **SimpleMarker**, is defined in:

installDir **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_ds_sm.c**.

The simple marker copies a specific DS value into each IP header DS field on all packets that match a filter and can also set the drop precedence on every packet that matches.

The members of the **ds_sm** class are defined as follows:

mask

This member determines if the marker should set the DS field and/or set the drop precedence. Construct the value of the mask by **AND**ing the following two **DS_SM**_*x* constants:

- **DS_SM_DS_VAL** - the DS value
- **DS_SM_DROP_P** - the drop precedence

ds_value

The DS value that the marker sets on each packet that matches (if **mask** has the **DS_SM_DS_VAL** bit set).

drop_precedence

The drop precedence (one of the **IPCOM_PKT_DROP_PRECEDENCE**_*x* constants) that the marker sets on each packet that matches (if **mask** has the **DS_SM_DROP_P** bit set).

Single-Rate Three-Color Marker

The single-rate, three-color marker, **srTCM**, is defined in:

installDir **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_ds_srtcm.c**

The single-rate, three-color marker meters the byte rate of a flow and marks packets as green, yellow, or red. The shaper prioritizes green packets over yellow packets and yellow packet over red packets. See RFC 2697 for a complete description of the **srTCM** meter/marker.

The members of the **ds_srtcm** class are as follows:

mode

The mode in which the **sr TCM** marker is operating. This can be either **DS_SRTCM_MODE_COLOR_BLIND** or **DS_SRTCM_MODE_COLOR_AWARE**.

CIR

The Committed Information Rate (bytes/second): the maximum, long term, data rate the flow can have and still have the marker mark it as green.

CBS

The Committed Burst Rate (bytes): the maximum size of the token bucket for green packets.

EBS

The Excess Burst Rate (bytes): the maximum size of the token bucket for yellow packets.

ds_green

The DS value that the marker gives to green packets.

ds_yellow

The DS value that the marker gives to yellow packets.

ds_red

The DS value that the marker gives to red packets.

Note: Set at least one of **CBS** or **EBS** to be greater than zero and at least as big as the largest possible packet in the flow.

Previous topic: Creating New Meter/Marker Entity Varieties on page 80
Next topic: Network Interface Output Queues on page 83

WNDRVR

8. Network Interface Output Queues

You can attach an interface output queue to every network interface in the network stack.

All packets that the network stack sends through such an interface pass through a queue, and the queue can meter and enforce a maximum throughput on the packet flow.

There are two varieties of interface output queues in the network stack:

- **Leaf queues**, in which the packets are stored. These cannot have child queues. (See Leaf Queues on page 87.)
- **Container queues**, which have one or more child queues (which can be either leaf queues or container queues) and a set of rules that determine which child queue to queue each packet in (see Container Queues on page 90).

The API that you use to create, control, and delete interface output queues is defined in *installDir* **/vxworks-7/pkgs_v2/net/ipnet/qos/h/ipnet_qos.h**.

ifqueue_qos class

To establish a queue, set the members of an object of the **ifqueue_qos** class and then attach this queue object to an interface using one of the techniques described in Adding an Interface Output Queue on page 84. The **ifqueue_qos** class.

The members of the **ifqueue_qos** class are as follows:

ifq_name

The name of the network interface that this queue attaches to, for instance: **"eth0"**.

ifq_type

The type of queue, for instance:

"fifo"

A first-in/first-out queue with a queue limit.

"dpaf" - see: Drop Precedence-Aware FIFO on page 88

A first-in/first-out queue with three drop precedence levels (low, medium, and high) and with a queue limit; packets marked "high" are dropped before those marked "medium" or "low".

"null"

A queue in which all packets are dropped; some DiffServ shapers need queues of this sort.

"none"

Indicates that the interface does not have an interface output queue.

If your system automatically attaches queues to all interfaces, you can effectively remove a queue from a particular interface (for instance, a pseudo-interface that you do not want to filter) by setting its queue type to **none**.

"netemu"

Can add latency, and can reorder, drop, and corrupt packets; you can use this to test various network conditions.

"mbc"

Holds an array of queues, arranged in order of priority (the lower the index in the array, the higher the priority); packets dequeue from the container in order of priority.

WNDRVR

"htbc"

Holds a set of queues that are not prioritized relative to each other; packets dequeue from these queues in a round-robin fashion.

ifq_id

The identifying number of the queue. In **GET** operations, if you set this to **IFQ_ID_NONE**, the operation returns the root queue, otherwise it returns the queue with this ID.

In **SET** operations, you can set this to a specific ID if you want to operate on a specific queue, or you can set this to **IFQ_ID_NONE** if you want the stack to select a unique ID. If the specified queue ID already exists, a **SET** operation replaces the queue. If you replace a container queue, this removes all of its children.

All queues attached to a particular interface have unique identifying numbers.

ifq_parent_id

The ID of the parent of this queue if this is a child queue, or **IFQ_ID_NONE**, if this is the root queue.

ifq_count

(Read-only.) The number of packets in this queue, if it is a leaf queue, or the sum of packets in all of its child queues, if it is a container queue.

ifq_data

An object that defines the characteristics of the particular type of queue. This may be an object of a queue class of your own invention, or one of the following:

- **ifqueue_fifo**
- **ifqueue_dpaf**
- **ifqueue_netemu**
- **ifqueue_mbc**
- **ifqueue_htbc**

Previous topic: Using Existing Meter/Marker Entity Varieties on page 81
Next topic: Adding an Interface Output Queue on page 84

9. Adding an Interface Output Queue

To add an interface output queue to a network interface, or to replace one that was previously added, you can either call a function from within a program or use a **qc** command interactively.

To invoke this operation from within a program, use the following call:

```
ioctl (sock_fd, SIOCSIFQUEUE, &queue);
```

Where **queue** is a **ifqueue_qos** object that describes the leaf or container queue you are adding.

If the **ifq_id** member of the **ifqueue_qos** object matches that of a queue that is already attached to the interface, this operation will replace that queue with the one described by **queue**.

You can also use the following **qc** command:

```
[vxWorks *] # qc [-V virtualRouter] queue add [parameters]
```

For instance, if you want to test how your system would work under conditions when there was 100 millisecond latency on interface **lo0**, you could create a network emulator queue on that device that simulates such latency, with the following **qc** command:

```
[vxWorks *] # qc queue add dev lo0 netemu limit 10 min_latency 100 max_latency 100
```

If you want to rate-limit all traffic on interface **eth0** to two Mbits per second, you could do this by establishing a rate-limited HTBC queue with the following command:

```
[vxWorks *] # qc queue add dev eth0 htbc rate 2000kbps
```

Previous topic: Network Interface Output Queues on page 83
Next topic: Getting an Object that Describes an Interface Output Queue on page 85

10. Getting an Object that Describes an Interface Output Queue

To get an object that describes an interface output queue on a network interface, you can either call a function from within a program or use a qc command interactively.

To invoke this operation from within a program, use the following call:

```
ioctl (sock_fd, SIOCGIFQUEUE, &queue);
```

Where **queue** is a **ifqueue_qos** object.

If you call this with the **ifq_id** field of **queue** set to **IFQ_ID_NONE**, this function will fill **queue** so that it describes the root queue on this interface, otherwise it will fill **queue** so that it describes the queue matching **ifq_id**.

You can also use a **qc** command, which has the following format:

```
[vxWorks *] # qc [-V virtualRouter] queue show [parameters]
```

Previous topic: Adding an Interface Output Queue on page 84
Next topic: Adding a Filter Rule to a Container Queue on page 85

11. Adding a Filter Rule to a Container Queue

To add a filter rule to a container queue on a network interface, you can either call a function from within a program or use a qc command interactively.

To invoke this operation from within a program, use the following call:

```
ioctl (sock_fd, SIOCXAIFQFILTER, &filter);
```

In this call **filter** is an **ifqueue_filter** object that describes the filter you are adding.

You can also use a qc command, which has the following format:

```
[vxWorks *] # qc filter add dev device parent queueID handle filterID [filterArgs]
flowid queueID
```

The arguments to this command are as follows:

dev *device*

The device to which you are attaching the filter, for instance **eth0**.

parent *queueID*

The identifying number of the container queue to which you are adding the filter.

handle *filterID* [*filterArgs*]

The *filterID* is the identifying number of the filter. You may use the following arguments in the *filterArgs* argument to describe the filter:

- proto *number*
- tclass *number*
- srcport *range*
- dstport *range*
- srcaddr *address*[/*prefix*]
- dstaddr *address*[/*prefix*]
- flowid *queueID*

The identifying number of the destination queue for packets that match the filter.

For example:

To add a filter identified by the number 5 to the container queue identified by the number 1, so that all TCP packets (protocol number 6) are filtered into the queue identified by the number 31, use the following qc command:

```
[vxWorks *] # qc filter add dev eth0 parent 1 handle 5 proto 6 flowid 31
```

To add a second filter (identified by the number three) to the same container queue that filters all UDP packets (protocol number 17) that are sent to 2001::/16 into the same queue, use the following qc command:

```
[vxWorks *] # qc filter add dev eth0 parent 1 handle 3 proto 17 srcaddr 2001::/16
flowid 31
```

Previous topic: Getting an Object that Describes an Interface Output Queue on page 85
Next topic: Deleting a Filter Rule from a Container Queue on page 86

12. Deleting a Filter Rule from a Container Queue

To delete a filter rule from a container queue on a network interface, you can either call a function from within a program or use a **qc** command interactively.

To invoke this operation from within a program, use the following call:

```
ioctl (sock_fd, SIOCXDIFQFILTER, filter);
```

In this call **filter** is an **ifqueue_filter** object that describes the filter you are removing (see Container Queues on page 90).

You can also use a qc command, which has the following format:

```
[vxWorks *] # qc [-V virtualRouter] filter del [parameters]
```

Previous topic: Adding a Filter Rule to a Container Queue on page 85
Next topic: Leaf Queues on page 87

13. Leaf Queues

Leaf queues cannot have children and you cannot assign filter rules to them.

The Wind River Network Stack includes the following kinds of leaf queues:

- None
- FIFO
- Drop Precedence-Aware FIFO
- Network Emulator

None

If you specify **none** as the queue type, the interface does not have an interface output queue. If your system automatically attaches queues to all interfaces, you can effectively remove a queue from a particular interface by setting its queue type to **none**.

- Queue name: **none**

FIFO

- Queue name: **fifo**
- File: *installDir* **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ ipnet_pkt_queue_fifo.c**
- **qc** command:

```
[vxWorks *] # qc queue add dev device fifo limit number
```

This is the default queue for all interfaces in the network stack.

You can use FIFOs as buffers to handle temporary peaks in traffic. You can also use them as leaf queues in more complex queue hierarchies.

Packets dequeue from a FIFO queue, in the same order as they arrived on the queue, without regard to the packets' individual properties.

The cost of queuing or dequeuing a packet on a FIFO queue is always O(1).

Creating a FIFO Queue Object on page 88 Creating a FIFO queue involves creating objects of classes ifqueue_qos( ) and ifqueue_fifo( ).
Drop Precedence-Aware FIFO on page 88 Drop Precedence-Aware FIFO queues (DPAFs) work as normal FIFOs until the maximum number of packets in the queue is exceeded.
Creating a Drop Precedence-Aware FIFO Queue Object on page 88 Creating the FIFO queue involves creating objects of the classes **ifqueue_qos** and **ifqueue_dpaf** .

Network Emulator on page 89 The network emulator is a testing and debugging tool. You can use it to introduce jitter and latency into a stream, which may result in packet reordering, or to drop and corrupt packets.

Previous topic: Deleting a Filter Rule from a Container Queue on page 86
Next topic: Container Queues on page 90

13.1. Creating a FIFO Queue Object

Creating a FIFO queue involves creating objects of classes ifqueue_qos( ) and ifqueue_fifo( ).

About This Task

Complete the following steps to create the object:

Procedure

1. Create an object of class ifqueue_qos( ) and fill its members appropriately.
2. Set the **ifq_type** member of that object to **"fifo"**.
3. Create an object of class ifqueue_fifo( ) and set its **fifo_limit** member to the maximum number of packets that can be stored in the queue.
4. Set the **ifq_data** member of the **ifqueue_qos** object to the **ifqueue_fifo** object.

Parent topic: Leaf Queues on page 87

13.2. Drop Precedence-Aware FIFO

Drop Precedence-Aware FIFO queues (DPAFs) work as normal FIFOs until the maximum number of packets in the queue is exceeded.

- Queue name: **dpaf**
- File: *installDir* **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_pkt_queue_dpaf.c**
- qc command:

```
[vxWorks *] # qc queue add dev device  dpaf limit  number
```

If the limit is exceeded, a DPAF checks the drop precedence of the new packet. The following outcomes are possible:

- Reject this packet if is has high drop precedence .
- Drop a high-drop-precedence packet from the queue and replace it with the new packet if this packet has medium or low drop precedence.
- Drop a medium-drop-precedence packet from the queue and replace it with the new packet if this packet has low drop precedence.
- Reject this packet if there are no packets in the queue with higher drop precedence that the DPAF can drop.

The extra overhead compared to FIFO makes this queue slower. The cost of queuing/dequeuing a packet is O(log(n)), where n is the number of packets in the queue.

Parent topic: Leaf Queues on page 87

13.3. Creating a Drop Precedence-Aware FIFO Queue Object

Creating the FIFO queue involves creating objects of the classes **ifqueue_qos** and **ifqueue_dpaf** .

WNDRVR

About This Task

Complete the following steps to create the FIFO object.

Procedure

1. Create an object of class **ifqueue_qos** and fill its members appropriately.
2. Set the **ifq_type** member of that object to **"dpaf"**.
3. Create an object of class **ifqueue_dpaf** and set its **dpaf_limit** member to the maximum number of packets that can be stored in the queue.
4. Set the **ifq_data** member of the **ifqueue_qos** object to the **ifqueue_dpaf** object.

Parent topic: Leaf Queues on page 87

13.4. Network Emulator

The network emulator is a testing and debugging tool. You can use it to introduce jitter and latency into a stream, which may result in packet reordering, or to drop and corrupt packets.

- Queue name: **netemu**
- File: *installDir* /**vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_pkt_queue_netemu.c**
- qc command:

```
[vxWorks *] # qc queue add dev device  netemu limit number [min_latency  msec] \
 [max_latency milliseconds] \
 [drop probability [random | pattern {0|1},{0|1}[,...,{0|1}]]] \
 [corrupt  probability [random]]
```

A **netemu** queue with zero latency, zero drops, and zero corruption is equivalent to a FIFO queue.

Creating a Network Emulator Queue Object on page 89 Creating the network emulator queue involves creating objects of the classes **ifqueue_qos** and **ifqueue_netemu**.

Parent topic: Leaf Queues on page 87

Creating a Network Emulator Queue Object

Creating the network emulator queue involves creating objects of the classes **ifqueue_qos** and **ifqueue_netemu**.

About This Task

Complete the following steps to create the network emulator object.

Procedure

1. Create an object of class **ifqueue_qos** and fill its members appropriately.
2. Set the **ifq_type** member of that object to **"netemu"**.
3. Create an object of class **ifqueue_netemu** and set its members appropriately (see below).
4. Set the **ifq_data** member of the **ifqueue_qos** object to the **ifqueue_netemu** object.

WNDRVR

Results

The members of the **ifqueue_netemu** class are as follows:

netemu_limit

The maximum number of packets that can be stored in this queue.

netemu_min_latency

netemu_max_latency

The minimum and maximum latency that the emulator adds to each packet, in milliseconds. The emulator will evenly distribute the latency on individual packets between [**netemu_min_latency**..**netemu_max_latency**].

netemu_random_drop

netemu_drop_probability

netemu_drop_pattern

netemu_drop_pattern_len

Set **netemu_random_drop** to **TRUE** to drop individual packets with the probability of 1/**netemu_drop_probability**. Set **netemu_random_drop** to **FALSE** to drop a packet every **netemu_drop_probability** packets. For example, if you set **netemu_drop_probability** to 4, setting **netemu_random_drop** to **FALSE** means the emulator drops every fourth packet, while setting it to **TRUE** means that for each packet there is a one-in-four probability that the emulator will drop it.

You can also set **netemu_drop_pattern** to a bitmask that represents a regular pattern of **netemu_drop_pattern_len** bits, with bits marked "1" representing dropped packets, so that, for instance, the pattern 00000011 (**netemu_drop_pattern** = 0x00000003, **netemu_drop_pattern_len** = 8)will drop the last two of every eight packets.

netemu_random_corrupt

netemu_corrupt_probability

Set **netemu_random_corrupt** to **TRUE** to corrupt individual packets with the probability of 1/**netemu_corrupt_probability**. Set **netemu_random_corrupt** to **FALSE** to corrupt a packet every **netemu_corrupt_probability** packets. For example, if you set **netemu_corrupt_probability** to 4, setting **netemu_random_corrupt** to **FALSE** means the emulator corrupts every fourth packet, while setting it to **TRUE** means that for each packet there is a one-in-four probability that the emulator will corrupt it.

Parent topic: Network Emulator on page 89

14. Container Queues

A container queue contains one or more child queues (child queues may be container queues or leaf queues, or a combination of both). Filter rules determine which child queue the container queue stores a particular packet in.

Container Superclass

All container classes are subclasses of the **ifqueue_container** class (which is to say that the first member of the structure that defines a container class is an **ifqueue_container** structure). The members of this class are as follows:

child_count

the number of child queues this container has (a child queue that is a container queue only counts as a single child even if it in turn has multiple children)

WNDRVR

child_ids

an array that contains the queue IDs of each of the child queues

Filter Rules

You may attach filter rules to container queues (see Adding a Filter Rule to a Container Queue on page 85). These rules control in which child queue a container queue should place packets. If a packet does not match any rule, the container queue queues the packet on its default child queue.

The **ifqueue_filter** class describes a filter rule. The members of this class are defined as follows:

filter_ifname

The name of the network interface this filter operates on.

filter_id

The ID of this rule. The stack sets this field when the filter is added; if you are deleting a filter, set this field to indicate which filter you want to delete.

filter_queue_id

The ID of the (container) queue this filter applies to.

filter_child_queue_id

The ID of the child queue into which packets matching this rule will be queued by the container queue.

filter_rule

The actual rule, in the form of an **classifier_rule** object (see About Differentiated Services on page 73).

Available Container Queues

The Wind River Network Stack includes the following container queues:

- Multiband Container (MBC)
  - Queue name: **MBC**
  - File: *installDir* **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_pkt_queue_mbc.c**
  - qc command:

```
[vxWorks *] # qc queue add dev device mbc bands number [default_band number]
```

An MBC container queue keeps an array of child queues (bands) in decreasing priority. An MBC queue always dequeues packets from the first non-empty queue in its array of queues.

You can use this variety of container queue when certain kinds of traffic, such as signalling, must transmit as quickly as possible, while other traffic, like e-mail, can wait for low-traffic conditions.

When you create an MBC queue, it initially has an array of FIFO child queues. You may replace these with queues of another variety by issuing an **SIOCSIFQUEUE** ioctl call (see Adding an Interface Output Queue on page 84).

Define an MBC queue by setting the **ifq_type** member of the **ifqueue_qos** structure to **"mbc"** and the **ifq_data** member of the **ifqueue_qos** structure to an object of the **ifqueue_mbc** class. The members of this class are defined as follows:

mbc_container

The base class for container queues.

WNDRVR

mbc_bands

The number of bands (child queues) that this container queue manages. You may only set this member prior to the time you create the MBC queue.

mbc_default_band

Which of the bands (child queues) the MBC container queue puts a packet into if that packet does not match any of the filter rules. This is an index value in the range [0..**mbc_bands**).

- Hierarchy Token Bucket Container (HTBC)
  - Queue name: **HTBC**
  - File: *installDir* **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_pkt_queue_htbc.c**
  - qc command:

```
[vxWorks *] # qc queue add dev device  htbc rate rate [burst number]
```

Use the HTBC queue when you want to control the bandwidth usage on an interface. The queue calculates the data rate by taking the sum of bandwidth used by all children; it calculates this when it dequeues packets. It considers all children to have the same priority, so it dequeues packets from the child queues in a round-robin fashion.

You can add or delete child queues from an HTBC container queue throughout the lifetime of the HTBC. When you create an HTBC, it comes with a single child FIFO queue, the default queue, attached to it. You can add more queues, or replace this default queue with one of another variety, by using the **SIOCSIFQUEUE** ioctl call (see Adding an Interface Output Queue on page 84).

Define an HTBC queue by setting the **ifq_type** member of the **ifqueue_qos** structure to **"htbc"** and the **ifq_data** member of the **ifqueue_qos** structure **ifqueue_htbc** class. The members of this class are defined as follows:

htbc_container

The base class for containers.

htbc_byterate

The maximum bandwidth, in bytes per second, at which this queue may send.

htbc_token_limit

The maximum number of tokens this queue can have. For each byte that an HTBC container queue dequeues from one of its children and sends, it consumes one token. You must set this value to be greater than the MTU (maximum transmission unit) of the interface you attach the queue to. A reasonable value might be in the range **htbc_byterate**/100 to **htbc_byterate**. A larger value results in HTBC dropping fewer packets during temporary bursts in the flow, so the actual value depends on the acceptable level of "burstiness."

htbc_default_id

The ID number of the default queue, into which HTBC places all packets that do not match any filter rule.

Previous topic: Leaf Queues on page 87
Next topic: Adding a New Leaf Queue Type on page 92

15. Adding a New Leaf Queue Type

Create a new leaf queue variety by adding its structure to the **ifqueue_data** union, implementing all functions in the **Ipnet_pkt_queue** structure, and registering an instance of that type with the ipnet_pkt_queue_register( ) function.

The Wind River Network Stack registers its queues, like the FIFO and Hierarchy Token Bucket Container (HTBC) queues, in the ipnet_pkt_queue_init( ) function, which is located in *installDir* **/vxworks-7/pkgs_v2/net/ipnet/qos/src/ipnet_pkt_queue.c**. You may want to register any new queues you add at the same time, which you can do by changing this function to make additional registration calls.

The members of this structure are defined as follows:

type

The name of the queue type.

impl_size

The size of the structure used by this queue, which may be an **Ipnet_pkt_queue** structure or a subclass structure that derives from it.

c_ops

If this queue is a container queue, it must implement this interface. Write these functions to return 0 (zero) on success, or an **IPNET_ERRNO**_*x* error code on failure (except for **q_get**, which returns a queue structure, or **IP_NULL** if it finds no queue that matches the ID).

q_get- retrieves a queue by ID number

```
Ipnet_pkt_queue_struct * my_q_get
 (Ipnet_pkt_queue_struct * containerQueue, int queueID)
```

q_insert- adds a queue to the container

```
int my_q_insert (Ipnet_pkt_queue_struct * containerQueue,
 Ipnet_pkt_queue_struct * addThisQueue);
```

q_remove - removes a queue from the container

```
int my_q_remove (Ipnet_pkt_queue_struct * containerQueue,
 Ipnet_pkt_queue_struct * removeThisQueue);
```

f_insert - adds a filter to a queue

```
int my_f_insert (Ipnet_pkt_queue_struct * containerQueue,
 int filterID, classifier_rule * rule, int childQueueID);
```

f_remove - removes a filter from a queue

```
int my_f_remove (Ipnet_pkt_queue_struct * containerQueue,
 int filterID)
```

enqueue or **enqueue_locked**

Enqueues a packet on this queue. This function has the following prototype:

```
int myEnqueue (struct Ipnet_pkt_queue_struct * queue, Ipcom_pkt * packet)
```

Write this function to return 0 (zero) on success, or an **IPNET_ERRNO**_*x* error code on failure.

dequeue or **dequeue_locked**

Dequeues a packet from this queue. This function has the following prototype:

WNDRVR

```
    Ipcom_pkt * myDequeue (Ipnet_pkt_queue_struct * queue)
```

Write this function to return the next packet (according to the rules of the queue), or **IP_NULL** if the queue is empty.

requeue or **requeue_locked**

Puts a packet back on the queue that was removed from the queue with the **dequeue** function. This function has the following prototype:

```
    void myRequeue (Ipnet_pkt_queue_struct * queue,
Ipcom_pkt * packet);
```

count or **count_locked**

Returns the number of packets in this queue, or the sum of packets in all child queues if this is a container queue. This function has the following prototype:

```
    int myCount (Ipnet_pkt_queue_struct * queue);
```

reset

Removes all packets from the queue and resets the internal state of the queue. This function has the following prototype:

```
    void myReset (Ipnet_pkt_queue_struct * queue);
```

dump

Fills an **ifqueue**_*x* structure with the current configuration of this queue. This function has the following prototype:

```
void myDump (Ipnet_pkt_queue_struct * queue, union ifqueue_data * data);
```

Write this function so that it fills the **data** structure with the configuration information specific to this queue.

configure

Configures the queue based on the **ifqueue**_*x* structure for this queue, found in the **data** parameter. This function has the following prototype:

```
int myConfigure (Ipnet_pkt_queue_struct * queue,
        union ifqueue_data * data);
```

Write this function so that it returns the number of elements that may be placed in the queue.

init

The function that initializes the queue after memory has been allocated for it. This function has the following prototype:

```
    int myInit (Ipnet_pkt_queue_struct * queue);
```

Write this functions to return 0 (zero) on success, or an **IPNET_ERRNO**_*x* error code on failure.

destroy

The function that frees all resources allocated by this queue. This function has the following prototype:

```
    void myDestroy (Ipnet_pkt_queue_struct * queue);
```

Note: Do not set the remaining fields of this structure, such as **id**, **parent_id**, **netif**, **prev**, and **next**.

Previous topic: Container Queues on page 90
Next topic: Example Reserving Bandwidth for an Application on page 95

WNDRVR

16. Example Reserving Bandwidth for an Application

This example demonstrates how to use an HTBC interface queue to reserve bandwidth for an application. To specify the application, we identify its specific UDP or TCP port--in this example, port 5001.

About This Task

Procedure

1. Create an MBC.

   Using the **qc** command, create an MBC with two bands--one for traffic with unlimited bandwidth and one for the application for which you want to reserve bandwidth. Use the first band as the default (used if there is no matching rule).

   1. Issue the following command to create the MBC:

   ```
   [vxWorks *] # qc queue add dev eth0 mbc bands 2 default_band 0
   ```

   2. Issue a **qc show** command to verify that the MBC has been created:

   ```
   [vxWorks *] # qc queue show dev eth0
         mbc/1[0] queue at eth0
           bands: 2, default_band: 0
          fifo/1000[1] queue at eth0
                   limit: 16
          fifo/1001[1] queue at eth0
                   limit: 16
   ```

2. Attach an HTBC to the second band of the MBC.

   The child queue in that band has ID 1001. Replace it with one with an HTBC with a limit of 1 Mbit per sec.

   1. Issue the following command to create the child queue:

   ```
   [vxWorks *] # qc queue add dev eth0 parent 1 handle 1001 htbc rate 1Mbit burst 100kb
   ```

   The specified **burst** parameter must be low enough to improve the accuracy of the rate limiter, yet high enough to create a bucket container of adequate size for the specified rate, which is set in relation to the clock speed of the operating system.

   2. Issue another **qc show** command to verify the creation of this queue.

   ```
   [vxWorks *] # qc queue show dev eth0
               mbc/1[0] queue at eth0
                    bands: 2, default_band: 0
                fifo/1000[1] queue at eth0
                    limit: 16
                   htbc/1001[1] queue at eth0
                     rate: 125kbps, burst: 100k, default id: 1002
                           fifo/1002[1001] queue at eth0
                            limit: 16
   ```

   Note: The speed is always shown as kilobytes per second, so 1 Mbit/sec is shown as 125 KB/sec.

3. Create a filter rule for the application.

WNDRVR

Create a filter rule that puts all traffic for the application into the band with the HTBC connected to it and all other traffic into the other band.

1. Issue the following command to create the rule:

```
[vxWorks *] # qc filter add dev eth0 parent 1 dstport 5001 flowid 1001
```

This command adds a filter rule to a queue that is attached to device **eth0**.

The network stack will apply this rule when something is queued at the MBC queue (parent 1, which is the ID of the MBC). If the rule matches, it will use the child queue with ID 1001 (flowid 1001). The rule specifies a destination port of 5001. (Because no protocol is defined, the rule applies to both UDP and TCP port 5001.)

2. Issue a ttcp command to verify the result. The ttcp command must use TCP port 5001 by default.

```
[vxWorks *] # ttcp -n 1000 192.168.130.4
    ttcp-t: fd=28, buflen=8192, nbuf=1000, align=16/0, port=5001,
sockbufsize=32767  TCP -> 192.168.130.4
     ttcp-t: socket
     ttcp-t: setsockopt(IP_SO_REUSEADDR)
     ttcp-t: bind IPv4 0
     ttcp-t: setsockopt(sndbuf)
     ttcp-t: connect
     ttcp-t: 8192000 bytes in 68581 milliseconds = 116 KB/sec, 119 B/msec
+++
     ttcp-t: 1000 I/O calls, msec/call = 68, calls/sec = 14
```

The throughput of 116 KB/sec is a bit slower than the configured value of 125 KB/sec, but that is expected because the limit includes all headers, while **ttcp** measures the throughput of the TCP payload.

4. Change the port to 5002, which is not bandwidth-limited:

```
[vxWorks *] # ttcp -n 1000 -p 5002 192.168.130.4
    ttcp-t: fd=29, buflen=8192, nbuf=1000, align=16/0, port=5002,
sockbufsize=32767 TCP  -> 192.168.130.4
    ttcp-t: socket
    ttcp-t: setsockopt(IP_SO_REUSEADDR)
    ttcp-t: bind IPv4 0
    ttcp-t: setsockopt(sndbuf)
    ttcp-t: connect
    ttcp-t: 8192000 bytes in 447 milliseconds = 17897 KB/sec, 18326 B/msec +++
    ttcp-t: 1000 I/O calls, msec/call = 0, calls/sec = 2237
```

The throughput changes to 18 MB/sec. The sample output shown above was prepared using a debug build of the network stack running in VxSim. The throughput would be even higher on a network stack prepared for deployment on an actual target.

Results

Specifying the burst Parameter

The **burst** parameter specifies the size of the "bucket" that defines the capacity of an HTBC. Packets received on a queue are added to this bucket, and packets transmitted on a queue are subtracted from this bucket, both at the rate specified by the **rate** parameter.

If incoming packets exceed the **burst** parameter, the excess packets are dropped. If outgoing packets are transmitted at a rate that drains the bucket, transmission stops until incoming packets have filled the bucket in an amount equal to the amount of outgoing

packets in the queue. The bucket must therefore be sized so that is neither completely drained nor completely filled by the traffic anticipated on the queue, given the specified **rate** parameter.

To set the **burst** parameter, you must take the tick rate of the VxWorks operating system into account, which is about 60 Hz. Note further that the network stack might begin transmitting packets on the queue just after a tick, so the bucket must be large enough to allow transmission for two full ticks.

You can calculate the value for this parameter as follows:

R/60 * 2 = B

where

R = rate

B = burst

For example, if you set the **rate** parameter to 1 Mbit, your calculation for the **burst** parameter is as follows:

1 * 10242/60 * 2 = 35 KB

You can set the **burst** parameter to a higher value to allow temporary bursts of traffic, but the sustained rate can never exceed the value specified by the **rate** parameter, regardless of the value of the **burst** parameter. If you specify too low a value for the **burst** parameter, the shaped bandwidth will be lower than the value configured by the **rate** parameter.

Previous topic: Adding a New Leaf Queue Type on page 92
Next topic: qc Command on page 97

17. qc Command

The qc command configures the network interface output queue(s).

Name

Synopsis

qc [**-V** *vr*] **queue** [**show** | **add** | **del**] **dev** *if* [**root** | **parent** *q-id*][**handle** *q-id*] *type* [*type_specific_opt*]
qc [**-V** *vr*] **filter** [**show** | **add** | **del**] **dev** *if* **parent** *q-id* **handle** *f-id* [*filter_args*] **flowid** *q-id*

Description

qc is used to control which queues are added to network interfaces and is used to control the filter rules on container queues.

-V *vr*

Apply the operation on the specified virtual router.

WNDRVR

queue

Create a queue, using the specified parameters.

filter

Create a filter, using the specified parameters.

show

Show the configuration for a queue or filter.

add

Add/replace a queue or filter.

del

Delete a queue or filter.

dev *if*

Specifies which network interface (or network device) the queue or filter applies to.

root

Specifies that this queue has no parent queue and is rooted directly at the network interface.

parent

Specifies which queue this queue is a child to. The parent queue must be one with a name ending with a c (mbc, htbc). The c stands for container and means that the queue is a container for one or more child queues.

handle

Specifies the ID (or handle) of this queue. It is only needed for replace operations.

Adding a Multiband Container (MBC) with two bands will automatically add two FIFO queues as children to the MBC. Those children will automatically be given unique IDs (or handles). Any of the children may be replaced by some other queue by using **handle** *id-of-child* with an add command. (There is no special replace operation; adding a queue with an ID that already exists means that the queue will be replaced.)

flowid *q-id*

Specifies the child queue to which a packet is queued when the packet meets the criteria specified by a filter.

A filter definition consist of a number of fields that must match a packet, like source address equal to *src_addr*, ToS equal to *type_of_service*, etc.

A queue container such as an MBC holds one or more children. Let's assume it has two children, with the IDs I1 and I2.

The flowid of a filter attached to the MBC specifies the children to which a matching packet is queued. So *q-id* would be I1 or I2 in this example.

Every queue container has an implicit filter that matches any packet. Use **default_band** *id* to specify the *id* value of that child when you create an MBC.

q-id

ID of an output queue. Must be unique per interface, unless it should replace an existing queue.

f-id

ID of a filter, must be unique per queue unless it should replace an existing filter.

type

Queue type, for example, FIFO, Hierarchy Token Bucket Container (HTBC).

range

Numeric range, format is **a**, **a-**, **-b** or **a-b**.

addr

IPv4 or IPv6 address. Note that source and destination address must be specified for the same domain per rule.

prefix

Number of bites used for the network identifier.

The filter parser accepts the following fields for *filter_args*:

- **proto** *number*
- **tclass** *number* (**tclass** can be substituted for **tos**
- **srcport** *range*
- **dstport** *range*
- **srcaddr** *addr*[**/** *prefix*]
- **dstaddr** *addr*[**/** *prefix*]

Examples:

Add an HTBC as root to eth0, max bandwidth is 2 Mbit with a burst of 100 KB:

```
qc queue add dev eth0 root handle 3 htbc rate 2Mbit burst 100kb
```

Add a FIFO that can queue up to 50 packets to the Hierarchy Token Bucket Container (HTBC) added in Example

```
qc queue add dev eth0 parent 3 handle 31 fifo limit 50
```

Add a filter with ID 5 to queue (1) so that all Transmission Control Protocol (TCP) packets end up in Example :

```
qc filter add dev eth0 parent 1 handle 5 proto 6 flowid 31
```

Add another filter to queue (1) so that all User Datagram Protocol (UDP) packets sent to 2001::/16 end up in Example :

```
qc filter add dev eth0 parent 1 handle 3 proto 17 srcaddr 2001::/16 flowid 31
```

Previous topic: Example Reserving Bandwidth for an Application on page 95
Next topic: qos Command on page 99

18. qos Command

Name

qos - controls the QoS features in the Wind River Network Stack

Synopsis

```
qos [option]...
op optype [args]
```

Description

The **qos** command is used to configure and view the Quality of Service features within the network stack, except interface queues, which use **qc**.

The following options exist:

-4

Use IPv4 as address domain (default).

-6

Use IPv6 as address domain.

-V *tab*

Operate on virtual router with index *tab*.

The following operations (*op*) exist:

add

Add something.

delete

Delete something.

get

Lookup and display something.

list

Lookup and display something.

The following operation types (*optype*) exist:

policy

Policy routing.

Policy routing arguments:

Add operation:

from *addr*

Apply this rule if the source address is *addr*.

to *addr*

Apply this rule if the destination address is *addr*.

proto *p*

Apply this rule if the transport proto is *p* .

tos *n*

Apply this rule if the TOS is *n* .

tclass *n*

Apply this rule if the traffic class is *n* .

flow *f*

Apply this rule if the traffic flow is *f* .

ifindex *if*

Apply this rule if the **ifindex** of the incoming interface is *if* .

flags *f*

Apply this rule if the packet has the appropriate flags set or unset. The flags are comma-separated, and to require a flag to be not set, it should be prepended with an explanation point (**!**). Existing flags are **forwarded**, **multicast**, **tunneled**, **broadcast**, **fragment**, **mf ipv4**, **ipv6**.

scope *s*

Apply this rule if the scope of the address is *s* .

reverse

If this rule is applied, do the lookup based on *src* instead of *dst* .

table *t*

Use the routing table with policy ID *t* if the rule matches.

last

Do not look at any more rules, even if there is no match on this table.

prio *p*

The priority of the rule to add [-32768..32767].

Delete/get operation:

id *i*

The ID of the rule to delete or show.

Differentiated services arguments:

Add/Delete operation:

src *addr*

Apply this filter if the source address is *addr* .

dst *addr*

Apply this filter if the destination address is *addr* .

proto *p*

Apply this filter if the transport protocol is *p* .

tos *n*

Apply this filter if the TOS is *n* . This option is for IPv4.

tclass *n*

Apply this filter if the traffic class is *n* . This option is for IPv6.

srcport *n*

Apply this filter if the source port is *n* or from **srcport** *n* to **srcportmax** *n* .

dstport *n*

Apply this filter if the destination port is *n* or from **dstport** *n* to **dstportmax** *n* .

srcportmax *n*

Apply this filter if the source port from **srcport** *n* to **srcportmax** *n* .

dstportmax *n*

Apply this filter if the destination port from **dstport** *n* to **dstportmax** *n* .

local

Apply this filter for locally generated packets.

handler *str*

Apply this filter using *str* handler.

Handler arguments:

SimpleMarker

Apply simple marker if match this filter.

srTCM

Apply single rate Three Color Marker if match this filter.

Handler **SimpleMarker** arguments:

ds-val *n*

Set DSCP with *n* if match this filter.

drop-precedence *p*

Set drop precedence with *p* if match this filter.

Handler **srTCM** arguments:

mode *m*

Use mode <m> if match this filter.

CIR *n*

Set Committed Information Rate to *n* if match this filter.

CBS *n*

Set Committed Burst Size to *n* if match this filter.

EBS *n*

Set Excess Burst Size to *n* if match this filter.

ds-green *n*

Set DSCP with *n* for green packets if match this filter.

ds-yellow *n*

Set DSCP with *n* for yellow packets if match this filter.

ds-red *n*

Set DSCP with *n* for red packets if match this filter.

Examples:

- Set all the local generated packets matching destination 192.168.1.0/24 to a TOS value with 0x10:

```
# qos add filter dst
192.168.1.0/24 local handler SimpleMarker ds-val 0x10
```

Set all the local generated packets matching source port 21 to a TOS value with 0x10:

```
# qos add filter srcport
21 local handler SimpleMarker ds-val 0x10
```

Previous topic: qc Command on page 97

9. TUNNELING

1. Tunneling

Tunneling through an IPv4 Internet refers to the encapsulation of data (such as an IPv4 or an IPv6 packet for a VPN connection) in an IPv4 packet that is then transmitted to an IPv4-addressed destination. At the destination, the data is retrieved from the IPv4 packet and processed. If the data retrieved is an IPv6 packet, and the receiving stack is a dual IPv4/IPv6 stack, the stack can forward the IPv6 packet out onto the IPv6 Internet.

A tunnel attaches to the stack as a network interface. Tunneling interfaces are created using the ifconfig shell command.

> Note: IPsec does its own IP tunneling for associations running in tunneling mode; therefore, an IPsec tunnel will not show up as a regular network interface.

Tunnels can be either configured or automatic . A configured tunnel determines the endpoint addresses by configuration information on the encapsulating node. An automatic tunnel determines the IPv4 endpoints from the addresses of the embedded IPv6 datagram.

> Note: You can also set up host-to-router tunnels, router-to-router tunnels, and other permutations.

Next topic: Configuring VxWorks for Tunneling on page 104

2. Configuring VxWorks for Tunneling

The VxWorks 7 Network Stack tunneling components:

- GIF Tunnel Interface Driver (RFC 1853, RFC 2473)
- GRE Tunnel Interface Driver (RFC 2002, RFC 2784)
- 6over4 Tunnel Interface Driver (RFC 2529)
- 6to4 Tunnel Interface Driver (RFC 3056)
- SIT Tunnel Interface Driver

The VxWorks source build (VSB) component COMPONENT_TUNNEL_SUPPORT enables these.

GIF Tunnel Interface Driver

The VSB component COMPONENT_TUNNEL_SUPPORT compiles modules that implement the **gif** tunneling pseudo-device for IPv4 and IPv6. GIF can tunnel IPv4 and IPv6 over IPv4 or IPv6. GIF tunneling is configured, not automatic, which means you must specify the endpoints when the tunnel is created, rather than relying on the endpoints being extracted from the addresses of the protocol being tunneled; you can configure endpoints per route entry. This means, after having created the tunnel network interface you can route commands to direct traffic to any number of various destination hosts or subnets to use the tunnel interface.

The GIF component has no configuration parameters and is automatically included when you enable tunneling support.

Use ifconfig to create GIF tunnel interfaces. Interfaces for GIF tunnels must begin with "**gif**". For example:

```
_-> ifconfig "gif0 create"
```


WNDRVR

To set tunnel endpoints on a GIF tunnel, use ifconfig. For details, see reference documentation on the ifconfig command. For example, to set the endpoints on a GIF tunnel over IPv4:

```
-> ifconfig "gif0 inet tunnel 192.168.0.10 10.1.2.3"
```

To set the endpoints on a GIF tunnel over IPv6:

```
-> ifconfig "gif0 inet6 tunnel 2001:10::10 2001:20::1"
```

GRE Tunnel Interface Driver

To set tunnel endpoints on a GRE tunnel, use ifconfig:

```
ifconfig "interfaceName [inet | inet6] tunnel localAddress remoteAddress"
```

For example:

```
-> ifconfig "greTunnel inet6 tunnel 2001:10::10 2001:20::1"
```

The VSB component COMPONENT_TUNNEL_SUPPORT compiles modules that support the **gre** tunneling pseudo-devices for IPv4 and IPv6. GRE can tunnel IPv4 and IPv6 over IPv4 or IPv6. Like GIF tunneling, GRE tunnels are *configured* as opposed to *automatic* (which is to say that you must specify the endpoints when the tunnel is created, rather than relying on the endpoints being extracted from the addresses of the protocol being tunneled; you can configure endpoints per route entry. GRE has a version field set to 0 and provides an optional payload checksum.

GRE can be run in minimal encapsulation when tunneling IPv4 over IPv4 (described in RFC 2002).

Use ifconfig to create tunnel interfaces. Interfaces for GRE tunnels must begin with "**gre**". For example:

```
-> ifconfig "greTunnel create"
```

6over4 Tunnel Interface Driver

The VSB component COMPONENT_TUNNEL_SUPPORT compiles modules that support the 6over4 tunneling pseudo-device for IPv6.

A 6over4 tunnel is an IPv4 multicast tunnel that requires a fully functional IPv4 multicast infrastructure.

6over4 requires the INCLUDE_IPCOM_USE_INET6 component and has no configuration parameters.

Use ifconfig to create tunnel interfaces. Interfaces for 6over4 tunnels must begin with "**6over4**". For example:

```
-> ifconfig "6over4Lan create"
```

To set a local IPv4 address for a 6over4 tunnel, use ifconfig:

```
-> ifconfig " interfaceName  inet tunnel localAddress remoteAddress"
```

As shown above, ifconfig( ) requires a peer address, even though it does not use it in this case; provide a dummy peer address, such as the following:

```
-> ifconfig "6over4Lan inet tunnel 192.168.0.10 0.0.0.0"
```

6to4 Tunnel Interface Driver

The VSB component COMPONENT_TUNNEL_SUPPORT compiles modules that support the 6to4 tunneling pseudo-device for IPv6. Unlike GIF and GRE, 6to4 is an *automatic* tunnel: tunnel endpoints are extracted from the encapsulated IPv6 datagram, and so you do not need to configure them manually.

6to4 tunnels use a prefix of the form "2002:*tunnelIPv4address*::/48" (for instance, "2002:a01:203::1") to tunnel IPv6 traffic over IPv4 (see RFC 3056). Routers advertise a prefix of the form "2002:[IPv4]:*xxxx*/64" to IPv6 clients.

This component requires the INCLUDE_IPCOM_USE_INET6 component and has no configuration parameters.

Use **ifconfig** to create tunnel interfaces. Interfaces for 6to4 tunnels must begin with "**6to4**". For example:

```
-> ifconfig "6to4tun create"
```

SIT Tunnel Interface Driver

Like 6to4, the Simple Internet Tunnel (SIT) tunneling device for IPv6 is an automatic tunnel; tunnel endpoints are extracted from the encapsulated IPv6 datagram.

SIT uses IPv4-compatible IPv6 addresses (for instance, "::10.1.2.3") to tunnel IPv6 traffic over IPv4. The IPv4 address is in the 32 least-significant bits in the IPv6 address. Such an address uses the prefix "::/96".

Use ifconfig to create tunnel interfaces. Interfaces for SIT tunnels must begin with "**sit**". For example:

```
-> ifconfig "sit0 create"
```

Note: The Internet Engineering Task Force (IETF) has deprecated the use of this tunnel type.

Previous topic: Tunneling on page 104
Next topic: Creating 6to4 Tunnels for IPv6 Packets on page 106

3. Creating 6to4 Tunnels for IPv6 Packets

In the VxWorks 7 Network Stack, a 6to4 pseudo-device is an automatic tunnel and one of the many IPv6 transition mechanisms.

Figure 2. 6to4 Addresses

2002	IPv4 Address	SLA ID	Interface ID
16 bits	32 bits	16 bits	64 bits

Consider the following setup code for automatic tunneling on a VxWorks 7 Network Stack dual stack.

```
/* code for 6to4 tunnel setup */
#include "vxWorks.h"
#include "net/utils/ifconfig.h"
#include "net/utils/routeCmd.h"

void tunnel6to4Test ( )
 {
 /* Create and attach the tunnel */
 ifconfig ("6to4tun create");
```

```
/* Add IPv6 address to the tunnel */
ifconfig ("6to4tun inet6 add 6to4AddressForLocalInterface prefixlen 128");
/* Bring up the tunnel */
ifconfig ("6to4tun up");
/* Route all packets to the 2002::/16 network through tunnel "6to4tun"*/
routec ("add -inet6 -net -dev 6to4tun -prefixlen 16 2002::");
}
```

The second ifconfig( ) call creates a route table entry that catches packets with IPv6 destinations that match the first 128 bits of the *6to4AddressForLocalInterface*. This is the IPv4 address of the local network interface prefixed with an IPv6 6to4 prefix. This 6to4 IPv6 address implies a local IPv6 address space of 80 bits.

By convention, the 16 bits just after the IPv4 segment of the address are interpreted as a Site-Level Aggregation (SLA) ID, which you can set to any value. You may use this to organize the local space into several (up to 65,536) subnets. Because the site as a whole is identified to other sites by the first 48 bits of the address, you can use the remaining bits at your convenience without affecting the ability of remote machines to communicate with the site.

If you intend the systems at the tunnel ends to function as routers, make sure that the network stacks at those tunnel ends are configured to forward IPv6 packets. In the Wind River Network Stack, call sysctl( ) for this purpose:

```
-> sysctl "net.inet6.ip6.forwarding=1"
```

Creating RFC 2893-Style Configured Tunnels

The RFC 2893-style configured tunnels based on **gif** devices are point-to-point links. They are similar to point-to-point links over a serial cable except that the transmission medium is the Internet. Through the mediation of a **gif** device instance, you can use a configured tunnel as a direct connection to an endpoint in the IPv6 address space. Unlike with **stf**-based tunnels, you do not need to define the tunnel endpoints in terms of 6to4 IP addresses. Both endpoints need to support dual IPv4/IPv6 stacks, and both tunnel endpoints need an IPv4 address in addition to whatever IPv6 addresses you associate with the tunnel endpoints.

You set a **gif**-based configured tunnel in much the same way as you set up an **stf**-based automatic tunnel. To create a **gif** device instance and bind it to the IPv6 stack, use code modeled after the following:

```
/* code for gif tunnel-over-IPv4 setup. The tunnel will be
 * configured with  an IPv6 address, but it is possible to add an IPv4
 * address as well */
#include "vxWorks.h"
#include "net/utils/ifconfig.h"

void tunnelGifTest ( )
 {
 /* Create and attach the tunnel */
 ifconfig ("gif0 create");
 /* Configure the tunnel to tunnel over IPv4 */
 ifconfig ("gif0 inet tunnel localIPv4Address remoteIPv4Address");
 /* Add IPv6 address to the tunnel */
 ifconfig ("gif0 inet6 add IPv6AddressForTunnel");
 /* Bring up the tunnel */
 ifconfig ("gif0 up");
 }
```

Packets sent to this **gif** device are always transmitted from the specified local interface to the specified remote interface.

Note that the IPv6 addresses you supply in the code, like the example above, need not be 6to4 addresses. They can be any valid IPv6 addresses (of proper scope) validly associated with the tunnel endpoints.

To direct IPv6 packets to this device by default, add an IPv6 default entry to the route table. For example:

```
[vxWorks *] # route "add -inet6 default tunnelEndpointIPv6Address "
```

Alternatively, you can direct only some IPv6 packets to the interface. For example, the following entry captures traffic destined for 2001:DB8:1::/64:

```
[vxWorks *] # route "add -inet6 2001:DB8:1:: tunnelEndpointIPv6Address  -prefixlen 64"
```

Configuration Example

In this example, assume the following:

- you own the IPv4 address 10.1.0.1 and the IPv6 address 2001:DB8:1234::1
- the owner of IPv4 address 10.2.0.1 has agreed to route your IPv6 traffic for 2001:DB8:3333::/32
- the IPv4 node at 10.2.0.1 is a dual IPv4/IPv6 stack with an IPv6 address of 2001:DB8:5678::1

Create a GIF tunnel between 10.1.0.1 and 10.2.0.1 as follows:

```
-> ifconfig "gif0 create up"
-> ifconfig "gif0 inet tunnel 10.1.0.1 10.2.0.1"
```

Then configure the interfaces and route table as follows:

```
-> ifconfig    "fei0    inet6    2001:DB8:1234::1"
-> ifconfig    "gif0    inet6    2001:DB8:1234::1    2001:DB8:5678::1    prefixlen    128"
-> route    "add    -inet6    2001:DB8:3333::2001:DB8:5678::1    -prefixlen   48"
```

Your local node now supports the following devices and route table entries:

	Addresses	Configured
fei0	10.1.0.1	
fei0	2001:DB8:1234::1	
gif0	2001:DB8:1234::1 -> 2001:DB8:5678::1	10.1.0.1 ->10.2.0.1
	Routes	
	10.1.0.1/8	**fei0**
	2001:DB8:1234::/64	**fei0**
	2001:DB8:3333::/32	**gif0**

If you negotiate for routing services from more than one remote IPv6 router that supports a dual IPv4/IPv6 stack, you can create additional **gif** devices to manage configured tunnels through those routers. You would also want to set up your route table to control which IPv6 packets go to which router.

For example, if the second remote dual IPv4/IPv6 stack is at 10.3.0.1 IPv4 and 2001:DB8:9999::1 IPv6, use the following setup code:

WNDRVR

```
-> ifconfig "gif0 create up"
-> ifconfig "gif1 create up"
-> ifconfig "gif0 inet tunnel 10.1.0.1 10.2.0.1"
-> ifconfig "gif1 inet tunnel 10.1.0.1 10.3.0.1"
-> ifconfig "fei0 inet6 add 2001:DB8:1234::1"
-> ifconfig "gif0 inet6 add 2001:DB8:1234::1 prefixlen 128"
-> ifconfig "gif1 inet6 add 2001:DB8:1234::1 prefixlen 128"
```

This sets up the following devices:

Devices	Addresses	Configured
gif0	2001:DB8:1234::1 -> 2001:DB8:5678::1	10.1.0.1 -> 10.2.0.1
gif1	2001:DB8:1234::1 -> 2001:DB8:9999::1	10.1.0.1 -> 10.3.0.1

To add a route over **gif0** use 5678::1 in a route( ) command. For example:

```
[vxWorks *] # route "add -inet6 2001:DB8:1::  2001:DB8:5678::1 -prefixlen 64"
```

To add a route over **gif1** use 2001:DB8:9999::1 in a route( ) command. For example:

```
[vxWorks *] # route "add -inet6 2001:DB8:4444  2001:DB8:9999::1 -prefixlen 48"
```

Thus, the route table would include the following entries:

Routes	Device
2001:DB8:9999:1::/64	**gif0**
2001:DB8:4444::/48	**gif1**

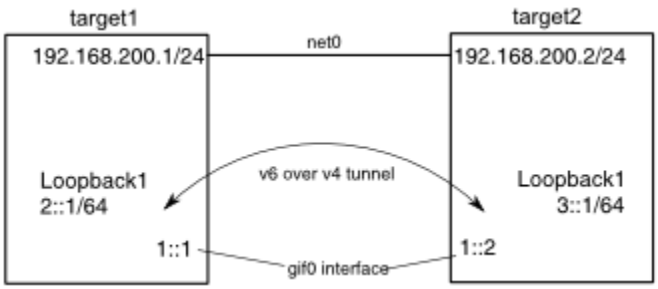
Previous topic: Configuring VxWorks for Tunneling on page 104
Next topic: Tunnel Example on page 109

4. Tunnel Example

How to establish a generic tunnel interface (gif) tunnel to tunnel IPv6 traffic over an IPv4 connection.

About This Task

Figure 3. Two Targets Establish a gif Tunnel



Procedure

1. Establish the Network

   To create this topology, do the following:

   1. On **target1**, issue the following command:

```
    [vxWorks *] # ifconfig gif0 create inet tunnel 192.168.200.1 192.168.200.2 inet6 add 1::
1 up
```

   2. On **target2**, issue the following command:

```
    [vxWorks *] # ifconfig gif0 create inet tunnel 192.168.200.2 192.168.200.1 inet6 add 1::
2 up
```

   This creates a new interface called **gif0**: a virtual interface that connects endpoint 192.168.200.1 to 192.168.200.2 on **target1** and 192.168.200.2 to 192.168.200.1 on **target2**.

   You can verify that the tunnel's endpoints are IPv4 by looking at the interface type in the output from the ifconfig gif0 command:

```
[vxWorks *] # ifconfig gif0
gif0    Link type:Tunnel  Queue:none  IPv[4|6]-over-IPv4
192.168.200.2 --> 192.168.200.1  ttl:64
            inet 224.0.0.1  mask 240.0.0.0
            inet6 unicast FE80::C0A8:C802%gif0 prefixlen 64  automatic
            inet6 unicast 1::2  prefixlen 64
            inet6 unicast FE80::%gif0  prefixlen 64  anycast
            inet6 unicast 1::  prefixlen 64  anycast
            inet6 multicast FF02::1:FF00:2%gif0 prefixlen 16
            inet6 multicast FF02::1:FF00:0%gif0 prefixlen 16
            inet6 multicast FF02::1%gif0  prefixlen 16  automatic
            inet6 multicast FF02::1:FFA8:C802%gif0 prefixlen 16
            UP RUNNING SIMPLEX POINTOPOINT MULTICAST NOARP
            MTU:1480  metric:1  VR:0
            RX packets:0 mcast:0 errors:0 dropped:0
            TX packets:10 mcast:10 errors:0
            collisions:0 unsupported proto:0
            RX bytes:0  TX bytes:688
```

   Notice that the inet6 address of **gif0** is **1::2** (**inet6 unicast**), which means that the tunnel will forward all packets destined to network **1::/64**.

   From **target2**, ping the IPv6 address of the **target1** interface (**1::1**):

```
[vxWorks *] # ping6 ::1

    Pinging ::1 (::1) with 64 bytes of data:
    Reply from ::1 bytes=64 time=0ms hlim=64
    Reply from ::1 bytes=64 time=0ms hlim=64
    Reply from ::1 bytes=64 time=0ms hlim=64
    Reply from ::1 bytes=64 time=0ms hlim=64

    --- ::1 ping statistics ---
    4 packets transmitted, 4 received, 0% packet loss, time 4080 ms rtt
    min/avg/max = 0/0/0 ms
```

2. Add Loopback Interfaces and Default Routes.

   Since the default loopback interface (**lo0**) is used by many system calls and socket applications you will add a new loopback interface (**lo1**). This interface represents a different network.

   In most cases a tunnel is used to connect networks that cannot be directly connected. This example shows how to add a loopback interface on each target and apply an IPv6 address to each interface. Then it shows how to use the route command to tell the network device about remote networks and how to reach those networks using a default gateway.

   On **target1**, issue the following command to add a loopback interface:

```
[vxWorks *] # ifconfig lo1 create inet6 add 2::1 up
```

   Verify the new interface:

```
[vxWorks *] # ifconfig lo1
lo1     Link type:Local loopback  Queue:none
            inet 224.0.0.1  mask 240.0.0.0
            inet6 unicast FE80::1%lo1  prefixlen 64  automatic
            inet6 unicast 2::1  prefixlen 64
            inet6 multicast FF02::1:FF00:1%lo1 prefixlen 16
            inet6 multicast FF02::1%lo1  prefixlen 16  automatic
            UP RUNNING LOOPBACK MULTICAST
            MTU:1500  metric:1  VR:0
            RX packets:3 mcast:0 errors:0 dropped:3
            TX packets:3 mcast:3 errors:0
            collisions:0 unsupported proto:0
            RX bytes:168  TX bytes:168
```

   On **target2**, issue the following command to add a loopback interface:

```
[vxWorks *] # ifconfig lo1 create inet6 add 3::1 up
```

   Verify the new interface:

```
[vxWorks *] # ifconfig lo1
lo1     Link type:Local loopback  Queue:none
        inet 224.0.0.1  mask 240.0.0.0
        inet6 unicast FE80::1%lo1  prefixlen 64  automatic
        inet6 unicast 3::1  prefixlen 64
        inet6 multicast FF02::1:FF00:1%lo1 prefixlen 16
        inet6 multicast FF02::1%lo1  prefixlen 16  automatic
        UP RUNNING LOOPBACK MULTICAST
```

```
        MTU:1500  metric:1  VR:0
        RX packets:3 mcast:0 errors:0 dropped:3
        TX packets:3 mcast:3 errors:0
        collisions:0 unsupported proto:0
        RX bytes:168  TX bytes:168
```

From **target2**, ping the loopback interface of **target1** (**2::1/64**):

```
[vxWorks *] # ping6 2::1
Pinging 2::1 (2::1) with 64 bytes of data:
Echo request operation failed: Network is unreachable (51)
```

You cannot ping interfaces on the network **2::/64** since you do not have a route to that network. Look at the routing table of **target2** by issuing the route show command:

```
[vxWorks *] # route show

    INET route table - vr: 0, table: 254
    Destination      Gateway           Flags  Use   If    Metric
    0.0.0.0/0        192.168.200.254   UGS    0     net0  0
    127.0.0.0/8      localhost         UR     0     lo0   0
    localhost        localhost         UH    12     lo0   0
    192.168.200.0/24 link#2            UC     1     net0  0
    192.168.200.1    7a:7a:c0:a8:c8:01 UHL   10     net0  1
    target2          link#1            UH    10     lo0   0

    INET6 route table - vr: 0, table: 254
    Destination     Gateway    Flags Use   If    Metric
    ::              link#1     UHRS  0     lo0   0
    ::1             ::1        UH   48     lo0   0
    1::/64          link#4     U     0     gif0  0
    1::2            link#1     UH    0     lo0   0
    3FFE:1:2:3::/64 link#2     UC    0     net0  0
    3FFE:1:2:3::4   link#1     UH    0     lo0   0
    FE80::%lo0/64   link#1     UC    0     lo0   0
    FE80::%net0/64  link#2     UC    0     net0  0
    FE80::%gif0/64  link#4     U     0     gif0  0
    FE80::1%lo0     link#1     UH    0     lo0   0
```

Or you can use the --inet6 flag to view only IPv6 route configuration:

```
    [vxWorks *] # route show -inet6

    INET6 route table - vr: 0, table: 254
    Destination      Gateway    Flags Use   If    Metric
    ::               link#1     UHRS   0   lo0    0
    ::1              ::1        UH    48   lo0    0
    1::/64           link#4     U      0   gif0   0
    1::2             link#1     UH     0   lo0    0
    3FFE:1:2:3::/64  link#2     UC     0   net0   0
    3FFE:1:2:3::4    link#1     UH     0   lo0    0
    FE80::%lo0/64    link#1     UC     0   lo0    0
    FE80::%net0/64   link#2     UC     0   net0   0
    FE80::%gif0/64   link#4     U      0   gif0   0
    FE80::1%lo0      link#1     UH     0   lo0    0
```

3. Add a Default Gateway

   Notice that IPv6 does not have a default gateway (a default gateway is notated by the letter "G" in the flags field). Since we have only one exit point from the device, we can configure the tunnel as a default gateway for any unknown IPv6 destination.

   Issue the following command to add a default gateway to target2:

```
    [vxWorks *] # route add -inet6 default 1::1
        add net ::: netmask ::: gateway 1::1
```

   Now, look at the route table again:

```
    [vxWorks *] # route show -inet6

    INET6 route table - vr: 0, table: 254
    Destination      Gateway    Flags Use   If    Metric
    ::               link#1     UHRS  0     lo0    0
    ::/0             1::1       UGS   0     gif0   0
    ::1              ::1        UH    48    lo0    0
    1::/64           link#4     U     0     gif0   0
    1::2             link#1     UH    0     lo0    0
    3FFE:1:2:3::/64  link#2     UC    0     net0   0
    3FFE:1:2:3::4    link#1     UH    0     lo0    0
    FE80::%lo0/64    link#1     UC    0     lo0    0
    FE80::%net0/64   link#2     UC    0     net0   0
```

   Add a default gateway to **target1**:

```
    [vxWorks *] # route add -inet6 default 1::2
        add net ::: netmask ::: gateway 1::2
```

4. Test the Tunnel

   With a default gateway and a v6-over-v4 tunnel you can ping the loopback interfaces. From **target1**:

```
    [vxWorks *] # ping6 3::1
    Pinging 3::1 (3::1) with 64 bytes of data:
    Reply from 3::1 bytes=64 time=0ms hlim=64
    Reply from 3::1 bytes=64 time=0ms hlim=64
    Reply from 3::1 bytes=64 time=0ms hlim=64
    Reply from 3::1 bytes=64 time=0ms hlim=64

    --- 3::1 ping statistics ---
    4 packets transmitted, 4 received, 0% packet
loss, time 4080 ms rtt min/avg/max = 0/0/0 ms
```

   This shows ping-v6 packets encapsulated in IPv4 packets and carried from **target1** to **target2** over an IPv4 link.

Previous topic: Creating 6to4 Tunnels for IPv6 Packets on page 106

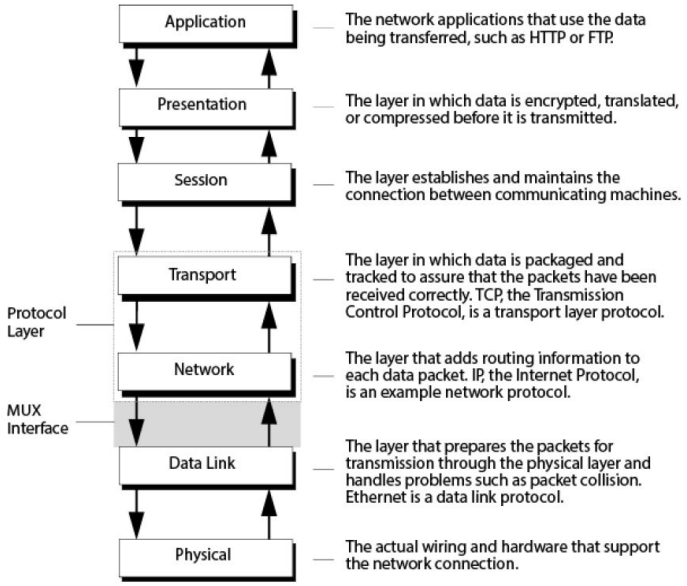
10. INTEGRATING A NEW NETWORK SERVICE

1. Introduction

This chapter describes how to integrate a new network service with the Wind River Network Stack. By a network service, or equivalently a network protocol, we mean here an implementation of the network layer (Layer 3) of the OSI network model. The service may also implement the transport layer or higher levels of a network stack; however it is the network layer that interfaces with the MUX.

> Note: By using “network service” in this sense, we are explicitly excluding TCP or UDP socket applications running on top of the Wind River Network Stack. Such services do not interact directly with the MUX; the Wind River Network Stack's IP protocols (and related protocols such as ARP) are what we would consider the "network services" in this case.

The following figure shows how network services communicate with the data link layer through the MUX interface. Part of porting a new network service to the network stack is porting its data link layer access code to use the MUX interface. Everything specific to the network interface is handled in the drivers of the data link layer.



Next topic: Implementing the MUX/Network Service Interface on page 114

2. Implementing the MUX/Network Service Interface

A network service sends and receives packets through the MUX interface. At minimum, to work with the MUX, your service must implement an initialization routine and routines that support packet transfer and error reporting.

Previous topic: Introduction on page 114

3. Initializing the Interface

Besides loading a network device into the MUX and starting it, you usually want to attach a network stack—that is, a collection of network protocols—to the device. A stack must provide a means of attaching itself to a network device. For example, you can attach the current version of the Wind River Network Stack to a network device by calling either of the following routines:

```
ipcom_drv_eth_init (
const char * drvname, Ip_u32 drvunit,
const char * ifname
)
```

```
ipAttach (int drvunit, char * drvname)
```

The arguments are in slightly different formats in these routines. ipcom_drv_eth_init( ) is the lowest-level function, and ipAttach( ) is provided for compatibility with previous versions of the Wind River Network Stack.

The ipcom_drv_eth_init( ) routine queries the **IFF_NOARP** flag to determine if the interface is Ethernet or non-Ethernet and sets the network interface structure contents accordingly. The ipnet_eth_if_init( ) routine initializes Ethernet interfaces, whereas the ipnet_noeth_if_init( ) routine initializes non-Ethernet interfaces.

The *drvname* and *drvunit* arguments are the device name and unit number, respectively, as seen by the MUX and the network driver. If you call ipcom_drv_eth_init( ), you can optionally specify an interface name, to be used by the network stack, that differs from the driver-level name. For example, the following call attaches to the **fei2** network interface, but calls it **eth0** at the stack level:

```
ipcom_drv_eth_init ("fei", 2, "eth0");
```

On the other hand, the following call (which may be less confusing) attaches the network stack to the interface and uses the same interface name, **fei2**, in the stack that it uses at the driver level:

```
ipcom_drv_eth_init ("fei", 2, IP_NULL);
```

An ipAttach( ) counterpart to this second call would be:

```
ipAttach (2, "fei")
```

Calling ipcom_drv_eth_init( ) or ipAttach( ) makes the stack aware of the interface and causes the stack to bind its protocols to the device. For example, for an IPv4-capable stack, at least the IPv4 and ARP protocols would be bound to the device, and possibly others.

Whether you write a stack of several protocols, or simply a single network service, you need to provide a similar routine that informs your service (or stack) of the network device and causes it to bind to that device. For convenience, assume this routine is called *x*Attach( ) and that *x* is replaced with your service's name. You can also modify the startup code to insert calls to *x*Attach( ) for particular devices. There is no standard interface to accomplish this. However, Wind River recommends that you create a component descriptor file (**.cdf** file) and a configlette so that your service can be handled by the Workbench Kernel Configuration Editor. For more information, see the *VxWorks 7 Application Programmer's Guide: VxWorks Header Files*.

Although it is not required, it is recommended practice to also provide an *x*Detach( ) to unbind your service from a network device and release internal service resources that refer to that device.

Next topic: Binding to a Network Device on page 116

4. Binding to a Network Device

A network service must bind to a network device (that has been loaded into the MUX) before it can send and receive packets through it. A service binds to a loaded network device by calling one of the three routines muxBind( ), muxTkBind( ), or mux2Bind( ). The first two of these routines are for **M_BLK**-oriented protocols, while mux2Bind( ) is provided to bind an **Ipcom_pkt**-oriented protocol.

Note: The IPNET portion of the Wind River Network Stack binds its protocols using mux2Bind( ).) **M_BLK**-oriented protocols that bind using muxBind( ) or muxTkBind( ) can bind to IPNET-native network devices; and **Ipcom_pkt**-oriented prototols that bind using mux2Bind( ) can bind to **M_BLK**-oriented devices. However, in both of these cases, there is some packet format translation overhead incurred on each send and receive. There is also some minimal API translation overhead when a protocol binds using muxTkBind( ) to an **M_BLK**-oriented (END-style) device. There is no translation overhead, however, when a protocol binds using muxBind( ) to an **M_BLK**-oriented driver, or uses mux2Bind( ) to bind to an IPNET-native driver.

Let us consider the (somewhat similar) arguments to muxBind( ), muxTkBind( ), and mux2Bind( ):

```
PROTO_COOKIE muxBind
    (
    char * pName,
    int    unit,
    BOOL   (*stackRcvRtn) (void*, long, M_BLK_ID, LL_HDR_INFO *, void*),
    STATUS (*stackShutdownRtn) (void*, void*),
    STATUS (*stackTxRestartRtn) (void*, void*),
    void   (*stackErrorRtn) (END_OBJ*, END_ERR*, void*),
    long   type,
    char*  pProtoName,
    void*  pSpare
    );

PROTO_COOKIE muxTkBind
    (
    char *      pName,
    int         unit,
    FUNCPTR     stackRcvRtn,
    FUNCPTR     stackShutdownRtn,
    FUNCPTR     stackTxRestartRtn,
    VOIDFUNCPTR stackErrorRtn,
    long        type,
    char *      pProtoName,
    void *      pNetCallbackId,
    void *      pNetSvcInfo,
    void *      pNetDrvInfo
    );

void * mux2Bind
    (
    char * name,
    int     unit,
    BOOL   (*stackRcvRtn) (void * callbackArg, struct Ipcom_pkt_struct * pkt),
    STATUS (*stackShutdownRtn) (PROTO_COOKIE cookie, void * callbackArg),
    STATUS (*stackTxRestartRtn) (void * callbackArg),
    void   (*stackErrorRtn) (void * callbackArg, END_ERR * err),
    unsigned short type,
    char * pProtoName,
    void * callbackArg
    );
```

Each binding routine takes an initial two arguments specifying the driver name and device unit number of the device to bind to. Each routine also takes four function pointers to callback routines that the MUX will call. The purpose of these routines is to:

- deliver packets to the protocol
- command the protocol to detach itself from the device
- notify the protocol that transmission may continue after a send attempt "stalled"
- notify the protocol of various other events relating to the device it is bound to

The primary differences between the binding methods is the signatures of the callback functions. The muxTkBind( ) prototype, unfortunately, does not indicate the actual effective calling signature of its callbacks, but they are as follows:

```
BOOL   (*stackRcvRtn) (void*, long, M_BLK_ID, void *);
STATUS (*stackShutdownRtn) (void *);
STATUS (*stackTxRestartRtn) (void *);
void   (*stackErrorRtn) (void*, END_ERR*);
```

Each routine also takes a *type* argument, which is the network service type. This is either an “ethertype” value identifying the Layer 3 protocol (such as 0x0800 for IPv4, 0x0806 for ARP, and so on), or one of three special service types not associated with a particular Layer 3 protocol number. This issue is discussed more fully below.

Note: Ethertype values are defined by RFC 1700 and other sources.

Each bind routine is also passed a string naming the protocol, which is used in muxShow( ) output, and a **void *** argument (variously called *pSpare* , *pNetCallbackId* , or *callbackArg*) that is passed to each of the callback functions as one of its arguments.

A service may bind to more than one network interface; we call the pairing of a network interface with the service bound to it a "binding instance." Each of the routines returns an opaque value—a protocol binding cookie—that identifies the binding instance to the MUX. The return value may be used in calls to MUX routines that expect a protocol binding cookie argument, such as muxSend( ), muxTkSend( ), muxTkPollReceive( ), muxIoctl( ), muxUnbind( ), muxProtoInfoGet( ), and others.

The muxTkBind( ) API has two additional arguments, *pNetSvcInfo* and *pNetDrvInfo* , that relate to the very rarely used **END_BIND_QUERY** mechanism. This mechanism allows a driver to be notified of protocols binding to its devices and to potentially exchange information between a protocol that is binding and the device driver.

With regard to the network service type value, a service is called a “normal service” or a “typed protocol” if this value is an ethertype identifying a particular Layer 3 protocol. Otherwise, the network service type must be one of the following three special values:

- A **MUX_PROTO_SNARF** type service (or snarf protocol) sees all the received packets that are processed by any device to which it is bound, and that are not consumed by an earlier-bound snarf service.
- A **MUX_PROTO_PROMISC** type service (promiscuous protocol) sees all received packets that are processed by any device to which it is bound, that are not consumed by any snarf, normal, or earlier-bound promiscuous protocol.
- A **MUX_PROTO_OUTPUT** type service sees outgoing rather than incoming packets.

A normal service (or “typed” protocol) sees only packets received by the devices that it is bound to, that match its type, and that were not consumed by any snarf service. At most one typed protocol of a given network service type may be bound to a network device. For any packet received on a device, any snarf services bound to the device see the packet first, then the bound normal service that matches the packet’s service type (if any), then any promiscuous protocol bound to the device; always with the proviso that if an earlier service consumes the packet, it will not be seen by any later service.

A service consumes a packet by returning **TRUE** (or any nonzero value) from its receive routine; it is then responsible for freeing the packet. A service receive routine that does not consume a packet passed to its receive routine returns **FALSE** and should not modify or free the packet.

Previous topic: Initializing the Interface on page 115
Next topic: Using MUX/Service Interface Routines on page 118

WNDRVR

5. Using MUX/Service Interface Routines

The subsections that follow provide an overview of how a network service handles the following tasks:

- sending packets
- controlling devices
- shutting down an interface

Sending Packets

A service sends packets over a network device by calling one of the three routines:

```
int mux2Send (END_OBJ * pEnd, struct Ipcom_pkt_struct * pkt);
```

```
STATUS muxSend (PROTO_COOKIE cookie, M_BLK_ID pMblk);
```

```
STATUS muxTkSend (PROTO_COOKIE cookie, M_BLK_ID pNBuff, char * destAddr,
                                  USHORT svcType, void * extra);
```

Note: Here we consider only normal sends, not polled mode sends.

The first routine represents the packet as an Ipcom_pkt structure (or a chain of such structures), while the latter two routines represent a packet as an **M_BLK** tuple (or chain of such tuples).

First Argument

The first argument of each routine identifies the network device over which to send the packet. For mux2Send( ), this argument is explicitly a pointer to an **END_OBJ**, while the latter two functions expect a protocol binding cookie. In this release, muxSend( ) and muxTkSend( ) will accept an **END_OBJ** pointer in place of the protocol cookie, so, in fact, all of the functions can be called with an **END_OBJ** pointer as the first argument. But mux2Send( ) may not be passed a protocol binding cookie as its first argument.

muxTkSend( ) Additional Arguments

The muxTkSend( ) routine differs in that it has additional arguments. When the *destAddr* argument is not **NULL**, the packet in the **pNBuff** buffer does not have a link header; *destAddr* points at a link-layer destination address (such as a 6-byte ethernet MAC address), and *svcType* contains the network service type for the outgoing packet, already converted to network byte order. The MUX (or an NPT-style driver, no longer supported) is expected to prepend a full link-layer header to the provided packet. When *destAddr* is NULL, the packet **pNBuff** already contains a full link-level header. The extra argument is a remnant from the no-longer-supported NPT driver style; it provided a way that a muxTkSend( ) could pass additional data to the NPT driver send routine. However, since VxWorks 6.9, this argument is unused and should be set to **NULL**.

Return Value

All three "send" routines return **OK** (zero) if the send succeeds; in that case, the driver takes ownership of the packet, and frees it when transmission completes.

But if mux2Send( ) returns **-IP_ERRNO_EWOULDBLOCK**, or if either of the other two routines returns **END_ERR_BLOCK**, this indicates that the device does not have sufficient resources—usually, space in the TX DMA ring—to complete the send at present. This is called a "transmit stall." The caller maintains ownership of the packet, and the driver arranges to call muxTxRestart( ) later, when the send might succeed if retried. The muxTxRestart( ) call by the driver results in a call to the transmit restart routines of all services bound to the device, as discussed below.

Any other return value is unexpected and probably indicates a defect. The driver's send routine frees the packet in this case. One condition that could cause such an error in some drivers, and might crash others, is if the packet that the caller is attempting to send is longer than the maximum frame size for the device (such as 1518 bytes for ethernet when VLAN tags are allowed and jumbo frames are not enabled). It is the responsibility of the caller to respect the device's MTU.

Note:

The device's current MTU may be fetched using the **EIOCGIFMTU** END ioctl command, or set (within the device limit) using the **EIOCSIFMTU END** ioctl. However, coordination between multiple services attached to a device when one of them changes the configured MTU of the device is not automatic, and the coordination may be problematic. For such shared devices, it may be best not to change the device from its default MTU, unless some sort of out-of-band coordination between the bound services is arranged.

Constructing Link Level Headers

Both mux2Send( ) and muxSend( ) require full packets with link-level headers included. muxTkSend( ) allows omitting the link-level header, but it is more efficient for a service to construct the link header itself rather than to allow muxTkSend( ) to construct the link header. This is primarily because if muxTkSend( ) constructs the link header itself, and the driver send returns **END_ERR_BLOCK**, then muxTkSend( ) would have to restore the packet to its original state without a link header. Since the packet will either be dropped or queued for resending by the service, this is probably wasted effort.

If a service will add the link header to a network-layer datagram, it must first perform any necessary conversion from service addresses to the link-layer addresses appropriate to the device being used. Knowing the destination and source link-layer addresses and the network service type, an **M_BLK**-oriented service may call muxAddressForm( ) or muxLinkHeaderCreate( ) to prefix a link header to a packet, making use of the device's *x*FormAddress( ) routine. A mux2Bind( ) service bound to an IPNET-native device could use the analogous **formLinkHdr** function pointer member of the device's **END_OBJ** to do the same thing, using **Ipcom_pkt** packets. Alternatively, if the service knows the type of the END interface being used and the format of the link header for that specific interface type, the service may use its own means to construct a link header; however, the service might not automatically work with other types of device drivers that provide their own special *x*FormAddress( ) or formLinkHdr( ) routines. The IPNET protocols currently follow the latter approach and construct link headers themselves.

Shutting Down an Interface

Relatively few applications need to unload a network device from the MUX, but for those that do, the MUX provides the function muxDevUnload( ). If your application calls muxDevUnload( ), the MUX calls the *x*StackShutdownRtn( ) routine registered at bind time for each network service that is still bound to the device. See the reference entry for *x*StackShutdownRtn( ).

Warning:

Do not call muxDevUnload( ) for a device managed by a VxBus driver. VxBus drivers expect to call muxDevUnload( ) themselves in their {vxbDrvUnlink}( )methods, and instability may result if muxDevUnload( ) is called for a VxBus network device instance by other code. See the VxWorks Device Driver Developer's Guide for more information about unloading VxBus network devices.

Within this shutdown routine, the network service must start the process of detaching from the device. This process includes calling muxUnbind( ) to unbind the network service from the device. It is not necessary that the shutdown routine actually complete the detach process (that is, call muxUnbind( )) within the shutdown routine itself, but the shutdown routine must at least schedule this work. If any of the *x*StackShutdownRtn( ) calls returns a value other than **OK**, muxDevUnload( ) immediately returns **ERROR**. If all of the bound service *x*StackShutdownRtn( ) calls return **OK**, then when all services have completed unbinding from the device, and when any additional references to the device that were acquired using muxDevAcquire( ) are released using muxDevRelease( ), the device is removed from the MUX, and the driver's unload routine is called.

Warning:

It is not safe to call muxDevUnload( ) for an interface unless all the protocols that are bound to it support a working *x*StackShutdownRtn( ) callback routine.

As it may be difficult for some services to accomplish all the work necessary to close and detach from a network interface entirely in the context of the synchronous *x*StackShutdownRtn( ) callback, some applications may find it more convenient to start the process of

detaching services from a network interface first (using service-specific detach routines), calling muxDevUnload( ) only after all such problematic services have completed unbinding from the interface.

Previous topic: Binding to a Network Device on page 116
Next topic: Service Routines Registered When Binding to a Device on page 120

6. Service Routines Registered When Binding to a Device

When a network service binds to a network driver through the MUX, it must provide references to routines that the MUX can call to handle the following:

- passing a packet into the service
- passing exceptional event notifications to the service
- shutting down the network service

The prototypes of the routines that you specify to handle these cases differ depending on which routine you call to bind the service to a network interface in the MUX (mux2Bind( ), muxTkBind( ), or muxBind( )). This section describes the four MUX interface routines that a service implements and references in a mux[Tk]Bind( ) call.

xStackShutdownRtn( )

If your application calls muxDevUnload( ) for a loaded network device, muxDevUnload( ) in turn calls the *x*StackShutdownRtn( ) of each service that is bound to that device.

Within this routine, the network service must do whatever is necessary to initiate the process of detaching the service from the device. This process includes releasing any internal references the service has to the device and calling muxUnbind( ) to unbind the service from the device. However, the process need not be completed in *x*StackShutdownRtn( ). The current implementation uses a reference count mechanism, and completes the unload process when all services have unbound from the device and any other references acquired using muxDevAcquire( ) have been released.

muxTkBind( ) Version

For a service bound with muxTkBind( ), the *x*StackShutdownRtn( ) prototype is:

```
    STATUS xStackShutdownRtn
        (
        void *  netCallbackId  /* the handle/ID installed at bind time */
        )
```

The MUX passes this routine a single argument, which is the identifier that the service passed into the MUX during the muxTkBind( ) call. This value is opaque to the MUX, but the network service understands it and uses it to identify the particular network interface. It is typically a pointer to the service's data structure that represents that interface.

muxBind( ) Version

For a service bound with muxBind( ), the *x*StackShutdownRtn( ) prototype is:

```
    STATUS xStackShutdownRtn
        (
        void *  pBindCookie,     /* binding cookie returned from muxBind() */
```

```
        void *  netCallbackId    /* the handle/ID installed at bind time */
        )
```

The MUX passes this routine two arguments:

- the binding instance cookie that was returned from muxBind( )
- The identifier that the service passed into the MUX during the muxBind( ) call. This value is opaque to the MUX, but the network service understands it and uses it to identify the particular END interface. It is typically a pointer to the service's data structure that represents that interface.

xStackRcvRtn( )

The MUX delivers the packets it receives from the network device to a service by calling the *x* StackRcvRtn( ) callback that the service registered when it bound to the device. The *x* StackRcvRtn( ) is called only in the context of the network job queue used by the network interface.

The *netCallbackId* parameter that the MUX passes into *x* StackRcvRtn( ) is the callback argument that the service specified at bind time. The *type* parameter specifies the network service type of the packet. The *pPkt* parameter points to an **M_BLK** tuple (or **Ipcom_pkt**) that describes the packet. Currently, drivers must always provide received packets in a single contiguous data segment.

If a network service accepts the packet by returning **TRUE**, it takes ownership of the packet and is responsible for freeing the given **M_BLK** chain when the service is finished with it. If it returns **FALSE**, it should neither free nor modify the packet.

mux2Bind( ) Version

For a service bound with mux2Bind( ), the *x* StackRcvRtn( ) prototype is:

```
    BOOL xStackRcvRtn
        (
        void * netCallbackId,          /* identifies the device to the service */
        struct Ipcom_pkt_struct * pPkt /* the packet being delivered */
        )
```

The *netCallbackId* parameter passed to mux2Bind( ) identifies the device to the service.

The *pPkt* parameter is a pointer to an **Ipcom_pkt** describing the packet being delivered. The network service type of the packet is available in the VxWorks-only **net_svc** member of the **Ipcom_pkt** structure. For more information on how packets are represented by **Ipcom_pkt** structures, see the VxWorks Device Driver Developer's Guide.

muxTkBind( ) Version

For a service bound with **muxTkBind( )** the *x* **StackRcvRtn( )** prototype is:

```
    BOOL xStackRcvRtn
        (
        void *    netCallbackId,  /* the handle/ID installed at bind time */
        long      type,           /* network service type of the packet */
        M_BLK_ID  pPkt,           /* the packet as an M_BLK tuple chain*/
        void *    pSpareData      /* pointer to optional data from driver */
        )
```

For services bound with **muxTkBind( )**, the link header is always present in the cluster in the first tuple of the chain describing the packet, but the lead **M_BLK** may be adjusted to skip over the link header:

- For normal typed protocols, **pPkt->mBlkHdr.mData** is advanced by the size of the link header, while **pPkt->mBlkHdr.mLen** and **pPkt->mBlkPktHdr.len** are decreased by the size of the link header.
- For **MUX_PROTO_SNARF** and **MUX_PROTO_PROMISC** services, the lead **M_BLK** is not adjusted, that is **pPkt->mBlkHdr.mData** still points to the start of the link header, and **pPkt->mBlkHdr.mLen** and **pPkt->mBlkPktHdr.len** still include the length of the link header.
- For **MUX_PROTO_OUTPUT** services bound to **END**s, **pPkt->mBlkHdr.mData** is advanced by the size of the link header, while **pPkt->mBlkHdr.mLen** and **pPkt->mBlkPktHdr.len** are decreased by the size of the link header. A pointer to the destination MAC address in the header is placed in **pPkt->mBlkPktHdr.rcvif**.
- The size of the link header (the network service offset) is stored in **pPkt->mBlkHdr.offset1**.

muxBind( ) Versions

For a service bound with muxBind( )the *x*StackRcvRtn( ) prototype is:

```
BOOL xStackRcvRtn
    (
    void *         pBindCookie,   /* returned by muxBind() */
    long           type,          /* the network service type of the packet*/
    M_BLK_ID       pPkt,          /* the packet as an M_BLK tuple chain */
    LL_HDR_INFO *  pLLHInfo,      /* link-level header info structure */
    void *         pCallbackId    /* the handle/ID installed at bind time */
    )
```

In this version of *x*StackRcvRtn( ), *pBindCookie* is the binding cookie that is returned by muxBind( ). As this value is opaque to the service, it is probably less useful than the *netCallbackId* value passed into the other version of *x*StackRcvRtn( ).

For all services bound with muxBind( ), the link-level header is present at the start of the packet. It is described in the **LL_HDR_INFO** structure pointed to by the *pLLHInfo* argument. The MUX calls the **END** device's *x*PacketDataGet( ) routine to parse the link header and fill out this structure. The most important information in this structure is the size of the link header, but it also holds the offsets and sizes of the source and destination link-level addresses, as well as (redundantly) the network service type of the packet.

xStackErrorRtn( )

A device notifies the MUX of various exceptional events it encounters by calling muxError( ), and the MUX forwards these to the network services that are bound to the device by calling *x*StackErrorRtn( ). It is the responsibility of the network service to take any necessary action when it receives the event notification.

The MUX passes this routine a pointer to an **END_ERR** structure and the callback argument *netCallbackId* that was passed at bind time.

mux2Bind( ) Version

For a service bound with mux2Bind( ), the *x*StackErrorRtn( ) prototype is:

```
void xStackErrorRtn
    (
    void * netCallbackId,  /* identifies the device to the service */
    END_ERR * err          /* identifies the event */
    )
```

muxTkBind( ) Version

For a service bound with muxTkBind( ) the *x*StackErrorRtn( ) prototype is:

```
void xStackErrorRtn
    (
    void *    netCallbackId,  /* the handle/ID installed at bind time */
    END_ERR * pError          /* pointer to structure containing error */
    )
```

muxBind( ) Version

For a service bound with muxBind( ), the *x*StackErrorRtn( ) prototype is:

```
void xStackErrorRtn
    (
    void *     pEND,           /* END_OBJ passed to the MUX by the driver */
    END_ERR *  pError,         /* pointer to structure containing error */
    void *     netCallbackId   /* the handle/ID installed at bind time */
    )
```

This version of the *x*StackErrorRtn( ) routine takes an additional argument, *pEnd*, which describes the **END** device.

xStackRestartRtn( )

The MUX calls the *x*StackRestartRtn( ) routine to restart transmission over a network device by any network services bound to the device that care to do so.

When n device's *x*Send( ) routine returns **END_ERR_BLOCK** (for an **M_BLK**-oriented driver) or **-IP_ERRNO_EWOULDBLOCK** (for an IPNET-native driver), it is indicating that it cannot schedule the packet for transmission immediately (usually due to a temporary lack of space in the transmit ring). The sending service may choose to drop the packet, or to hold on to it for later retransmission. Having returned **END_ERR_BLOCK** (or **-IP_ERRNO_EWOULDBLOCK**), the driver guarantees that it will call muxTxRestart( ) when the device is again ready to accept packets for transmission. muxTxRestart( ) calls the *x*StackRestartRtn( )routine for each service that is bound to the device and that provided such a routine. The *x*StackRestartRtn( ) routine may respond by sending any packets that it has queued for the device until it sends them all or the send routine returns **END_ERR_BLOCK** (or **-IP_ERRNO_EWOULDBLOCK**) once more.

The MUX passes this routine the callback argument value that the service passed to the bind routine.

mux2Bind( ) Version

For a service bound with mux2Bind( ), the *x*StackRestartRtn( ) prototype is:

```
STATUS xStackRestartRtn
    (
    void * netCallbackId    /* identifies the device to the service */
    )
```

muxTkBind( ) Version

For a service bound with muxTkBind( ), the *x*StackRestartRtn( ) prototype is:

```
STATUS xStackRestartRtn
    (
    void *  netCallbackId  /* the handle/ID installed at bind time */
    )
```

muxBind( ) Version

For a service bound with muxBind( ) the *x*StackRestartRtn( ) prototype is:

```
STATUS xStackRestartRtn
    (
    void *  pEND,          /* END_OBJ passed to the MUX by the driver */
    void *  netCallbackId  /* the handle/ID installed at bind time */
    )
```

This version of the *x*StackRestartRtn( ) routine takes an additional argument, *pEnd*, which describes the **END** device.

Previous topic: Using MUX/Service Interface Routines on page 118

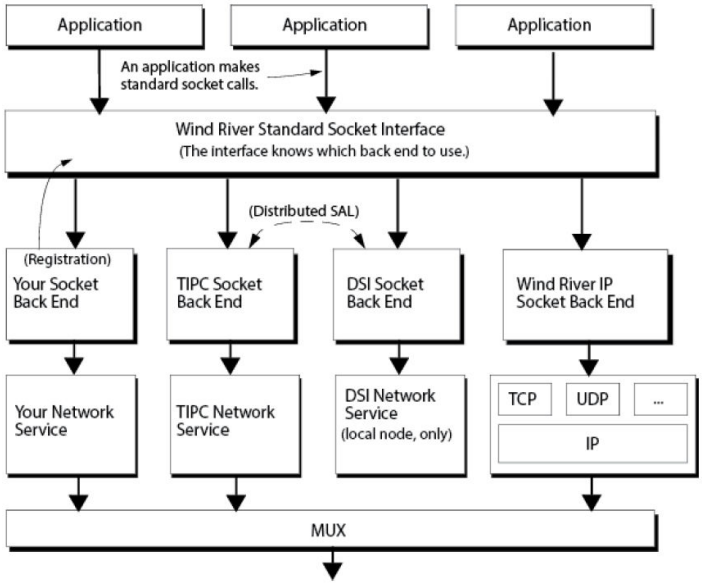
7. Adding a Socket Interface to Your Service

One way to allow applications to access your network service is to add sockets support to the service. In order to make it easier for you to write a network service that includes sockets support, the Wind River Network Stack includes a standard sockets interface.

With the standard socket interface, you can add new socket back ends to access your network service, and through it, the network. This allows developers who are already familiar with the standard sockets API to use your service more easily.

With the standard sockets interface, an application can create and use sockets of different address families, which may be managed by different back end service implementations. A layered architecture makes this possible. The Wind River standard sockets interface, implemented by **sockLib**, is a layer above your back end socket layer, as shown in the folllowing figure.

This section shows you how to implement a sockets back end.



Process Overview

To create a socket, an application calls the standard function socket( ), and receives a socket descriptor in return. The socket( )routine looks up the correct back end implementation to use based upon the specified domain (address family) argument. If it finds a registered back end that handles that address family, socket( )calls the back end's socket creation function, obtaining back a data structure that represents the socket. The socket( ) routine then completes the data structure by allocating a descriptor from the I/O system and associating it with the socket data structure, and returns the descriptor to the caller.

The socket data structure contains a pointer to a table of pointers to functions that the back end provides to implement all the various socket operations (see Socket Functional Interface on page 126). You implicitly specify this table when you register the socket back end by calling sockLibAdd( ), as discussed in Registering a Socket Back End on page 125. When an application makes sockets API calls other than socket( ), the socket descriptor returned by socket( )is one of the arguments. The **sockLib** implementation for each such sockets API call converts the socket descriptor to the underlying socket data structure using I/O system descriptor look-up facilities, fetches the pointer to the back end's function table, and calls the appropriate back-end function to complete the call.

The I/O system functions read( ), write( ), ioctl( ), and close( ) may also be used on socket descriptors. In this case, the I/O system converts the descriptor to the underlying socket data structure, calls the registered **sockLib** read, write, ioctl, or close function, which in turn calls the back end's read, write, ioctl, or close handler via the back end function table. **sockLib** registers a single I/O system driver to handle all socket descriptors.

⚠ Warning:

In the present release, the socket data structure used by **sockLib** is in fact a struct socket, declared in *installDir* **/components/ip_net2-6.***x* **/vxcoreip/include/net/socketvar.h**. This is primarily for historical reasons. The struct socket structure contains many members, only a few of which are needed by **sockLib** itself; and some socket back ends will prefer not to use the other members of struct socket. It is very likely that a future release will replace (or modify) struct socket with a much smaller structure (which would be embeddable in a back end's private data structure representing a socket). The following members from struct socket are likely to remain in the new, smaller structure:

```
struct sockFunc * pSockFuncTbl;  /* socket back-end function table */
     int               so_fd;         /* the socket file descriptor */
     void *            so_bkendaux;   /* socket back-end auxiliary data */
```

If your back end uses other members of struct socket, then after the change to the smaller structure, you may have to move these members to your back end's own private socket data structure.

Generally, the interface between **sockLib** and socket back ends, as well as the interface for registering socket back ends, although described here as it presently exists, should be considered somewhat fluid. Wind River may modify these interfaces in future releases.

For further information on setting socket options, see the *VxWorks 7 Sockets Programmer's Guide*.

8. Registering a Socket Back End

You can register a socket back end implementation by calling sockLibAdd( ) some time during system start-up after **sockLib** is itself initialized by a call to sockLibInit( ). The sockLibAdd( ) routine has the following prototype:

```
extern STATUS   sockLibAdd (SOCK_LIB_INIT_RTN sockLibInitRtn, int domainMap,
                                              int domainReal);
```

The routine returns **OK**, or **ERROR** if the routine could not add the socket back end. The routine takes three parameters:

sockLibInitRtn

A pointer to your xSockLibInit( ) routine that sockLibAdd( ) invokes. sockLibAdd( ) calls this routine as if it had the following prototype:

```
SOCK_FUNC * xSockLibInit (void);
```

That is, it is passed no arguments, and it is expected to return a pointer to an initialized **SOCK_FUNC** structure, which is the table of function pointers for the back end (see Socket Functional Interface on page 126).

domainMap

sockLibAdd( ) ignores this parameter.

domainReal

This parameter specifies the address family that this back end implements. A back end may support more than one address family, but in such a case you must call sockLibAdd( ) multiple times, once per address family. Allowed address families are in the range from **1** to **AF_MAX-1**, and the **AF_** constants that identify these address families (**AF_INET**, and so forth) are declared in *installDir* **/components/ip_net2-6.***x* **/vxcoreip/include/sys/socket.h**.

At most one back end will handle sockets of any given address family. For example, the native Wind River Network Stack normally handles sockets of the **AF_INET**, **AF_INET6**, **AF_ROUTE**, and **AF_PACKET** families. A socket back end of your own implementation may not handle one of these address families without disabling handling of that family by the native stack.

Next topic: Socket Functional Interface on page 126

9. Socket Functional Interface

The socket functional interface is the set of implementations of standard socket routines that a particular socket back end supports. **SOCK_FUNC** is declared in *installDir* **/components/ip_net2-6.***x* **/vxcoreip/include/sockFunc.h** as follows:

```
    struct sockFunc                 /* SOCK_FUNC */
            {
            SOCK_FUNC_INIT              libInitRtn; /* Unused */
            SOCK_FUNC_ACCEPT            acceptRtn;
            SOCK_FUNC_BIND              bindRtn;
            SOCK_FUNC_CONNECT           connectRtn;
            SOCK_FUNC_CONN_TIMEO        connectWithTimeoutRtn;
            SOCK_FUNC_GETPEERNAME       getpeernameRtn;
            SOCK_FUNC_GETSOCKNAME       getsocknameRtn;
            SOCK_FUNC_LISTEN            listenRtn;
            SOCK_FUNC_RECV              recvRtn;
            SOCK_FUNC_RECVFROM          recvfromRtn;
            SOCK_FUNC_RECVMSG           recvmsgRtn;
            SOCK_FUNC_SEND              sendRtn;
            SOCK_FUNC_SENDTO            sendtoRtn;
            SOCK_FUNC_SENDMSG           sendmsgRtn;
            SOCK_FUNC_SHUTDOWN          shutdownRtn;
            SOCK_FUNC_SOCKET            socketRtn;
            SOCK_FUNC_GETSOCKOPT        getsockoptRtn;
            SOCK_FUNC_SETSOCKOPT        setsockoptRtn;
            SOCK_FUNC_ZBUF              zbufRtn;

            /*
             * The following IO-system handlers are called via wrappers in
```

```
             * sockLib.c:
             */
            SOCK_FUNC_CLOSE             closeRtn;
            SOCK_FUNC_READ              readRtn;
            SOCK_FUNC_WRITE             writeRtn;
            SOCK_FUNC_IOCTL             ioctlRtn;
            };
```

The **sockFunc** header also has typedefs for each of the function types used in this structure, such as **SOCK_FUNC_ACCEPT**:

```
    typedef int (*SOCK_FUNC_ACCEPT)
        (struct socket **pso,
        struct sockaddr * addr,
        socklen_t * pAddrlen);
```

In most cases, you can guess the type of each **SOCK_FUNC** routine by considering the corresponding standard sockets API function prototype in **sockLib.h**, or the corresponding I/O system function prototype in **ioLib.h**, and replacing the integer socket descriptor argument with a struct socket * argument. For instance, the connect API is prototyped in **sockLib.h** as the following:

```
extern STATUS   connect (int s, const struct sockaddr *name, socklen_t namelen);
```

The back end function called through the **connectRtn** function pointer is called as if it had the following prototype:

```
STATUS xConnectRtn (struct socket * so, const struct sockaddr * name, socklen_t namelen);
```

The **sockLib** connect( ) implementation converts the integer socket descriptor passed as its first argument, *s* , to the struct socket pointer so, and calls the back end's *x* ConnectRtn( ) routine, replacing s with *so* and passing the *name* and *namelen* arguments unchanged. From the *so* argument, the socket back end can find its own private data for the socket; the **so_bkendaux** member of **struct socket** is intended as a pointer to such private data. (Alternatively, since the back end's socketRtn( ) is responsible for allocating the **struct socket**, the back end may choose to embed the **struct socket** in a larger structure containing also the private data.)

The return value from the back end *x* ConnectRtn( ) is same as the return value from connect( ).

Consult the **sockLib** reference pages for additional information on the intended behavior of the sockets API functions. External sockets API information is also very useful; for instance IEEE Std 1003.1 contains the official POSIX descriptions of sockets API functions and data structures. (Historically, the VxWorks sockets API prototypes did not always match exactly those defined by POSIX; but in VxWorks 6.9, the socket API prototypes were made consistent with the POSIX sockets prototypes.) For general background on sockets programming, see W. Richard Stevens, Unix Network Programming - Networking APIs: Sockets and XTI, Volume 1.

There are exceptions to the above rule of thumb. These are socketRtn( ), acceptRtn( ), and ioctlRtn( ).

xSocketRtn( )

The *x* SocketRtn( ) routine has the following prototype:

```
STATUS xSocketRtn
        (
        int              domain,     /* socket domain or address family number */
        int              type,       /* socket's nature, e.g. SOCK_DGRAM */
        int              protocol,   /* the protocol variety of the socket */
        struct socket ** ppSo        /* the socket structure */
        )
```


WNDRVR

The back end's *x*SocketRtn( ) is responsible for allocating and initializing a **struct socket** and any other structures that the back end needs to represent a socket of the specified domain, type, and protocol (these are passed directly from the corresponding arguments to the socket( ) routine). The allocation may be done using the kernel heap, a **netBufLib** pool, or a back-end specific method. If the *x*SocketRtn( ) cannot allocate and initialize a socket of the specified kind, *x*SocketRtn( ) must free any partial allocations, set *errno* to the appropriate value (such as **ENOMEM**, **ENOBUFS**, **EAFNOSUPPORT**, **EPROTONOSUPPORT**, **EPROTOTYPE**), and return **ERROR**.

Otherwise, *x*SocketRtn( ) stores a pointer to the allocated **struct socket** at the address specified by *ppSo* , and returns **OK**. The socket( ) code will then itself store a pointer to the back end's **SOCK_FUNC** table in the returned **struct socket**'s **pSockFuncTbl** member, then attempt to allocate a file descriptor from the I/O system, associating it with the socket. The socket( ) call may still fail if the calling RTP (possibly the kernel) is out of file descriptors. In this case, the socket( ) code will immediately call the back end's *x*CloseRtn( ) function, to destroy the socket, and return **ERROR**. On the other hand, if a file descriptor is successfully allocated, the socket( ) code stores that file descriptor in the **so_fd** member, and return the file descriptor as its result.

sockLib( ) itself has no requirements as to how the back end initializes the **struct socket** structure that it allocates; however, the back end will probably want to initialize any members (other than **pSockFuncTbl** and **so_fd**) that it needs, in particular setting **so_bkendaux** to point to any auxiliary private data the back end wishes to maintain for the socket.

xAcceptRtn( )

The *x*AcceptRtn( ) routine has the following prototype:

```
STATUS xAcceptRtn
        (
        struct socket ** ppSo,  /* IN: parent socket. OUT: child socket. */
        struct sockaddr * addr, /* Address of child's peer. */
        socklen_t * addrlen     /* Length of child's peer's address. */
        );
```

The back end's *x*AcceptRtn( ) is called by the accept( ) code in **sockLib**. This accept( ) code does some checking, however, before it calls *x*AcceptRtn( ). If addr is non-**NULL** but *addrlen* is **NULL**, accept( ) returns an error. Otherwise, accept( ) attempts to convert the descriptor passed as its first argument to a **struct socket**. If the descriptor is not a valid socket descriptor, accept( ) again returns an error. If the back end does not provide any accept handler, that is, if the **acceptRtn** member of the back end's **SOCK_FUNC** structure is **NULL**, accept( ) again returns **ERROR**.

Otherwise, the accept( ) code calls the back end's *x*AcceptRtn( ) function, passing as the first argument the address of a pointer to the **struct socket**, converted from the socket descriptor passed to accept( ). The other two arguments to *x*AcceptRtn( ) are passed directly from the corresponding arguments of accept( ).

The back end's *x*AcceptRtn( ) must check that the socket passed in by the first argument is in fact a listening parent socket, capable of providing child socket connections to accept( ). If not, *x*AcceptRtn( ) must return **ERROR** and set *errno* appropriately (see IEEE Std 1003.1). If the parent can provide child connections, but no completed child socket connection is presently queued for the parent, then *x*AcceptRtn( ) must do one of the following:

- set *errno* to **EAGAIN** or **EWOULDBLOCK** and return **ERROR**, if the parent socket is non-blocking
- if the parent socket is blocking, pend until either a completed child socket connection becomes available (or optionally: until a timeout, signal, or other implementation defined event occurs, in which case *errno* should be set appropriately, and **ERROR** returned).

If a completed child socket connection becomes available, *x*AcceptRtn( ) allocates a **struct socket** for it (and any other needed private data structures). It also stores a pointer to the child **struct socket** at the address passed in the *ppSo* argument, overwriting the previous pointer to the parent's **struct socket**. If the *addr* argument is non-**NULL**, *x*AcceptRtn( ) obtains the child's peer's socket address and copies it (truncating to the length specified in the *int* pointed to by *addrlen* , if necessary) to the specified address *addr* ; and finally storing the actual address length in the integer pointed to by *addrlen* . *x*AcceptRtn( ) then returns **OK**.

If *x*AcceptRtn( ) does not return **ERROR**, the accept( ) code stores a pointer to the back end's **SOCK_FUNC** table in the child socket's **pSockFuncTbl** member, then goes on to attempt to allocate a socket descriptor from the I/O system for the child socket. If this fails, accept( ) immediately calls the back end's *x*CloseRtn( ) routine and returns **ERROR**.

Otherwise, accept( ) stores the allocated socket descriptor in the child **struct socket**'s *so_fd* member, and returns that socket descriptor as its result.

xIoctlRtn( )

The *x*IoctlRtn( ) routine has the following prototype:

```
int xIoctlRtn
        (
        struct socket * so, /* the socket */
        int code,           /* ioctl command code */
        _Vx_usr_arg_t data  /* ioctl argument */
        void * mode         /* indicates if the call is from user space */
        );
```

The back end's *x*IoctlRtn( ) routine is called when the I/O system processes an ioctl( ) call made on a socket descriptor. The routine is called as the rule of thumb would suggest, passing the **struct socket** pointer *so* instead of a socket file descriptor and passing the **ioctl** command and data arguments unchanged, except that *x*IoctlRtn( ) is also passed another argument mode. This argument is **NULL** if ioctl( ) was called from the kernel; it is an arbitrary non-**NULL** value if ioctl( ) was called from a user-space RTP. This indication is intended to help support validation of user-space **ioctl** arguments.

Previous topic: Registering a Socket Back End on page 125
Next topic: Memory Validation and Socket Ioctls on page 129

10. Memory Validation and Socket Ioctls

Socket APIs are expected to validate their arguments for proper memory access when called from a user-space RTP application. This is accomplished for most socket calls in the **socketScLib** shim library. This library contains the system call handlers for sockets API system calls; these handlers execute in the kernel and perform argument memory access validation before calling the appropriate kernel socket APIs in **sockLib**. (Memory validation is done using the scMemValidate( )routine; see its reference entry for more information.)

Memory validation for the read( ) and write( ) buffer and length arguments is done by the I/O system's system call handler code. However, ioctl( ) calls made on socket descriptors are a special case. The I/O system level does not have knowledge about the form and use of the ioctl arguments passed to ioctl operations implemented by the lower-level "driver" code, such as the socket back ends, and so cannot do the memory validation. In VxWorks, the lower-level drivers (including socket back ends) are expected to do memory validation for the ioctls they implement, which may be called by RTP applications.

sockScLib provides a pair of functions that can aid a socket back end in doing ioctl argument memory validation. These functions should only be called through the following function pointers:

```
int (*pSockIoctlMemVal)
    (
    unsigned int cmd,
    void *       data
    );

STATUS (*pUnixIoctlMemVal)
    (
    unsigned int cmd,
```

```
    const void * pData
    );
```

The function pointers are only non-**NULL** when RTP support is included in the VxWorks image. They should only be called when the mode argument passed to *x*IoctlRtn( ) is non-**NULL**, indicating an ioctl( ) call from user space.

Most (although unfortunately not all) ioctl commands on sockets encode the length and usage of the command argument in the ioctl command code itself. Ioctl codes following the conventions in **target/h/sys/ioctl.h** or *installDir* **/components/ip_net2-6.***x* **/vxcoreip/include/ipnet/ipioctl.h** encode whether the ioctl argument is a pointer to a buffer that is read into the kernel, written to by the kernel, or both; and if so, how large the buffer is. The top-level memory validation for such ioctl codes may be performed by calling pUnixIoctlMemVal( ), passing it the command code and the ioctl data argument. This routine returns OK if memory validation succeeds, otherwise it sets *errno* appropriately and returns **ERROR**.

There are some limitations of the function referenced by pUnixIoctlMemVal( ):

- It does not check that the ioctl code is supported by the back end.
- It assumes without any check that the code follows the conventions encoding the parameter length and usage, as described in **target/h/sys/ioctl.h**.
- It only does top-level checking: if the ioctl parameter is a pointer to a buffer holding a data structure that contains additional pointers, these other pointers are not validated.

The pSockIoctlMemVal( ) function pointer behaves similarly to pUnixIoctlMemVal( ), except that it additionally validates memory for a small number of ioctl( ) codes that do not follow the conventions upon which pUnixIoctlMemVal( ) depends. In the present release, these additional ioctl codes are **FIONBIO**, **FIONREAD**, **FIOSELECT**, and **FIOUNSELECT**. It also supports **SIOCMUXPASSTHRU** and **SIOCMUXL2PASSTHRU**, doing second-level validation of the embedded MUX ioctl commands in these two ioctls' arguments. For any other ioctl code passed to pSockIoctlMemVal( ), it simply calls pUnixIoctlMemVal( ).

Previous topic: Socket Functional Interface on page 126

11. VLANS

1. About VxWorks 7 Network Stack VLAN Support

How you can implement and manage a virtual local area network (VLAN), using VLAN tagging as defined in (IEEE) Std 802.1Q - Virtual LANs and how to configure a stacked VLAN, as defined in the Institute of Electrical and Electronics Engineers (IEEE) standard 802.1ad - Provider Bridges.

It is assumed that you are familiar with the principles and operations of IEEE 802.1Q VLAN. For further information on this technology, see How VLAN Works on page 155.

For further information on the Wind River implementation of a VLAN, see VLAN: The Wind River Implementation on page 157.

Next topic: Building the VLAN Libraries on page 131

2. Building the VLAN Libraries

When you implement a virtual local area network (VLAN) using the VxWorks Network Stack, you must include the component INCLUDE_MUX_L2 (MUX Layer 2 support) in your VxWorks image. Including this component initializes the MUX-L2 infrastructure for VLAN support.

To allow the network stack to inter-operate with MUX-L2, you must rebuild it with the IPCOM_VXWORKS_USE_MUX_L2 flag. Use the following method:

- Enable the command #define IPCOM_VXWORKS_USE_MUX_L2 found in **ipcom/port/vxworks/config/ipcom_pconfig.h**.

Enabling Support for VLAN Stacking

If you plan to configure the VxWorks kernel with support for VLAN stacking, you must enable the VSB component COMPONENT_VLAN_STACKING and rebuild the network stack.

Enabling 802.1Q Virtual LAN Support

If you plan to configure the VxWorks kernel with support for 802.1Q VLAN, you must enable the VSB component COMPONENT_USE_VLAN and rebuild the network stack.

Previous topic: About VxWorks 7 Network Stack VLAN Support on page 131
Next topic: Building VxWorks with VLAN Support on page 131

3. Building VxWorks with VLAN Support

To configure your VxWorks image project (VIP) with support for a virtual local area network (VLAN), you must include the required components. These include the VLAN component itself, plus required support components for MUX-L2 infrastructure and a layer 2 configuration utility.

Include the INCLUDE_IPNET_USE_VLAN (**VLAN Pseudo Interface support**) component in your build if you want it to include Layer 2 subnet-based VLANs. Including this component initializes the FreeBSD-style VLAN pseudo-interface for subnet-based VLAN support.

MUX Layer 2 Support on page 132 The INCLUDE_MUX_L2 (**MUX Layer 2 support**) component pulls in the MUX network interface library for layer 2. Including this component initializes the MUX-L2 infrastructure for virtual local area network (VLAN) support.
VxWorks 7 Network Stack L2 Configuration on page 132 VxWorks 7 Network Stack L2 Configuration
Adding Support for VLAN Stacking on page 132 To configure VxWorks with support for virtual local area network (VLAN) stacking, you must include the following components:

Previous topic: Building the VLAN Libraries on page 131
Next topic: Managing a VLAN at Run Time on page 133

3.1. MUX Layer 2 Support

The INCLUDE_MUX_L2 (**MUX Layer 2 support**) component pulls in the MUX network interface library for layer 2. Including this component initializes the MUX-L2 infrastructure for virtual local area network (VLAN) support.

The table below lists the INCLUDE_MUX_L2 component parameters.

Parameter	Default Value
MUX_L2_MAX_VLANS_CFG Specifies the maximum number of 802.1Q VLANs supported on the target.	16
MUX_L2_NUM_PORTS_CFG Specifies the maximum number of physical ports available to the target.	16

The INCLUDE_MUX_L2 component requires the following components:

- INCLUDE_END (**END interface support**)
- INCLUDE_ETHERNET (**Ethernet multicast library support**)

For further information on programmatically controlling this support at run time, see Controlling MUX Extensions for Layer 2 VLAN Support on page 152.

Parent topic: Building VxWorks with VLAN Support on page 131

3.2. VxWorks 7 Network Stack L2 Configuration

VxWorks 7 Network Stack L2 Configuration

The INCLUDE_L2CONFIG (**l2config**) component provides support for the layer 2 configuration utility. Including this component initializes the L2 command-line configuration utility. This component requires the INCLUDE_MUX_L2 component.

Parent topic: Building VxWorks with VLAN Support on page 131

3.3. Adding Support for VLAN Stacking

To configure VxWorks with support for virtual local area network (VLAN) stacking, you must include the following components:

INCLUDE_IPNET_USE_VLAN_STACKING

This component provides support for VLAN stacking. It requires the component INCLUDE_IPCOM_USE_ETHERNET.

INCLUDE_IPVCONFIG_CMD

This component provides support for the vconfig shell command.

Specifying the Tag Protocol ID

INCLUDE_IPNET_USE_VLAN_STACKING contains one component for specifying the service VLAN tag protocol ID (TPID).

IPNET_VLAN_S_VLAN_TPID

Identifies the service VLAN tag (see *IEEE 802.1ad - Provider Bridges*). The default value is 0x88a8 as defined in Institute of Electrical and Electronics Engineers (IEEE) 802.1ad. The service VLAN (S-VLAN) TPID must be a different value than the customer VLAN (C-VLAN) TPID value of 0x8100. You can also specify this parameter at run time, using the sysvar ipnet.vlan.svlan.tpid.

Parent topic: Building VxWorks with VLAN Support on page 131

4. Managing a VLAN at Run Time

You can configure a virtual local area network (VLAN) at run time with VxWorks 7 Network Stack, using a variety of techniques.

- Use the l2config utility to manage MUX-L2 VLAN functionality. For further information, see MUX-L2 VLAN Management on page 134.
- Use the ifconfig shell command or ifconfig( ) API to configure the pseudo-interface that implements a subnet-based VLAN. For further information, see Subnet Based VLAN Management on page 135.
- Use sockets to manage such VLAN properties as the VLAN ID (VID) or user priority. For further information, see Example of Subnet-based VLAN Management on page 136.
- Use the vconfig shell command to manage the processing of stacked VLANs or to configure a provider edge node. For further information, see Configuring a Stacked VLAN on page 140.

If you are configuring a stacked VLAN, you cannot configure a subnet-based VLAN and a stacked VLAN on the same port.

MUX-L2 VLAN Management on page 134 If you enable MUX-L2 functionality when you build the VxWorks 7 Network Stack, you can use the l2config utility to access the layer 2 set functions supported by muxL2Ioctl( ).
Sample MUX-L2 Configuration on page 134 This example attaches the **fei1** port to MUX-L2.
Subnet Based VLAN Management on page 135 To support virtual local area network (VLAN) routing, FreeBSD uses the VLAN pseudo-interface to de-multiplex VLAN-tagged frames into logical VLAN network interfaces. The VxWorks 7 Network Stack adapts this technique to support subnet based VLAN configuration.
Example of Subnet-based VLAN Management on page 136 These examples show how to create virtual local area network (VLAN) pseudo-interfaces. The first examples rely on the compact interface naming style. The examples after that rely on the restrictive interface naming style.
Configuring a Stacked VLAN on page 140 You can display a variety of stacking virtual local area network (VLAN) parameters at run time, using the ifconfig shell command.
Debugging MUX-L2 Interaction on page 149

Previous topic: Building VxWorks with VLAN Support on page 131
Next topic: Controlling MUX Extensions for Layer 2 VLAN Support on page 152

4.1. MUX-L2 VLAN Management

If you enable MUX-L2 functionality when you build the VxWorks 7 Network Stack, you can use the l2config utility to access the layer 2 set functions supported by muxL2Ioctl( ).

The l2config utility does not give you access to the muxL2Ioctl( ) "get" functionality, which, because of its use of structures, is more suited to programmatic use. However, you can access much of the same information from the command line using muxL2Show( ) or muxL2VlanShow( ), which are described in their respective reference entries and in Debugging MUX-L2 Interaction on page 149.

For port-oriented configuration needs, l2config allows you to do the following:

- Attach/detach a port to/from MUX-L2.
- Set the default port VID (PVID).
- Set the default user priority.
- Set the acceptable ingress frame type.
- Set the ingress frame filter.

For configuration needs involving both the port and a virtual local area network (VLAN), l2config allows you to do the following:

- Join a port to a VLAN.
- Set the egress frame type for the VLAN.
- Leave a VLAN that a port joined previously.

For more information, see the examples below and the reference entry for l2config.

Parent topic: Managing a VLAN at Run Time on page 133

4.2. Sample MUX-L2 Configuration

This example attaches the **fei1** port to MUX-L2.

```
-> l2config "vlandev fei1 attach" value = 0 = 0x0
```

The following example enables the ingress frame filter for port **fei1**. It also sets the **fei1** ingress acceptable frame filter type to ADMIT_TAGGED_ONLY_FRAMES:

```
-> l2config "vlandev fei1 infilter on ingress admittag" value = 0 = 0x0
```

The following example joins the **fei1** port to the virtual local area network (VLAN) with VID 20 and configures the egress frame type for the VLAN to transmit VLAN-tagged frames only:

```
-> l2config "vlandev fei1 join vid 20 egress tagged" value = 0 = 0x0
```

The following example removes the **fei1** port from VLAN 20, the VLAN to which it was joined previously:

```
-> l2config "vlandev fei1 leave vid 20" value = 0 = 0x0
```

The following example shows how you can issue multiple command options at the same time, using l2config. The attach and join commands above can be combined into a single command:

```
-> l2config "vlandev fei1 attach join vid 20 egress tagged" value = 0 = 0x0
```


WNDRVR

Parent topic: Managing a VLAN at Run Time on page 133

4.3. Subnet Based VLAN Management

To support virtual local area network (VLAN) routing, FreeBSD uses the VLAN pseudo-interface to de-multiplex VLAN-tagged frames into logical VLAN network interfaces. The VxWorks 7 Network Stack adapts this technique to support subnet based VLAN configuration.

Each VLAN pseudo-interface can be created at run time by using the ifconfig( ) create command. For each pseudo-interface, call ifconfig to assign a VLAN, a parent interface, and a numeric VID. The parent interface must be a physical interface that is attached to the IP layer at the time you create the VLAN pseudo-interface.

A single parent interface can support multiple VLAN pseudo-interfaces provided that the pseudo-interfaces have different VIDs. The parent interface must be a member of the VLAN for the VID assigned to the VLAN pseudo-interface.

To configure the VLAN pseudo-interface, use the following three ifconfig options:

vlanInterfaceName create

Create the specified VLAN pseudo-interface named by *vlanInterfaceName*. *vlanInterfaceName* must start with the letters "vlan"; for example: **vlan**, **vlan10**, or **vlanPrivate**.

vlanInterfaceName **destroy**

Delete the specified VLAN pseudo-interface from the network stack.

vlanInterfaceName **vlan** *vlanID* **vlanif** *interfaceName* **vlanpri** *priority*

Set the VLAN ID to *vlanID*, associate the physical interface *interfaceName* with *vlanInterfaceName*, and assign the Class Of Service value *priority* to the VLAN tag for this VLAN pseudo-interface. *vlanID* is a 16-bit number between 1-4094 that is used to create an 802.1Q VLAN header for packets the stack sends from the VLAN pseudo-interface. *priority* is a three-bit value.

Packets transmitted through the VLAN interface will be diverted to the physical interface *interfaceName* with 802.1Q VLAN encapsulation. Packets with 802.1Q encapsulation received by the physical interface with the correct VLAN ID will be diverted to the associated VLAN pseudo-interface. The VLAN interface is assigned a copy of the physical interface's flags and Ethernet address. If the VLAN interface already has a physical interface associated with it, this command fails. To change the association to another physical interface, you must first clear the existing association.

Note that you must set **vlan** and **vlanif** at the same time.

Consequences of Changing the VID

You can change the parent interface and the VID only when the VLAN interface is down. For example, to change the parent interface to **fei1** and the VID for the VLAN pseudo-interface to **1234** on a created, configured, and up VLAN interface, type the following:

```
[vxWorks *] # ifconfig vlanLab down vlanif fei1 vlan 1234 up
```

Be aware that changing the VID for the VLAN pseudo-interface does not automatically remove the parent interface from the VLAN membership associated with the old VID. It also does not automatically add the parent interface to the member set specified by the new VID. Therefore, the parent interface must be a member of the VLAN specified by the new VID before the new VID can be assigned. The parent interface remains a member of the VLAN specified by the old VID unless the membership is explicitly removed.

Parent topic: Managing a VLAN at Run Time on page 133

4.4. Example of Subnet-based VLAN Management

These examples show how to create virtual local area network (VLAN) pseudo-interfaces. The first examples rely on the compact interface naming style. The examples after that rely on the restrictive interface naming style.

If you have built the network stack to support MUX-L2, when you create the VLAN pseudo-interface the network stack will implicitly attach the physical parent interface to MUX-L2 and will join the parent interface to the VLAN that you have configured for the VLAN pseudo-interface. Once the stack joins the port to MUX-L2, MUX-L2 enforces strict ingress and egress VLAN rules (as described in sections Enabling VLAN Support for a Port on page 152 and Enabling VLAN Support for a Port on page 152). When you attach the parent physical interface (port) to MUX-L2, you can manage the port by calling l2config( ) (for instance, to set the port ingress or egress properties).

Examples Using the Compact Creation API

The following example uses the compact interface naming style to create a VLAN pseudo interface **fei1.50** with IP address 190.0.2.234/24. It also specifies the parent interface **fei1** and VID 50 for the VLAN pseudo-interface. You must have already attached **fei1** to the network stack.

```
-> ifconfig "fei1.50 create inet 190.0.2.234/24"
value = 0 = 0x0
-> ifconfig "fei1.50"
fei1.50: flags=48043<UP,BROADCAST,RUNNING,MULTICAST,INET_UP> mtu 1496
 inet 190.0.2.234 netmask 0xffffff00 broadcast 190.0.2.255
 ether 00:08:c7:c9:24:76
 vlan: 50 user priority: 0 parent interface: fei1
value = 0 = 0x0
```

The following example destroys the **fei1.50** VLAN pseudo interface previously created.

```
-> ifconfig "fei1.50 destroy"
value = 0 = 0x0
```

The following example uses the compact interface naming style to create a VLAN pseudo interface, **fei0.20**, with IP address 190.0.4.234/24. It also specifies the parent interface **fei0**, VID 20, and user priority 5 for the VLAN pseudo-interface. You must have already attached **fei0** to the network stack.

```
-> ifconfig "fei0.20 create"
value = 0 = 0x0

-> ifconfig "fei0.20 190.0.4.234/24"
value = 0 = 0x0

-> ifconfig "fei0.20 vlanpri 5"
value = 0 = 0x0

-> ifconfig "fei0.20"
fei0.20: flags=48043<UP,BROADCAST,RUNNING,MULTICAST,INET_UP> mtu 1496
                inet 190.0.4.234 netmask 0xffffff00 broadcast 190.0.4.255
                ether 00:03:47:b0:d7:17
                vlan: 20 user priority: 5 parent interface: fei0
value = 0 = 0x0
```

Examples Using the Name-Restrictive Creation API

The following example creates a VLAN pseudo-interface, **vlan0**, with IP address 190.0.2.123/24, assigns the pseudo-interface with VID 20, and associates it with the parent interface **fei1**. You must have already attached **fei1** to the network stack.

```
-> ifconfig "vlan0 create"
value = 0 = 0x0

-> ifconfig "vlan0"
vlan0 Link type:Layer 2 virtual LAN HWaddr
00:01:02:03:04:10 Queue:none
 vlan: 20 parent: fei0
 inet 190.0.2.123 mask 255.255.255.0 broadcast 190.0.2.255
 inet 224.0.0.1 mask 240.0.0.0
 inet6 unicast FE80::201:2FF:FE03:410%vlan10
prefixlen 64 automatic
 inet6 unicast FE80::%vlan10 prefixlen 64 anycast
 inet6 multicast FF02::1%vlan10 prefixlen 16 automatic
 inet6 multicast FF02::1:FF03:410%vlan10 prefixlen 16
 inet6 multicast FF02::1:FF00:0%vlan10 prefixlen 16
 UP RUNNING SIMPLEX BROADCAST MULTICAST
 MTU:1496 metric:0 VR:0
 RX packets:0 mcast:0 errors:0 dropped:0
 TX packets:13 mcast:15 errors:0
 collisions:0 unsupported proto:0
 RX bytes:0 TX bytes:1138
-> ifconfig "vlan0 vlan 20 vlanif fei1"
value = 0 = 0x0

-> ifconfig "vlan0 190.0.2.123/24"
value = 0 = 0x0

-> ifconfig "vlan0"
vlan0: flags=48043<UP,BROADCAST,RUNNING,MULTICAST,INET_UP> mtu 1496
 inet 190.0.2.123 netmask 0xffffff00 broadcast 190.0.2.255
 ether 00:08:c7:c9:24:76
 vlan: 20 user priority: 0 parent interface: fei1
value = 0 = 0x0
```

The following example destroys the **vlan1** VLAN pseudo-interface previously created.

```
-> ifconfig "vlan0 destroy"
value = 0 = 0x0
```

Socket-based VLAN Management

The socket API includes a socket option, SO_VLAN, and a structure, **sovlan**, that carry all the information relevant to VLAN configuration. If you treat the VID and the user priority as socket-level configuration options, you can use these extensions by calling getsockopt( ) and setsockopt( ) and thus get or set VLAN membership information.

SO_VLAN can only be used for send operations.

The **sovlan** structure is defined as follows:

After an application creates a socket, it can call setsockopt( ) ( )to configure the VID and/or user priority for the socket. In order to transmit a VLAN-tagged or priority-tagged frame, the port/interface that the socket is bound to must already be attached to the

MUX-L2, as described previously. If a port transmits a VLAN-tagged frame, the port must also be a member of the VLAN for which the socket-based VLAN is configured.

```
struct sovlan
 {
 /*
 * If so_onff is set, the vlan id and/or user priority will be copied
 * to the socket structure and SO_VLAN so_option will be set. If so_onff
 * is not set, the SO_VLAN so_option for the socket will be cleared.
 */
 int vlan_onoff;  /* on/off option */

 /*
 * The priority_tagged boolean must be set to true if application using
 * socket-based vlan requires to egress 802.1P priority-tagged frame
 * (i.e. the value of vid is zero). Defaults to false. If set to true,
 * the value specified by the vid will be ignored.
 */

 BOOL priority_tagged;
 unsigned short vid; /* VLAN ID, valid vid: 1..4094 */
 unsigned short upriority; /* User Priority, valid priority: 0..7 */
 };
```

Setting User Priority for Transmitted Priority-Tagged Frames

The following code fragment is an example of how to call setsockopt( ) in order to configure the user priority for a socket and configure that socket to transmit priority-tagged frames.

```
struct sovlan vl;

/* set up the vlan_onoff to indicate that the VID and or User Priority
 * are valid */

vl.vlan_onoff = 1;
/*
 * Informs lower-layers (such as subnet-based VLAN) that the
 * information provided is for Priority-tagged frame and that lower-layers
 * must not alter the VLAN control information for VID configuration
 */

vl.priority_tagged = TRUE;


/* VID is not applicable for Priority-tagged frame */

vl.vid = 0;

/* Configure the Priority-tagged frame for User Priority with value 7 */

vl.upriority = 7;
if (setsockopt (s, SOL_SOCKET, SO_VLAN, (char *) &vl, sizeof (struct sovlan))
   < 0)
   printf( "setsockopt SO_VLAN for socket %d failed\n", s);
```

Setting User Priority for Transmitted VLAN-tagged Frames

The following code fragment is an example of how to call setsockopt( ) ( )in order to configure the user priority for the specified socket and to configure that socket to transmit VLAN-tagged frames.

```
struct sovlan vl;

/* setup vlan_onoff to indicate that VID and/or User Priority * are valid */

vl.vlan_onoff = 1;

/*
 * Informs lower-layers (such as Subnet-based VLAN) that the
 * information provided is for VLAN-tagged frame and that lower-layers should
 * alter the VLAN control information for VID if the VID is not specified
 */

vl.priority_tagged = FALSE;

/*
 * Specifies VID with value of 0 to allow lower-layers (such as Subnet-
 * based VLAN) to insert the appropriate VID to the VLAN
 * control information for the outgoing VLAN-tagged frame.
 */

vl.vid = 0;

/* Configure the VLAN-tagged frame for User Priority with value 3 */
vl.upriority = 3;

if (setsockopt (s, SOL_SOCKET, SO_VLAN, (char *)&vl, sizeof (struct sovlan))
 < 0)
 printf ("setsockopt SO_VLAN for socket %d failed\n", s);
```

Clearing the VLAN Configuration for the Socket

Consider the following code fragment calls setsockopt( ) to clear the VLAN configuration for a socket:

```
struct sovlan vl;

bzero ((char *) &vl, sizeof (struct sovlan));

/* reset all the VLAN control information for the socket */

vl.vlan_onoff = 0;

if (setsockopt (s, SOL_SOCKET, SO_VLAN, (char *) &vl, sizeof (struct sovlan))
 < 0)
 printf ("setsockopt SO_VLAN for socket %d failed\n", s);
```

Getting Configuration Information

Note: For the VLAN-tagged frame, if getsockopt( ) returns a VID value of 0, this implies that the application uses the socket-based VLAN, chooses not to configure the VID, and relies on the lower-layers (such as subnet-based VLAN) to insert the appropriate VID to the VLAN control information for the outgoing VLAN-tagged frame.

The following code fragment is an example of how to call getsockopt( ) ( )to retrieve the VLAN configuration for a socket.

```
struct sovlan vl;
int vsize = sizeof (struct sovlan);

bzero ((char *) &vl, sizeof (struct sovlan));
if (getsockopt (s, SOL_SOCKET, SO_VLAN, (char *) &vl, &vsize) < 0)
    printf ("getsockopt SO_VLAN for socket %d failed\n", s);
if (0 == vl.vlan_onoff)
    printf ("No VLAN control info for socket %d\n", s);
else
   {
   if (vl.priority_tagged)
   printf ("Socket %d Priority-Tagged User Priority %d\n", s,
            vl.upriority);
else
   printf ("Socket %d VLAN-Tagged VID %d User Priority %d\n", s, vl.vid,
            vl.upriority);
 }
```

Parent topic: Managing a VLAN at Run Time on page 133

4.5. Configuring a Stacked VLAN

You can display a variety of stacking virtual local area network (VLAN) parameters at run time, using the ifconfig shell command.

You can also set these parameters, using the vconfig shell command. These parameters include:

- Service VLAN tag protocol ID.
- VLAN stacking port type, such as a customer edge/network port on a provider edge node, provider network port on a service VLAN (S-VLAN) node, provider network port on a provider edge node.
- Port VID for each port on a provider edge node.
- Customer VIDs allowed on a customer edge/network port.
- Service VIDs mapped from C-VIDs, and S-VIDs on a provider network port.
- Ingress filtering rule on a provider edge node: admit all frames, admit tagged frames, admit priority-tagged and untagged frames. (Refer to IEEE Std 802.1Q for ingress rule definition.)
- Egress filtering rule on a provider edge node: tagged, untagged. (Refer to IEEE Std 802.1Q for egress rule definition.)
- Receiving VLAN frame priority to service VLAN priority mapping on a provider edge node.

You can also use vconfig to display the current service TPID value.

The following tutorials guide you through these specific use cases:

- Displaying the Current VLAN Parameters
- Configuring a Service TPID
- Configuring a Provider Edge Node—Use Case 1

- Configuring a Provider Edge Node—Use Case 1
- Configuring a Provider Edge Node—Use Case 3
- Configuring an S-VLAN Node

Note: For some of the commands in these tutorials, you must use the kernel shell in C-interpreter mode (with a **->** prompt). For others, you must use the shell in command-interpreter mode (with a **[vxWorks *] #** prompt). The tutorial will specify which mode you must use to run each command.

For a detailed explanation of the options and syntax for the vconfig shell command, see the reference entry at vconfig Command on page 158.

Displaying the Current VLAN Parameters

Use the ifconfig command with the **-s** flag to display the current VLAN parameters for the specified interface. For example:

```
-> ifconfig "-s eth1"
eth1 Link type:Ethernet HWaddr 00:0f:b7:10:64:65 Queue:none
         capabilities: TXCSUM VLAN_MTU
                 VLAN stacking type: customer edge/network port  pvid: 1
                 ingress rule: admit-all  egress rule: tagged
                 c-vid: 1  s-vid: 1forwarding to:  eth2
                                priority mapping:
                                         0 --> 0  1 --> 1  2 --> 2  3 --> 3
                                         4 --> 4  5 --> 5  6 --> 6  7 --> 7
                 c-vid: 10 s-vid: 20forwarding to:  eth2
                               priority mapping:
                                         0 --> 0  1 --> 1  2 --> 2 3--> 3
                                         4 --> 4  5 --> 5  6 --> 6 7 --> 7
                 c-vid: 20  s-vid: 30forwarding to:  eth2
                                priority mapping:
                                         0 --> 0  1 --> 1  2 --> 2 3 --> 3
                                         4 --> 4 5 --> 5   6 --> 6 7 --> 7
                c-vid: 30  s-vid: 20forwarding to:  eth2
                                priority mapping:
                                         0 --> 0  1 --> 1  2 --> 2 3 --> 3
                                         4 --> 4  5 --> 5  6 --> 6  7 --> 7
                inet 100.1.1.2  mask 255.255.255.0  broadcast 100.1.1.255
                inet6 unicast fe80::20f:b7ff:fe10:6465%eth1  prefixlen
 64  automatic
                         UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
                         MTU:1500  metric:1  VR:0  ifindex:3
                         RX packets:0 mcast:0 errors:0 dropped:0
                         TX packets:9 mcast:8 errors:0
                         collisions:0 unsupported proto:0
                         RX bytes:0
                 TX bytes:690
                         HW Forwarder
                         Tx Packets: 9, Bytes: 690
                         Rx Packets: 0, Bytes: 0
                               Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                               Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

Configuring a Provider Edge Node Use Case 1 on page 142 Various configurations are suitable for a provider edge node, depending on the system requirements.

WNDRVR

Configuring a Provider Edge Node Use Case 2 on page 145 Subscriber traffic may be grouped by type into service provider VLANs (S-VLANs). For example, video data may be on S-VLAN1, voice on S-VLAN2, and data on S-VLAN3. Not all subscribers may receive all types of service, so the edge provider must be able to identify the traffic types and map them to the appropriate S-VLANs.

Configuring a Provider Edge Node Use Case 3 on page 145 This scenario involves multicast virtual local area network (M-VLAN) traffic, which is common in applications such as Internet Protocol television (IPTV).

Parent topic: Managing a VLAN at Run Time on page 133

Configuring a Provider Edge Node Use Case 1

Various configurations are suitable for a provider edge node, depending on the system requirements.

About This Task

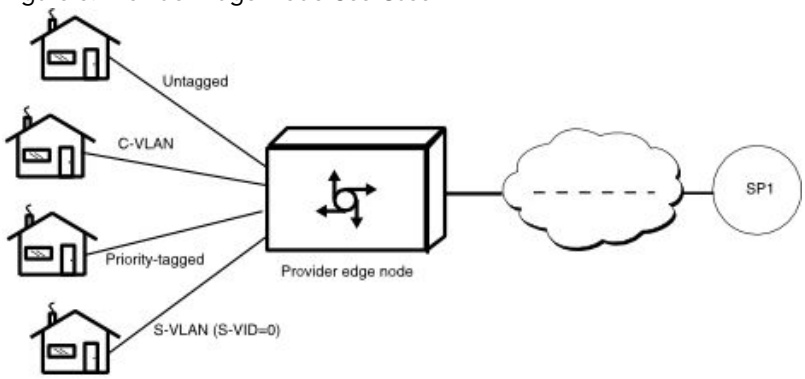
In one scenario, subscriber services may or may not be VLA aware. The provider edge node must be capable of mapping the following frame types:

- priority-tagged VLAN frames (with no C-VID)
- customer VLAN (C-VLAN) tagged frames
- service VLAN (S-VLAN) tagged frames (S-VID = 0)
- untagged frames to a provider VLAN ID (S-VLAN)

The node must also support flow in the opposite direction.

The figure below illustrates this configuration.

Figure 6. Provider Edge Node Use Case 1



To implement this configuration, use the vconfig command to create a mapping between Customer VID (C-VID) and Service VID (S-VID). When a frame does not carry a C-VID, the default port VID is used. You must also create the mapping from the default VID to an S-VID.

To configure the provider edge node, follow this procedure:

Procedure

1. Configure two Ethernet interfaces:

```
->  ipAttach 1,"eth"
->  cmd ifconfig eth1 100.1.1.2/24 up
->  ipAttach 2,"eth"
->  ifconfig eth2 up
```

2. Switch the shell to command-interpreter mode:

```
-> cmd
```

3. Configure provider network port **eth2** with S_VID 20, 30:

```
[vxWorks *] # vconfig eth2 vlantype 2 svid 20

[vxWorks *] # vconfig eth2 vlantype 2 svid 30
```

4. Configure customer edge/network port **eth1** with the following mappings:

   C-VID 10: S-VID 20

   C-VID 30: S-VID 20

   C-VID 20: S-VID 30

   PVID 1: S-VID 1

   To establish these mappings, issue the following commands:

```
[vxWorks *] # vconfig eth1 vlantype 1 cvid 10 svid 20

[vxWorks *] # vconfig eth1 vlantype 1 cvid 30 svid 20

[vxWorks *] # vconfig eth1 vlantype 1 cvid 20 svid 30

[vxWorks *] # vconfig eth1 vlantype 1 cvid 1 svid 1
```

5. Display the **eth1** configuration:

```
[vxWorks *] # ifconfig -s eth1
eth1 Link type:Ethernet HWaddr 00:0f:b7:10:64:65 Queue:none
                capabilities: TXCSUM VLAN_MTU
                        VLAN stacking type: customer edge/network  pvid: 1
                ingress rule: admit-all
               egress rule: tagged
                        c-vid: 1 s-vid: 1forwarding to: eth2
                                priority mapping:
                                        0 --> 0 1 --> 1 2 --> 2 3 --> 3
                                        4 --> 4 5 --> 5 6 --> 6 7 --> 7
                        c-vid: 10 s-vid: 20forwarding to: eth2
                                priority mapping:
                                        0 --> 0 1 --> 1 2 --> 2 3 --> 3
                                        4 --> 4 5 --> 5 6 --> 6 7 --> 7
                        c-vid: 20 s-vid: 30forwarding to: eth2
                                priority mapping:
                                        0 --> 0 1 --> 1 2 --> 2 3 --> 3
                                        4 --> 4 5 --> 5 6 --> 6 7 --> 7
                        c-vid: 30 s-vid: 20forwarding to: eth2
                                priority mapping:
```

```
                                        0 --> 0 1 --> 1 2 --> 2 3 --> 3
                                        4 --> 4 5 --> 5 6 --> 6 7 --> 7
                        inet 100.1.1.2 mask 255.255.255.0 broadcast 100.1.1.255
                        inet6 unicast fe80::20f:b7ff:fe10:6465%eth1 prefixlen
64 automatic
                        UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
                        MTU:1500 metric:1 VR:0 ifindex:3
                        RX packets:0 mcast:0 errors:0 dropped:0
                        TX packets:9 mcast:8 errors:0
                        collisions:0 unsupported proto:0
                        RX bytes:0 TX bytes:690
                        HW Forwarder
                        Tx Packets: 9, Bytes: 690
                        Rx Packets: 0, Bytes: 0
                        Frames size: 64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                        Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

6. Display the **eth2** configuration:

```
[vxWorks *] # ifconfig  -s eth2
eth2  Link type:Ethernet  HWaddr 00:0f:b7:10:64:66  Queue:none
                capabilities: TXCSUM VLAN_MTU
                        VLAN stacking type: provider network port on provide edge
                                node pvid: 1
                                ingress rule: admit-all egress rule: tagged
                                s-vid: 20
                                c-vid: 10forwarding to: eth1
                                c-vid: 30forwarding to: eth1
                                s-vid: 30
                                c-vid: 20forwarding to: eth1
                                s-vid: 1
                                c-vid: 1forwarding to: eth1
                        inet6 unicast fe80::20f:b7ff:fe10:6466%eth2  prefixlen
                                64 tentative automatic
                        UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
                        MTU:1500  metric:1  VR:0  ifindex:4
                        RX packets:0 mcast:0 errors:0 dropped:0
                        TX packets:0 mcast:0 errors:0
                        collisions:0 unsupported proto:0
                        RX bytes:0  TX bytes:0
                        HW Forwarder
                        Tx Packets: 0, Bytes: 0
                                                Rx Packets: 0, Bytes: 0
                                        Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0,
1519: 0
                                        Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

A received VLAN frame is admitted by the VLAN ingress rule. Depending on the VLAN egress filtering rule, a C-VLAN tag is kept, added, removed. Depending upon the egress VLAN port type, when a S-VLAN tag is added, or removed. The destination and source Media Access Control (MAC) addresses are kept the same as in the original frame.

Parent topic: Configuring a Stacked VLAN on page 140

Configuring a Provider Edge Node Use Case 2

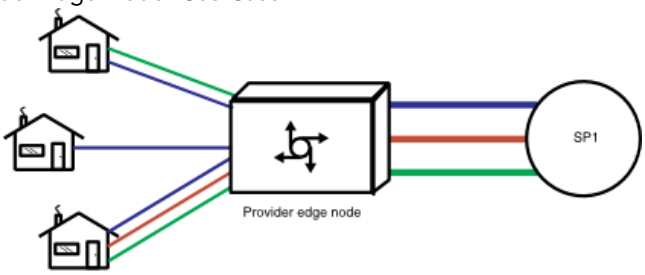
Subscriber traffic may be grouped by type into service provider VLANs (S-VLANs). For example, video data may be on S-VLAN1, voice on S-VLAN2, and data on S-VLAN3. Not all subscribers may receive all types of service, so the edge provider must be able to identify the traffic types and map them to the appropriate S-VLANs.

The traffic types are differentiated by customer VLANs (C-VLANs) with a different C-VID inserted by the subscriber when a frame is sent out. In this scenario, provider equipment on the customer premises maps different traffic to different C-VIDs assigned by the provider. For example, video data is sent out on C-VLAN1, voice on C-VLAN2, and data on C-VLAN3. The provider edge node does not examine the header information in layers 3-7 to determine the traffic types. Instead, you must create mappings between C-VIDs and S-VIDs, using the vconfig command.

Figure 1 on page 145 illustrates this configuration.

The configuration for this use case is similar to the one described in Configuring a Provider Edge Node Use Case 1 on page 142, except the traffic types are differentiated by C-VLANs, with different C-VID inserted by the subscriber when a frame is sent out.

Figure 7. Provider Edge Node--Use Case 2



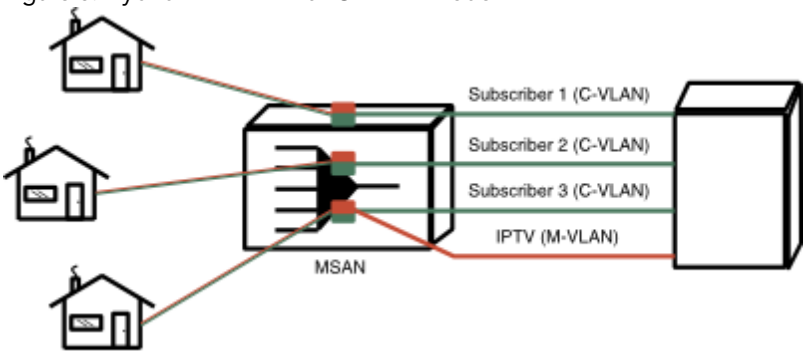
Parent topic: Configuring a Stacked VLAN on page 140

Configuring a Provider Edge Node Use Case 3

This scenario involves multicast virtual local area network (M-VLAN) traffic, which is common in applications such as Internet Protocol television (IPTV).

This architecture combines the strength of a customer VLAN (C-VLAN) with an M-VLAN in a node referred to as a multiservice access node (MSAN). The MSAN provides a single policy enforcement point and efficient multicast delivery. Figure 1 on page 145 illustrates this configuration.

Figure 8. Hybrid M-VLAN with C-VLAN Model



This model utilizes multiple VLANs as follows:

- A subscriber-dedicated C-VLAN carries unicast traffic such as Internet access and voice over IP (VoIP) between the access node and the broadband services router (BSR).

- An S-VLAN carries broadcast television traffic to each MSAN. Because the S-VLAN carries IPTV traffic exclusively, it is often referred to as an M-VLAN.

This model assumes that each subscriber operates its own C-VLAN, which carries its own unicast traffic (Internet access, VoIP, and VoD). In addition, one VLAN carries multicast traffic to all subscribers. The network supports all of these VLANs--each C-VLAN plus the VLAN dedicated to multicast traffic.

When a multicast VLAN-tagged frame arrives at a VLAN stacking port, by default it will be forwarded to all ports that have same VLAN membership at an S-VLAN node, or to the configured VLAN forwarding table between the C-VLAN and S-VLAN on a provider edge node. If multicast optimization is selected, a multicast VLAN-tagged frame is forwarded to all ports that have same VLAN membership and have multicast listeners on the link. The system uses manual configuration to determine VLAN membership. It determines multicast membership by listening to Internet Group Management Protocol (IGMP)/Multicast Listener Discovery (MLD) multicast protocol messages.

Configuring an S-VLAN Node on page 146IEEE Std 802.1ad describes a use case for an service provider VLAN (S-VLAN).
Managing VLAN Priority on page 148A virtual local area network (VLAN) ID as shown in is 12 bits. Its range is from 0 to 4095, with two reserved values.

Parent topic: Configuring a Stacked VLAN on page 140

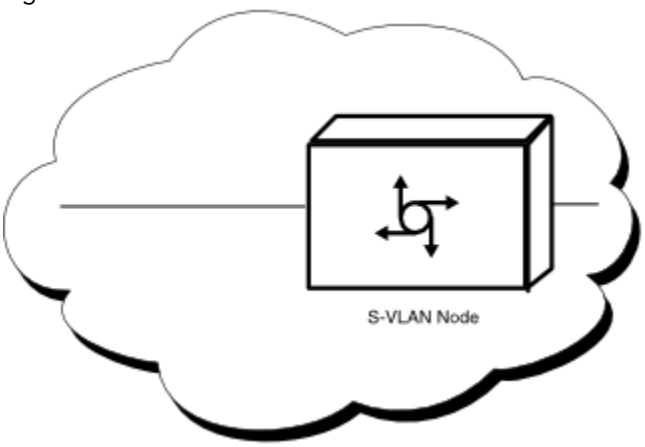
Configuring an S-VLAN Node

IEEE Std 802.1ad describes a use case for an service provider VLAN (S-VLAN).

About This Task

The node is located in the provider network cloud and transfers virtual local area network (VLAN) traffic across the provider network between provider edge nodes. Figure 1 on page 146 illustrates this configuration.

Figure 9. S-VLAN Node



This node receives S-VLAN-tagged frames. The provider network ports are manually configured with S-VID according to the port VLAN membership. The VLAN forwarding table is established for the ports with the same VLAN membership. The S-VLAN node forwards traffic received on one port to other ports according to the S-VID and the destination address.

To configure an S-VLAN node for this use case, follow this procedure:

Procedure

1. Configure two Ethernet interfaces:

WNDRVR

```
->  ipAttach1,"eth"
->  cmd ifconfig eth1 up
->  ipAttach2,"eth"
->  cmd ifconfig eth2 up
-> cmd
```

2. Configure provider network port **eth1** with S_VID 20:

```
[vxWorks *] # vconfig eth1 vlantype 3 svid 20
```

3. Configure provider network port **eth2** with S_VID 20:

```
[vxWorks *] # vconfig eth2 vlantype 3 svid 20
```

4. Display the **eth1** configuration:

```
[vxWorks *] # ifconfig -s eth1
eth1 Link type:Ethernet HWaddr 00:0f:b7:10:64:65 Queue:none
                        capabilities: TXCSUM VLAN_MTU
                        VLAN stacking type: provider network port on S-VLAN node
                s-vid: 20 forwarding to: eth2
                        inet6 unicast fe80::20f:b7ff:fe10:6465%eth1 prefixlen 64 automatic
                        UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
                        MTU:1500 metric:1 VR:0 ifindex:3
                        RX packets:0 mcast:0 errors:0 dropped:0
                        TX packets:9 mcast:8 errors:0
                        collisions:0 unsupported proto:0
                        RX bytes:0 TX bytes:690
                        HW Forwarder
                        Tx Packets: 9, Bytes: 690
                        Rx Packets: 0, Bytes: 0
                        Frames size 64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                        Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

5. Display the **eth2** configuration:

```
[vxWorks *] # ifconfig -s eth2
eth2  Link type:Ethernet  HWaddr 00:0f:b7:10:64:66  Queue:none
                        capabilities: TXCSUM VLAN_MTU
                        VLAN stacking type: provider network port on S-VLAN node
                        s-vid: 20 forwarding to: eth1
                        inet6 unicast fe80::20f:b7ff:fe10:6466%eth2 prefixlen 64 tentative auto
matic
                        UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
                        MTU:1500 metric:1 VR:0 ifindex:4
                        RX packets:0 mcast:0 errors:0 dropped:0
                        TX packets:0 mcast:0 errors:0
                        collisions:0 unsupported proto:0
                        RX bytes:0 TX bytes:0
                        HW Forwarder
                        Tx Packets: 0, Bytes: 0
                        Rx Packets: 0, Bytes: 0
                        Frames size: 64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                        Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

Parent topic: Configuring a Provider Edge Node Use Case 3 on page 145

Managing VLAN Priority

A virtual local area network (VLAN) ID as shown in is 12 bits. Its range is from 0 to 4095, with two reserved values.

About This Task

VLAN reserved values:

- 0 indicates a priority-tagged frame,
- 4095 is reserved but not currently used

No matter what the virtual local area network ID (VID) value is, a network user can set the user priority bits in a VLAN frame to indicate the priority of the frame. A service provider's network administrator can configure a priority regeneration table on a port to remap the user priority carried in a VLAN tag and queue a frame at different transmission queues according to different priorities.

A network administrator can remap priorities at the provider edge node by creating an entry in the priority regeneration table for each configured C-VID. The default priority mapping is directly mapped --that is, incoming priority 1 maps to outgoing network priority 1, 2 maps to 2, etc. You can use the vconfig shell command to change the default mapping. Once the customer priority is mapped to a network priority, the system can assign outgoing frames to different egress transmission queues, according to the priority of the frame.

The Wind River Network Stack only supports priority mapping for egress transmission queues.

The VLAN tag carries three user priority bits to indicate the priority of the frame . On a customer edge/network port on a provider edge node, these customer priority bits are remapped to different network priorities based on the priority regeneration table configured on the reception port. The remapped network priority bits are inserted and carried in the S-VLAN tag. The default priority remapping is direct mapping -that is, customer priority 1 maps to network priority 1, customer priority 2 maps to network priority 2, etc. The configuration command **vconfig** is provided to change the default priority mapping. The priority remapping table is per port per C-VID, so the configuration is also based on per port per C-VID.

To configure priority remapping on a customer edge/network port, follow this procedure:

Procedure

1. Configure an Ethernet interface:

```
-> ipAttach1,"eth"
-> ifconfig eth1 100.1.1.2/24 up
-> cmd
```

2. Configure the customer edge/network port **eth1** with the following mapping:

   C-VID 10: S-VID 20

```
[vxWorks *] # vconfig eth1 vlantype 1 cvid 10 svid 20
```

3. Display the default priority remapping table:

```
[vxWorks *] # ifconfig -s eth1
eth1    Link type:Ethernet  HWaddr 00:0f:b7:10:64:65 Queue:none
                        capabilities: TXCSUM VLAN_MTU
                        VLAN stacking type: customer edge/network port     pvid: 1
                        ingress rule: admit-all  egress rule: tagged
                        c-vid: 10    s-vid: 20
                        priority mapping:
```

```
                                0 --> 0   1 --> 1   2 --> 2   3 --> 3
                                4 --> 4   5 --> 5   6 --> 6   7 --> 7
                        inet 100.1.1.2  mask 255.255.255.0  broadcast  100.1.1.255
                        inet6 unicast fe80::20f:b7ff:fe10:6465%eth1 prefixlen 64  automatic
                        UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
                        MTU:1500  metric:1  VR:0  ifindex:3
                        RX packets:0 mcast:0 errors:0 dropped:0
                        TX packets:9 mcast:8 errors:0
                        collisions:0 unsupported proto:0
                        RX bytes:0  TX bytes:690
                        HW Forwarder
                        Tx Packets: 9, Bytes: 690
                        Rx Packets: 0, Bytes: 0
                        Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                        Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

4. Set priority remapping 3 to 4 for C-VID 10 on eth1:

```
[vxWorks *] # vconfig eth1 cvid 10 inpri 3 spri 4
```

5. Redisplay the priority remapping table to confirm the remapping:

```
[vxWorks *] #ifconfig -s eth1
eth1 Link type:Ethernet HWaddr 00:0f:b7:10:64:65 Queue:none
                capabilities: TXCSUM VLAN_MTU
                VLAN stacking type: customer edge/network port
          pvid: 1
                ingress rule: admit-all  egress rule: tagged
                c-vid: 10
          s-vid: 20
                priority mapping:
                        0 --> 0 1 --> 1 2 --> 2 3 --> 4
                        4 --> 4 5 --> 5 6 --> 6 7 --> 7
                inet 100.1.1.2 mask 255.255.255.0 broadcast 100.1.1.255
                inet6 unicast fe80::20f:b7ff:fe10:6465%eth1 prefixlen 64 automatic
                UP RUNNING SIMPLEX BROADCAST MULTICAST  PROMISC
                MTU:1500 metric:1 VR:0 ifindex:3
                RX packets:0 mcast:0 errors:0 dropped:0
                TX packets:9 mcast:8 errors:0
                collisions:0 unsupported proto:0
                RX bytes:0 TX bytes:690
                HW Forwarder
                Tx Packets: 9, Bytes: 690
                Rx Packets: 0, Bytes: 0
                Frames size: 64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

Parent topic: Configuring a Provider Edge Node Use Case 3 on page 145

4.6. Debugging MUX-L2 Interaction

For debugging and diagnostic purposes, the MUX-L2 provides muxL2Show( ), **muxL2StatShow( )**, **muxL2VlanShow( )**, and **muxL2VlanStatShow( )**.

muxL2Show( )

Call muxL2Show( ) to display the configuration of ports registered with the MUX-L2.

```
-> muxL2Show
max number of physical ports: 16
max number of vlans device supports: 16
number of vlans configured in device: 3
number of ports attached to MUX-L2: 2

Device: <fei> Unit: <1> L2 Port Object: 0x10cc040 endObjId: 1
Port VID (PVID): 1 Port User Priority: 0
Port ingress filter: TRUE
Port ingress acceptable frame filter type: Admit vlan-tagged frames only
Hardware supports vlan tagging: FALSE
Hardware supports vlan mtu: FALSE
Number of vid configured for the port: 2
Port VLAN membership:
 VID 1 Egress: untagged-tagged
 VID 20 Egress: vlan-tagged

Device: <fei> Unit: <0> L2 Port Object: 0x10e40a0 endObjId: 2
Port VID (PVID): 1 Port User Priority: 0
Port ingress filter: TRUE
Port ingress acceptable frame filter type: Admit All Frames
Hardware supports vlan tagging: FALSE
Hardware supports vlan mtu: FALSE
Number of vid configured for the port: 2
Port VLAN membership:
 VID 1  Egress: untagged-tagged
 VID 100 Egress: vlan-tagged
value = 1 = 0x1
```

muxL2VlanShow( )

Calls muxL2VlanShow( ) to display the VLAN configurations maintained by the MUX-L2 on a per-VLAN basis.

```
-> muxL2VlanShow

VLAN  1:  Number  of  Members:  2  Egress  Untagged:  2
VLAN  20: Number  of  Members:  1  Egress  Untagged:  0
VLAN 100: Number  of  Members:  1  Egress  Untagged:  0

Port to device name mapping:
  Port  1  ->  fei1
  Port  2  ->  fei0

VLAN Membership information:
(Legend: 'M' = Member '-' = Unspecified)
  VLAN  1   :  MM--------------
  VLAN  20  :  M---------------
  VLAN  100 :  -M--------------

VLAN Egress Frame Type:
(Legend: 'T' = Vlan-Tagged 'U' = Untagged '-' = Unspecified)
```

```
  VLAN  1  :  UU--------------
  VLAN  20 :  T---------------
  VLAN 100 :  -T--------------

value  =  1  =  0x1
```

muxL2StatShow( ) and muxL2VlanStatShow( )

The MUX-L2 also maintains various VLAN statistics on a per-port per-VLAN basis. These statistics are disabled by default for performance reasons. To include these statistics, build MUX-L2 with MUX_L2_VLAN_STATS defined. If VLAN statistics are included, muxL2VlanStatShow( ) and **muxL2StatShow( )** can be used to monitor the traffic on a per-port per-VLAN basis.

```
-> muxL2StatShow
fei1 Port Statistics:
  Number of received frames discarded due to non-vlan
  reasons (i.e. Discard Ingress Filtering): 9
  Number of egress frames discarded due to Egress
  Rules violation: 0

  VLAN 1 staticstics:
  Number of frames received: 0
  Number of frames transmitted: 0
  Number of received frames discarded: 0

  VLAN  20  staticstics:
  Number of frames received: 166
  Number of frames transmitted: 226
  Number of received frames discarded: 0

fei0  Port  Statistics:
  Number of received frames discarded due to non-vlan
  reasons (i.e. Discard Ingress Filtering): 0
  Number of egress frames discarded due to Egress
  Rules violation: 0

  VLAN  1  staticstics:
  Number of frames received: 0
  Number of frames transmitted: 0
  Number of received frames discarded: 0

  VLAN  100  staticstics:
  Number of frames received: 393
  Number of frames transmitted: 486
  Number of received frames discarded: 0

value = 0 = 0x0

-> muxL2VlanStatShow
fei1  VLAN  1  staticstics
  Number of frames received: 0
  Number of frames transmitted: 0
  Number of received frames discarded: 0

fei0  VLAN  1  staticstics
  Number of frames received:
  Number of frames transmitted: 0
```

```
  Number of received frames discarded: 0

fei1  VLAN  20  staticstics
  Number of frames received: 166
  Number of frames transmitted: 226
  Number of received frames discarded: 0

fei0  VLAN  100  staticstics
  Number of frames received: 393
  Number of frames transmitted: 486
  Number of received frames discarded: 0

value = 0 = 0x0
```

Parent topic: Managing a VLAN at Run Time on page 133

5. Controlling MUX Extensions for Layer 2 VLAN Support

How to programmatically control MUX-L2 virtual local area network (VLAN) support if you have built your network stack with MUX-L2 interoperability.

Overview of MUX-L2 VLAN Management

The MUX-L2 extensions allow you to manage the VLAN membership for a VxWorks target. These extensions support the following 802.1Q characteristics:

- VLAN classification of untagged, priority-tagged, or VLAN tagged ingress frames.
- Port-based VLAN classification as the default ingress rule (that is, all untagged and priority-tagged frames that a port receives are classified as belonging to the VLAN whose VID is associated with that port).
- Tagging of egress frames as VLAN-tagged, priority tagged, or untagged frames on a per-port basis.
- Enabling a port to be a member of multiple VLANs.
- Ingress Filter and Ingress Acceptable Frame Type configuration on a per-port basis.
- Ingress Filter configuration on a per-port basis.
- Ingress Acceptable Frame Type configuration on a per-port basis.
- The ability to send untagged frames for some VLANs and VLAN-tagged frames for others on a per-port basis.
- The assignment of all VLAN-enabled ports to the default Port VLAN ID (PVID) of 1. The PVID value of a port is configurable.

You can access and control this functionality through the muxL2( )... ( ) functions, through a socket interface, or with a VLAN pseudo-interface.

Enabling VLAN Support for a Port on page 152 For an END device loaded to MUX, call the muxL2PortAttach( ) function to enable VLAN support for the port.

Previous topic: Managing a VLAN at Run Time on page 133
Next topic: How VLAN Works on page 155

5.1. Enabling VLAN Support for a Port

For an END device loaded to MUX, call the muxL2PortAttach( ) function to enable VLAN support for the port.

WNDRVR

Note: The muxL2PortAttach( ) function assumes an Ethernet device. If you are using a non-Ethernet device, call muxL2PortAltAttach( ) instead.

The muxL2PortAttach( ) function prepares the port for VLAN support as follows:

- It joins the port to the default VLAN with a VID of 1, according to the 802.1Q requirement. It also configures the port to transmit untagged frames on the default VLAN. You can change the egress tagging state for the default Port VLAN ID (PVID) by calling muxL2Ioctl( ) with the MUXL2IOCSPORTVLAN control command.
- It initializes the port-specific attributes with the following defaults:
- Default Port User priority: 0
- Ingress Acceptable Frame Filter Type: admits all frame types
- Ingress Filter: False
- It queries the hardware for its VLAN capabilities.

A driver that supports hardware VLAN tagging must indicate this by setting flags in the **cap_available** member of the **END_CAPABILITIES** structure the driver returns in response to a **EIOCGIFCAP** message to its *x* Ioctl( ) function. Those flags are as follows:

- **IPCOM_IF_DRV_CAP_VLAN_MTU**- indicates that the driver can handle slightly larger than normal frames (that is, frames with a VLAN tag). This notifies the MUX-L2 that it can leave the MTU for the port at the normal setting. If the **IPCOM_IF_DRV_CAP_VLAN_MTU** flag is not set and software VLAN-tagging is required, the MUX-L2 decreases the hardware MTU by 4-bytes.
- **IPCOM_IF_DRV_CAP_VLAN_HWTAGGING_TX** - indicates that the driver can insert the VLAN tag into a frame on egress, by using the information that it reads from the **pkt->link_cookie** field in host-byte order.
- **IPCOM_IF_DRV_CAP_VLAN_HWTAGGING_RX** - indicates that the driver can strip the VLAN tag from a received frame and store that tag information in the **pkt->link_cookie** field in host-byte order.
- It determines which type of Ethernet address format the device can support (Ethernet Type 2 encapsulation or 802.3 style length encapsulation). This information is required when the MUX-L2 assembles an Ethernet header for the egress frame.
- It saves the address of the original END driver's **pFuncTable** function table. The MUX-L2 will restore the original driver's function table during muxL2PortDetach( ).
- It replaces the driver's registered *x* PacketDataGet( )with muxL2IngressClassify( ), and the *x* FormAddress( ) function pointer with muxL2EgressClassify( ).

Note: As an alternative to muxL2PortAttach( ), you can call muxL2Ioctl(( ) using the **MUXL2IOCSPORTATTACH** control command.

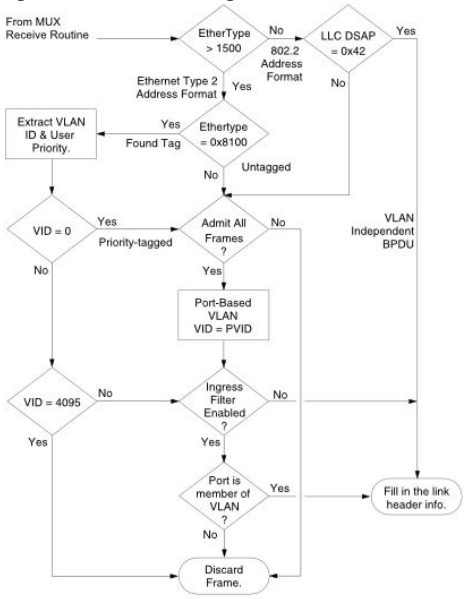
Disabling VLAN Support for a Port

To disable VLAN support for a port, call muxL2PortDetach(( ). This function detaches the port from the MUX-L2, removes the port from all the VLAN memberships it has joined, and restores the original driver's function table. If the port is removed from the MUX, muxDevUnload( ) calls muxL2PortDetach( ) as well. As an alternative to muxL2PortDetach( ), you can call muxL2Ioctl( ) using the **MUXL2IOCSPORTDETACH** control command.

MUX-L2 Ingress Rules

When a frame arrives on a port, the driver's ISR schedules the frame processing work to **tNet0** or another task. The MUX receive function would normally schedule a call to the driver's *x* PacketDataGet( ) to separate the address information and data in the frame. However, for a VLAN-enabled port, the MUX receive function schedules a call to muxL2IngressClassify( ) to filter the received frame according to its VLAN header tag. The figure below shows how the ingress filter handles an incoming frame.

Figure 10. MUX-L2 Ingress Rules



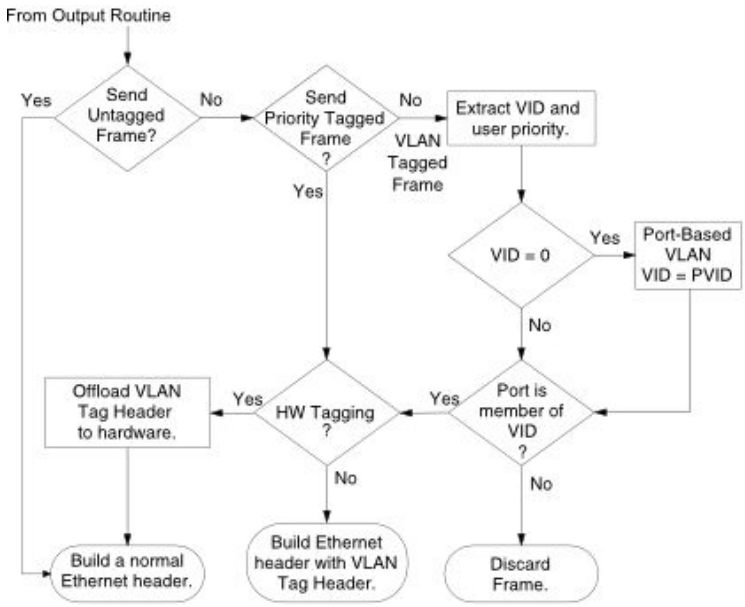
MUX-L2 Egress Rules

When the MUX needs to transmit a frame, it normally calls the driver's *x* FormAddress( )function to create and prepend a link-layer-appropriate frame header to the **M_BLK** chain containing outgoing data. However, for packets transmitted over a VLAN-enabled port, 802.1Q requires some additional preprocessing.

In addition, 802.1Q requires that a port transmits only VLAN-tagged frames or untagged frames but never transmits using both formats for the same VLAN. To support the egress tagging decision on a per-port per-VLAN basis, the MUX-L2 keeps track of the per-port egress tagging configuration for each VLAN.

To evaluate an outgoing frame in accordance with this information, the MUX calls muxL2EgressClassify( ) to determine whether the outgoing frame should be tagged or untagged, and then to build the link-layer header based on the tagging decision. The figure below shows how the egress filter handles an outgoing frame.

Figure 11. MUX-L2 Egress Rules



Accessing the MUX L2 Control Routines

Call muxL2Ioctl( ) to access the MUX-L2 control functionality. These control functions let you do such things as retrieve port information and set the ingress frame filter type. For more information, see the muxL2Ioctl( ) reference entry.

Parent topic: Controlling MUX Extensions for Layer 2 VLAN Support on page 152

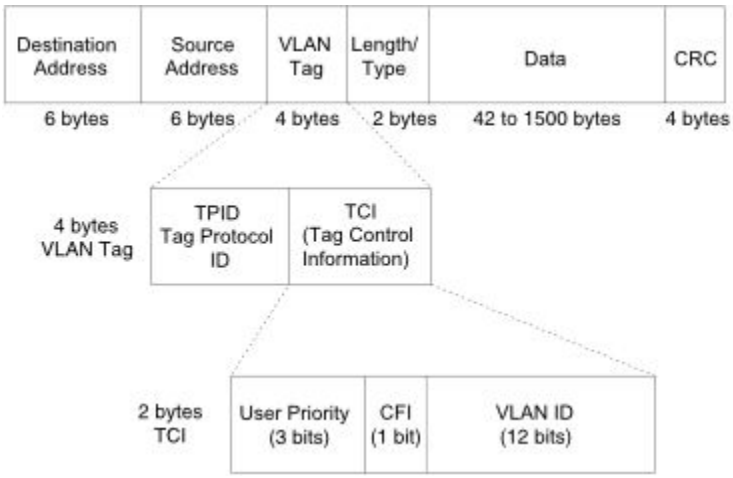
6. How VLAN Works

A VLAN represents a virtual local area network, which groups hosts with a common set of requirements that communicate as if they were attached to the broadcast domain regardless of their physical location. A VLAN has the same attributes as a physical LAN, but it allows for end stations to be grouped together even if they are not located on the same network switch. Network reconfiguration can be done through software instead of physically relocating devices.

The VLAN tag header is as shown in Figure 1 on page 155.

The 802.1Q tag is a four-byte field after the six-byte Source Address field and before the two-byte length/type field in the Ethernet header. An 802.1Q VLAN tagging implementation indicates that a frame is tagged by setting its Type field to the VLAN Identifier Protocol (0x8100). This means the next two bytes contain tag control information.

Figure 12. VLAN Tag Header Format on Ethernet

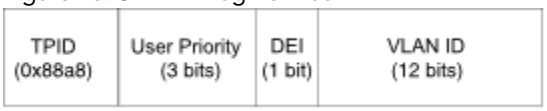


The two-byte tag control information consists of a 3-bit priority (0-7) value, a canonical format indicator (CFI) field (0 for Ethernet), and a 12-bit VLAN identifier (VID). The 12-bit VID field can take any value from 0 to 4095, but two of these values have special meanings according to the 802.1Q specification: The value of all ones (0xFFF) is reserved but currently unused; the value of all zeros (0x000) indicates a that the frame is priority-tagged and that no VID is present in the frame.

Stacked VLAN

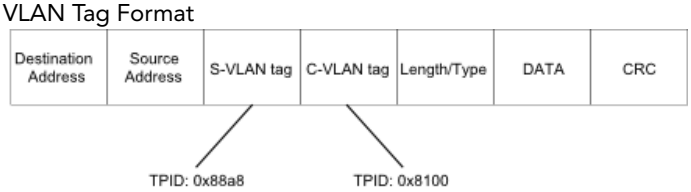
IEEE Std 802.1ad amends Std 802.1Q by inserting another service VLAN tag before the customer VLAN. The service VLAN tag is added to provide network service providers the flexibility to configure and administer the service network independently of the customer networks. Service providers can configure their networks without requiring that customers coordinate or change the VLAN IDs they use and without limiting the total number of VIDs across all customers to 4094, resulting in increased service transparency to the customer. The new VLAN tag introduced by IEEE Std 802.1ad has a different TPID value of 0x88a8. It also has a slightly different format from the C-VLAN tag, as shown in the figure below. The new element in this tag is the drop eligible indicator (DEI) bit.

Figure 13. S-VLAN Tag Format



IEEE Std 802.1ad defines at most two VLAN tags in a packet. To save the network bandwidth, IEEE 802.1ad allows a packet to only carry one S-VLAN tag without a C-VLAN tag. Since the S-VLAN tag has a different tag protocol identifier (TPID) value than the C-VLAN tag, there is no ambiguity about which VLAN tag it is if a packet only carries one VLAN tag. The figure below illustrates the format of a packet carrying two VLAN tags.

Figure 14. VLAN Tag Format



Previous topic: Controlling MUX Extensions for Layer 2 VLAN Support on page 152
Next topic: VLAN: The Wind River Implementation on page 157

7. VLAN: The Wind River Implementation

How to implement the virtual local area network (VLAN).

VLAN tagging is part of the network stack. You can access it by any of the following methods:

- muxL2( )... ( ) functions (if you have built the stack with MUX-L2 support)
- extensions to the socket interface
- a pseudo-interface with which you can manage the VLAN as a subnet
- a shell command by which you can configure VLAN nodes, map VLAN tags, and map priorities

Note: Building a boot ROM with a VLAN is not supported.

About the 802.1Q VLAN Tag Header

The Wind River VLAN implementation supports the following three frame types:

- **Untagged frames**-- frames that do not carry any identification of the VLAN to which they belong
- **Priority-tagged frames**-- frames that include a tag header carrying explicit user priority information but not identifying the frames as belonging to a specific VLAN
- **VLAN-tagged frames**-- frames that include an explicit identification of the VLAN to which they belong

Current MUX-L2 Limitations

The current MUX-L2 implementation has the following known limitations:

- {{"Basically, MUX-L2 only support statically configured VLANs. Since GVRP is not supported, dynamically learned VLAN by GVRP is not supported as well." -KC}}
- There is no support for the automatic distribution of VLAN configuration using the Generic Attribute Registration Protocol (GARP) VLAN Registration Protocol (GVRP). The MUX-L2 support for VLAN is limited to those VLANs that are created statically.
- The MUX-L2 does not configure and operate the address learning process described in the 802.1Q specification. Therefore, it is not capable of broadcasting or multicasting frames to multiple ports belonging to a VLAN.
- Although the MUX-L2 allows the 802.1P User Priority to be specified with an egress VLAN-tagged frame, it does not support user priority to traffic class mapping described in the 802.1Q specification. It also does not provide any mechanism to perform the ingress user priority regeneration as described in the 802.1Q specification.
- Although the MUX-L2 implements selected RFC 2674 static VLAN objects, the VLAN configuration methodology is not compatible with the RFC 2674 MIB. RFC 2674 VLAN management is VLAN-centric and requires a port list bitmap specifying the ports belonging to a VLAN. The VLAN management for the MUX-L2 is port-centric and achieves VLAN configuration on a per-port basis.
- The support for 802.1Q VLAN tagging is currently implemented for END drivers only.
- The MUX-L2 implementation is provided in the context of Ethernet as the underlying data link technology. Because the fundamental VLAN operation and behavior are independent of the underlying data link, the implementation can be easily modified to adapt to a non-Ethernet environment.
- Wind River Learning Bridge is not compatible with the MUX-L2.

Conformance to Standards on page 158

Previous topic: How VLAN Works on page 155
Next topic: vconfig Command on page 158

7.1. Conformance to Standards

The virtual local area network (VLAN) implementation of the Wind River Network Stack conforms to the following standards:

- RFC 2674: *Definitions of Managed Objects for Bridges with Traffic Classes, Multicast Filtering and Virtual LAN Extensions* is implemented with exceptions. See Supported RFCs for the VxWorks 7 Network Stack on page 2 for further information.
- The provisions of *IEEE Std 802.1ad - Provider Bridges* related to VLAN stacking and MAC learning are implemented.
- The provisions of *802.1Q - Virtual LANs* are supported, except for bridge-related features, such as STP, GVRP, or GMRP.
- The VLAN stacking is implemented independently of the Wind River Learning Bridge, not in the bridge.
- VLAN is only supported on Ethernet devices.
- The PPPoE with VLAN is not supported. There is a check in the VLAN configuration code that prevents the configuration of a VLAN on a PPPoE interface.
- The Q-MIB is not required or supported. VLAN statistics are counted as part of interface statistics.

Parent topic: VLAN: The Wind River Implementation on page 157

8. vconfig Command

The vconfig command and options.

Name

vconfig - VLAN stacking configuration

Synopsis

```
vconfig [ options ]
    vconfig stpid value
    vconfig interface vlantype value cvid tag svid tag
    vconfig interface pvid tag
    vconfig interface ingress rule
    vconfig interface egress rule
    vconfig interface cvid tag inpri value spri value
    vconfig interface remove tag
```

Description

vconfig configures virtual local area network (VLAN) stacking parameters.

Commands

-h

Display usage help message.

-l

Display service VLAN tag protocol ID.

stpid

Set service VLAN tag protocol ID (1-33023,33025-65535).

vlantype *value*

The VLAN stacking type:

- 1 = customer edge/network port
- 2 = provider network port on provider edge node
- 3 = provider network port on Service Provider VLAN (S-VLAN) node

pvid *tag*

The port VID (1-4094).

cvid *tag*

The customer VID (1-4094).

svid *tag*

The service VID (1-4094).

remove *tag*

Delete a VLAN stacking VID (1-4094).

ingress *rule*

The VLAN ingress rule.

- 0 = Admit-All
- 1 = Admit-Tag
- 2 = Admit-Untag+PriorityTag

egress *rule*

The VLAN egress rule.

- 0 = Tagged
- 1 = Untagged

inpri *value*

The priority bits in a receiving VLAN frame.

spri *value*

The priority bits mapped from a receiving VLAN frame, and inserted to the S-VLAN tag in a transmitting VLAN frame, on a customer edge/network port.

Previous topic: VLAN: The Wind River Implementation on page 157

12. STACKED VLANS WITH 802.1AD

1. About Stacked VLANs

How to configure a stacked virtual local area network (VLAN), as defined in Institute of Electrical and Electronics Engineers (IEEE) Std 802.1ad - Provider Bridges. It assumes that you are familiar with the principles and operations of IEEE 802.1Q VLAN. For further information on this technology, see 10. Adding Support for VLANs.

This section also assumes you are familiar with the principles and operations of IEEE 802.1ad VLAN. For an overview of stacked VLAN technology, see How Stacked VLANs Work on page 174. For further information on the Wind River implementation of a stacked VLAN, see Wind River Stacked VLANs on page 175.

Note: You cannot configure a subnet based 802.1Q VLAN and a stacked VLAN on the same interface.

Next topic: Building a Stacked VLAN on page 160

2. Building a Stacked VLAN

To build a stacked virtual local area network (VLAN), you must build the required libraries and configure VxWorks with the necessary components.

Building the Stacked VLAN Libraries

To implement a stacked VLAN, you must enable the VxWorks source build (VSB) component **COMPONENT_VLAN_STACKING**.

Enhancing Debug Support

In addition to the various log utilities and the build verbosity option, you can get extra debug messages from VLAN stacking. To do so, use the ADDED_CFLAGS compiler flag to turn on the macro **SVLAN_DEBUG**. Alternately, you can edit a specific S-VLAN file if you want extra debug messages from that file only.

Configuring VxWorks with Stacked VLAN Support

To configure VxWorks with support for VLAN stacking, you must configure VxWorks with following components:

INCLUDE_IPNET_USE_VLAN_STACKING

This component provides support for VLAN stacking. It requires the component INCLUDE_IPCOM_USE_ETHERNET.

INCLUDE_IPVCONFIG_CMD

This component provides support for the vconfig shell command.

Specifying the Tag Protocol ID

INCLUDE_IPNET_USE_VLAN_STACKING contains one component for specifying the service VLAN tag protocol ID (TPID).

IPNET_VLAN_S_VLAN_TPID

Identifies the service VLAN tag (see *IEEE 802.1ad - Provider Bridges*). The default value is 0x88a8 as defined in IEEE 802.1ad. The service VLAN (S-VLAN) TPID must be a different value than the customer VLAN (C-VLAN) TPID value of 0x8100.

You can also modify this parameter at run time, using the vconfig shell command or the sysvar ipnet.vlan.svlan.tpid.

Previous topic: About Stacked VLANs on page 160
Next topic: Managing a Stacked VLAN on page 161

3. Managing a Stacked VLAN

You can display a variety of stacking virtual local area network (VLAN) parameters at run time, using the ifconfig shell command. You can also set these parameters, using the vconfig shell command.

These parameters include:

- service VLAN tag protocol ID
- VLAN stacking port type, such as a customer edge/network port on a provider edge node, provider network port on a S-VLAN node, or provider network port on a provider edge node
- port VID for each port on a provider edge node. You must configure the VLAN stacking port before setting the Port VLAN ID (PVID).
- customer VIDs allowed on a customer edge/network port
- service VIDs mapped from C-VIDs, and S-VIDs on a provider network port
- ingress filtering rule on a provider edge node: admit all frames, admit tagged frames, admit priority-tagged and untagged frames (see IEEE Std 802.1Q for ingress rule definition)
- egress filtering rule on a provider edge node: tagged, untagged (see IEEE Std 802.1Q for egress rule definition)
- receiving VLAN frame priority to service VLAN priority mapping on a provider edge node

You can also use **vconfig** to display the current service tag protocol identifier (TPID) value.

The following tutorials guide you through these specific use cases:

- Displaying the Current VLAN Parameters on page 162
- Configuring a Service TPID on page 163
- Configuring a Provider Edge Node Use Case 1 on page 164
- Configuring a Provider Edge Node Use Case 2 on page 166
- Configuring a Provider Edge Node Use Case 3 on page 167
- Configuring an S-VLAN Node Use Case 4 on page 169
- Configuring an S-VLAN Node Use Case 5 on page 171

Note: For some of the commands in these tutorials, you must use the kernel shell in C-interpreter mode (with a **->** prompt). For others, you must use the shell in command-interpreter mode (with a **[vxWorks *] #** prompt). The tutorial will specify which mode you must use to run each command.

For a detailed explanation of the options and syntax for the vconfig shell command, see the reference entry in vconfig Command on page 158.

Previous topic: Building a Stacked VLAN on page 160
Next topic: Displaying the Current VLAN Parameters on page 162

4. Displaying the Current VLAN Parameters

Using the ifconfig command, you can display either the virtual local area network (VLAN) stacking port type, or detailed parameters for an interface.

About This Task

Use ifconfig to display only the VLAN stacking port type for the specified interface. For example:

```
-> cmd ifconfig eth1

eth1 Link type:Ethernet HWaddr 00:0f:b7:10:64:65 Queue:none
                capabilities: TXCSUM VLAN_MTU
                VLAN stacking type: customer edge/network port
                inet 100.1.1.2 mask 255.255.255.0 broadcast 100.1.1.255
                inet6 unicast fe80::20f:b7ff:fe10:6465%eth1 prefixlen 64 automatic
                UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
                MTU:1500 metric:1 VR:0 ifindex:3
                RX packets:0 mcast:0 errors:0 dropped:0
                TX packets:9 mcast:8 errors:0
                collisions:0 unsupported proto:0
                RX bytes:0 TX bytes:690
                HW Forwarder
                Tx Packets: 9, Bytes: 690
                Rx Packets: 0, Bytes: 0
                 Frames size: 64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                 Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

Use the ifconfig command with the -s flag to display the detailed VLAN parameters for the specified interface. For example:

```
-> cmd ifconfig "-s eth1"
eth1 Link type:Ethernet HWaddr 00:0f:b7:10:64:65 Queue:none
                capabilities: TXCSUM VLAN_MTU
                VLAN stacking type: customer edge/network port  pvid: 1
                ingress rule: admit-all egress rule: tagged
                c-vid: 1  s-vid: 1  forwarding to:  eth2
                priority mapping:
                        0 --> 0  1 --> 1  2 --> 2  3 --> 3
                        4 --> 4  5 --> 5  6 --> 6  7 --> 7
                c-vid: 10  s-vid: 20  forwarding to:  eth2
                priority mapping:
                        0 --> 0  1 --> 1  2 --> 2  3 --> 3
                        4 --> 4  5 --> 5  6 --> 6  7 --> 7
                c-vid: 20  s-vid: 30 forwarding to: eth2
                priority mapping:
                        0 --> 0  1 --> 1  2 --> 2  3 --> 3
                        4 --> 4  5 --> 5  6 --> 6  7 --> 7
                c-vid: 30 s-vid: 20 forwarding to: eth2
                priority mapping:
                        0 --> 0  1 --> 1  2 --> 2  3 --> 3
                        4 --> 4  5 --> 5  6 --> 6  7 --> 7
                inet 100.1.1.2 mask 255.255.255.0 broadcast 100.1.1.255
                inet6 unicast fe80::20f:b7ff:fe10:6465%eth1 prefixlen 64  automatic
                UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
                MTU:1500  metric:1  VR:0  ifindex:3
```

```
                RX packets:0 mcast:0 errors:0 dropped:0
                TX packets:9 mcast:8 errors:0
                collisions:0 unsupported proto:0
                RX bytes:0  TX bytes:690
                W Forwarder
                        Tx Packets: 9, Bytes: 690
                        Rx Packets: 0, Bytes: 0
                Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

Previous topic: Managing a Stacked VLAN on page 161
Next topic: Configuring a Service TPID on page 163

5. Configuring a Service TPID

You can also modify the service tag protocol ID (TPID) at run time, using either the vconfig shell command or a sysvar.

About This Task

To specify a new service TPID with vconfig, follow this procedure:

Procedure

1. Switch the shell to command-interpreter mode, if necessary:

```
-> cmd
```

2. Display the current service TPID value:

```
[vxWorks *] # vconfig -l
                                VLAN service Tag Protocol ID: 0x88a8
```

3. Set the service TPID value to 0x9100:

```
[vxWorks *] # vconfig stpid 37120
Service TPID is set to 0x9100
```

4. Display the new service TPID value:

```
[vxWorks *] # vconfig -l
                                VLAN service Tag Protocol ID: 0x9100
```

Alternately, you can specify the service VLAN TPID at run time, using the sysvar ipnet.vlan.svlan.tpid.

Previous topic: Displaying the Current VLAN Parameters on page 162
Next topic: Configuring a Provider Edge Node Use Case 1 on page 164

6. Configuring a Provider Edge Node Use Case 1

Various configurations are suitable for a provider edge node, depending on the system requirements. In one scenario, subscriber services may or may not be virtual local area network (VLAN) aware.

About This Task

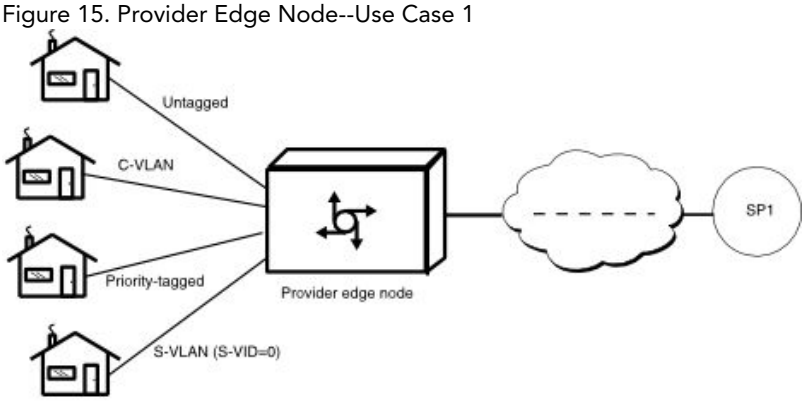
The provider edge node must be capable of mapping the following frame types:

- priority-tagged VLAN frames (with no C-VID)
- customer VLAN (C-VLAN)-tagged frames
- Service Provider VLAN (S-VLAN)-tagged frames (S-VID = 0)
- untagged frames to a provider VLAN ID (S-VLAN)

The node must also support flow in the opposite direction.

The figure below illustrates this configuration.

Figure 15. Provider Edge Node--Use Case 1



To implement this configuration, use the vconfig command to create a mapping between Customer VID (C-VID) and Service VID (S-VID). When a frame does not carry a C-VID, the default port VID is used. You must also create the mapping from the default VID to an S-VID.

To configure the provider edge node, follow this procedure:

Procedure

1. Configure two Ethernet interfaces:

```
-> ipAttach (1,"eth")
-> ifconfig "eth1 100.1.1.2/24 up"
-> ipAttach (2,"eth")
-> ifconfig "eth2 up"
```

2. Switch the shell to command-interpreter mode:

```
-> cmd
```

3. Configure the provider network port **eth2** with S-VID 20, 30, and 1:

```
[vxWorks *] # vconfig eth2 vlantype 2 svid 20
[vxWorks *] # vconfig eth2 vlantype 2 svid 30
[vxWorks *] # vconfig eth2 vlantype 2 svid 1
```

4. Configure customer edge/network port **eth1** with the following mappings:

   C-VID 10: S-VID 20

   C-VID 30: S-VID 20

   C-VID 20: S-VID 30

   PVID 1: S-VID 1

   To establish these mappings, issue the following commands:

```
[vxWorks *] # vconfig eth1 vlantype 1 cvid 10 svid 20
[vxWorks *] # vconfig eth1 vlantype 1 cvid 30 svid 20
[vxWorks *] # vconfig eth1 vlantype 1 cvid 20 svid 30
[vxWorks *] # vconfig eth1 vlantype 1 cvid 1 svid 1
```

5. Display the **eth1** configuration:

```
[vxWorks *] # ifconfig -s eth1
eth1  Link type:Ethernet  HWaddr 00:0f:b7:10:64:65  Queue:none
         capabilities: TXCSUM VLAN_MTU
         VLAN stacking type: customer edge/network port  pvid: 1
         ingress rule: admit-all egress rule: tagged
         c-vid: 1  s-vid: 1  forwarding to:  eth2
                 priority mapping:
                         0 --> 0  1 --> 1  2 --> 2  3 --> 3
                         4 --> 4  5 --> 5  6 --> 6  7 --> 7
         c-vid: 10  s-vid: 20  forwarding to:  eth2
                 priority mapping:
                         0 --> 0  1 --> 1  2 --> 2  3 --> 3
                         4 --> 4  5 --> 5  6 --> 6  7 --> 7
         c-vid: 20  s-vid: 30  forwarding to:  eth2
                 priority mapping:
                         0 --> 0  1 --> 1  2 --> 2  3 --> 3
                         4 --> 4  5 --> 5  6 --> 6  7 --> 7
         c-vid: 30  s-vid: 20  forwarding to:  eth2
                 priority mapping:
                         0 --> 0  1 --> 1  2 --> 2  3 --> 3
                         4 --> 4  5 --> 5  6 --> 6  7 --> 7
         inet 100.1.1.2  mask 255.255.255.0  broadcast 100.1.1.255
         inet6 unicast fe80::20f:b7ff:fe10:6465%eth1 prefixlen 64  automatic
         UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
         MTU:1500  metric:1  VR:0  ifindex:3
         RX packets:0 mcast:0 errors:0 dropped:0
         TX packets:9 mcast:8 errors:0
         collisions:0 unsupported proto:0
         RX bytes:0  TX bytes:690
         HW Forwarder
         Tx Packets: 9, Bytes: 690
         Rx Packets: 0, Bytes: 0
                        Frames size: 64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                        Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```


WNDRVR

6. Display the **eth2** configuration:

```
[vxWorks *] # ifconfig eth2
eth2  Link type:Ethernet  HWaddr 00:0f:b7:10:64:66  Queue:none
         capabilities: TXCSUM VLAN_MTU
         VLAN stacking type: provider network port on provide edge node  pvid: 1
         ingress rule: admit-all  egress rule: tagged
         s-vid: 20
                c-vid: 10  forwarding to:  eth1
                c-vid: 30  forwarding to:  eth1
         s-vid: 30
                 c-vid: 20  forwarding to:  eth1
         s-vid: 1
                 c-vid: 1  forwarding to:  eth1
         inet6 unicast fe80::20f:b7ff:fe10:6466%eth2  prefixlen 64  tentative  automatic
         UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
         MTU:1500  metric:1  VR:0  ifindex:4
         RX packets:0 mcast:0 errors:0 dropped:0
         TX packets:0 mcast:0 errors:0
         collisions:0 unsupported proto:0
         RX bytes:0  TX bytes:0
         HW Forwarder
         Tx Packets: 0, Bytes: 0
         Rx Packets: 0, Bytes: 0
                        Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
                        Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

As described in Managing a Stacked VLAN on page 161, a received VLAN frame is admitted by the VLAN ingress rule. Depending on the VLAN egress filtering rule, a C-VLAN tag is kept, added, or removed. Depending upon the egress VLAN port type, a S-VLAN tag is added, or removed. The destination and source Media Access Control (MAC) addresses are kept the same as in the original frame. See *Egress Data Flow*, for a description of this data flow.

Previous topic: Configuring a Service TPID on page 163
Next topic: Configuring a Provider Edge Node Use Case 2 on page 166

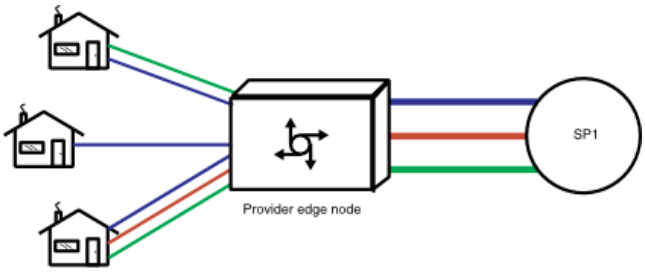
7. Configuring a Provider Edge Node Use Case 2

In another scenario, subscriber traffic may be grouped by type into S-VLANs. For example, video data may be on S-VLAN1, voice on S-VLAN2, and data on S-VLAN3.

About This Task

Not all subscribers may receive all types of service, so the edge provider must be able to identify the traffic types and map them to the appropriate S-VLANs.

Figure 16. Provider Edge Node--Use Case 2



The traffic types are differentiated by C-VLANs with a different C-VID inserted by the subscriber when a frame is sent out. In this scenario, provider equipment on the customer premises maps different traffic to different C-VIDs assigned by the provider. For example, video data is sent out on C-VLAN1, voice on C-VLAN2, and data on C-VLAN3. The provider edge node does not examine the header information in layers 3-7 to determine the traffic types. Instead, you must create mappings between C-VIDs and S-VIDs, using the vconfig command. Figure 1 on page 166 illustrates this configuration.

The configuration for this use case is similar to the one described in Configuring a Provider Edge Node Use Case 1 on page 164, except the traffic types are differentiated by C-VLANs, with different C-VID inserted by the subscriber when a frame is sent out.

Previous topic: Configuring a Provider Edge Node Use Case 1 on page 164
Next topic: Configuring a Provider Edge Node Use Case 3 on page 167

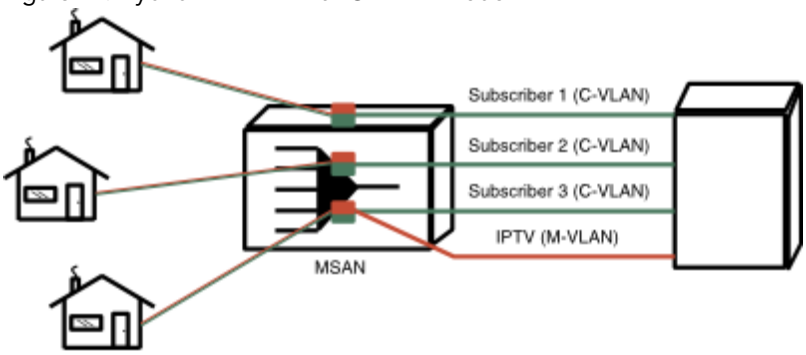
8. Configuring a Provider Edge Node Use Case 3

A third scenario involves multicast VLAN traffic, which is common in applications such as Internet Protocol TV (IPTV).

About This Task

This architecture combines the strength of a ustomer VLAN (C-VLAN) with a multicast VLAN (M-VLAN) in a node referred to as a multiservice access node (MSAN). The MSAN provides a single policy enforcement point and efficient multicast delivery. Figure 1 on page 167 illustrates this configuration.

Figure 17. Hybrid M-VLAN with C-VLAN Model



This model utilizes multiple VLANs as follows:

- A subscriber-dedicated C-VLAN carries unicast traffic such as Internet access and voice-over IP (VOIP) between the access node and the BSR.
- An S-VLAN carries broadcast television traffic to each MSAN. Because the S-VLAN carries IPTV traffic exclusively, it is often referred to as an M-VLAN.

This model assumes that each subscriber operates its own C-VLAN, which carries its own unicast traffic (Internet access, VoIP, and VoD). In addition, one VLAN carries multicast traffic to all subscribers. The network supports all of these VLANs--each C-VLAN plus the VLAN dedicated to multicast traffic.

When a multicast VLAN-tagged frame arrives at a VLAN stacking port, by default it will be forwarded to all ports that have same VLAN membership at an S-VLAN node, or to the configured VLAN forwarding table between the C-VLAN and S-VLAN on a provider edge node. The system uses manual configuration to determine VLAN membership.

In Figure 1 on page 167, each unicast traffic C-VLAN maps to different S-VLAN channels, and all multicast traffic M-VLAN maps to the same S-VLAN channel. The S-VLAN configuration commands would be similar to those used in Configuring a Provider Edge Node Use Case 1 on page 164.

To configure this provider edge node, follow this procedure:

Procedure

1. Configure four Ethernet interfaces:

```
-> ipAttach (1,"eth")
-> ifconfig "eth1 up"
-> ipAttach (2,"eth")
-> ifconfig "eth2 up"
-> ipAttach (3,"eth")
-> ifconfig "eth3 up"
-> ipAttach (4,"eth")
-> ifconfig "eth4 up"
```

2. Switch the shell to command-interpreter mode:

```
-> cmd
```

3. Configure S-VLAN 10, 20, 30, 50 at **eth4**:

```
[vxWorks *] # vconfig eth4 vlantype 2 svid 10
[vxWorks *] # vconfig eth4 vlantype 2 svid 20
[vxWorks *] # vconfig eth4 vlantype 2 svid 30
[vxWorks *] # vconfig eth4 vlantype 2 svid 50
```

Note: S-VLAN 50 is for multicast traffic.

4. Map the unicast traffic C-VLAN 10 at eth1 to S-VLAN 10 at **eth4**:

```
[vxWorks *] # vconfig eth1 vlantype 1 cvid 10 svid 10
```

5. Map the unicast traffic C-VLAN 20 at eth2 to S-VLAN 20 at **eth4**:

```
[vxWorks *] # vconfig eth2 vlantype 1 cvid 20 svid 20
```

6. Map the unicast traffic C-VLAN 30 at eth3 to S-VLAN 30 at **eth4**:

```
[vxWorks *] # vconfig eth3 vlantype 1 cvid 30 svid 30
```

7. Map the multicast traffic C-VLAN 50 at eth1, eth2, eth3, to S-VLAN 50 at **eth4**:

```
[vxWorks *] # vconfig eth1 vlantype 1 cvid 50 svid 50
[vxWorks *] # vconfig eth2 vlantype 1 cvid 50 svid 50
[vxWorks *] # vconfig eth3 vlantype 1 cvid 50 svid 50
```

Previous topic: Configuring a Provider Edge Node Use Case 2 on page 166
Next topic: Configuring an S-VLAN Node Use Case 4 on page 169

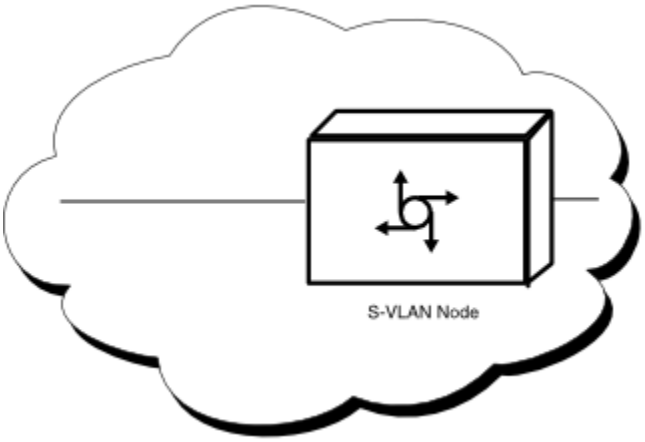
9. Configuring an S-VLAN Node Use Case 4

Institute of Electrical and Electronics Engineers (IEEE) Std 802.1ad describes a use case for a Service Provider VLAN (S-VLAN). The node is located in the provider network cloud and transfers virtual local area network (VLAN) traffic across the provider network between provider edge nodes.

About This Task

Figure 1 on page 169 illustrates this configuration.

Figure 18. S-VLAN Node



This node receives S-VLAN-tagged frames. The provider network ports are manually configured with S-VID according to the port VLAN membership. The VLAN forwarding table is established for the ports with the same VLAN membership. The S-VLAN node forwards traffic received on one port to other ports according to the S-VID and the destination address.

To configure an S-VLAN node for this use case, follow this procedure:

Procedure

1. Configure two Ethernet interfaces:

```
-> ipAttach (1,"eth")
-> ifconfig "eth1 up"
-> ipAttach (2,"eth")
-> ifconfig "eth2 up"
```

2. Switch the shell to command interpreter mode:

```
-> cmd
```

3. Configure provider network port **eth1** with S_VID 20:

```
[vxWorks *] # vconfig eth1 vlantype 3 svid 20
```

4. Configure provider network port **eth2** with S_VID 20:

```
[vxWorks *] # vconfig eth2 vlantype 3 svid 20
```

5. Display the **eth1** configuration:

```
[vxWorks *] # ifconfig -s eth1
eth1  Link type:Ethernet  HWaddr 00:0f:b7:10:64:65  Queue:none
            capabilities: TXCSUM VLAN_MTU
            VLAN stacking type: provider network port on S-VLAN node
            s-vid: 20  forwarding to: eth2
            inet6 unicast fe80::20f:b7ff:fe10:6465%eth1  prefixlen 64  automatic
            UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
            MTU:1500  metric:1  VR:0  ifindex:3
            RX packets:0 mcast:0 errors:0 dropped:0
            TX packets:9 mcast:8 errors:0
            collisions:0 unsupported proto:0
            RX bytes:0  TX bytes:690
            HW Forwarder
            Tx Packets: 9, Bytes: 690
            Rx Packets: 0, Bytes: 0
             Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
           Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

6. Display the **eth2** configuration:

```
[vxWorks *] # ifconfig -s eth2
eth2  Link type:Ethernet  HWaddr 00:0f:b7:10:64:66  Queue:none
            capabilities: TXCSUM VLAN_MTU
            VLAN stacking type: provider network port on S-VLAN node
            s-vid: 20  forwarding to:  eth1
            inet6 unicast fe80::20f:b7ff:fe10:6466%eth2  prefixlen 64  tentative  automatic
            UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
            MTU:1500  metric:1  VR:0  ifindex:4
            RX packets:0 mcast:0 errors:0 dropped:0
            TX packets:0 mcast:0 errors:0
            collisions:0 unsupported proto:0
            RX bytes:0  TX bytes:0
            HW Forwarder
                    Tx Packets: 0, Bytes: 0
                    Rx Packets: 0, Bytes: 0
            Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
            Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

Previous topic: Configuring a Provider Edge Node Use Case 3 on page 167
Next topic: Configuring an S-VLAN Node Use Case 5 on page 171

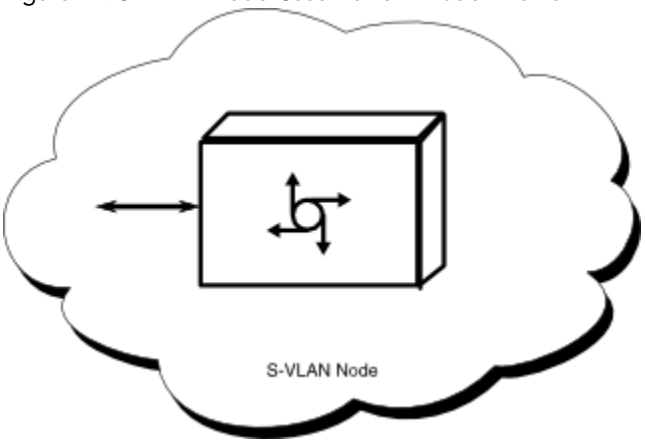
10. Configuring an S-VLAN Node Use Case 5

A Service Provider VLAN (S-VLAN) node is located in the provider network cloud. S-VLAN traffic is terminated at the S-VLAN node inside the provider network.

About This Task

An S-VLAN node is located in the provider network cloud. S-VLAN traffic is terminated at the S-VLAN node inside the provider network, as shown in Figure 1 on page 171.

Figure 19. S-VLAN Node Case As Termination Point



In this use case, the receiving traffic could be IP traffic carrying S-VLAN and customer VLAN (C-VLAN) tags. A provider network port is manually configured with S-VID and IP address. The IP traffic targeted to the provider network port is terminated at the S-VLAN node and forwarded to the higher level at which the packet is terminated. The S-VLAN tag with PVID as S-VID is inserted into the packets originating at the S-VLAN node and transmitted out through the provider network port.

To configure an S-VLAN node for this use case, follow this procedure:

Procedure

1. Configure an Ethernet interface:

```
-> ipAttach (1,"eth")
-> ifconfig "eth1 up"
```

2. Switch the shell to command-interpreter mode:

```
-> cmd
```

3. Configure provider network port eth1 with S_VID 20:

```
[vxWorks *] # vconfig eth1 vlantype 3 svid 20
```

4. Specify the PVID for **eth1**:

```
[vxWorks *] # vconfig eth1 pvid 20
```

5. Configure the IP address on the provider network port **eth1**:

```
[vxWorks *] # ifconfig eth1 192.168.3.3 network 255.255.0.0
```

6. Display the **eth1** configuration:

```
[vxWorks *] # ifconfig -s eth1
eth1  Link type:Ethernet  HWaddr 00:0f:b7:10:64:65  Queue:none
            capabilities: TXCSUM VLAN_MTU
            VLAN stacking type: provider network port on S-VLAN node
            s-vid: 2
            inet 192.168.3.3  mask 255.255.0.0  broadcast 192.168.255.255
            inet6 unicast fe80::20f:b7ff:fe10:6465%eth1  prefixlen 64  automatic
            UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
            MTU:1500  metric:1  VR:0  ifindex:3
            RX packets:0 mcast:0 errors:0 dropped:0
            TX packets:9 mcast:8 errors:0
            collisions:0 unsupported proto:0
            RX bytes:0  TX bytes:690
            HW Forwarder
                    Tx Packets: 9, Bytes: 690
                    Rx Packets: 0, Bytes: 0
            Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
            Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

Previous topic: Configuring an S-VLAN Node Use Case 4 on page 169
Next topic: Managing VLAN Priority on page 172

11. Managing VLAN Priority

How to manage virtual local area network (VLAN) priorities.

About This Task

A VLAN ID as shown in the figure below is 12 bits. Its range is from 0 to 4095, with two reserved values as follows:

- 0 indicates a priority-tagged frame
- 4095 is reserved but not currently used

No matter what the VLAN ID (VID) value is, a network user can set the user priority bits in a VLAN frame to indicate the priority of the frame. A service provider's network administrator can configure a priority regeneration table on a port to remap the user priority carried in a VLAN tag and queue a frame at different transmission queues according to different priorities.

A network administrator can remap priorities at the provider edge node by creating an entry in the priority regeneration table for each configured C-VID. The default priority mapping is directly mapped--that is, incoming priority 1 maps to outgoing network priority 1, 2 maps to 2, etc. You can use the vconfig shell command to change the default mapping. Once the customer priority is mapped to a network priority, the system can assign outgoing frames to different egress transmission queues, according to the priority of the frame.

The Wind River Network Stack only supports priority remapping between the receiving user's priority and the priority in the S-VLAN tag. It does not support egress transmission queues.

The VLAN tag carries three user priority bits to indicate the priority of the frame. On a customer edge/network port on a provider edge node, these customer priority bits are remapped to different network priorities based on the priority regeneration table configured on the reception port. The remapped network priority bits are inserted and carried in the S-VLAN tag. The default priority

remapping is direct mapping--that is, customer priority 1 maps to network priority 1, customer priority 2 maps to network priority 2, etc. The configuration command vconfig is provided to change the default priority mapping. The priority remapping table is per port per Customer VID (C-VID), so the configuration is also based on per port per C-VID.

To configure priority remapping on a customer edge/network port, follow this procedure:

Procedure

1. Configure an Ethernet interface:

```
-> ipAttach (1,"eth")
-> ifconfig "eth1 100.1.1.2/24up"
```

2. Switch the shell to command interpreter mode:

```
-> cmd
```

3. Configure the customer edge/network port **eth1** with the following mapping:

   C-VID 10: S-VID 20

```
[vxWorks *] # vconfig eth1 vlantype 1 cvid 10 svid 20
```

4. Display the default priority remapping table:

```
[vxWorks *] # ifconfig -s eth1
eth1  Link type:Ethernet  HWaddr 00:0f:b7:10:64:65  Queue:none
            capabilities: TXCSUM VLAN_MTU
            VLAN stacking type: customer edge/network port  pvid: 1
            ingress rule: admit-all  egress rule: tagged
            c-vid: 10  s-vid: 20
            priority mapping:
                     0 --> 0  1 --> 1  2 --> 2  3 --> 3
                     4 --> 4  5 --> 5  6 --> 6  7 --> 7
            inet 100.1.1.2  mask 255.255.255.0  broadcast 100.1.1.255
            inet6 unicast fe80::20f:b7ff:fe10:6465%eth1  prefixlen 64  automatic
            UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
            MTU:1500  metric:1  VR:0  ifindex:3
            RX packets:0 mcast:0 errors:0 dropped:0
            TX packets:9 mcast:8 errors:0
            collisions:0 unsupported proto:0
            RX bytes:0  TX bytes:690
            HW Forwarder
            Tx Packets: 9, Bytes: 690
            Rx Packets: 0, Bytes: 0
            Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
            Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

5. Set priority remapping 3 to 4 for C-VID 10 on eth1:

```
[vxWorks *] # vconfig eth1 cvid 10 inpri 3 spri 4
```

6. Redisplay the priority remapping table to confirm the remapping:

```
[vxWorks *] # ifconfig -s eth1
eth1  Link type:Ethernet  HWaddr 00:0f:b7:10:64:65  Queue:none
            capabilities: TXCSUM VLAN_MTU
```


WNDRVR

```
VLAN stacking type: customer edge/network port  pvid: 1
ingress rule: admit-all  egress rule: tagged
c-vid: 10  s-vid: 20
priority mapping:
        0 --> 0  1 --> 1  2 --> 2  3 --> 4
        4 --> 4  5 --> 5  6 --> 6  7 --> 7
inet 100.1.1.2  mask 255.255.255.0  broadcast 100.1.1.255
net6 unicast fe80::20f:b7ff:fe10:6465%eth1  prefixlen 64  automatic
UP RUNNING SIMPLEX BROADCAST MULTICAST PROMISC
MTU:1500  metric:1  VR:0  ifindex:3
RX packets:0 mcast:0 errors:0 dropped:0
TX packets:9 mcast:8 errors:0
collisions:0 unsupported proto:0
RX bytes:0  TX bytes:690
HW Forwarder
        Tx Packets: 9, Bytes: 690
        Rx Packets: 0, Bytes: 0
Frames size:  64: 0, 128: 0, 256: 0, 512: 0, 1024: 0, 1519: 0
Broadcast: 0, Multicast: 0, Dropped: 0 (0)
```

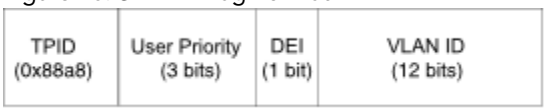
Previous topic: Configuring an S-VLAN Node Use Case 5 on page 171
Next topic: How Stacked VLANs Work on page 174

12. How Stacked VLANs Work

Institute of Electrical and Electronics Engineers (IEEE) Std 802.1ad amends IEEE Std 802.1Q by inserting another service virtual local area network (VLAN) tag before the customer VLAN.

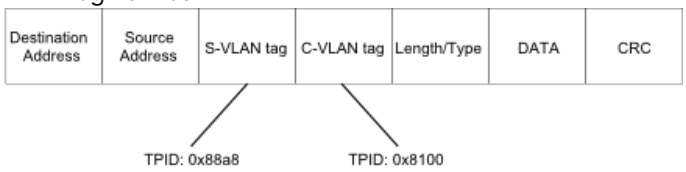
The service VLAN tag is added to give network service providers the flexibility to configure and administer the service network independently of the customer networks. Service providers can configure their networks without requiring that customers coordinate or change the VLAN IDs (VIDs) they use and without limiting the total number of VIDs across all customers to 4094, resulting in increased service transparency to the customer.

Figure 20. S-VLAN Tag Format



The new VLAN tag introduced by IEEE Std 802.1ad has a different tag protocol identifier (TPID) value of 0x88a8. It also has a slightly different format from the customer VLAN (C-VLAN) tag, as shown in the figure below. The new element in this tag is the drop eligible indicator (DEI) bit.

Figure 21. VLAN Tag Format



IEEE Std 802.1ad defines at most two VLAN tags in a packet. To save the network bandwidth, IEEE 802.1ad allows a packet to only carry one Service Provider VLAN (S-VLAN) tag without a C-VLAN tag. Since the S-VLAN tag has a different TPID value than the C-

VLAN tag, there is no ambiguity about which VLAN tag it is if a packet only carries one VLAN tag. The figure *VLAN Tag Format on page 174* illustrates the format of a packet carrying two VLAN tags.

Previous topic: Managing VLAN Priority on page 172
Next topic: Wind River Stacked VLANs on page 175

13. Wind River Stacked VLANs

virtual local area network (VLAN) stacking in conformance with Institute of Electrical and Electronics Engineers (IEEE) Std 802.1ad is only available in the Wind River VxWorks Platforms builds of the network stack. The Wind River General Purpose Platform, VxWorks Edition, does not include 802.1ad VLAN stacking.

Node Types

As specified in IEEE Std 802.1ad, a provider node can be a provider edge node or an Service Provider VLAN (S-VLAN) node. A provider edge node is at the edge of the provider network connecting customer equipment. This node aggregates customer VLAN (C-VLAN) traffic into S-VLAN traffic and forwards the combined traffic flows to the provider network. In the reverse direction, a provider edge node receives S-VLAN traffic from the provider network, strips off the S-VLAN tag, and forwards frames according to the Customer VID (C-VID) and the destination Media Access Control (MAC) address. The default port VID is used if a frame does not have a C-VLAN tag.

An S-VLAN node is located in the provider network. On a provider edge node, a port connecting to customer equipment is called a customer edge or network port. The difference between a customer edge port and a customer network port is that besides C-VLAN-tagged, priority-tagged, and untagged frames, a customer network port can also receive and process S-VLAN-tagged frames (but only frames with S-VID = 0). A port connecting the provider network on a provider edge node, or a port on a S-VLAN node is called a provider network port.

Port Types

Supported port types include the following:

- customer edge/network port at a provider edge node
- provider network port at a provider edge node
- provider network port at a S-VLAN node

Frame Types

At most two VLAN tags are supported. The supported combinations for VLAN tagging is one of the following cases:

- One S-VLAN tag and one C-VLAN tag.
- One S-VLAN tag without any C-VLAN tag.
- One C-VLAN tag without any S-VLAN tag.

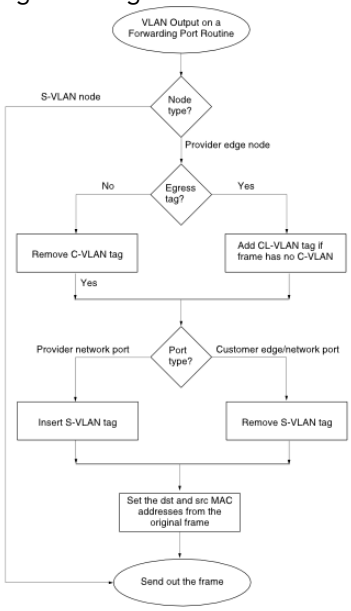
Other combinations, such as one S-VLAN tag with multiple C-VLAN tags, or multiple C-VLAN tags, or multiple S-VLAN tags, are not supported.

On a customer edge/network port, the frame could be C-VLAN-tagged, priority-tagged, untagged frames, and S-VLAN tag frames (only with S-VID = 0). On a provider network port, the frame could be one S-VLAN tag and one C-VLAN tag, and one S-VLAN tag without any C-VLAN tag.

Egress Data Flow

On the transmitting side, the configured stacked VLAN egress filtering rule can only be configured on a provider edge node. It only applies to VLAN forwarding traffic and has no impact to other traffic. The hardware does not support inserting and extracting an S-VLAN tag, so the software inserts and removes an S-VLAN tag.

Figure 22. Egress VLAN Data Flow



On a customer edge/network port, the egress-untagged option means to remove all VLAN tags and transmit frames out to the customer edge/network port with no VLAN tag. The egress-tagged option means to transmit with a C-VLAN tag. On a provider network port on a provider edge node, egress-untagged means to remove the C-VLAN tag and transmit frames out to the provider network port with only one S-VLAN tag. Egress-tagged means to transmit the frame with one S-VLAN tag inserted and one C-VLAN tag from the original frame or with the PVID added. Figure 1 on page 176 illustrates the VLAN processing at the transmitting side.

Previous topic: How Stacked VLANs Work on page 174
Next topic: Conformance to Standards on page 176

14. Conformance to Standards

The stacked virtual local area network (VLAN) implementation of the Wind River Network Stack conforms to these standards.

- The implementation is VLAN stacking according to Institute of Electrical and Electronics Engineers (IEEE) Std 802.1ad. MAC learning is also implemented. The other bridge-related features, such as Spanning Tree Protocol (STP), Generic Attribute Registration Protocol (GARP), or GARP Multicast Registration Protocol (GMRP) in IEEE Std 802.1Q, are not supported.
- The VLAN stacking is implemented independently of the Wind River Learning Bridge, not in the learning bridge.
- The VLAN is only supported on Ethernet devices.
- The Q-MIB is not required and supported. VLAN statistics is counted as part of interface statistics.

Previous topic: Wind River Stacked VLANs on page 175

13. VIRTUAL ROUTING

1. About Virtual Routing

Virtual routing increases network efficiency by allowing one physical router to service many independent networks.

The most efficient networks implement virtual routing by instantiating routing tables, instead of multiplying Transmission Control Protocol/Internet Protocol (TCP/IP) stacks, which would cause a significant drain on system resources. The Wind River Network Stack implements virtual routing in this fashion.

Note: The virtual router (VR) implementation is not related to the Virtual Router Redundancy Protocol (VRRP).

Virtual routing makes it possible to partition route nodes so they appear and behave as multiple physical router nodes when viewed from the outside. For example, a router with four VR instances would look like four physical routers when viewed from outside the network. A related concept is the *VR domain*, which would denote all nodes visible to a router in a particular VR instance. The benefits of virtual routing include lower hardware costs and simpler administration, since one system can act as multiple routers.

Virtual routing is enabled by default in the network stack for the following reasons:

- It does not impose any algorithmic speed penalty. All virtual router (VR) operations costs are constant (O(1)).This means that all virtual router operations are independent of the number of defined VR instances.
- It requires very little extra code.
- VR-specific structures are allocated when the VR is created and freed if the VR is removed.
- Virtual routing requires only one 16-bit field in structures that need to identify the VR to which they belong. These structures are sockets, network interfaces, and network packet headers.

The same port and/or address can be used on two different VRs without any collision because the following entities are separate for each VR:

- forwarding information base (FIB)
- IPv4 and IPv6 network interface addresses
- Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) ports
- sockets

Every network interface attached to the stack belongs to exactly one VR. Every network interface is initially assigned to the default VR, which always has **VR ID 0**. The default VR is created at boot time and cannot be deleted.

All network stack processes may belong to independent VR domains, each having its own set of configuration parameters and, most importantly, interfaces, routes, ARP/NDP entries, addresses, and sockets. The VR domains are completely separated and therefore capable of using duplicate IP addresses, TCP/IP connections with the same ports, interfaces with the same names, and so on. Because of this complete separation, multiple telnet servers and Web servers, as well as advanced routing software suites, can run on the same target with no software modifications required. Socket system call API changes are not required. One separate process per VR is created, running the same network application software but using only one virtualized TCP/IP stack.

With virtualization, each user process must belong to a specific VR domain, identified by the process-specific environment variable **VR**. If a process does not have the VR environment variable set, it automatically belongs to **VR 0**, the default, or main, VR. Since processes inherit all environment variables from the parent upon creation, the child processes belong to the same VR as the parent. To separate multiple processes created with the same names but belonging to unique VR domains, each process has the string **#** *vr* automatically appended to its name, where *vr* is the VR ID.

Interface Management

A VR domain can contain many interfaces, but each interface can belong to only one VR. If an interface is dynamically created by a process, it inherits its VR settings from the process that created it. You can also programmatically move an interface from one VR domain to another, using a call to an **ioctl( )** socket extension. In such a case, the VR domain for this interface changes while the network interface ID remains the same.

If an interface is moved to another VR, all its routes are moved as well. Interfaces may even use the same names as long as they belong to independent VR domains, because system calls operating on interfaces use the VR identifier and interface name in combination to identify an interface. The VR identifier used by a socket system call is normally taken from the socket. Because of this, advanced socket system calls, like routing sockets, operate on routes, addresses, and interfaces belonging to the same VR only. If no socket is used in the system call, the VR from the calling process is used (the **VR** environment variable).

For example, if you use the if_nameindex( ) routine to list all interfaces from a process belonging to VR 2, the list will only contain interfaces belonging to VR 2. Note also that each VR has a loopback interface named **lo0**. A list of interfaces for all VRs will show multiple loopback interfaces named **lo0**,with each **lo0** interface distinguished by its VR ID and network interface index. This feature provides full support for separated loopback communication.

Conformance to Standards

The Wind River implementation of virtual routing is not based on any published standard. Virtual routing is not associated with the VRRP.

Next topic: Managing Virtual Routers on page 178

2. Managing Virtual Routers

A virtual router (VR) is created and destroyed at run time. A newly created VR contains an empty forwarding information base (FIB) for every network protocol for which the network stack was built (IPv4, IPv6, and MPLS).

The network stack uses the VR ID to uniquely identify the VR, but you can assign names of your choice to all VRs. In your application code, you can reference the VR by its VR ID or by the assigned name.

Every VR contains at least one FIB, the default FIB. The default FIB is always assigned table ID IPCOM_ROUTE_TABLE_DEFAULT. This ID is configurable, but has a default value of 254. All lookups are done in the default table, unless another table is assigned by a policy routing rule.

To add or delete a VR, add or delete a FIB, add a VR by name, name a tuple (VR, table), and map a name to a tuple (VR, table), use the following socket ioctl options. See the appropriate reference entries for more information on these options.

- SIOCADDVR
- SIOCADDROUTETAB
- SIOCDELROUTETAB
- SIOCGETROUTETAB
- SIOCSROUTETABNAME
- SIOCGROUTETABNAME

A network interface is assigned to a specific VR using the socketioctl( ) option SIOCSIFVR. A socket is assigned to a specific VR using setsockopt( ) option SO_X_VR. A socket returned by accept( ) inherits the VR ID from the parent socket.

Managing a VR with AF_ROUTE Socket Messages

A VR can also be created and destroyed using the **AF_ROUTE** socket messages **RTM_NEWVR** and **RTM_DELVR**.

You can both read and write **RTM_NEWVR** messages. This message adds a new virtual routing domain. The header for an **RTM_NEWVR** message is described by an **rt_msghdr** structure, which is defined in **net/ route.h**.

You can both read and write **RTM_DELVR** messages. This message deletes a virtual routing domain. The default route table (for VR 0) cannot be removed. The header for an **RTM_DELVR** message is described by an **rt_msghdr** structure, which is defined in **net/ route.h**.

For more information, see *VxWorks 7 Sockets Programmer's Guide*.

Creating Virtual Routers (VRs) and Assigning Interfaces on page 179 Before creating a virtual router, you must include the **COMPONENT_VIRTUAL_ROUTER** component in your VSB.
Working with Virtual Routers (VRs) in Applications on page 180 How to use VRs in applications.

Previous topic: About Virtual Routing on page 177
Next topic: Virtual Router Code Examples on page 181

2.1. Creating Virtual Routers (VRs) and Assigning Interfaces

Before creating a virtual router, you must include the **COMPONENT_VIRTUAL_ROUTER** component in your VSB.

See Source Build Configuration Options on page 6 for more information on VSB configuration requirements.

You can create VRs and assign interfaces by using the using the route and ifconfig shell commands. The following example shows how to create two VRs, four VLAN network interfaces, and assign two VLANs to each VR.

vlanExt1 and **vlanExt2** symbolize the network interfaces that would normally be connected to the public Internet. **vlanInt1** and **vlanInt2** symbolize the corporate LAN.

Use the route shell command to create VRs. The following commands create two VRs with VR ID 1 and 2 using the AF_ROUTE( ) socket API. The result would be exactly the same if the ioctl( ) API was used.

```
[vxWorks *] # route vr -add 1
[vxWorks *] # route vr -add 2
```

The following commands create four VLANs and assign **vlanExt1/vlanInt1** to VR 1 and **vlanExt2/vlanInt2** to VR 2:

```
[vxWorks *] # ifconfig vlanExt1 create vlanif eth0 vlan 10 vr 1
[vxWorks *] # ifconfig vlanExt2 create vlanif eth0 vlan 11 vr 2
[vxWorks *] # ifconfig vlanInt1 create vlanif eth0 vlan 12 vr 1
[vxWorks *] # ifconfig vlanInt2 create vlanif eth0 vlan 13 vr 2
```

Using the **ifconfig -a** command does not show any of the VLAN interfaces created, because the command only shows interfaces assigned to the current VR. To show all interfaces assigned to VR 1, for example, use the following command:

```
[vxWorks *] # ifconfig -V 1 -a
```

The following commands assign IPv4 addresses to the interfaces:

```
[vxWorks *] # ifconfig -V 1 vlanExt1 inet 10.1.1.10 up
[vxWorks *] # ifconfig -V 2 vlanExt2 inet 10.1.1.11 up
```

```
[vxWorks *] # ifconfig -V 1 vlanInt1 inet 192.168.1.1 up
[vxWorks *] # ifconfig -V 2 vlanInt2 inet 192.168.1.1 up
```

Even though **vlanInet1** and **vlanInet2** are assigned the same address, there are no delivery problems because the FIB on VR #1 does not contain information about **vlanInet2** and vice versa.

The ping and **ping6** commands are also VR-aware, and a specific VR can be used by specifying it with the **-V n** switch.

Parent topic: Managing Virtual Routers on page 178

2.2. Working with Virtual Routers (VRs) in Applications

How to use VRs in applications.

This code fragment starts two processes, each belonging to a separate VR. The process opens a socket.

```
void my_server(void)
{
    int sock;

    /* Open a socket. Note: the socket will automatically belong
     * to the same VR as the creator (specified by the environment
     * variable "VR")
     */
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock >= 0)
    {
        for(;;)
        {
            /* Network code */
        }
    }
}

void example(void)
{
    /* Create and start my_server belonging
to VR 1 */
    (void)putenv("VR=1");
    (void)taskSpawn("my_server",
                    50,
                    VX_PRIVATE_ENV,
                    4096,
                    (FUNCPTR)my_server,
                    0, 0, 0, 0,
                    0, 0, 0, 0,
                    0, 0);

    /* Create and start my_server belonging
to VR 2 */
    (void)putenv("VR=2");
    (void)taskSpawn("my_server",
                    50,
                    VX_PRIVATE_ENV,
                    4096,
                    (FUNCPTR)my_server,
```

```
                        0, 0, 0, 0,
                        0, 0, 0, 0,
                        0, 0);
}
```

Parent topic: Managing Virtual Routers on page 178

3. Virtual Router Code Examples

The following code examples show how to implement a virtual router (VR) with ioctl calls or with the **AF_ROUTE** socket.

The embedded comments in the example code provide the implementation details.

ioctl Calls

The example code in vr_ioctl( )provides the following functionality for managing a VR with ioctls:

- Opening a socket
- Adding a virtual router
- Assigning a name to the VR
- Reading back and printing the name of the VR
- Assigning an interface to the VR
- Reassigning the interface to the default VR
- Deleting the VR
- Closing the socket

```
#include <sysctlLib.h>
#include <taskLib.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <net/if.h>
#include <net/route.h>
#include <ipnet/ipioctl.h>
#include <ipnet/ipioctl_var.h>

/*
 * vr_ioctl
 *
 * Using ioctl calls, opens a new socket and assign the socket a specified
 * network interface to a new virtual router(VR). The VR is assigned a name,
 * and finally the VR is deleted again
 *
 * ifname : String of existing network interface name
 * vrid   : Virtual Router id
 * vrname : User assigned name to the VR
 */
```

```
int vr_ioctl(char *ifname, int vrid, char*
vrname)
{
        int  sock;
        struct ifreq  ifreq;
        short int  vr;
        int  ret;
        struct sioc_route_table  sioc_rt;

        /* Open a socket of your choice */

        sock = socket(AF_INET, SOCK_DGRAM, 0);
        if (sock < 0)
        {
                /* failed */
                return sock;
        }

        /* Add a new virtual router */

        vr = (short int) vrid;
        if((ret = ioctl(sock, SIOCADDVR, &vr)) < 0)
        {
                /* failed */
                printf("Failed to add VR\n");
                return ret;
        }

        /* Assign a name to the virtual router */

        memset(&sioc_rt, 0, sizeof(sioc_rt));
        sioc_rt.vr = (short int) vrid;
        strcpy(sioc_rt.name, vrname);
        if((ret = ioctl(sock, SIOCSROUTETABNAME, &sioc_rt)) < 0)
        {
                /* failed */
                printf("Failed to set VR name\n");
                return ret;
        }

        /* Read back the name of the virtual router */

        memset(&sioc_rt, 0, sizeof(sioc_rt));
        sioc_rt.vr = (short int) vrid;
        if((ret = ioctl(sock, SIOCGROUTETABNAME, &sioc_rt)) < 0)
        {
                /* failed */
                printf("Failed to get VR name\n");
                return ret;
        }
        printf("VR name set to %s\n", sioc_rt.name);

        /* Assign the interface to the virtual router */

        strcpy(ifreq.ifr_name, ifname);
        ifreq.ifr_ifru.ifru_vr = (short int)vrid;
        if ((ret = ioctl(sock, SIOCSIFVR, &ifreq)) < 0)
```

```
        {
                /* failed */
                printf("Failed to add interface to VR\n");
                return ret;
        }

        /* Set socket option to assign the socket to the new VR */

        if ((ret = setsockopt(sock, SOL_SOCKET, SO_X_VR, &vrid, sizeof(vrid))) < 0)
        {
                /* failed */
                printf("Failed to assign socket to VR\n");
                return ret;
        }

        /*
         *
         * Here you could add code to do other networking stuff with the socket
         * (or other sockets) that is now assigned on the new VR
         *
         */

        /* Clean up */

        /* Re-assign the interface to the default virtual router (0) */

        strcpy(ifreq.ifr_name, ifname);
        ifreq.ifr_ifru.ifru_vr = (short int) 0;
        if ((ret = ioctl(sock, SIOCSIFVR, &ifreq)) < 0)
        {
                /* failed */
                printf("Failed to restore interface to default VR\n");
                return ret;
        }

        /* Delete VR*/

        vr = (short int) vrid;
        if((ret = ioctl(sock, SIOCDELVR, &vr)) < 0)
        {
                /* failed */
                printf("Failed to delete VR\n");
                return ret;
        }

        /* Close socket */

        if((ret = close(sock)) < 0)
        {
                /* failed */
                printf("Failed to close socket\n");
                return ret;
        }

        return 0;
}
```

AF_ROUTE Socket

The example code in vr_routesock( )provides the following functionality for managing a VR with an **AF_ROUTE** socket:

- Opening an AF_ROUTE socket
- Assigning the socket to the VR
- Adding a VR
- Deleting a VR
- Closing the socket

```
#include <sysctlLib.h>
#include <taskLib.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <net/if.h>
#include <net/route.h>
#include <ipnet/ipioctl.h>
#include <ipnet/ipioctl_var.h>

/*
 * vr_routesock
 *
 * Add or delete a new Virtual Router Using an AF_ROUTE socket
 *
 * vrid   : Virtual Router identifier
 * delete : delete = 0 to add new VR, delete = 1 to delete existing VR
 */

int vr_routesock(int vrid, int delete)
{
                int  rsock;
        struct rt_msghdr *rtm;
        char             buf[sizeof(struct rt_msghdr)];
        pid_t            pid = taskIdSelf();
        int              seq = rand();

        /* Open an AF_ROUTE socket */

        rsock = socket(AF_ROUTE, SOCK_RAW, 0);
        if(rsock < 0)
        {
                /* Failed */
                printf("Failed to create AF_ROUTE socket\n");
                return -1;
        }

        /* Assign the socket to the VR */

        if (setsockopt(rsock, SOL_SOCKET, SO_X_VR, &vrid, sizeof(vrid)) != 0)
        {
                /* Failed */
                printf("Failed set VR sockopt\n");
```

```
                return -1;
        }

        /* Send message to add/delete a virtual router
         * The Virtual Router ID is supplied implicitly via the socket where it was
         * previously set. Together with the rtm_table it uniquely identifies a
         * route table (FIB) */

        rtm = (struct rt_msghdr *) buf;
        memset(rtm, 0, sizeof(*rtm));
        rtm->rtm_msglen = sizeof(*rtm);
        rtm->rtm_version = 4;
        if(delete == 1)
                rtm->rtm_type = RTM_DELVR;
        else
                rtm->rtm_type = RTM_NEWVR;
        rtm->rtm_pid = pid;
        rtm->rtm_seq = seq;
        rtm->rtm_table = 254;  /* default FIB ID */
        if (send(rsock, (void *) rtm, rtm->rtm_msglen, 0) < 0)
        {
                /* Failed */
                printf("Failed to %s rtm message\n", (delete == 1)?"delete":"add");
                return -1;
        }

        /*

     * The VR has now been created. Routes can now be assigned to the new VR as
     * described in Wind River Network Stack Programmer's Guide

     */

        /* Close socket */

        if(close(rsock) < 0)
        {
                /* failed */
                printf("Failed to close socket\n");
                return -1;
        }
        return 0;
}
```

Previous topic: Managing Virtual Routers on page 178

14. VIRTUAL ROUTING REDUNDANCY PROTOCOL

1. About Virtual Router Redundancy Protocol (VRRP)

Virtual Router Redundancy Protocol (VRRP) specifies an election protocol that dynamically assigns responsibility for a virtual router to one of the VRRP routers on a LAN.

The Wind River network stack supports version 3 of the Virtual Router Redundancy Protocol (VRRP) for IPv4 and IPv6, version 2 of the protocol for IPv4, and version 1 for IPv4.

VRRP assigns a single IP address to multiple IP routers. One of those routers is designated the master router for that address, and it becomes the only router to serve that address. The other routers become backup routers. Nodes on that network use this VRRP address as a gateway.

If the VRRP address master fails, one of the backup routers starts serving that address. The nodes do not have to change anything, and their connectivity continues to work. VRRP only affects router nodes. Client nodes run without modification and do not need to be configured with VRRP.

VRRP is the protocol that makes sure that only one router serves the address. If you have a statically configured router on a LAN, you can use VRRP to assign backup routers using virtual IP addresses. This practice allows failover if the master router fails.

VRRP is fully described in RFCs 2338, 3768 and 5798.

Terminology

In this guide and in the context of VRRP, the term *virtual router* is used to describe an abstract object managed by VRRP that acts as a default router for hosts on a shared LAN. This is not the same as the Wind River virtual router implementation, which allows multiple routing tables per network stack instance. The Wind River implementation of virtual routing is not based on any published standard and is not associated with VRRP.

Next topic: Including VRRP in a Build on page 186

2. Including VRRP in a Build

To enable virtual router redundancy protocol (VRRP) in a VxWorks source build (VSB), enable the IPNET_VRRP option.

To manage VRRP from your VIP, include the **INCLUDE_IPVRRPD** component.

Configure VRRP as required with the parameters listed in table titled *VRRP Configuration Parameters and System Variables*. Be sure to select the appropriated version of VRRP with the VRRP_VERSION parameter.

Previous topic: About Virtual Router Redundancy Protocol (VRRP) on page 186
Next topic: VRRP Configuration Parameters and System Variables on page 186

3. VRRP Configuration Parameters and System Variables

To use the Virtual Router Redundancy Protocol (VRRP), you must configure address and interface lists on which to perform VRRP operations.

You can also specify an optional password. The table below lists the parameters used to configure VRRP at build time and the system variables used for runtime configuration.

Parameter Name, System Variable Name, and Description	Default and Type
VRRP_VERSION ipnet.vrrp.version Defines the VRRP version that will be used. The Wind River network stack supports version 3 of VRRP for IPv4 and IPv6 (as implemented in RFC 5798), version 2 of the protocol for IPv4 (as implemented in RFC 3768), and version 1 for IPv4 (as implemented in RFC 2338). To select the appropriate version, set the *VRRP_VERSION* parameter to either *1* , *2* , or *3* as required.	"2" char *
VRRP_IFLIST_VRIDS_IPADDR ipnet.vrrp.*VRRP_IFLIST_VRIDS.* IP_Addresses This component specifies one or more IPv4 addresses associated with this virtual router for each interface VRID pair. The IP address list is specified using the format *ifparam =value* . Each pair of *ifparam =value* is semicolon-separated. For example: **eth0.1=10.130.2.254;eth1.2=10.130.3.254**	NULL char *
VRRP_IFNAME_LIST ipnet.vrrp.interfaces This component specifies a list of interfaces where this router should perform VRRP operations. The format is a space-separated list of interface names. For example: eth0 eth1. The default list is empty, which means the daemon shuts down if it is started.	" "
VRRP_IFLIST_VRIDS ipnet.vrrp.*VRRP_IFNAME_LIST* .vrids This component specifies a list of virtual IDs defined on this interface. Must be >= 1 and <= 255. The vrids list is specified using the format *ifparam =value* . Each pair of *ifparam =value* is semicolon-separated. For example: eth0=1;eth1=2	NULL char *
VRRP_AUTH_DATA ipnet.vrrp.*VRRP_IFLIST_VRIDS.* Authentication_Data This component specifies the VRRP password. Maximum 8 characters. For example: eth0.1=secret1;eth1.2=secret2.	NULL

Parameter Name, System Variable Name, and Description	Default and Type
VRRP_AUTH_TYPE ipnet.vrrp.*VRRP_IFLIST_VRIDS.* Authentication_Type This component specifies the VRRP password type (none = *0*, plain text = *1*). It is only supported in VRRP version 1. For example: eth0.1=0;eth1.2=0.	NULL
VRRP_IFLIST_VRIDS_ADV_INTERVAL ipnet.vrrp.*VRRP_IFLIST_VRIDS* .Advertisement_Interval This component specifies the time interval (in centiseconds) between VRRP advertisement messages for each interface VRID pair. The value must be between 1 and 4095. The default is 100. The advertisement interval list is specified using the format *ifparam* =*value* . Each pair of *ifparam* =*value* is semicolon-separated. For example: eth0.1=1;eth1.2=100.	NULL
VRRP_IFLIST_VRIDS_PREEMPT_MODE ipnet.vrrp.*VRRP_IFLIST_VRIDS* .Preempt_Mode This component controls whether a higher priority backup router preempts a lower priority master for each interface VRID pair. The preempt mode list is specified using the format *ifparam* =*value* , where 1=true. Each pair of *ifparam* =*value* is semicolon-separated. For example: eth0.1=1;eth1.2=1	NULL
VRRP_IFLIST_VRIDS_PRIORITY ipnet.vrrp.*VRRP_IFLIST_VRIDS* .Priority This component specifies the priority value to be used in master election by this virtual router for each interface VRID pair. The value must be between 1 and 254. This range is available for virtual routers backing up the master virtual router. 255 is reserved by the system; it means that this router owns the address and starts as the master. The default priority is 100. The priority list is specified using the format *ifparam* =*value* . Each pair of *ifparam* =*value* is semicolon-separated. For example: eth0.1=100;eth1.2=254. Do not set the priority for the virtual router that owns the protected IPv4/IPv6 address. The system recognizes that it is the address owner and assigns the priority of 255 to it automatically. In addition, note that ipnet_vrrpd_show( ) only ( )shows the configured priority, which is different from actual the runtime priority. This means that for the IP address owner, ipnet_vrrpd_show( ) shows the default priority of 100, but it actually runs with the priority of 255.	NULL

Parameter Name, System Variable Name, and Description	Default and Type
Do not set the priority for the virtual router that owns the protected IPv4/IPv6 address. The system recognizes that it is the address owner and automatically assigns the priority of 255 to it. In addition, note that ipnet_vrrpd_show( ) only shows the configured priority, which is different from the actual run-time priority. This means that the IP address owner, ipnet_vrrpd_show( ) shows the default priority of 100, but it actually runs with the priority of 255.	
VRRP_INSTALL_STATE_CHANGED_CALLBACK_HOOK This parameter determines if the user-defined VRRP callback routine as specified by the macro **VRRP_STATE_CHANGED_CALLBACK_HOOK** should be installed. If FALSE, the **VRRP_STATE_CHANGED_CALLBACK_HOOK** configuration is not applicable.	BOOL FALSE
VRRP_STATE_CHANGED_CALLBACK_HOOK This user-defined VRRP callback routine is called when the VRRP group is changed. The input parameter is the VRRP configuration entry and the new state.	ipnet_vrrpd_statechange_default_callback FUNCPTR
VRRP_TASK_PRIORITY This parameter sets the priority of the task. Changing the value has risks.	
VRRP_DEBUG_ENABLE This parameter enables debugging tasks. Set the value to 1 to enable debugging.	

Iv6 Parameter Name, System Variable Name, and Description	Default and Type
VRRP_IFLIST_VRIDS_IPV6ADDR ipnet.vrrp. *VRRP_IFLIST_VRIDS* .IPV6_Addresses This component specifies one or more IPv6 addresses associated with this virtual router for each interface VRID pair. The first address must be the IPv6 link-local address associated with the virtual router. The IPv6 address list is specified using the format *ifparam* =*value* . Each pair of *ifparam* =*value* is semicolon-separated. For example:	NULL

WNDRVR

Iv6 Parameter Name, System Variable Name, and Description	Default and Type
eth0.1=fe80::787a:c0ff:fea8:c803,1::100;eth1.2=fe80::787a:c0ff:fea8:c804,2::100	
VRRP_IFLIST_VRIDS_IPV6_PRIORITY ipnet.vrrp.*VRRP_IFLIST_VRIDS.* IPV6_Priority This parameter specifies the priority value for IPv6 VRRP groups used in the master election by the virtual router for each interface VRID pair. The value must be between 1-254. Values 1-254 are available for virtual routers to back up the master virtual router. The value 255 is reserved by the system. This means this router owns the address and starts as master. The default priority is 100. The priority list is specified using the format *ifparam=value* . Each pair of *ifparam=value* is separated by a semicolon. For example, eth0.1=100;eth1.2=254.	NULL char *
VRRP_IFLIST_VRIDS_IPV6_PREEMPT_MODE ipnet.vrrp.*VRRP_IFLIST_VRIDS.IPV6_* Preempt_Mode This parameter controls whether a higher priority backup router preempts a lower priority master for each interface VRID pair. Default: 1 (true). The preempt mode list is specified using the format *ifparam=value* . Each pair of *ifparam=value* is semicolon separated. For example, eth0.1=1;eth1.2=1.	NULL char *
VRRP_IFLIST_VRIDS_IPV6_ADV_INTERVAL ipnet.vrrp.*VRRP_IFLIST_VRIDS.IPV6* _Advertisement_Interval This parameter time interval (in centiseconds) for IPv6 VRRP groups between VRRP advertisement messages for each interface VRID pair. The value must be 1- 4095. The default value is 100. The advertisement interval list is specified using the format *ifparam=value* . Each pair of *ifparam=value* is semicolon separated. For example, eth0.1=100;eth1.2=100".	NULL char *

Previous topic: Including VRRP in a Build on page 186
Next topic: Starting the VRRP Daemon on page 190

4. Starting the VRRP Daemon

The Virtual Router Redundancy Protocol (VRRP) daemon does not start automatically at boot time.

You can start it in any of the following ways:

- From the shell with the command interpreter as follows:

```
[vxWorks *] # ipd start ipnet_vrrpd
```

- From the shell with C interpreter as follows:

```
-> ipcom_ipd_start("ipnet_vrrpd")
```

- Programmatically as follows:

```
ipcom_ipd_start("ipnet_vrrpd");
```

It is useful to check VRRP daemon status with the ipd list command (**INCLUDE_IPD_CMD**), because the VRRP daemon will not start successfully if the VRRP parameters are not configured correctly. For example, if the VRRP_IFLIST_VRIDS_PRIORITY parameter is set to **255**, then the VRRP daemon will not start.

Previous topic: VRRP Configuration Parameters and System Variables on page 186
Next topic: Getting VRRP Status on page 191

5. Getting VRRP Status

Use ipnet_vrrpd_show( ) to display the virtual router redundancy protocol (VRRP) daemon status.

For example:

```
-> ipnet_vrrpd_show

interface   vrid   adv-interval   priority
  virtual-address              state
-------------------------------------------------------------------------------
simnet0     1      300            100    192.168.200.100              BACKUP

simnet0     2      300            150    fe80::787a:c0ff:fea8:c811    BACKUP
                                           1::100
value = 0 = 0x0
->
```

Previous topic: Starting the VRRP Daemon on page 190

15. TELNET SERVER

1. About the Telnet Server

Telnet provides command line login sessions between hosts on a network. The Wind River implementation of telnet works over both IPv4 and IPv6.

You can use either the VxWorks native shell or the IPCOM shell with the Telnet server.

2. Configuring and Building Telnet

Several telnet components must be included and configured to build functional Telnet support..

The Wind River Network Stack has the following Telnet configuration components:

- TELNET client (INCLUDE_TELNET_CLIENT)
- TELNET/FTP password protection (INCLUDE_SECURITY)
- Telnet Server (INCLUDE_IPTELNETS)

TELNET client

The **INCLUDE_TELNET_CLIENT** component pulls in the **telnetcLib** module and the VxWorks **telnet** client, a lightweight implementation of the client side of the Telnet protocol (RFC 854). This client works over both IPv4 and IPv6, provides all the essential features of Network Virtual Terminal (NVT), and can function as an interface between terminal and terminal-oriented processes. For details, see the **telnetcLib** reference entry.

Note: By default, the client sends an **ECHO** at start up. It works in default mode only when it receives **DO** and **DONOT** from a remote **telnet** server. An **ECHO** instructs the server to retransmit data characters it receives over the TELNET connection back to the sender.

Telnet Authentication

To implement authentication for Telnet, you must include the INCLUDE_SECURITY component.

3. Telnet Server

The telnet server enables other machines or users to remotely log in to the target and access the target's shell.

This allows those remote users to execute shell commands and programs whose output and input are displayed on the remote terminal. The telnet server connects by default to the VxWorks shell.

The telnet server implements the following features:

- allows multiple connections
- listens to connections from both IPv4 and IPv6 sockets if these are available in the TCP/IP stack
- authenticates users

- allows users to connect to an underlying shell
- optionally echoes user-typed characters

The table below describes the parameters you can use to configure the Telnet server.

Parameter Name, System Variable Name, and Description	**Default and Type**
IPCOM_TELNET_CFGFILE The full path of the configuration file that VxWorks will use to configure the Telnet server parameters. When no file is specified (default setting), the Telnet server parameter values are based on the VIP configuration. For more information, see *File-Based Configuration of the Telnet Server on page 195*.	char *
IPCOM_TELNET_DELAYEDSTART When set to FALSE, the Telnet server is automatically started upon booting VxWorks. When set to TRUE, the Telnet server is not started upon booting VxWorks. The Telnet server can be started with an ipd command. For more information, see *File-Based Configuration of the Telnet Server on page 195*.	**FALSE** BOOL
IPCOM_TELNET_PORT **ipcom.telnet.port**	**"23"** char *
IPCOM_TELNET_MAX_CLIENTS **ipcom.telnet.max_sessions** Controls the maximum number of simultaneous telnet clients. If set to 0, the number is unlimited.	**"0"** char *
ipcom.telnet.authenticate Available as a system variable only, this parameter defines if users must authenticate (**1** = authentication is enabled and **0** = authentication disabled).	**"0"** char *
ipcom.telnet.task_priority Available as a system variable only, this parameter sets the priority of the ipcom_telnetd task. The range is from 1 - 7 (IPCOM_PROC_PRIO_1 to IPCOM_PROC_PRIO_7).	**"4"** char *

Parameter Name, System Variable Name, and Description	Default and Type
ipcom.shell.prompt Available as a system variable only, this parameter specifies the IPCOM shell prompt. The supported characters are as follows: • \p - shell process name • \P - shell process id • \i - the IP address of the target • \w - the current working directory • \W - the basename of the current working directory • \V - ipcom_getenv( )(**"VR"**)	"\\i#\\V> " char *
ipcom.shell.greeting Available as a system variable only, this parameter specifies the welcome text printed upon starting the IPCOM shell.	\n\nWelcome to IPCOM SHELL. Type help for a list of commands.\n\n"" char *

16. FILE-BASED CONFIGURATION OF THE TELNET SERVER

1. About File Based Configuration of the Telnet Server

By default, the Telnet server parameter values are set at VxWorks image project (VIP) configuration time. The file-based configuration functionality allows you to reconfigure Telnet server parameter values at boot time.

You reconfigure Telnet server parameter values with a list of system variables stored in a text-based configuration file. The file path and file name are specified by a VIP parameter. This configuration method provides an easy way to adapt a system to new networking environments by simply updating the configuration file contents and then transferring the file to the target before starting the Telnet server.

A delayed start functionality is also available to prevent the Telnet server from starting automatically during the VxWorks boot process. The delayed start functionality can be used to explicitly start the Telnet server after VxWorks has booted or at runtime from your application.

Next topic: Enabling and Disabling File-Based Configuration on page 195

2. Enabling and Disabling File-Based Configuration

To enable the configuration of the Telnet server from a configuration file, you must specify the full path to the configuration file as part of the IPCOM_TELNET_CFGFILE VIP parameter.

To disable the configuration of the Telnet server from a configuration file, set the IPCOM_TELNET_CFGFILE VIP parameter to NULL (**""**). The Telnet server configuration is then based on the VIP configuration.

For information on configuring the IPCOM_TELNET_CFGFILE VIP parameter, see *Configuring and Building Telnet*.

For information on startup scenarios for the Telnet server, see Startup Scenarios for the Telnet Server on page 198.

For information on operational requirements for the target and Telnet server, see Operational Requirements on page 199.

Previous topic: About File Based Configuration of the Telnet Server on page 195
Next topic: Sample Telnet Server Configuration File on page 195

3. Sample Telnet Server Configuration File

Wind River provides a sample Telnet server configuration file that contains the list of valid system variables that can be configured from a configuration file.

The sample configuration file is at:

installDir **/vxworks-7/pkgs_v2/net/ipnet/coreip/sample_cfg/etc/telnet.conf**

> Note: Only the specified system variables in the sample file are configurable from a configuration file.

To correlate Telnet server VxWorks image project (VIP) parameter names to Telnet system variables listed in the sample configuration file, see *Configuring and Building Telnet on page 192*.

Note: To prevent unnecessary reconfiguration of the VIP parameters, the configuration file should only contain system variables that must be changed. The remaining parameter values will be taken from the VIP configuration.

Previous topic: Enabling and Disabling File-Based Configuration on page 195
Next topic: Configuration File Format on page 196

4. Configuration File Format

The configuration file is a text-based file you can edit with a standard text editor.

To configure a parameter from a configuration file, a valid system variable name must be followed by a valid value (system variable / value pair) and be on a single line.

A configuration file can also contain comments.

Note: The configuration file does not accept Escape characters (\).

System Variable and Value Pair on page 196A system variable/value pair consists of a system variable name and a value. Valid system variables are provided in the sample configuration file.
Comment Delimiter on page 197 The comment delimiter for the configuration file is the pound (**#**) character.

Previous topic: Sample Telnet Server Configuration File on page 195
Next topic: Configuration File Processing on page 197

4.1. System Variable and Value Pair

A system variable/value pair consists of a system variable name and a value. Valid system variables are provided in the sample configuration file.

A system variable/value pair can start in any column of a given line. A system variable and name must be separated by least one space or tab.

Each line can only contain a single system variable/value pair.

Boolean, integer, floating point, and string values can be listed with or without double quotation marks. Example valid line content follows:

```
systemVariableName_A   TRUE
systemVariableName_B   2
systemVariableName_C   192.168.1.1
systemVariableName_D   String

systemVariableName_A   "TRUE"
systemVariableName_B   "2"
systemVariableName_C   "192.168.1.1"
systemVariableName_D   "String"
```

Parent topic: Configuration File Format on page 196

4.2. Comment Delimiter

The comment delimiter for the configuration file is the pound (**#**) character.

The character is used as follows:

```
# This is a comment line.
```

You can add a comment on the same line as a system variable/name pair. For example:

```
systemVariableName_A   FALSE  # This is a boolean value.
systemVariableName_B   22  # This is an integer value.
systemVariableName_C   192.168.2.2 # This is a floating point value.
systemVariableName_D   String  # This is a string value.
```

Parent topic: Configuration File Format on page 196

5. Configuration File Processing

Understanding configuration file processing rules can help you to avoid errors and troubleshoot effectively.

- No empty lines or comment lines are processed
- If a configuration file is not found, no processing is done and the system variables retain their values from the VxWorks image project (VIP) configuration.
- If a configuration file is located, the system processes the configuration file line-by-line. The values in the configuration file overwrite the values set at VIP configuration time.
- Lines with errors such as misspelled system variable names and missing parameter values are ignored (skipped). Hence, a system variable associated with a faulty line entry retains the value set at VIP configuration time.

Note: If a system variable/value pair is listed more than once in a file, the last valid occurrence is used to configure the system variable. No error is reported as part of the diagnostic output.

Previous topic: Configuration File Format on page 196
Next topic: Diagnostics on page 197

6. Diagnostics

Enable and disable diagnostics from within the configuration file. When enabled, the configuration status of the system variables is sent to the console.

To enable diagnostics, set the **cfgfile.diag** variable in the configuration file to **TRUE**. To disable, set to **FALSE**. For example:

```
cfgfile.diags      FALSE
```

When enabled, the diagnostics process generates output and provides status on the action performed on each system variable.

An updated system variable is labeled as **Updated**. The output format is as follows:

```
Variable: systemVariableName Value:
value Result: Updated
```

A skipped update for a system variable is labeled as **Unrecognized variable (Skipping)**. The output format is as follows:

```
Variable: systemVariableName Value:
value Result: Unrecognized variable (Skipping)
```

Previous topic: Configuration File Processing on page 197
Next topic: Verifying System Variables on page 198

7. Verifying System Variables

To verify the configuration of the system variables, you can display the system variables and their content from the command shell.

To view system variables, use the following command:

```
vxWorks# sysvar list serviceName
```

Note: To access the sysvar shell command, you must add the INCLUDE_IPCOM_SYSVAR_CMD component to your VIP.

Previous topic: Diagnostics on page 197
Next topic: Startup Scenarios for the Telnet Server on page 198

8. Startup Scenarios for the Telnet Server

The table below lists the possible startup scenarios set by the IPCOM_TELNET_CFGFILE and IPCOM_TELNET_DELAYEDSTART VIP configuration parameters.

IPCOM_TELNET_CFGFILE VIP Parameter Setting	IPCOM_TELNET_DELAYEDSTART VIP Parameter Setting	Operation
Null file path	FALSE	The Telnet server starts automatically at boot time. Telnet server parameter values are based on the VIP configuration. This is the default configuration.
Valid file path	FALSE	The Telnet server starts automatically at boot time. Telnet server parameter values are based on valid system variable names and values listed in the configuration file.
Null file path	TRUE	The Telnet server is not started at boot time.

IPCOM_TELNET_CFGFILE VIP Parameter Setting	IPCOM_TELNET_DELAYEDSTART VIP Parameter Setting	Operation
		The Telnet server can be started with an ipd start command executed from a startup script or programmatically. Telnet server parameter values are based on the VIP configuration.
Valid file path	**TRUE**	The Telnet server is not started at boot time. The Telnet server can be started with an ipd start command executed from a startup script or programmatically. Telnet server parameters are based on valid Telnet server system variable names and values listed in the configuration file.

Note: To access the ipd shell command, you must add the INCLUDE_IPD_CMD component to your VIP.

Previous topic: Verifying System Variables on page 198
Next topic: Operational Requirements on page 199

9. Operational Requirements

To enable parameter configuration from a file, the target hosting the Telnet server must include read/write mass storage capability.

A configuration file can be modified by an external host and transferred to the target using one of the following methods:

- By transferring the file from a file transfer protocol (FTP) server to the target.
- By mounting the target's mass storage device as a USB disk and using desktop system facilities to update the configuration file.
- By using a removable mass storage device supported by the target.

Previous topic: Startup Scenarios for the Telnet Server on page 198

17. NETWORK ADDRESS EVENT HOOKS

1. Network Address Event Hook Facility

The **ipaddrEventLib** library allows you to add and manage hook functions for IP network address events.

Wind River recommends that the hook functions themselves only be used to print or record information.

The use of APIs in hook functions must be restricted to the same APIs as are allowed for ISRs (for information in this regard, see the *VxWorks 7 Platform Programmer's Guide: Interrupt Service Routines: ISRs*). Hook functions must not make any calls into the network stack. For example, use of sockLib( ), selectLib( ), and ioct( ) calls on sockets are prohibited. Calls that may block could cause the network stack process to hang, leaving the system in an undefined state.

The address events to which hook functions can be attached are the following:

- an IPv4 address is added
- a conflict for an IPv4 address is detected
- an IPv4 address passes duplicate address detection
- if an IPv4 address is removed before DAD_SUCCESS or DAD_FAILED has been generated
- an IPv6 address is added
- a conflict for an IPv6 address is detected
- an IPv6 address passes duplicate address detection
- if an IPv6 address is removed before DAD_SUCCESS or DAD_FAILED has been generated

The **ipaddrEventLib** library provides the following functions:

- ipAddressEventHookAdd( ) adds a user-defined hook function to be called when a specific type of address event occurs.
- ipAddressEventHookDelete( ) removes a hook function.

Next topic: Network Address Event Hook Code Example on page 200

2. Network Address Event Hook Code Example

```
/*
* Must call ipAddressEventHookAdd(myIpAddressEventHook) to install the
 * user-implemented function myIpAddressEventHook() as a hook.
*/

#include <vxWorks.h>
#include <stdlib.h>
#include <stdio.h>
#include <errnoLib.h>
#include <ipaddrEventLib.h>
#include <netinet/in.h>
#include <netinet6/in6.h>

void myIpAddressEventHook
    (
    int eventType,
```

```
    unsigned int ifindex,
    void * address,
    void * mac
    )
    {
    /* define your local variables */

     if ((IP_ADDREVENT_BASE >= eventType) || (IP_ADDREVENT_MAX <= eventType))
        return; /* do nothing and return */
     if ((0 == ifindex) || (NULL == address) || (NULL == mac))
        return; /* do nothing and return */

switch (eventType)
    {
    case IP_ADDREVENT_INET_DADBEGIN:
       /* add print code here */
       break;
     case IP_ADDREVENT_INET_DADFAILED:
       /* add print code here */
       break;
     case IP_ADDREVENT_INET_DADSUCCESS:
       /* add print code here */
       break;
     case IP_ADDREVENT_INET_DADABORTED:
       /* add print code here */
       break;
     case IP_ADDREVENT_INET6_DADBEGIN:
       /* add print code here */
       break;
     case IP_ADDREVENT_INET6_DADFAILED:
       /* add print code here */
       break;
     case IP_ADDREVENT_INET6_DADSUCCESS:
       /* add print code here */
       break;
     case IP_ADDREVENT_INET6_DADABORTED:
       /* add print code here */
       break;
     default:
       /* add print code here */
       return; /* do nothing and return */
     }
return;
}
```

Previous topic: Network Address Event Hook Facility on page 200

18. SYSTEM LOG CONFIGURATION

1. About the System Log Facility

The system logging daemon captures all run time debugging information. It uses the same debug levels as Linux. You can use the system logging facility to diagnose complex networking problems. This facility is accessible via the syslog shell command or public application program interfaces (APIs).

You have three opportunities to configure debug levels--when you compile the networking source code, when you build your VxWorks image project (VIP), and at run time.

- When you compile the networking source code, you must specify the default system logging level in your VxWorks source build (VSB) project. This setting determines the logging levels that will be compiled into the module. For further information on specifying the default debug level, see Source Build Configuration Options on page 6.
- When you build your VIP, you specify the default system logging level for all modules included in your project. This setting sets the initial logging level in effect at system boot. It can be further modified at run time. For further information, see Source Build Configuration Options on page 6.
- At run time, you specify the debug levels that are actually output to the system log. For further information, see Setting Priority Levels on page 203.

Before configuring the system log facility, you may need to recompile the source code with support for debugging. For further information, see Enabling Debug in the Source Code Build on page 202.

You must also configure your VIP with support for system logging. For further information, see Including System Logging in the Kernel on page 202.

Next topic: Enabling Debug in the Source Code Build on page 202

2. Enabling Debug in the Source Code Build

The syslog components can be used with any source build configuration, but the default debug levels are different in debug and speed builds.

By default, levels **INFO** and up are logged for debug builds, while levels **ERR** and up are logged for speed builds. Note also that only debug builds allow syslog output to the console. File output is available in all build modes.

To set the source code build mode, specify one of the following options for the FEATURE_IPNET_BUILD option:

- BUILD_MODE_speed
- BUILD_MODE_debug

Previous topic: About the System Log Facility on page 202
Next topic: Including System Logging in the Kernel on page 202

3. Including System Logging in the Kernel

When you configure your VxWorks image project (VIP), include the following components:

- INCLUDE_IPCOM_SYSLOGD_CMD
- INCLUDE_IPCOM_SYSLOGD_USE_LOG_FILE
- INCLUDE_DISK_UTIL

Set values for the following variables:

- IPCOM_SYSLOGD_DEFAULT_PRIORITY
- IPCOM_SYSLOGD_QUEUE_MAX
- IPCOM_SYSLOGD_LOG_FILE

With system logging enabled, you can then set priority levels for each individual module in your application.

Previous topic: Enabling Debug in the Source Code Build on page 202
Next topic: Setting Priority Levels on page 203

4. Setting Priority Levels

You can set priority levels when you compile the networking source code, when you build your VxWorks image project (VIP), or at run time.

You can also set a default priority level for all modules, or individual priority levels for specific modules.

Specifying Priority Levels at Build Time

The configuration component IPCOM_SYSLOGD_DEFAULT_PRIORITY sets the default debug level for all modules. The following debug levels are available:

IPCOM_LOG_EMERG

- Not used

IPCOM_LOG_CRIT

- Incorrect configuration
- Cannot start daemon
- Daemon encountered unrecoverable error, terminating.
- IPCOM unusable (crash, kernel panic, etc.)

IPCOM_LOG_ERR

- Local error (that is, not caused by peer) but possible to continue --for example, out of memory
- Bad configuration parameters given

IPCOM_LOG_WARNING

- Remote protocol error (peer sends invalid packet)
- API usage error (incorrect parameters provided by calling application)
- Deprecated API used (Only log first call). Note: this should only be logged if IPCOM_LOG_USE_OF_DEPRECATED_API is defined.
- Configurable system limit reached--for example, packet pool empty
- Maximum number of simultaneous connections reached

IPCOM_LOG_NOTICE

- Not used

IPCOM_LOG_INFO

Important events from a usage/external/sysadmin perspective--for example:

- New peer/client initiates communication with daemon--for example, Secure Socket Shell (SSH), File Transfer Protocol (FTP), Dynamic Host Control Protocol (DHCP), etc. Not used for lower-level protocols such as Transmission Control Protocol (TCP)/User Datagram Protocol (UDP).
- Peer/client releases connection/session with daemon--for example, SSH, FTP, DHCP, etc.
- IPD events--for example, start, stop and reconfiguration of daemons

IPCOM_LOG_DEBUG

The product must still be working properly under normal load with DEBUG enabled. The capacity will of course be affected by the logging.

- Application level message (both send and receive). With hex dump where applicable.
- Application payload data can be omitted--for example, IPsec payload data.
- Important implementation events--for example, timeouts

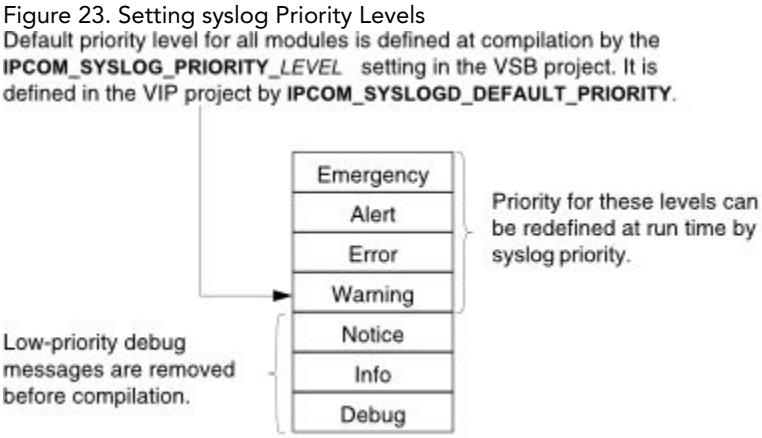
IPCOM_LOG_DEBUG2

This level should not be needed by most products. It is used for log entries that are excessive, which make the product not function robustly under normal load (due to the extra CPU load the logging generates).

- Function level trace
- Any implementation details, useful for debugging

Figure 1 on page 204 illustrates this functionality.

Figure 23. Setting syslog Priority Levels



Specifying Priority Levels by Module

You can also specify the default debug level for each individual module. There are two ways to set this level:

- in a VxWorks source build (VSB) project
- by editing the configuration file for an individual component

If a component supports system logging, you can instruct the component to use the default debug level defined by IPCOM_DEFAULT_SYSLOG_PRIORITY. If IPCOM_DEFAULT_SYSLOG_PRIORITY is not defined, the component uses the debug level defined in its configuration file. For further information, see Editing the Configuration File on page 205.

To instruct the component to use the debug level defined by IPCOM_DEFAULT_SYSLOG_PRIORITY, enable the *COMPONENT* _DEBUG parameter for that component. For example, for COMPONENT_IPEAP, enable IPEAP_DEBUG.

Note: Some components do not have a DEBUG parameter and thus only support the default level defined in the VSB configuration.

Editing the Configuration File

Some modules do not have VSB parameters that allow you to set the default debug level. In such a case, you set the debug level by editing in the configuration file*component* **_config.h**. This method can be used for all modules, even those having VSB options for the debug level.

There is one such file for each module, residing in one of the following locations:

- *installDir* **/vxworks-7/pkgs_v2/net/***layer***/cfg**
- *installDir***/vxworks-7/pkgs_v2/net/ipnet/***layer***/cfg**

For example, the configuration file for the SSH component is:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ssh-***x.x.x.x* **/h/ipssh_config.h**

Debug levels are specified in the following lines of code:

```
#ifdef IPSSH_DEBUG
#define IPSSH_SYSLOG_PRIORITY IPCOM_LOG_DEBUG
#else
#define IPSSH_SYSLOG_PRIORITY IPCOM_LOG_ERR
#endif
```

You can set separate levels for debug and optimized builds.

After altering this setting, you must recompile the VxWorks source code.

Note: The components COMPONENT_DOT1X and COMPONENT_BRIDGE do not support IPCOM system logging. These components use a separate method for system logging.

Listing Priority Levels for the Current Module

At run time, use the list option of the syslog shell command to list the priority levels of the current facilities:

```
[vxWorks *] # syslog list
syslog facility  priority
ipike       Error
ipl2tp       Error
ipssh       Debug
ipssl       Error
```

In the preceding example, IPIKE, IPL2TP, and IPSSL are set to priority Error, while IPSSH is set to priority Debug.

You can also get the log level for a particular facility, using the API ipcom_getlogmask_on( ). For example:

```
ipcom_getlogmask_on(IPCOM_LOG_IPSSH)
```

This function returns the priority of the specified facility, or -1 if the facility does not exist.

Specifying Priority Levels at Run Time

At run time, use sysvar "ipcom.syslogd.default_priority" to set the default priority level at run time. It controls the lowest level that will be logged after boot.

Specifying Priority Levels by Module

At run time, use the priority option of the syslog shell command to modify the priority for a specified facility.

First, a syslog list command displays the priorities for the current facilities:

```
[vxWorks *] # syslog list
syslog facility  priority
ipssh       Debug
ipssl        Error
```

Next, a syslog priority command modifies the priority for IPSSH to Critical.

```
[vxWorks *] # syslog priority ipssh Crit
syslog: facility ipssh priority set to Crit
```

Finally, a syslog list command verifies the new priority.

```
[vxWorks *] # syslog list
ipssh       Crit
ipssl       Error
```

You can also set the priority level for a particular facility, using the API ipcom_setlogmask_on( ). For example:

```
ipcom_setlogmask_on( 2, IPCOM_LOG_IPSSH)
```

This function returns the old priority if the priority was successfully updated, or -1 if the facility does not exist.

Previous topic: Including System Logging in the Kernel on page 202
Next topic: Sending System Log Messages on page 206

5. Sending System Log Messages

Use the echo option to print a message to the syslog with the specified priority.

For example, the following sequence of commands output the system log, prints **this is my string** with the priority level **Emergency**, and outputs the system log again, showing the new string.

The first command displays the existing log:

WNDRVR

```
[vxWorks *] # cat /ram/syslog
SAT MAY 26 05:00:12 2007: dhcps6[6068d120]: Error: Interface was not found
```

The second command prints the string, using the syslog echo command:

```
[vxWorks *] # syslog echo Emerg "this is my string"
```

The third command displays the log again, showing the new string:

```
[vxWorks *] #cat /ram/syslog
SAT MAY 26 05:00:12 2007: dhcps6[6068d120]: Error: Interface was not found
SUN MAY 27 08:53:00 2007: ipcom[605eafb0]: Emerg: this is my string
[vxWorks *] #
```

Specifying the Maximum Queue Depth for System Logging

At build time, use the parameter IPCOM_SYSLOGD_QUEUE_MAX to specify the maximum queue depth for the system logging facility.

Previous topic: Setting Priority Levels on page 203
Next topic: Specifying the System Log File on page 207

6. Specifying the System Log File

To set the name of the system log file at build time, specify a value for IPCOM_SYSLOGD_LOG_FILE. To disable the use of a log file, set this parameter to **IP_NULL**.

At run time, use the syslog log file command to change the name of the system log file.

```
[vxWorks *] # syslog log file /ram/mylog
```

You can also specify a log file name programmatically, using the ipcom_setlogfile( ) API.

Previous topic: Sending System Log Messages on page 206
Next topic: Clearing the System Log File on page 207

7. Clearing the System Log File

To clear the system log file, you must disable the syslog, manually remove the log file, and restart the syslog daemon.

The first command, syslog log file, closes the current file and opens a new file with the same name.

```
[vxWorks *] # cat /ram/syslog
MON MAY 28 06:04:35 2007: ipppp[6068f120]: Error: [ppp0 sec=0 lcp=1 auth=0
ipcp=0 ipv6cp=0] [ppp0] Failed to open driver: No such file or directory

[vxWorks *] # syslog log file /ram/syslog
```

The second command removes the log file:

```
[vxWorks *] # rm /ram/syslog
```

The third command restarts the syslog daemon:

```
[vxWorks *] # syslog log file /ram/syslog
```

Previous topic: Specifying the System Log File on page 207
Next topic: Specifying a UDP Address and Port for System Logging on page 208

8. Specifying a UDP Address and Port for System Logging

You can specify the User Datagram Protocol (UDP) address and port to which system log messages should be sent. Use ipcom_setlogudp( ).

For example:

```
ipcom_setlogudp("192.168.0.0", 514)
```

This function returns IPCOM_SUCCESS if the address and port were successfully set, or -1 if the address and port could not be updated.

You can also specify these settings programmatically, using the ipcom_setlogudp( ) function.

Previous topic: Clearing the System Log File on page 207

WNDRVR

19. RUN-TIME CONFIGURATION USING SYSTEM VARIABLES

1. About System Variables

The sysvar shell command lists and modifies global variables. The INCLUDE_IPCOM_SYSVAR_CMD component provides the sysvar command.

System variables are similar to UNIX environment variables, except that they are available throughout the system to any process. For example, if you are running an Internet Key Exchange (IKE) process, you can issue a sysvar command to alter a Network Address Translation (NAT) variable.

The **sysvar** command uses a treelike data structure for all the network components and services. For example, the system variable *iptcp.ConnectionTimeout* defines the number of seconds the network stack tries to create connection before giving up.

The system variable *ipssh.service.port_fwd* controls whether port forwarding can be used.

System variables are similar to the components you configure at build time using the Kernel Configuration Editor. Unlike kernel components, however, the sysvar command modifies parameters at run time.

> Note: Not all global variables can be changed at run time. Some parameters can only be reset at build time, using Workbench 4 or **wrtool**.

Next topic: Using System Variables on page 209

2. Using System Variables

Using system variables, you can configure many parameters in the network stack and other modules at run time. The sysvar shell command gives you access to these variables.

To make this command available, include INCLUDE_IPCOM_SYSVAR_CMD in your VIP.

The basic form of this command is:

```
[vxWorks *] # sysvar  subcommand  dynamicParameter  =  dynamicParameterValue
```

For example, to enable network proxy ARP, issue the following command:

```
[vxWorks *] # sysvar set -co ipnet.inet.EnableNetworkProxyArp 1
```

You can also use this command to list, get, set, and unset system variables. The following sections provide additional detail.

Listing System Variables

For example, to list system variables, issue the following command:

```
[vxWorks *] # sysvar list
System variables:
    ipcom.hostname=iptarget
    ipcom.syslogd.default_priority=2
```


WNDRVR

```
    ipcom.syslogd.queue.max=256
```
etc.

To restrict output to a particular branch of the tree structure, issue a command such as the following:

```
[vxWorks *] # sysvar list ipssh
System variables:
    ipssh.auth.max_fail=3
    ipssh.auth.pub_key.allowed=1
    ipssh.auth.pub_key.required=0
    ipssh.auth.pub_key_first=1
    ipssh.auth.pw.allowed=1
    ipssh.auth.pw.required=0
    ipssh.bind_addr=0.0.0.0
    ipssh.enc.3des=1
    ipssh.enc.aes=1
    ipssh.enc.arcfour=1
    ipssh.enc.blowfish=1
```
etc .

To further restrict output to a particular branch of the specified service name, issue a command such as the following:

```
[vxWorks *] # sysvar list ipssh.auth
System variables:
    ipssh.auth.max_fail=3
    ipssh.auth.pub_key.allowed=1
    ipssh.auth.pub_key.required=0
    ipssh.auth.pub_key_first=1
    ipssh.auth.pw.allowed=1
    ipssh.auth.pw.required=0
```

To filter selected fields, use a wildcard. The following example outputs only SSH authentication parameters beginning with **m**:

```
[vxWorks *] # sysvar list ipssh.auth.m*
System variables:
  ipssh.auth.max_fail=3
```

Creating a System Variable

The second command creates the new system variable and assigns it a value:

```
[vxWorks *] # sysvar set -c MyParam MyValue
sysvar: MyParam=MyValue ok
```

The third command verifies the creation of the system variable with the assigned value:

```
[vxWorks *] # sysvar get MyParam
sysvar: MyParam=MyValue
```

To create a read-only system variable, use the **-r** flag. After creation, this parameter cannot be modified or deleted.

The first command creates the new system variable and assigns it a value:

```
[vxWorks *] # sysvar set -cr NewParam NewValue
sysvar: NewParam=NewValue ok
```

The second command reads the value of the system variable:

```
[vxWorks *] # sysvar get NewParam
sysvar: NewParam=NewValue
```

The third command attempts to assign a new value to the system variable. This command fails because it is a read-only system variable.

```
[vxWorks *] # sysvar set NewParam VeryNewValue
sysvar: set failed : readonly entry
```

To create a new system variable, use the **set** option with the **-c** flag.

First, use a sysvar get command to search for the new system variable, in case it already exists:

```
[vxWorks *] # sysvar get MyParam
sysvar: 'MyParam' not found
```

Reading the Value of a System Variable

To read the value of a system variable, use the option **get**. The following example reads the value of **iptcp.ConnectionTimeout**:

```
[vxWorks *] # sysvar get iptcp.ConnectionTimeout
sysvar: iptcp.ConnectionTimeout=30
```

Deleting a System Variable

A subsequent **sysvar get** command verifies the removal:

```
[vxWorks *] # sysvar get iptcp.ConnectionTimeout
sysvar: 'iptcp.ConnectionTimeout' not found
```

Note: It is not possible to delete a read-only variable.

To remove a system variable, use the option unset. The following example removes the iptcp.ConnectionTimeout parameter:

```
[vxWorks *] # sysvar unset iptcp.ConnectionTimeout
sysvar: 'iptcp.ConnectionTimeout' unset ok
```

Assigning a Value to a System Variable

To assign a value to a system variable, use the option set. Use this option in conjunction with the **-o** flag and the options **list** and get to read a parameter, assign it a new value, and verify the assignment of the new value.

The first command displays the value for the *ipcom.hostname* variable:

```
[vxWorks *] # sysvar list ipcom.hostname
System variables:
    ipcom.hostname=iptarget
```

The second command assigns the value **MyTarget** to this variable, but fails because the parameter already exists:

```
[vxWorks *] # sysvar set ipcom.hostname MyTarget
sysvar: set failed : duplicate entry
```

The third command overwrites the existing value of *ipcom.hostname* with **MyTarget**:

```
[vxWorks *] #  sysvar set -o ipcom.hostname MyTarget
sysvar: ipcom.hostname=MyTarget ok
```

The fourth command verifies the new value:

```
[vxWorks *] # sysvar get ipcom.hostname MyTarget
sysvar: ipcom.hostname=MyTarget
```

For more information, see the complete reference entry on the sysvar shell command.

Previous topic: About System Variables on page 209

20. CONFIGURATION PARAMETER AND SYSTEM VARIABLE REFERENCE

1. About VxWorks Network Stack Configuration Parameters

Most networking protocols can be configured using two methods of configuration as follows:

- Build-time configuration, using either Workbench 4 or wrtool.
- Run-time configuration, using the sysvar shell command or other shell command.

In some cases, there is a one-to-one mapping between build-time configuration parameters and system variables. Where such a correspondence exists, you can configure the system variable either at build time or at run time. In other cases, a build-time configuration component or a system variable has no counterpart.

Next topic: IPv4 Network Stack Configuration Parameters on page 213

2. IPv4 Network Stack Configuration Parameters

VIP and System Variable Configuration Parameters

The table below lists the INCLUDE_IPCOM_USE_INET configuration parameters and their default values. Where applicable, the corresponding system variable is also noted for run-time configuration. Modify the values as needed.

Parameter Name, System Variable Name, and Description	Default
INET_BASE_HOP_LIMIT ipnet.inet.BaseHopLimit The default value for the time to live field for IPv4 unicast datagram packets.	"64 "
INET_BASE_REACHABLE_TIME ipnet.inet.BaseReachableTime Duration, in seconds, that an entry in the Address Resolution Protocol (ARP) cache is in reachable state.	"30 "
INET_BASE_RETRANSMIT_TIME ipnet.inet.BaseRetransmitTime Number of seconds to wait between retransmits.	"1 "

Parameter Name, System Variable Name, and Description	Default
INET_DELAY_FIRST_PROBE_TIME ipnet.inet.DelayFirstProbeTime Duration, in seconds, to wait for a stale Address Resolution Protocol (ARP) entry to become reachable before forcing a probe. Packets are sent using the stale entry during this time.	"5 "
INET_DELETE_ADDR_ON_DUPLICATE_DETECT **ipnet.inet.DeleteAddressOnDuplicateDetect** Automatically remove a configured IPv4 address upon duplicate address detection (gratuitous ARP).	"yes "
INET_DST_CACHE_TO_LIVE_TIME ipnet.inet.DstCacheToLive Number of seconds an entry is kept in the destination cache. The destination cache contains FIB entries created in response to redirect- and need-frag Internet Control Message Protocol (ICMP) messages.	"600 **PRESERVE nameliteral-uc"**
INET_DISABLE_BROADCAST_FWD ipnet.inet.DontForwardBroadcast Controls the forwarding of broadcasts. Set to 1 to disable the forwarding of broadcasts or set to 0 to enable.	"1 "
INET_ENABLE_PMTUD_RFC5927 ipnet.inet.EnablePmtudRfc5927 Hardens TCP against ICMP denial-of-service attacks. Set to "yes" to enable this algorithm.	"yes"
INET_ICMP_IGNORE_BROADCAST ipnet.inet.IcmpIgnoreBroadcast Set to 1 if the network stack should ignore ICMP requests from broadcast addresses.	"1 "

Parameter Name, System Variable Name, and Description	Default
INET_ICMP_IGNORE_ECHO_REQ ipnet.inet.IcmpIgnoreEchoRequest Controls if the stack should answer to ICMP echo request messages. Set to 0 if the stack should answer to echo requests.	"0 "
INET_ICMP_IGNORE_TIMESTAMP_REQ ipnet.inet.IcmpIgnoreTimestampRequest Controls if the stack should answer to ICMP timestamp request messages. Set to **"0"** if the stack should answer to timestamp requests.	"0 "
INET_ICMP_RATE_LIMIT_BUCKET_SIZE ipnet.inet.IcmpRatelimitBucketsize The number of ICMP messages that the stack is allowed to send in the interval specified by INET_ICMP_RATE_LIMIT_INTERVAL. For example, if INET_ICMP_RATE_LIMIT_BUCKET_SIZE="10" and INET_ICMP_RATE_LIMIT_INTERVAL=100, then up to ten ICMP messages can be sent in a 100 msec period. If more messages are sent, they are discarded. If no ICMP messages are sent for 1000 msec, the limit is still ten messages for the next 100 msec period.	"25000 "
INET_ICMP_RATE_LIMIT_INTERVAL ipnet.inet.IcmpRatelimitInterval The Icmp Ratelimit Interval millisecond interval.	"1000 "
INET_ICMP_REDIRECT_RECEIVE **ipnet.inet.IcmpRedirectReceive** Set to != 0 if redirect messages should be accepted and processed.	"1 "
INET_ICMP_REDIRECT_SEND ipnet.inet.IcmpRedirectSend Set to **2** if the stack should send a redirect and forward the original message (default). Set to **1** if the stack should send a redirect but discard the original message. Set to **0** if the stack should not send a redirect but should still forward the original message.	"2 "

Parameter Name, System Variable Name, and Description	Default
Set to **-1** if the stack should not send a redirect but should discard the original message.	
INET_ICMP_SEND_DST_UNREACHABLE ipnet.inet.IcmpSendDestinationUnreachable Set to **!= 0** if the stack should send destination unreachable messages.	"1 "
INET_ICMP_SEND_TIME_EXCEEDED **ipnet.inet.IcmpSendTimeExceeded** Set to **!= 0** if the stack should send time exceeded messages.	"1 "
INET_MAX_APP_SOLICIT ipnet.inet.MaxApplicationSolicit Maximum number of resolve messages that are sent to the routing/netlink sockets if ARP probes fail (or are disabled).	"1 "
INET_MAX_MULTICAST_SOLICIT ipnet.inet.MaxMulticastSolicit Maximum number of ARP request messages the stack should resend before staring to use application probes.	"9 "
INET_MAX_PKTS_PENDING ipnet.inet.MaxPending Maximum number of packets that can be waiting before IPv4 to link layer resolution is finished.	"3 "
INET_MAX_UNICAST_SOLICIT ipnet.inet.MaxUnicastSolicit Maximum number of unicast ARP request messages that the stack should send before starting to use multicast solicits.	"1 "
INET_MIN_MTU_SIZE Minimum MTU size, meaning the smallest path MTU expected in any route. All Internet routers should be able to handle 576 octets.	576

Parameter Name, System Variable Name, and Description	Default
This value is used by some ICMP messages that include data from the packet that caused an ICMP message to be sent.	
INET_NBR_CACHE_TO_LIVE_TIME ipnet.inet.NeighborCacheToLive Duration, in seconds, that can pass before a stale entry in the ARP cache is deleted. For more information, see *Configuring the ARP Entry Cache Time*.	"1200 "
INET_RANDOM_IP_ID Random ip id for IPv4. Set to 1 to enable this option.	"0 "
INET_SEND_GRATUITOUS_ARP ipnet.inet.SendGratuitousArp Send gratuitous ARP when addresses are added.	"yes "
INET_UDP_CHECKSUM Set to 0 to disable UDP checksum for IPv4.	"1 "

System Variable Only Configuration Parameters

the table below lists the INCLUDE_IPCOM_USE_INET component system variables and their descriptions for the system variables that have no configuration parameter counterpart.

System Variable Name and Description	Default
ipnet.inet.EnablePathMtuDiscovery Enable/disable path maximum transmission unit (MTU) discovery algorithm for IPv4.	1

Previous topic: About VxWorks Network Stack Configuration Parameters on page 213
Next topic: Global Configuration Parameters for Auto IP and Address Conflict Detection (ACD) on page 217

3. Global Configuration Parameters for Auto IP and Address Conflict Detection (ACD)

VIP and System Variable Configuration Parameters

The table below lists the configuration parameters for INCLUDE_IPAIP_GLOBAL_CONFIGS. You can modify these values at run time by using the indicated system variable.

Note: The values set for configuration parameters at build time or with the sysvar command at run time are used for both Auto IP and ACD.

Parameter Name, System Variable Name, and Description	Default
INET_IPAIP_ANNOUNCE_WAIT ipnet.inet.linklocal. ANNOUNCE_WAIT Time (in seconds) that Auto IP waits before it starts to announce the address it selects. It will not defend the address if it hears another announcement during this period.	"2" char *
INET_IPAIP_ANNOUNCE_INTERVAL ipnet.inet.linklocal. ANNOUNCE_INTERVAL Seconds between announcements.	"2" char *
INET_IPAIP_DEFEND_INTERVAL ipnet.inet.linklocal. DEFEND_INTERVAL Minimum interval between defensive ARPs.	"10" char *
ipnet.inet.AddressConflictDetect Enable or disable address conflict detection, 1 = enable.	"0" char *
INET_IPAIP_MAX_CONFLICTS ipnet.inet.linklocal. MAX_CONFLICTS Maximum number of address collisions that Auto IP allows before it applies rate-limiting when it creates the next address.	"10" char *
INET_IPAIP_PROBE_MAX ipnet.inet.linklocal. PROBE_MAX The maximum time (in seconds) to send the next probe. Auto IP sends the next probe between	"3" char *

Parameter Name, System Variable Name, and Description	Default
PROBE_MIN and PROBE_MAX seconds after the previous one.	
INET_IPAIP_PROBE_MIN ipnet.inet.linklocal. PROBE_MIN The minimum time (in seconds) to send the next probe. Auto IP sends the next probe between PROBE_MIN and PROBE_MAX seconds after the previous one.	"1" char *
INET_IPAIP_ANNOUNCE_NUM ipnet.inet.linklocal. ANNOUNCE_NUM Number of announcements.	"2" char *
INET_IPAIP_PROBE_NUM ipnet.inet.linklocal. PROBE_NUM The number of ARP requests the target will send to check a link-local address for uniqueness before it accepts the address.	"3" char *
INET_IPAIP_PROBE_WAIT ipnet.inet.linklocal. PROBE_WAIT The target will wait [0.. PROBE_WAIT] seconds before creating a link-local IPv4 address and checking it for uniqueness.	"1" char *
INET_IPAIP_RATE_LIMIT_INTERVAL ipnet.inet.linklocal. RATE_LIMIT_INTERVAL Delay between successive address regeneration attempts when the regeneration is rate-limited.	"60" char *

The table below lists the configuration parameters for INCLUDE_IPAIP_INTERFACE_CONFIGS. You can modify these values at run time by using the indicated system variable.

Note: To configure multiple interfaces using these parameters, use the format *ifname = if-specific-value*, using a semicolon to separate the entries. For example, to set the PROBE_NUM parameter to 1 for simnet0 and to 4 for simnet1, set INET_IPAIP_IFLIST_PROBE_NUM to the following string:

```
"simnet0=1;simnet1=4"
```

Parameter Name, System Variable Name, and Description	Default
INET_IPAIP_IFLIST_ANNOUNCE_WAIT *ifname* .inet.linklocal. ANNOUNCE_WAIT Time to wait (in seconds) before starting to announce the selected address. The address will not be defended if another announcement is heard during this period. The address has been assigned to the interface when the announce period starts. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	"" char *
INET_IPAIP_IFLIST_ANNOUNCE_INTERVAL *ifname* .inet.linklocal. ANNOUNCE_INTERVAL Seconds between announcements. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	"" char *
INET_IPAIP_IFLIST_DEFEND_INTERVAL *ifname* .inet.linklocal. DEFEND_INTERVAL The minimum interval between defensive ARPs. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	"" char *
INET_IPAIP_IFLIST_MAX_CONFLICTS *ifname* .ine.t.linklocal. MAX_CONFLICTS Maximum number of address collisions that are allowed before applying rate limiting to the creation of the next address. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	"" char *
INET_IPAIP_IFLIST_PROBE_MAX *ifname* .inet.linklocal. PROBE_MAX Maximum number of seconds to wait before sending the next probe. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	"" char *

Parameter Name, System Variable Name, and Description	Default
INET_IPAIP_IFLIST_PROBE_MIN *ifname* .inet.linklocal. PROBE_MIN Minimum number of seconds to wait before sending the next probe. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	" " char *
INET_IPAIP_IFLIST_ANNOUNCE_NUM *ifname* .inet.linklocal. ANNOUNCE_NUM The number of announcements to send. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	" " char *
INET_IPAIP_IFLIST_PROBE_NUM *ifname* .inet.linklocal.PROBE_NUM Number of times a link local address is checked for uniqueness before being assigned. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	" " char *
INET_IPAIP_IFLIST_PROBE_WAIT *ifname* .inet.linklocal. PROBE_WAIT The number of seconds a host will wait [0.. PROBE_WAIT] before creating a link local IPv4 address and checking it for uniqueness. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	" " char *
INET_IPAIP_IFLIST_RATE_LIMIT_INTERVAL *ifname* .inet.linklocal. RATE_LIMIT_INTERVAL The delay between successive attempts. This parameter can be specified for each interface specified in the INET_IPAIP_IFNAME_LIST in the form "*ifname*=*value*".	" " char *

Previous topic: IPv4 Network Stack Configuration Parameters on page 213
Next topic: IPv6 Configuration Parameters on page 221

4. IPv6 Configuration Parameters

VIP and System Variable Configuration Parameters

The table below lists the INCLUDE_IPCOM_USE_INET6 configuration parameters and their default values. Where applicable, a system variable is also documented for run-time configuration. Modify the values as needed.

Parameter Name, System Variable Name, and Description	Default
INET6_ACCEPT_RTADV ipnet.inet6.AcceptRtAdv Enable (1) or disable (0) the processing of router advertisement messages.	**"1"**
INET6_ADD_AUTONOMOUS_ADDR ipnet.inet6.AddAutonomousAddr Should this node create addresses from router advertisement messages.	**"yes"**
INET6_AUTO_CONFIG ipnet.inet6.AutoConf Enable/disable address auto-configuration.	**"1"**
INET6_BASE_HOP_LIMIT ipnet.inet6.BaseHopLimit The default hop limit for IPv6 packets.	**"64"**
INET6_BASE_REACHABLE_TIME ipnet.inet6.BaseReachableTime Number of seconds that an entry in the NDP cache is in reachable state.	**"30"**
INET6_BASE_RETRANSMIT_TIME ipnet.inet6.BaseRetransmitTime Number of seconds to wait between retransmits.	**"1"**
INET6_DELAY_FIRST_PROBE_TIME ipnet.inet6.DelayFirstProbeTime Number of seconds the stack should wait for a stale neighbor discovery (ND) entry to become reachable before forcing a	**"5"**

Parameter Name, System Variable Name, and Description	Default
probe. The stack sends packets using the stale entry during this time.	
INET6_DST_CACHE_TO_LIVE_TIME ipnet.inet6.DstCacheToLive Number of seconds an entry is kept in the destination cache. The destination cache contains FIB entries created in response to redirect- and need-frag ICMP messages.	**"600"**
INET6_ENABLE_PMTUD_RFC5927 ipnet.inet6.EnablePmtudRfc5927 Hardens TCP against ICMP denial-of-service attacks. Set to "yes" to enable this algorithm.	**"**yes**"**
INET6_ICMP_IGNORE_ECHO_REQ ipnet.inet6.IcmpIgnoreEchoRequest Set to **"0"** if the stack should answer to echo requests.	**"0"**
INET6_ICMP_RATE_LIMIT_BUCKET_SIZE ipnet.inet6.IcmpRatelimitBucketsize Number of ICMPv6 messages the stack is allowed to send per ICMP rate limit interval millisecond interval.	**"100"**
INET6_ICMP_RATE_LIMIT_INTERVAL ipnet.inet6.IcmpRatelimitInterval ICMP rate limit interval millisecond interval. Set to **"0"** to disable the rate limiter.	**"1000"**
INET6_ICMP_REDIRECT_RECEIVE ipnet.inet6.IcmpRedirectReceive Set to **!= 0** if the stack should accept redirect messages and process them.	**"1"**

Parameter Name, System Variable Name, and Description	Default
INET6_ICMP_REDIRECT_SEND ipnet.inet6.IcmpRedirectSend Set to **2** if the stack should send a redirect and forward the original message (default). Set to **1** if the stack should send a redirect but discard the original message. Set to **0** if the stack should not send a redirect but forward the original message. Set to **-1** if the stack should not send a redirect but should discard the original message.	**"2"**
INET6_ICMP_SEND_DST_UNREACHABLE ipnet.inet6.IcmpSendDestinationUnreachable Set to **!= 0** if the stack should send a destination unreachable message.	**"1"**
INET6_ICMP_SEND_TIME_EXCEEDED ipnet.inet6.IcmpSendTimeExceeded Set to **!= 0** if the stack should send time exceeded messages.	**"1"**
INET6_MAX_APP_SOLICIT Number of resolve messages the stack sends to routing/netlink sockets if ND probes fail (or are disabled).	**"1"**
INET6_MAX_MULTICAST_SOLICIT Number of neighbor discovery messages the stack should send before starting to use application solicit. Setting this to a value other than 0 causes IPv6 ready tests to fail, so exercise caution in changing this parameter.	**"0"**
INET6_MAX_PKTS_PENDING Number of packets that can be waiting before IPv6 to link layer resolution is finished.	**"3"**

Parameter Name, System Variable Name, and Description	Default
INET6_MAX_UNICAST_SOLICIT ipnet.inet6.MaxUnicastSolicit Number of unicast neighbor discovery messages that should be sent for ND entry update before starting to use multicast solicit.	**"3"**
INET6_MIN_MTU_SIZE The smallest path MTU expected in any route. All Internet routers should be able to handle 1280 octets. This value is used by some ICMP messages that include data from the packet that caused an ICMP message to be sent.	**"1280"**
INET6_NBR_CACHE_TO_LIVE_TIME ipnet.inet6.NeighborCacheToLive Number of seconds that can pass before a stale entry in the neighbor cache is deleted.	**"1200"**
INET6_DAD_TRANSMITS ipnet.inet6.DupAddrDetectTransmits Number of times the node should test the address for uniqueness before assigning it to the interface (set to 0 to turn off duplicate address detection).	**"1"**
INET6_ROUTER_LIFETIME *ifname* .inet6.AdvDefaultLifetime Number of seconds this router can be used as gateway after a router advertisement had been sent; only used when the network stack is configured as a router.	**"1800"**
INET6_RTHDR0 ipnet6.inet.RtHdr0 Enable (1) disable (0) the processing of Type 0 routing headers. For security purposes, it is disabled by default.	**"0"**
INET6_DISCARD_UDP_WITH_ZERO_CHECKSUM ipnet.inet6.DiscardUdpZeroChecksum Discard UDP packet with zero checksum for IPv6.	**"1"**

Parameter Name, System Variable Name, and Description	Default
To discard IPv6 UDP packets with zero checksum, set this parameter to 1. To accept IPv6 UDP packets with zero checksum, set this value to 0.	

Previous topic: Global Configuration Parameters for Auto IP and Address Conflict Detection (ACD) on page 217
Next topic: IPv6 Address Scope and Zone System Variables on page 226

5. IPv6 Address Scope and Zone System Variables

The table below describes the system variables used to set the IPv6 address scope and zone.

System Variable Name and Description	Default
ipnet.inet6.zone. {**node**|**link**|**subnet**|**admin**|**site**| **org**}**.Default** Zone ID of the specified node, link, subnet, admin, site, or organization. For further information, see *Assigning the Default Zone*.	node: 1 link: > 1 All others: 0
ifname **.inet6.zone.**{**node**|**link**|**subnet**|**admin**|**site**|**org**}**.Id** Zone ID of the specified network interface for the specified node, link, subnet, admin, site, or organization. For further information, see *Attaching an Interface to a Zone*.	0 for all scopes except link-local, where the default value is the *ifindex* value of the interface.

Previous topic: IPv6 Configuration Parameters on page 221
Next topic: Configuration Variables for Stateless Address Autoconfiguration on page 226

6. Configuration Variables for Stateless Address Autoconfiguration

The table below lists the system variables used to configure a node with stateless address auto-configuration.

For more information, see Privacy Extensions for Stateless Address Autoconfiguration on page 68.

Parameter Name, System Variable Name and Description	Default
INET6_ENABLE_PRIVACY_EXTENSONS [*ifname* .]**ipnet.inet6.EnablePrivacyExtensions** Enable/disable privacy extensions.	**"1"**
INET6_TEMP_VALID_LIFETIME ipnet.inet6.TempValidLifetime Maximum lifetime, in seconds, of temporary addresses.	**"604800"** (1 week)
INET6_TEMP_PREFERRED_LIFETIME ipnet.inet6.TempPreferredLifetime Maximum preferred lifetime, in seconds, of temporary addresses.	**"86400"** (1 day)
INET6_REGEN_ADVANCE ipnet.inet6.RegenAdvance Number of seconds that a new address will be generated before the current temporary address becomes non-preferred. Note: The default value specified by RFC 4941 is 5 seconds, but the Wind River Network Stack uses a value of 30 seconds.	**"30"**
ipnet.[*ifname* .]**inet6.TempIdgenRetries** Maximum number of times the node is allowed to regenerate the host ID as a consequence of duplicate address detection failure.	**"3"**

Previous topic: IPv6 Address Scope and Zone System Variables on page 226
Next topic: TCP Configuration Parameters on page 227

7. TCP Configuration Parameters

The INCLUDE_IPTCP component pulls in modules that provide support for Transmission Control Protocol/Internet Protocol (TCP/IP) in either the IPv4 or IPv6 domains. The table below lists the INCLUDE_IPTCP configuration parameters and their default values. Modify these values as needed.

For more information on TCP configuration, see *TCP*.

Parameter Name, System Variable Name, and Description	Default
TCP_ALLOW_TINY_WINDOW *iptcp.AllowTinyWindow* Enable or disable the acceptance of a peer window being set to a value less that one MSS in the ACK of the SYN/ACK. An attacker can use zero window to keep the node in persist time for a long time. Enabling can sometimes cause connection trouble in environments that contain traffic shapers which modify the peer windows of TCP segments. This is why this is allowed by default. To disable, set to 0.	**1**
TCP_ALLOW_ZERO_WINDOW iptcp.**AllowZeroWindow** Enable/disable acceptance of the peer window being set to zero in the ACK of the SYN/ACK. An attacker can use zero window to keep the node in persist time for a long time. The default is disabled. To enable, set to 1.	**0**
TCP_CHALLENGE_ACK_CNT *ipnet.tcp.ChallengeAckCnt* Define the number of challenge ACKs per second that is used to reply to potential "Blind Reset" or "Blind Data Injection" attacks.	**100**
TCP_CLAMP_MSS *iptcp.ClampMss* Maximum segment size (MSS) that TCP uses for sessions to peers that are not local to this node. Set to 0 to disable or set to a non-zero value to limit the maximum segment size. IPv4 requires at least 536 octets whereas IPv6 requires at least 1220 octets.	**0**
TCP_CONN_TIMEOUT *iptcp.ConnectionTimeout* The connection timeout, or number of seconds the stack attempts to create a TCP connection before giving up.	**75**
TCP_LAND_ATTACK_PROTECTION *iptcp.LandAttackProtection* Set to 1 to prevent LAND attacks on TCP implementation.	**1**

Parameter Name, System Variable Name, and Description	Default
TCP_KEEP_CNT *iptcp.KeepCnt* Defines the number of keep-alive probes to send before the connection is dropped. The send interval is defined by *iptcp.KeepIntvl*. This field also sets the default value of setsockopt( )(IPPROTO_TCP, TCP_KEEPCNT). The value is in seconds. The time-frame between the last received ACK from a peer to a connection being dropped due to a no-response on keep-alive is *(iptcp.KeepIdle + iptcp.KeepIntvl) * iptcp.KeepCnt*. Based on the default values, the time-frame is 72,750 seconds (just over 20 hours and 12 minutes).	**10**
TCP_KEEP_IDLE *iptcp.KeepIdle* Defines the idle time of TCP sessions before keep-alive probing starts. This field also sets the default value of setsockopt( )(IPPROTO_TCP, TCP_KEEPIDLE). The value is in seconds.	**7200**
TCP_KEEP_INTVL *iptcp.KeepIntvl* Defines the interval between keep-alive probes for which no response is received. This field also sets the default value of setsockopt( ) (IPPROTO_TCP, TCP_KEEPINTVL). The value is in seconds.	**75**
TCP_MAX_RETRANSMITS *iptcp.MaxRetransmits* Maximum number of times the stack resends a TCP segment before giving up and sending an **IPTCP_ERRNO_ETIMEDOUT** error. Exponential back off is applied before each resend (maximum time between two resends is 1 minute).	**10**
IPTCP_MAX_RETRANSMITS_IN_TRANSIENT *iptcp.MaxRetransmitsTransientStates* Number of retransmissions that are allowed for sessions in transient states when the total number of sessions in the transient state are greater than IPTCP_TRANSIENT_THRESH. See also *iptcp.MaxTransient* and TCP_MAX_RETRANSMITS **(iptcp.MaxRetransmits)**.	2

Parameter Name, System Variable Name, and Description	Default
iptcp.MaxTransient Maximum number of TCP sockets that are allowed to be in a state that is very short-lived under normal conditions. Large number of sockets in this states is an strong indication that this node is under some form of denial of service (DoS) attack. The session that spent the most time in a transient state will be dropped if this count is exceeded.	20
TCP_MSL **iptcp.Msl** TCP sockets that enter the **TIME_WAIT** state remain in that state for twice this value (2 X 30 seconds).	**30**
TCP_MAX_MSS *iptcp.MaxMss* Defines the largest maximum segment size (MSS) that TCP will ever suggest. The actual MSS suggested might be smaller than this number if it depends on the MTU for the interface. A value of 0 specifies the largest MSS allowed by the MTU.	**0**
TCP_PERSIST_TMO *iptcp.PersistTimeout* The maximum duration, in seconds, a peer can keep announcing a receive window of size zero. Default value is **0**, which means "infinite."	**0**
TCP_SEGMENT_MULTIPLIER Segment size multiplier used on outgoing segments when the outgoing interface supports TCP segmentation offload. Generally, the higher the value, the better the performance; however, factors higher than 2 are not guaranteed to work on all network cards.	**2**
TCP_DEFAULT_BACKLOG_SIZE Default backlock size, this is the value that will be used when 0 is passed as the second argument to listen( ).	5

Parameter Name, System Variable Name, and Description	Default
IPTCP_TRANSIENT_THRESH *iptcp.TransientTresh* This threshold refers to the number of sessions in a transient state. When this threshold is exceeded, the number of resend attempts is reduced to a value set by *iptcp.MaxRetransmitsTransientStates* . See also *iptcp.MaxTransient* .	5
TCP_USE_RFC1122_URGENT_DATA *iptcp.urg.Rfc1122* Determines which RFC to follow when setting urgent data. If set to **1**, use urgent data as described in RFC 1122. If set to **0**, use urgent data as described in RFC 793. Most stacks use **0**. However, this parameter must be set to **1** in order to successfully pass if Ixia ANVL TCP-CORE group 19 tests are used.	**0**
TCP_USE_TIMESTAMP *iptcp.opt.Timestamp* Determines whether the stack includes the timestamp option in all segments for peers that support it. Set to **0** if the stack should never use the timestamp option. Set to **1** to use the timestamp option.	**0**
TCP_ALLOW_SACK *iptcp.opt.AllowSack* Defines if selective acknowledgments is enabled (1) or disabled (0). When enabled, the data receiver informs the sender about all segments that have arrived successfully. Hence the sender only needs to retransmit the lost segments. See RFC 2018.	**1**
TCP_FORCE_IM_ACK_CNT *iptcp.ForceImAckCnt* Controls the number of delayed ACKs that can be received before transmission of an immediate ACK is forced.	**2**
TCP_MIN_RTO *iptcp.MinRTO* Minimum value of retransmission timeout, in msec. The default value is recommended according to RFC 6298.	**1000**

WNDRVR

Previous topic: Configuration Variables for Stateless Address Autoconfiguration on page 226
Next topic: Proxy Address Resolution Protocol (ARP) Configuration Parameters on page 232

8. Proxy Address Resolution Protocol (ARP) Configuration Parameters

The INCLUDE_IPPROXYARP component supplies configuration parameters to define how proxy ARP works over the network. The table below lists the INCLUDE_IPPROXYARP configuration parameters and their default values.

Where applicable, a system variable is also documented for run-time configuration. Modify these values as needed.

Parameter Name, System Variable Name, and Description	Default
INET_ACCEPT_SOURCE_ROUTE *ipnet.inet.AcceptSourceRoute* This parameter defines the behavior of proxy ARP with regard to source routed frames. When it is enabled, source routed frames are accepted.	"no"
INET_AUTO_PROXY_ARP *ipnet.inet.AutoProxyArp* This parameter defines the behavior of proxy ARP. When it is enabled, the network stack automatically tags all interface address network routes as proxy ARP. You must enable this parameter to configure a system with proxy ARP.	"no"
INET_IFLIST_AUTO_PROXY_ARP *ifname .inet.AutoProxyArp* This parameter defines the behavior of proxy ARP for individual interfaces. Default is disabled (**""**). When it is enabled, the network stack automatically tags the specified interface address network routes as proxy ARP. This parameter can be configured on a per-interface basis in the form "*ifparam=value*", for example **"eth0=1"**. Each "*ifparam=value*" pair must be separated by a semicolon.	" "
INET_ENABLE_PROXY_ARP *ipnet.inet.EnableNetworkProxyArp* This parameter defines the behavior of proxy ARP. When it is enabled, the network stack provides proxy ARP for network routes tagged with the proxy ARP flag. You must enable this parameter to configure a system with proxy ARP.	"yes"

Parameter Name, System Variable Name, and Description	Default
Set to 1 to enable.	
INET_IFLIST_ENABLE_PROXY_ARP *ifname .inet.EnableNetworkProxyArp* This parameter defines the behavior of proxy ARP on a per-interface basis. Default is disabled (**""**). When it is enabled, the network stack provides proxy ARP for network routes tagged with the proxy ARP flag. This parameter can be configured on a per-interface basis in the form "*ifparam=value*", for example **"eth0=1"**. Each "*ifparam=value*" pair must be separated by a semicolon.	" "

Previous topic: TCP Configuration Parameters on page 227
Next topic: SEND System Variables on page 233

9. SEND System Variables

You can configure SEND using the following system variables:

System Variable Name and Description	Default
ipnet.inet6.send.rsa_key_file The path to the RSA key used to sign secure NDP messages.	**" /ram/rsa.pem"**
ipnet.inet6.send.cert_file The path to X509 certificate (PEM format) file for the RSA key.	**" /ram/cert.pem"**
ipnet.inet6.send.ca_cert_file The path to the file holding trusted CA certificates (digital certificates issued by a Certificate Authority) in PEM format. The order of the certificates in the file is not relevant. Only one CA cert is supported.	**" /ram/ca_cert.pem"**
ipnet.inet6.send.intermediate_ca_cert_file The path to the file used for intermediate CAs in PEM format, used to create a certificate chain.	**" /ram/intermediate_ca.pem"**

System Variable Name and Description	Default
ipnet.inet6.send.timestamp_delta The allowed difference between local time and the timestamp in received signed NDP messages.	**300**
ipnet.inet6.[*if* **].send.mode** **"off"**. All outgoing messages are sent without signature; all incoming messages are received without signature verification. **"on"**. All outgoing messages are signed; all incoming messages must have a valid signature. **"transition"**. All outgoing messages are signed. Incoming signed values are validated, and unsigned messages are also allowed. This setting facilitates a transitional phase where some but not all nodes on a subnet are upgraded to support SEND. The *if* specification is optional and specifies a particular interface. If supplied, an *if* specification overrides the global setting specified by **ipnet.inet6.send.mode** for packets being transmitted or received on the specified interface.	
ipnet.inet6.send.cert_path_mode **"off"**. **"host"**. Host mode. A certification path solicitation message is sent when a router advertisement is received. **"router"**. Router mode. Certification path advertisements is sent in response to certification path solicitations.	

Previous topic: Proxy Address Resolution Protocol (ARP) Configuration Parameters on page 232

21. NETWORK STACK SHELL COMMAND REFERENCE

1. About Network Stack Shell Commands

This appendix provides reference information for the some of the shell commands you can use to configure the Wind River Network Stack at run time.

Most shell commands are specific to a particular protocol or technical area. A few are used across technologies or are generic to the network stack as a whole.

Next topic: arp on page 235

2. arp

arp - dynamically display and configure Address Resolution Protocol (ARP) and proxy ARP information, or for test purposes

Name

Synopsis

```
arp [-silent] [-V routetab] [-i ifname] -a
arp [-silent] [-V routetab] [-i ifname] -A
arp [-silent] [-V routetab] -d hostaddress
arp [-silent] [-V routetab] [-i ifname] [-p] [-t] -s hostaddress ether_addr
arp [-silent] [-V routetab] [-i ifname] -r hostaddress
```

Description

The ARP program displays and modifies the Internet-to-Ethernet address translation tables used by the ARP. With no flags, the program displays the current ARP entry for *hostname*. The host may be specified by name or by number, using Internet dot notation.

The arp options are:

-silent

Suppress output. MUST be the first argument.

-i *ifname*

Specify interface. If not specified, the first interface that uses ARP is used.

-t

Temporary ARP entry. The entry times out normally.

-p

Public ARP entry (proxy ARP entry). A proxy ARP entry is shown as a route with the proto2 flag set (shown as '2' with route/netstat).

-r

Send an ARP request to the host address.

-V *routetab*

Specify route table. 0 if not specified.

The ARP commands are:

-a

The program displays all of the current ARP entries.

-A

Erase all ARP entries on the interface.

-d

Delete an entry for the host called *hostaddress* .

-s *hostaddress ether_addr*

Create an ARP entry for the host having the specified IPv4 and Ethernet address. The Ethernet address is given in the standard format of six hex bytes separated by colons (for example, 01:02:03:04:05:06). The entry is permanent unless the -t flag is used, but can be removed by using the arp -d command. Public (proxy ARP) entries are added using the -p flag.

Previous topic: About Network Stack Shell Commands on page 235
Next topic: fwctrl on page 236

3. fwctrl

fwctrl - control and view forwarder properties

Name

Synopsis

```
fwctrl fw [enable|disable|start|stop]
fwctrl stats [netif] [reset]
fwctrl flow show
fwctrl flow invalidate {entry|all}
fwctrl sa show
fwctrl seed [value]
fwctrl util
```

Description

fw

Get or set the forwarding status.

enable

Use a forwarding cache to decide whether packets should be forwarded or terminated in this node.

disable

Poll interfaces, but send all packets to the network stack.

start

Cause forwarders on reserved cores to poll interfaces for packets.

stop

Block the forwarder and receive incoming packets using the standard interrupt-driven mechanism.

stats [*netif*]

Display forwarding statistics for the interface specified by *netif* .

reset

Reset the counters for the specified interface (or all interfaces if no interface is specified).

flow show

Display the contents of the forwarding cache.

flow invalidate

Remove the specified *entry* or all entries from the forwarding cache.

sa show

Displays the contents of the security association cache.

seed [*value*]

Sets or reads the current seed. The seed is used to calculate the tag value associated with a flow (a stream of packets with the same destination). The command can be used to see if an automatic update has happened, or to change the value manually (typically for debugging purposes).

util

Displays the current utilization level (in percent) of the forwarding hash table.

Previous topic: arp on page 235
Next topic: ifconfig on page 237

4. ifconfig

ifconfig - set or get configuration values for a network interface

Name

Synopsis

The shell command ifconfig can be used to assign information to a network interface, retrieve information on a network interface, and generate reports. This command is accessed from the kernel shell, which can be used in either of the following modes:

- command interpreter mode
- C interpreter mode

When the shell is set to C interpreter mode, all command arguments must be enclosed in double quotation marks ("). In this reference entry, all syntax diagrams and command examples are enclosed in double quotation marks. If you are using the shell in command interpreter mode, omit these quotation marks.

The syntax for the command varies by usage.

The valid syntax for assigning information to a network interface is as follows:

```
ifconfig " interface [protocol] [parameter parameterValue] "
```

The valid syntax for retrieving information on a network interface is as follows:

```
ifconfig "[flags] [interface] [protocol]"
```

The following combinations of flags are valid when using ifconfig to generate reports:

```
ifconfig "[-L] interface [ protocol ]"
ifconfig "-a [-L] [-d] [-u][ protocol ]"
ifconfig "-l [-d] [-u] [ protocol ]"
ifconfig "[-L] [-d] [-u][-s]"
```

Description

Use ifconfig to configure a network interface or to retrieve network interface configuration values. The ifconfig( ) API can be used programmatically.

As a convenience, failing to specify a value for *parameter* before the address is interpreted as indicating the default, add. Similarly, because of a prejudice in favor of IPv4, failing to specify a value for *protocol* is interpreted as indicating the default of inet, which specifies IPv4.

The correct usage of the elements of an ifconfig( ) call is as follows:

flags

The *flags* values apply when the ifconfig command (or ifconfig( ) call) is used to generate a report.

Use -a instead of specifying an *interface* value if you want a report containing information on all interfaces.

Use -d to limit the report to down interfaces.

Use -l to limit the report to a simple list of all interfaces without additional information.

Use -L to request lifetime information on an IPv6 address presented as a time offset string.

Use -u to limit the report to up interfaces.

Use -s to display VLAN stacking configuration.

interface

Use this parameter to specify the name of the network interface to which the command applies. The generic format of a network interface name is *UnitnameUnitnumber*, which results in names such as fei0.

protocol

This element lets you specify the network protocol associated with the call. The default *protocol* value is inet, which indicates the IPv4 protocol. The other valid value for *protocol* is inet6.

WNDRVR

address

If the *protocol* is inet, the *address* is either a host name already present in the host name database (see hosts( )), or an IPv4 Internet address expressed in Internet standard dot notation. You can also use the CIDR notation (also known as the slash notation) to specify netmask with the address. For example, using CIDR (slash) notation, 192.168.0.1/16 is a possible inet *address* string. However, if the protocol is inet6, the slash in an *address* string is interpreted as an indicator of the prefix length--for example, ::1/128. If you do not use slash notation to specify a prefix length in an inet6 *address*, ifconfig assumes a default value of 64. To override this default without using the slash notation, you can use the prefixlen parameter. See the entry under the *parameters* description for more information.

destAddr

This element of an ifconfig( ) call appears only when the call applies to the local end of a point-to-point link. In this case, the *destAddr* value indicates the address of the correspondent on the other end the link. See the add( ) entry under the *parameters* description below.

parameters

These elements of an ifconfig( ) call let you set the value of the configuration parameters associated with a network interface. The following is a list of valid values for *parameters*.

add

Add another network address to this interface. You may want to use this parameter when changing network numbers. It leaves the old address in place, which allows the interface to still accept packets destined to the old address. If the new address is on the same subnet as the first network address for this interface, you must specify a netmask of 0xffffffff. If this add applies to the local end of a point-to-point connection, you need to include the other end's destination address. The syntax is:

```
ifconfig " protocol  add  address  destAddr "
```

Note that the equivalent command alias is currently not supported.

delete

Remove the network address specified.

anycast

Specify that the IPv6 address configured is an anycast address. Based on the current specification, only routers may configure anycast addresses. An anycast address is not used as source address of any of outgoing IPv6 packets.

Note that the equivalent commands -alias and -remove are not currently not supported.

cga

The address is cryptographically generated (inet6 only). Available only when IPNET_USE_RFC3971 has been defined.

down

Mark an interface as down. If an interface is marked as down, IP does not transmit through that interface. All routes in the routing table associated with the interface, including static routes, are removed. If possible, ifconfig " *interface* down" also disables packet reception on the specified interface.

create

Create the specified network pseudo device. If the interface is given without a unit number, try to create a new device with an arbitrary unit number. If creation of an arbitrary device is successful, the new device name is printed to standard output.

destroy

Destroy the specified network pseudo device.

WNDRVR

mtu *n*

Set the maximum transmission unit (MTU) of the interface to *n* . The default MTU value is interface specific and may not be configurable.

preferred

Set the preferred lifetime for the specified IPv6 address.

prefixlen *length*

This *parameters* value is valid only when *protocol* is inet6. The *length* value specifies the number of address bits used for dividing networks into subnets. The *length* must be an integer value between 0 and 128. It is usually 64 under the current IPv6 assignment rule.

If the prefixlen is omitted, a default value of 64 is assumed. You can also use the "slash after the address" notation to specify the prefix length.

tentative

Set the tentative bit in the specified IPv6 address.

-tentative

Clear the tentative bit in the specified IPv6 address.

up

Mark an interface as up. Use this *parameters* value to undo a previous ifconfig " *interface* down" call. This status change is a software event only. Simply marking an interface as up cannot re-initialize the hardware associated with that interface.

valid

Set the valid lifetime for the specified IPv6 address.

dstaddr

Set remote address to 'a' (inet and PPP only).

lladdr *addr*

Set interface link address to *addr* .

vr *vr*

Set virtual router to *vr* .

dhcp

Enable DHCP autoconfiguration (inet only).

-dhcp

Disable DHCP autoconfiguration (inet only).

link

Enable special processing at the link level.

-link

Disable special processing at the link level.

promisc

Enable promiscuous mode at the interface.

-promisc

Disable promiscuous mode at the interface.

Wrapper Changes

The ifconfig backwards compatibility wrapper function does not support (or supports differently) the following parameters:

Command Name	Alternative	Description
alias	add	Add inet6 or inet addresses.
-alias	delete	Remove inet6 or inet addresses.
remove	delete	Remove inet6 or inet addresses.
arp	None	Enable the Address Resolution Protocol on the specified network interface.
-arp	None	Disable the Address Resolution Protocol on the specified network interface.
autoconf	None	Enable IPv6 auto configuration for the specified interface.
-autoconf	None	Disable IPv6 auto configuration for the specified interface.
broadcast	None	Specify the address used to broadcast to the network (inet only).
debug	None	Enable driver dependent debugging code.
-debug	None	Disable driver dependent debugging code.
deprecated	None	Set the IPv6 deprecated address bit.
-deprecated	None	Clear the IPv6 deprecated address bit.
media *type*	None	Set the media type of the interface.
mediaopt *opts*	None	Set media options of the interface.
vlan *vlan_tag*	None	Set the VLAN ID of the interface (1-4094).
vlanpri *value*	None	Set the VLAN User Priority (0-7).
vlandev *iface*	None	Associate a physical interface to a VLAN pseudo interface.

Command Name	Alternative	Description
-vlandev *iface*	None	Disassociate physical interface from a VLAN pseudo interface.
metric	None	Set the routing metric for the interface.
pltime	preferred	Preferred lifetime for the specified IPv6 address.
-tentative	None	Clear the tentative bit in the specified IPv6 address.
vltime	valid	Set the valid lifetime for the specified IPv6 address.
dstaddr	different	Set PPP remote address (inet only). Part of the add parameter.

Unsupported Wrappers

The following flags (used for report-generating ifconfig( ) calls) are not supported:

Flag	Alternative	Description
-d	None	Limit report to down interfaces.
-L	None	Request lifetime information on an IPv6 address presented as a time offset string.
-l	None	Limit the report to a simple list of all interfaces without additional information.
-u	None	Limit the report to up interfaces.

New Wrappers

The following are new commands:

Command	Alternative	Description
lladdr *addr*	New	Set interface link address to *addr* .
vr *vr*	New	Set virtual router to *vr* .
dhcp	New	Enable DHCP auto configuration (inet only).

Command	Alternative	Description
-dhcp	New	Disable DHCP auto configuration (inet only).
link[0-2]	New	Enable special processing at the link level.
-link[0-2]	New	Disable special processing at the link level.

Previous topic: fwctrl on page 236
Next topic: ipd on page 243

5. ipd

Most networking component initialization functions and daemons are automatically controlled by a central module called ipd.

The ipd shell command controls the other daemons, such as the multicasting proxy daemon. The component for ipd is INCLUDE_IPD_CMD.

Name

ipd - daemon process command

Synopsis

```
ipd  command  [ -options ]
```

Individual synopses are:

```
ipd [-V  vr] list
ipd [-V  vr] start  service
ipd [-V  vr] kill  service
ipd [-V  vr] reconfigure  service
ipd [-V  vr] #  service
```

Description

Command options are:

-V *vr*

Use the virtual router identified by *vr* .

list

List daemon services.

start

Start the specified service.

kill

Stop the specified service.

reconfigure

Reconfigure the specified service.

Previous topic: ifconfig on page 237
Next topic: ipnet_ctrl on page 244

6. ipnet_ctrl

ipnet_ctrl - control functions for IPNET

Name

Synopsis

```
ipnet_ctrl [-p] [-a] [-t][-r domain[: VR[:table]]] [-s sec]
```

Description

ipnet_ctrl is used to check the internal state of the IPNET stack.

The ipnet_ctrl options are as follows:

-p

List all allocated packets in the system.

-a

List all packets in the system.

-t

List all time outs.

-r *domain*[**:** *VR*[**:** *table*]]

Dump the routing table.

-s *sec*

Suspend the network stack for the specified number of seconds.

Previous topic: ipd on page 243
Next topic: ipversion on page 245

7. ipversion

The ipversion shell command displays the product versions. The component for ipversion is INCLUDE_IPVERSION_CMD.

Name

ipversion - show product versions

Synopsis

```
ipversion
```

Description

Shows product versions and copyrights for network stack and other technologies.

Previous topic: ipnet_ctrl on page 244
Next topic: mcastproxy on page 245

8. mcastproxy

mcastproxy - display the current status of the multicasting daemon

Name

Synopsis

```
            mcastproxy [-4|-6] {-i|-j|-r}
```

Description

This command displays a list of active multicast groups for each interface available. The MLD daemon shows the MLD state per interface, for example, **QUERIER** or **NON QUERIER**. The default, with no options, is to show both the IPv4 and IPv6 status.

The mcastproxy options are:

-4

Show IPv4 only.

-6

Show IPv6 only.

-i

Show interface statistics.

-j

Show groups joined by clients.

-r

Show route statistics.

Examples

This example shows the statistics when the multicast proxy is routing IP datagrams sent to group 225.0.0.11, arriving at vlan4 to vlan5, vlan6, and vlan7.

To show the IPv4 interface statistics:

```
192.168.200.20> mcastproxy -4 -i
Upstream interface IPv4 statistics
    Name  Pkts  Bytes
    vlan4 5     235
Downstream interface IPv4 statistics
    Name  Pkts  Bytes  State
    vlan5 5     235    QUERIER
    vlan6 5     235    QUERIER
    vlan7 5     235    QUERIER
```

To show the IPv4 groups joined by clients:

```
192.168.200.10> mcastproxy -4 -j
IPv4 groups joined at vlan5
    Group            State
    225.0.0.11       ACTIVE
IPv4 groups joined at vlan6
    Group            State
    225.0.0.11       ACTIVE
IPv4 groups joined at vlan7
    Group            State
    225.0.0.11        ACTIVE
```

To show the IPv4 route statistics:

```
192.168.200.10> mcastproxy -4 -r
IPv4 routes
    Group        Source        Pkt  Bytes  Dwnstr  ifs
    225.0.0.11   10.30.200.4   5    235    vlan5,  vlan6, vlan7
```

Previous topic: ipversion on page 245
Next topic: mib2 on page 246

9. mib2

mib2 - Print MIB-2 statistics

Name

Synopsis

```
mib2 cmd [group |table |variable ] [value ]
```

Description

The mib2 shell command displays MIB-2 statistics.

Available commands are:

clear

Clear all MIB-2 statistics.

help

Show this info.

show {*group* | *table* }

Display a MIB-II group or table.

set *variable value1* [*value2*] [*value3*] [*value4*] [*value5*]

Write a MIB-2 variable.

Previous topic: mcastproxy on page 245
Next topic: ndp on page 247

10. ndp

ndp - dynamically display and configure NDP information, or for test purposes

Name

Synopsis

```
ndp [-silent] [-V routetab ] -a
ndp [-silent] [-V routetab ] -A
ndp [-silent] [-V routetab ] -c
ndp [-silent] [-V routetab ] [-1ifname ] -d hostaddress
ndp [-silent] [-V routetab ] -p
ndp [-silent] [-V routetab ] -r
ndp [-silent][-V routetab ] -R
ndp [-silent][-V routetab ][-i ifname ] -s nodeaddress ether_addr
```

Description

The ndp command manipulates the address-mapping table used by NDP.

The ndp options are:

-silent

Suppress output. MUST be the first argument.

-V

Specify route table to use. 0 if not specified.

-a

Dump the currently existing NDP entries.

-A

Erase all the NDP entries.

-d

Delete specified NDP entry.

-p

Show prefix list.

-r

Show default router list.

-R

Flush all the entries in the default router list.

-s

Register an NDP entry for a node. The entry is permanent.

-i

Specify interface. Selected by the stack based on the address otherwise.

Previous topic: mib2 on page 246
Next topic: netstat on page 248

11. netstat

netstat - show network status

Name

netstat - show network status

WNDRVR

Synopsis

```
netshtat -h
netstat [-an] [-f address_family][-V vr] [-p protocol]
netstat -r [-nl][-f address_family] [-V vr]
netstat -s [raw|ip|icmp|sctp|udp|tcp|ip6|icmp6]
```

Description

The netstat command symbolically displays the contents of various network-related data structures.

The netstat options are:

-a

Show the state of all sockets; normally sockets used by server processes are not shown.

-f *address_family*

Limit statistics or address control block reports to those of the specified address family. The following address families are recognized: **inet** for **AF_INET** and **inet6** for **AF_INET6**.

-h

Help (this usage menu).

-n

Show network addresses as numbers. Default is to lookup symbolic names for the addresses.

-p *protocol*

Show statistics about protocol, which is either a well-known name for a protocol or an alias for it.

-r

Show the routing tables

-V *vr*

Specify virtual router index. 0 if not specified.

-T *table*

Specify route table ID. If not otherwise specified, the value of IPCOM_ROUTE_TABLE_DEFAULT is used by default.

-l

Do not show routes where the LLINFO flag is set.

-s

Display networking statistics.

Previous topic: ndp on page 247
Next topic: passwd on page 250

WNDRVR

12. passwd

Name

passwd - Set or change a user's password

Synopsis

```
passwd [loginName]
```

Description

The passwd command sets or changes a user's password. If no argument is specified, the password is changed for the user currently logged in.

Command options are as follows:

loginName

User name of the specified user. The user name can be up to 80 bytes long.

Previous topic: netstat on page 248
Next topic: pcap on page 250

13. pcap

pcap - start/stop packet capture on a network interface

Name

Synopsis

```
pcap [ -f filename ] ifname { start|stop }
```

Description

All packets sent or received on the interface are written to a file that can be imported into Wireshark (formerly known as Ethereal).

-f *filename*

Store all captured packets in a target-resident file named by filename.

ifname

An interface.

start

Start capture on an interface.

stop

Stop capture on an interface.

Previous topic: passwd on page 250
Next topic: ping on page 251

14. ping

ping - send ICMPv4 ECHO_REQUEST packets to network hosts

Name

Synopsis

```
ping [-silent] [-ADbnrx][-I ifname] [-S src_addr] [-T tsonly|tsandaddr | tsprespec]
[-Q tos] [-x][-V vr] [-c count] [-s packetsize][-t ttl] [-w ms] host
```

Description

ping uses the ICMPv4 protocol's mandatory ECHO_REQUEST datagram to elicit an ECHO_REPLY from a host or gateway.

The ping options are:

-A

Add router alert option to each sent ping request.

-b

Allow pinging a broadcast address.

-c *count*

Stop after sending count ECHO_RESPONSE packets.

-D

Set the DF (don't fragment) flag on each sent ping request.

-I *ifname*

Specify outgoing interface.

-n

Numeric output only. No attempts are made to look up names for host addresses.

-Q *tos*

Specify the Type of Service field in the IPv4 header.

-r

Bypass the normal routing tables and send directly to a host on an attached network.

-s *packetsize*

Specify the number of data bytes to be sent. The default is 56, which translates into 64 ICMP data bytes when combined with the 8 bytes of ICMP header data.

-S *src_addr*

Specify the address to use as source address when sending.

-t *ttl*

Use the specified time-to-live.

-T *timestamp_option*

Include IP timestamp option. *timestamp_option* can be tsonly (only timestamp), tsandaddr (address and timestamp), or tsprespec *host1* [*host2* [*host3* [*host4*]]] (timestamp-specific hops).

-x

Send an ICMP timestamp request instead of an ECHO request.

-V *vr*

Specify virtual router. 0 if not specified.

-w *timeout*

Timeout in milliseconds: the default is 1,000 ms.

host

The host to which ECHO_REQUEST packets shall be sent.

-silent

No output; returns success or failure.

Previous topic: pcap on page 250
Next topic: ping6 on page 252

15. ping6

ping6 - send ICMPv6 ECHO_REQUEST packets to network hosts

Name

Synopsis

```
ping6 [-Tnrt] [-c count][-h hoplimit] [-m ifname] [-s packetsize]-w timeout] [-F label]
[-S      src_addr][-Q tclass] [-V vr] [hop ...] host
```

Description

ping6 uses the ICMPv6 protocol's mandatory ICMP6_ECHO_REQUEST datagram to elicit an ICMP6_ECHO_REPLY from a host or gateway. ICMP6_ECHO_REQUEST datagrams ("pings") have an IPv6 header, and ICMPv6 header formatted as documented in RFC

2463. It is possible to specify additional nodes that the ping request must visit before reaching the final destination. This is accomplished by adding a routing header to the packet.

The ping6 options are:

-c *count*

Stop after sending count ECHO_RESPONSE packets.

-F *label*

Specify the flow label in the IPv6 header.

-h *hoplimit*

Set the IPv6 hoplimit.

-m *ifname*

Name of outgoing interface for multicast addresses.

-n

Numeric output only. No attempts are made to look up names for host addresses.

-Q *tclass*

Specify the traffic class in the IPv6 header.

-r

Bypass the normal routing tables and send directly to a host on an attached network.

-S *src_addr*

Specifies the address to use as source address when sending.

-s *packetsize*

Specifies the number of data bytes to be sent. The default is 56, which translates into 64 ICMP data bytes when combined with the 8 bytes of ICMP header data.

-t

Ping the specified host until stopped.

-T

Prefer temporary addresses as source.

-V *vr*

Specify virtual router. vr = 0 if not specified.

-w *timeout*

Timeout in milliseconds, default is 1000 ms.

host

The host to which ICMP6_ECHO_REQUEST packets shall be sent.

hop ...

Zero or more nodes that the echo request must pass, in order, before reaching the final host node.

Previous topic: ping on page 251

WNDRVR

Next topic: radvdconfig on page 254

16. radvdconfig

radvdconfig - configure router advertisement daemon

Name

Synopsis

```
radvdconfig [-silent] [-V vr] interface add id prefix /prefix_len [optional_parameters]
radvdconfig [-silent][-V vr] interface {enable|disable|delete} id
```

Description

radvdconfig is used to add, delete, enable, or disable prefixes that will be announced in an IPv6 router advertisement message.

Options

-silent

Suppresses output. MUST be first argument.

Commands

add *id prefix / prefix_len*

Adds a new prefix in disabled state.

delete *id*

Deletes a prefix entry.

enable *id*

Enables a prefix entry.

disable *id*

Disables a prefix entry.

Values

interface

The name of an attached network interface.

id

String that identifies the prefix.

prefix

Prefix specified as an IPv6 address.

prefixlen

Number of bits in *prefix* that is used for network ID.

Optional Parameters

valid *sec*

Number of seconds the prefix is valid.

preferred *sec*

Number of seconds the prefix is preferred.

A

The prefix can be used for autonomous address configuration.

-A

The prefix cannot be used for autonomous address configuration.

L

The prefix can be used for on-link determination.

-L

The prefix cannot be used for on-link determination.

Previous topic: ping6 on page 252
Next topic: route and routec on page 255

17. route and routec

route and routec - a utility to manually manipulate network routing tables

Name

Synopsis

```
route [-V routetab] [-n] command [[modifiers] args]
```

Use routec from the C shell.

Description

This utility supports a limited number of general options, but a rich command language enables the user to specify any arbitrary request that could be delivered via the programmatic interface.

Route Command Options

The route command options are as follows.

add

Add a route.

delete

Delete a route.

change

Change aspects of a route (such as its gateway).

get

Look up and display the route for a destination.

show

Print out the route table (similar to **netstat REVIEW: EN dash in source converted to regular hyphen-r**).

monitor

Continuously report any changes to the routing information base, routing lookup misses, or suspected network partitionings.

vr

Add or delete a virtual route table.

The monitor command has the syntax:

```
route [-n] monitor
```

The vr command has the syntax:

```
route vr [[-T table] -add | -delete | -addbyname] vr_index
```

vr_index **0** (the default route table) cannot be added or deleted. The new route table is completely empty.

The following options are supported:

-T *table*

Create a new FIB within the VR.

-add

Add a new VR.

-delete

Delete a VR.

-addbyname

Add a new VR. *vr_index* is the name of the VR.

WNDRVR

Route Utility Commands

The route utility commands have the following syntax:

```
route [-n] command [-net | -host] [-option ...] dest_ip [gate_ip]
```

Use routec from the C shell.

Options for these route commands include the following:

[-net | -host]

Forces the destination to be interpreted as a network or a host, respectively.

-net sets the IPNET_RTF_HOST flag and sets the mask to all ones (/32 for IPv4 and /128 for IPv6).

-host clears the IPNET_RTF_HOST flag and ensures the mask has at least one zero (i.e., -prefixlen is < 32 for IPv4 and < 128 for IPv6).

dest_ip

The IPv4/IPv6 destination host or network address.

gate_ip

The IPv4/IPv6 host serving as the gateway.

The following additional options are supported:

-V *vr*

Specifies the virtual router index. If no VR index is specified, **0** is used. Must be the first switch if present.

Generic Flags

-n

Disables DNS address lookup.

-silent

Enables silent mode.

-q

Enables silent mode.

-dev *ifname*

Device name. This option must be used if *gate_ip* is not specified. It may also be used if *gate_ip* is specified and *gate_ip* can be reached by more than one link; in such a case, use -dev *ifname* to specify which link should be used.

-nollinfo

Do not show routes which have the IPNET_RTM_LLINFO flag set.

IP Domain

-inet

IPv4 route (default)

WNDRVR

-inet6

IPv6 route

-host

Forces the destination to be interpreted as a host.

-net

Forces the destination to be interpreted as a network.

Mask

-netmask *a.b.c.d*

Specifies an IPv4 destination netmask for -net routes. The netmask is stated in dotted decimal notation (*a.b.c.d*), where each portion of the mask is an integer between 0-255, representing the value of the bytes in that position in the mask.

-prefixlen *val*

Prefix length for -net routes.

If the destination is directly reachable via an interface requiring no intermediary system to act as a gateway, specify the -iface route flag; the gateway given is the address of this host on the common network, indicating the interface to be used for transmission. For further information on -iface, see MAKE REF!

The optional -netmask qualifier is used to manually add subnet routes with netmasks different from that of the implied network interface. Specify an additional address parameter (to be interpreted as a network mask). You can override the implicit network mask generated in the AF_INET case by using this option after the destination parameter. You can also use the modifier -prefixlen for similar purposes in the IPv6 case.

Route Flags

Routes have associated flags, which influence operation of the protocols when the router transmits packets to destinations matched by those routes. To set (or sometimes clear) these flags, indicate the corresponding modifiers as follows:

-cloning

IPNET_RTF_CLONING - generates a new route on use.

-iface

~ IPNET_RTF_GATEWAY - destination is directly reachable.

-notdone

~IPNET_RTF_DONE - unresolved ARP/NDP entry.

-static

IPNET_RTF_STATIC - manually added route.

-nostatic

~ IPNET_RTF_STATIC - pretend route added by kernel or daemon.

-reject

IPNET_RTF_REJECT - emit an ICMP unreachable when matched.

-blackhole

IPNET_RTF_BLACKHOLE - silently discard packets (during updates).

-llinfo

IPNET_RTF_LLINFO - validly translate proto address to link address.

-xresolve

IPNET_RTF_XRESOLVE - emit a message on use (for external lookup).

-proto1

IPNET_RTF_PROTO1 - set protocol specific routing flag #1.

-proto2

IPNET_RTF_PROTO2 - set protocol specific routing flag #2.

-pref

IPNET_RTF_PREF - always prefer this route.

-tunnelend

~IPNET_RTF_TUNNELEND - gateway specifies tunnel endpoint.

-mbinding

~IPNET_RTF_MBINDING - Packet sent along this route must be sent to multiple egress interfaces and/or multiple gateways.

-skip

~IPNET_RTF_SKIP - Stop searching this FIB and move to next one.

Metric Flags

-rtt

Provides an initial value for round trip time to quantities maintained in the routing entry by transport level protocols.

-rttvar

Provides an initial value for round trip time variance to quantities maintained in the routing entry by transport level protocols.

-mtu

Provides an initial value for maximum transmission unit to quantities maintained in the routing entry by transport level protocols.

-hopcount

Provides an initial value for hopcount to quantities maintained in the routing entry by transport level protocols.

-expire

Lifetime of a route entry.

Vr Flags

-V *vr*

Specifies virtual router index. 0 if not specified. Virtual route #0 is the default VR instance that is always present and cannot be deleted.

-add

Used in conjunction with the **vr** sub command to **route**. Adds a new VR.

For example, the command **route vr -add 1** adds VR#1 to the node. VR#1 is assigned one FIB with table ID == IPCOM_ROUTE_TABLE_DEFAULT, which is 254 by default. The command **route vr -T 123 -add 1** adds another FIB to VR#1; this FIB has a table ID of **123**. The only way a FIB with a table ID != IPCOM_ROUTE_TABLE_DEFAULT will be used is if a policy routing rule that says if (PRED), then do the lookup in the table with ID == X.

where PRED is something like:

- true if the ingress interface is X
- true if the source address is X
- true if the destination network is X and the ingress interface is Y
- true if the IP protocol is TCP and the source port is between X and Y

-delete

Deletes a VR and all its FIBs or deletes a single FIB if used with the -T *table* switch. Note: The only way to delete the default FIB is to delete the whole VR.

-addbyname

Adds a VR that is referenced by name instead of number. This switch is similar to -add, but the *vr_index* is a (node-unique) name instead of a number. The stack will select an unused VR# for this VR.

Table Index

-T *table*

Specifies the route table ID. If no route table ID is specified, the default (defined by IPCOM_ROUTE_TABLE_DEFAULT) is used.

Optional Modifiers

The optional modifiers -rtt, -rttvar, -mtu, -hopcount, and -expire provide initial values to quantities maintained in the routing entry by transport level protocols, such as TCP. In a **change** or **add** command where the destination and gateway are not sufficient to specify the route (as in the ISO case, where several interfaces may have the same address), the router can use the -ifp or -ifa modifiers to determine the interface or interface address.

Examples

Add the default IPv4 gateway to 10.1.1.1:

```
# route add default 10.1.1.1
```

Add the IPv4 network route 14.1/16 on the specified interface:

```
# route add -dev eth0 -net -netmask 255.255.0.0 14.1.0.0
```

Add an IPv4 route for host 15.1.6.7 to gateway 10.1.1.22:

```
# route add -host 15.1.6.7 10.1.1.22
```

Add the default IPv6 gateway to FEC0::1:0:0:0:6:

```
# route add -inet6 default FEC0::1:0:0:0:6
```

Add a new virtual router:

```
# route vr -add 1
```

Previous topic: radvdconfig on page 254
Next topic: syslog on page 261

18. syslog

The syslog shell command is used to send system log messages. The component for syslog is INCLUDE_IPCOM_SYSLOGD_CMD.

Name

syslog - system log command

Synopsis

```
syslog echo prio message
syslog list
syslog priority facility prio
syslog log file [logfile]
```

Description

Command options are:

prio

Priority level of the specified message. Options include:

- Emerg
- Crit
- Error
- Warning
- Notice
- Info
- Debug
- Debug2

message

The specified message.

facility prio

Priority level of the specified facility. Some of the following options may not be available with your product. Options for *facility* include:

- kern
- user
- daemon

- auth
- syslog
- ipcom
- ipcom_shell
- ipcom_telnet
- ipcrypto
- ipipsec
- ipike
- ipl2tp
- ipnat
- ippppoe
- ipradius
- iprip
- ipssh
- ipssl
- ipftp
- ipfirewall
- ipdhcpd
- ipdhcpc
- ipnet
- iptftp
- ipsntp
- ipdhcps
- ipdhcps6
- ipsnmp
- ipdhcpr
- ipcom_drv_eth
- ipppp
- ipmip
- ipappl
- iptcp
- ipfreescale
- ipmcp
- ip8021x
- ipeap
- iphwcrypto
- ipnetsnmp
- ipdhcpc6
- ipdiameter
- ***ILLEGAL***

logfile

The specified log file.

The table below lists the syslog debug levels.

Level Name	No.	Workbench 4 Configuration Parameter	Priority
EMERGENCY	0	IPCOM_LOG_EMERG	Highest priority.
CRITICAL	1	IPCOM_LOG_CRIT	
ALERT		Not supported in VxWorks--only in Linux.	
ERROR	2	IPCOM_LOG_ERR	Default log output level at run time, as defined by **IPCOM_SYSLOGD_DEFAULT_PRIORITY**.
WARNNING	3	IPCOM_LOG_WARNING	This level and higher compiled in for normal builds.
NOTICE	4	IPCOM_LOG_NOTICE	
INFO	5	IPCOM_LOG_INFO	
DEBUG	6	IPCOM_LOG_DEBUG	Lowest priority. This level and higher compiled in for debug builds.
DEBUG 2	7	IPCOM_LOG_DEBUG2	Default level when debug flag is used.

Previous topic: route and routec on page 255
Next topic: sysvar on page 263

19. sysvar

The sysvar command lists and modifies global variables.

System variables are similar to UNIX environment variables, except that they are available throughout the system to any process. For example, if you are running an IKE process, you can issue a sysvar command to alter a network address translation (NAT) variable.

The sysvar command uses a treelike data structure for all the network components and services. For example, the system variable *iptcp.ConnectionTimeout* defines the number of seconds the network stack tries to create connection before giving up.

The system variable *ipssh.service.port_fwd* controls whether port forwarding can be used.

System variables are similar to the components you configure at build time using the Kernel Configuration Editor. Unlike kernel components, however, the sysvar command modifies parameters at run time.

> Note: Not all global variables can be changed at run time. Some parameters can only be reset at build time, using Workbench 4 or **wrtool**.

To include this command in your project, include INCLUDE_IPCOM_SYSVAR_CMD.

Name

sysvar - lists, gets, and defines system variables

Synopsis

```
sysvar list [name [*]]
sysvar get  name
sysvar unset  name [*]
sysvar set [-c | -o | -r] name value
```

Description

Command options are:

name

Name of a system variable.

-c

OK to create.

-o

OK to overwrite.

-r

Flag read-only.

value

Value of a system variable.

Previous topic: syslog on page 261
Next topic: tcpao on page 264

20. tcpao

tcpao - TCP-AO command

Name

Synopsis

```
tcpao add  keys  [optional]
tcpao delete  keys  [optional]
tcpao update keys  [optional]
tcpao flush keys  [optional]
tcpao dump keys  [optional]
```

Description

The tcpao command adds, deletes, updates, flushes, and dumps master key tuples (MKTs).

tcpao add

For tcpao add, the *keys* are as follows:

-src *ip*

Source IP address.

-dst *ip*

Destination IP address.

-srcport *port*

Source port.

-dstport *port*

Destination port.

-mkey *key*

Master key (in string format).

For tcpao add, the *options* are as follows:

-srcmax *ip*

Source maximum IP address (for linear range).

-dstmax *ip*

Destination maximum IP address (for linear range).

-srcmask *mask*

Source mask. The default is all 1s.

-dstmask *mask*

Destination mask. The default is all 1s.

-srcmaxport *port*

Source maximum port (for linear range).

-dstmaxport *port*

Destination maximum port (for linear range).

- auth [sha1 |aescmac]

Authentication algorithm. The default is sha1.

-kdf [sha1 |aescmac]

Key Derivation Function (KDF). The default is sha1.

-sndid *val*

Send key ID (0-255). The default is 0.

WNDRVR

-rcvid *val*

Receive key ID (0-255). The default is 0.

-optflag [enable | disable]

Whether options are in MAC. The default is enable.

tcpao delete

For tcpao delete, the *keys* are as follows:

-src *ip*

Source IP address.

-dst *ip*

Destination IP address.

-srcport *port*

Source port.

-dstport *port*

Destination port.

For tcpao delete, the *options* are as follows:

-srcmax *ip*

Source maximum IP address (for linear range).

-dstmax *ip*

Destination maximum IP address (for linear range).

-srcmask *mask*

Source mask. The default is all 1s.

-dstmask *mask*

Destination mask. The default is all 1s.

-srcmaxport *port*

Source maximum port (for linear range).

-dstmaxport *port*

Destination maximum port (for linear range).

-sndid *val*

Send key ID (0-255). The default is 0.

tcpao update

For tcpao update, the *keys* are as follows:

-src *ip*

Source IP address.

-dst *ip*

Destination IP address.

-srcport *port*

Source port.

-dstport *port*

Destination port.

For tcpao update, the *options* are as follows:

-srcmax *ip*

Source maximum IP address (for linear range).

-dstmax *ip*

Destination maximum IP address (for linear range).

-srcmask *mask*

Source mask. The default is all 1s.

-dstmask *mask*

Destination mask. The default is all 1s.

-srcmaxport *port*

Source maximum port (for linear range).

-dstmaxport *port*

Destination maximum port (for linear range).

-sndid *val*

Send key ID (0-255). The default is 0.

For tcpao update, the *modifiers* are as follows:

- auth [sha1 |aescmac]

Authentication algorithm. The default is sha1.

-kdf [sha1 |aescmac]

Key Derivation Function (KDF). The default is sha1.

-rcvid *val*

Receive key ID (0-255). The default is 0.

-mkey *key*

Master key (in string format).

-optflag [enable | disable]

Whether options are in MAC. The default is enable.

tcpao flush

For tcpao flush, the *options* are as follows:

-4

Only flush IPv4 MKT.

-6

Only flush IPv6 MKT.

tcpao dump

For tcpao dump, the *options* are as follows:

-4

Only dump IPv4 MKT.

-6

Only dump IPv6 MKT.

-src *ip*

Source IP address.

-dst *ip*

Destination IP address.

-srcport *port*

Source port.

-dstport *port*

Destination port.

-srcmax *ip*

Source maximum IP address (for linear range).

-dstmax *ip*

Destination maximum IP address (for linear range).

-srcmask *mask*

Source mask. The default is all 1s.

-dstmask *mask*

Destination mask. The default is all 1s.

-srcmaxport *port*

Source maximum port (for linear range)

-dstmaxport *port*

Destination maximum port (for linear range)

-sndid *val*

Send Key ID (0-255). The default is 0.

Previous topic: sysvar on page 263

Next topic: traceroute on page 269

21. traceroute

traceroute - Trace route shell command for IPv4 and IPv6

Name

Synopsis

```
traceroute [options] node
```

Description

-c *num*

Set number of probes per hop limit (default 3)

-n

Do not perform DNS lookup on addresses

-F

Set the "don't fragment" bit.

-I

Use ICMP ECHO instead of UDP datagrams.

-v

Verbose output. For each hop, the size and the destination of the response packets is displayed. ICMP packets received other than TIME_EXCEEDED and UNREACHABLE are also listed.

-f *first_ttl*

Set the starting TTL value to *first_ttl* to override the default value of 1. traceroute skips processing for those intermediate gateways which are less than *first_ttl* hops away.

-m *max_ttl*

Set the maximum TTL used in outgoing probe packets. Default: 30 hops.

-p *port*

Specifies the starting destination port number when traceroute is used in UDP mode. This parameter specifies a port number that is not in use by the destination so that it can return an ICMP_PORT_UNREACHABLE message to terminate the route tracing. The range of valid values is 2,048 - 60,000. Default: 33,434.

-w *waittime*

Set the time, in seconds, to wait for a response to a probe. Default: 5 sec.

Previous topic: tcpao on page 264
Next topic: ttcp on page 270

22. ttcp

ttcp - test TCP/UDP transmission rate

Name

Synopsis

```
ttcp -t [- options] host [ <in ]
ttcp -r [-options > out]
```

Description

This command tests the TCP connection. It makes a connection on default port 5001 and transfers fabricated buffers or data copied from stdin.

The ttcp options are:

-x

Use IPv6 instead of IPv4.

-l *buf_len*

Length of buffers read from or written to network (default 8192).

-U

Use UDP and IPCOM zero copy API instead of TCP.

-u

Use UDP instead of TCP.

-p *port_num*

Port number to send to or listen at (default DEFAULT_PORT).

-s - t:

Source a pattern to network.

-r

Sink (discard) all data from network.

-e *pkt_num*

Number of packets to end UDP transmission.

-m

Use event logging (not supported on all platforms).

-A

Align the start of buffers to this modulus (default 16384).

-O

Start buffers at this offset from the modulus (default 0).

-v

Verbose; print more statistics.

-d

Set SO_DEBUG socket option (if supported).

-b *buf_size*

Set socket buffer size (if supported).

The -t flag has the following options:

-n *buf_num*

Number of source buffers written to network (default 2048).

-D

Don't buffer TCP writes (sets TCP_NODELAY socket option).

The -r flag has the following options:

-B (for -s only)

Output full blocks as specified by -l (for TAR).

-T

Touch, or access, each byte as it is read.

-V

Virtual router index (if supported).

Previous topic: traceroute on page 269
Next topic: user on page 271

23. user

Name

user - VxWorks user management shell command

Synopsis

```
user add [-p password] [-u user_Id] [-g groupId] loginName
user del loginName
user mod [-g newGroupId] [-l newLoginName] [-d] [-e] loginName
```


WNDRVR

Description

The user add command registers a new user allowed to log in the system. The compulsory argument is the login name of that new user.

Command options are as follows:

password

The new user's cleartext password. No password is set for the user if this option is not used.

user_Id

User ID of the specified user. This argument must be a unique positive numerical value greater than 0 and lower than 65536.

groupId

Group ID of the specified user. This argument must be a unique positive numerical value greater than 0 and lower than 65536.

loginName

User name of the specified user. The user name can be up to 80 bytes long.

The user del command deletes an existing user from the system. The compulsory argument is the login name of that existing user.

The user mod command modifies some of an existing user's information. The compulsory argument is the login name of that existing user.

Command options are as follows:

-g

The user's new primary group ID. This argument must be a unique positive numerical value greater than 0 and lower than 65536.

-l

The user's new login name. This name must not already be in use.

-d

Disables the account.

-e

Enables the account.

loginName

Name of the existing user.

Previous topic: ttcp on page 270
Next topic: whoami on page 272

24. whoami

Name

whoami - Display the name of the current shell session's user

Synopsis

```
whoami
```

Description

The whoami command displays the login name corresponding to the user logged in the current shell session.

Previous topic: user on page 271